Vorabfragen

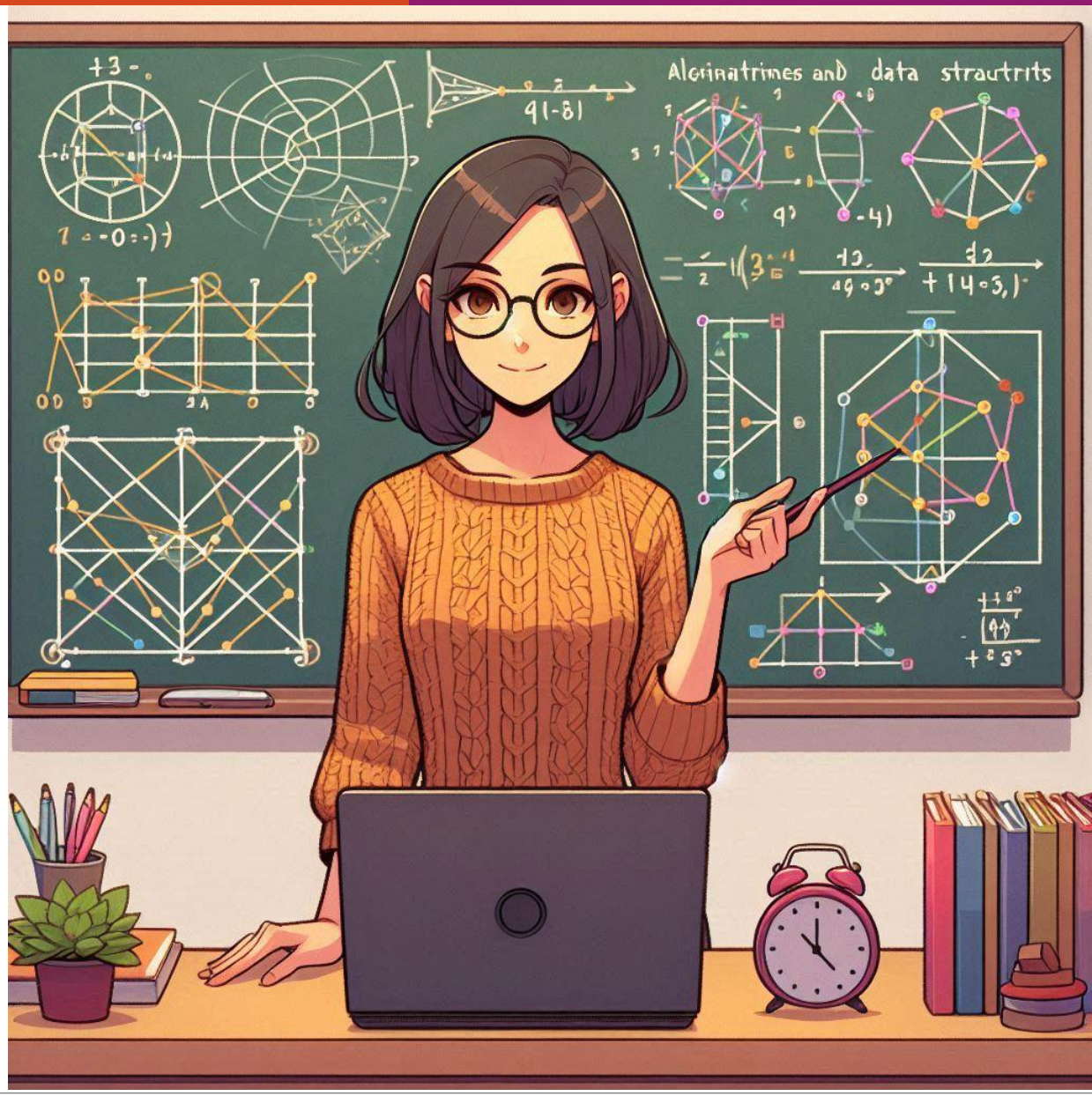
- Wie würden Sie die Voraussetzungen im Modulhandbuch als Graph modellieren? Was sind die Knoten, was die Kanten? Ist der Graph gerichtet oder ungerichtet? Gewichtet oder ungewichtet?

- Sie wollen prüfen, ob es in einem Graphen einen Pfad von s nach t gibt. Dazu probieren Sie rekursiv alle Nachbarn von s aus, dann deren Nachbarn usw. Was kann passieren, wenn der Graph einen Zyklus enthält? Wie könnten Sie das verhindern?

- Auf einer Karte hat jede Straße eine Länge. Die Breitensuche findet den Weg mit den wenigsten Kanten. Ist das automatisch auch der kürzeste Weg? Warum (nicht)? Und was könnte schiefgehen, wenn eine Kante ein negatives Gewicht hätte?

Algorithmik – WS 26/27

Gelesen von Prof. Dr. Daniel Gaida



Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

DALL-E 3: Mit KI erstellt
11. September 2024 um 10:06 AM

Technology
Arts Sciences
TH Köln

Lernraum III: Graphen

- Graphen (un-/gerichtet, un-/gewichtet)
- Traversierung von Graphen
  - Tiefensuche
  - Breitensuche
-
  -
-
  -
  -
-

Lernraum III: Graphen

- Graphen (un-/gerichtet, un-/gewichtet)
- Traversierung von Graphen
  - Tiefensuche
  - Breitensuche
- Gerichtete Graphen
  - Topologische Sortierung
- Kürzeste Pfade
  - Ungewichteter Graph: Breitensuche
  - Gewichteter Graph: Algorithmus von Dijkstra
- Minimale Spannbäume

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Technology Arts Sciences TH Köln

Übersicht über heute

- Graphen
  - Einführung/Anwendungen/Begriffe
- Traversierung von Graphen
  - Erreichbarkeitsproblem
  - Tiefensuche
  - Breitensuche
-
  -
-
  -
  -

Übersicht über heute

- Graphen
  - Einführung/Anwendungen/Begriffe
- Traversierung von Graphen
  - Erreichbarkeitsproblem
  - Tiefensuche
  - Breitensuche
- Gerichtete Graphen
  - Topologische Sortierung
- Kürzeste Pfade
  - Ungewichteter Graph: Breitensuche
  - Gewichteter Graph: Algorithmus von Dijkstra

Beziehungen zwischen Daten

Arrays

Einem Datentyp zugehörig

Manchmal sortiert

Typischerweise unabhängig voneinander

Elemente speichern nur reine Daten, keine Verbindungsinformationen

0	1	2
A	B	C

Bäume

Graphen

Prof. Dr. Daniel Gaida

Professor für Cyber-Physische Systeme

Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

CSE 373 18 SP - KASEY CHAMPION

Beziehungen zwischen Daten

Arrays

Einem Datentyp zugehörig

Manchmal sortiert

Typischerweise unabhängig voneinander

Elemente speichern nur reine Daten, keine Verbindungsinformationen

0	1	2
A	B	C

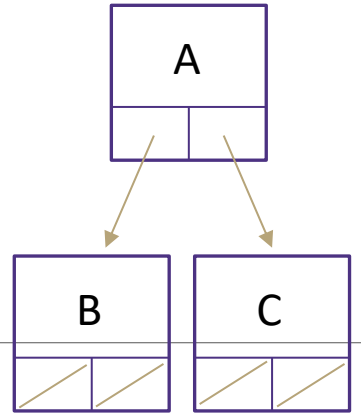
Bäume

Direktionale Beziehungen

Geordnet für einfachen Zugriff

Begrenzte Verbindungen

Knoten speichern Daten und Verbindungsinformationen



Graphen

Prof. Dr. Daniel Gaida

Professor für Cyber-Physische Systeme

Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

CSE 373 18 SP - KASEY CHAMPION

Beziehungen zwischen Daten

Arrays

Einem Datentyp zugehörig

Manchmal sortiert

Typischerweise unabhängig voneinander

Elemente speichern nur reine Daten, keine Verbindungsinformationen

0	1	2
A	B	C

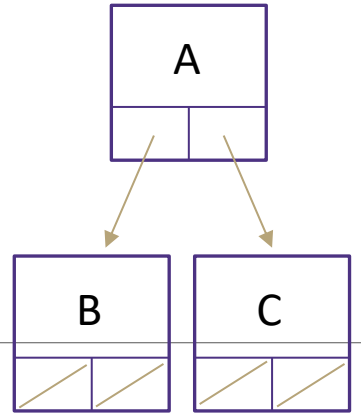
Bäume

Direktionale Beziehungen

Geordnet für einfachen Zugriff

Begrenzte Verbindungen

Knoten speichern Daten und Verbindungsinformationen



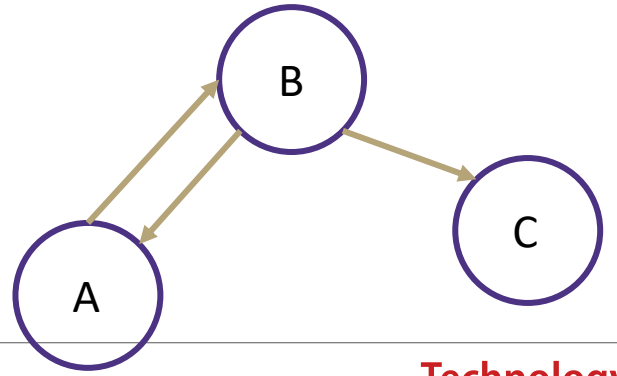
Graphen

Mehrere Beziehungen

Beziehungen diktieren die Struktur

Verbindungsfreiheit!

Sowohl Knoten als auch Kanten können Daten speichern



Prof. Dr. Daniel Gaida

Professor für Cyber-Physische Systeme

Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

CSE 373 18 SP - KASEY CHAMPION

Beziehungen zwischen Daten

Arrays

Einem Datentyp zugehörig

Manchmal sortiert

Typischerweise unabhängig voneinander

Elemente speichern nur reine Daten, keine Verbindungsinformationen

0	1	2
A	B	C

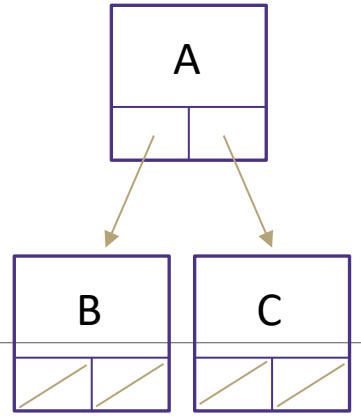
Bäume

Direktionale Beziehungen

Geordnet für einfachen Zugriff

Begrenzte Verbindungen

Knoten speichern Daten und Verbindungsinformationen



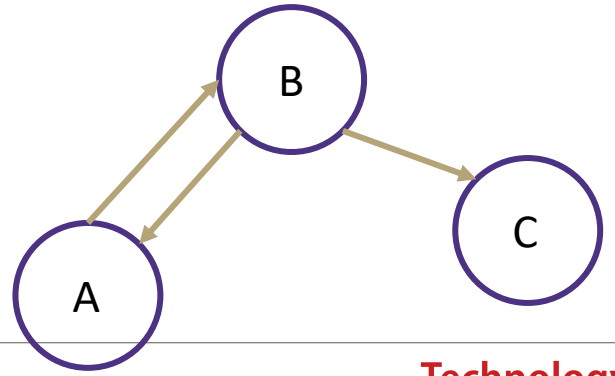
Graphen

Mehrere Beziehungen

Beziehungen diktieren die Struktur

Verbindungsfreiheit!

Sowohl Knoten als auch Kanten können Daten speichern



Prof. Dr. Daniel Gaida

Professor für Cyber-Physische Systeme

Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

CSE 373 18 SP - KASEY CHAMPION

Graphen

Alles ist ein Graph.

Die meisten Dinge, die wir in diesem Modul untersucht haben, lassen sich durch Graphen darstellen.

- Binäre Suchbäume, Verkettete Listen, Heaps:
  - Können alle als Graph dargestellt werden.

Graphen

Alles ist ein Graph.

Die meisten Dinge, die wir in diesem Modul untersucht haben, lassen sich durch Graphen darstellen.

- Binäre Suchbäume, Verkettete Listen, Heaps:
  - Können alle als Graph dargestellt werden.

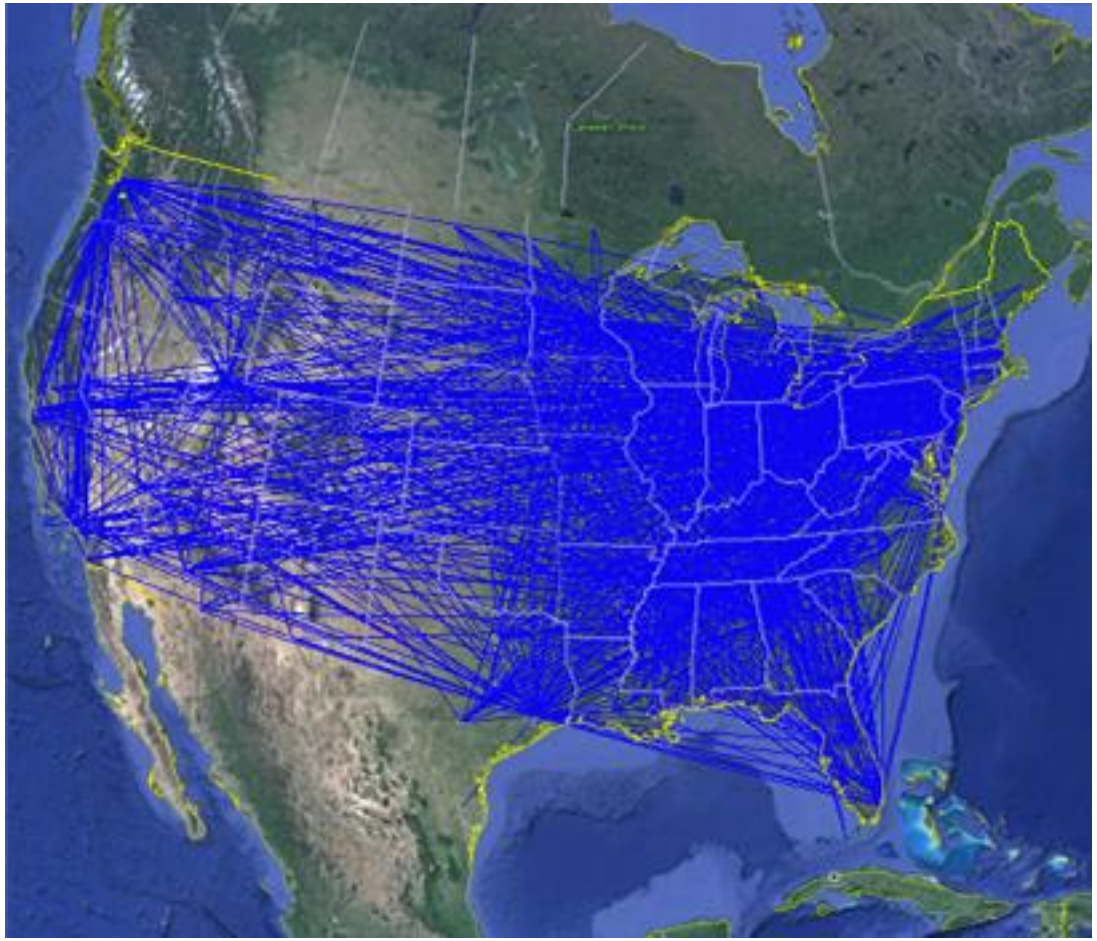
Aber es sind nicht nur Datenstrukturen, die wir besprochen haben...

- Google Maps? Ein Graph.
- Facebook, Twitter (X)? Ein Graph.
- Gitlab, Github: Historie eines Repositories (Forks, Branches, …)? Graph.
- Modul-Voraussetzungen im Modulhandbuch? Graph.
- Stammbaum? Ein Graph.

Graphen Anwendungen (1/4)

Physikalische Karten

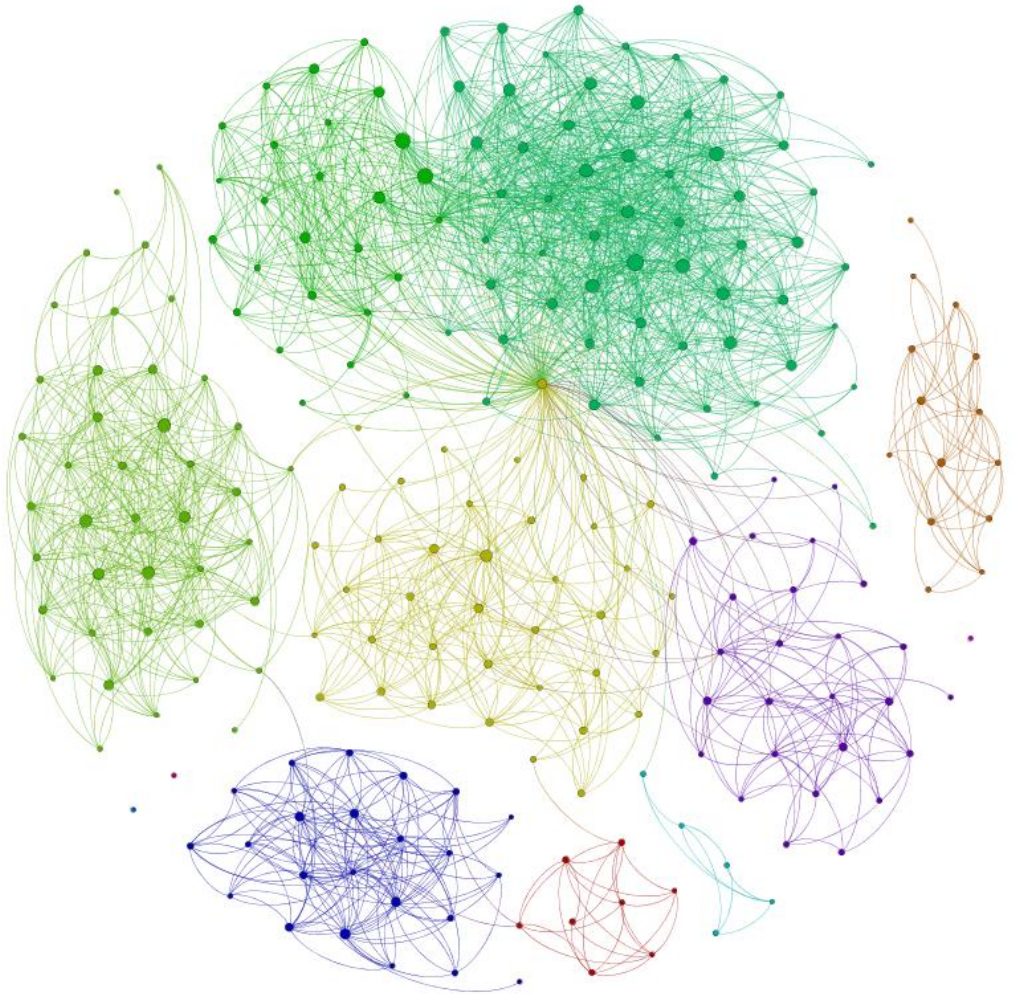
- Fluglinienkarten
  - Knoten sind Flughäfen, Kanten sind Flugrouten
- Verkehr
  - Knoten sind Adressen, Kanten sind Straßen
- Telefon-, Strom-/Gas-, Kanalnetz
  - Knoten sind Häuser, Kanten sind Kabel/Rohre



Graphen Anwendungen (2/4)

Beziehungen

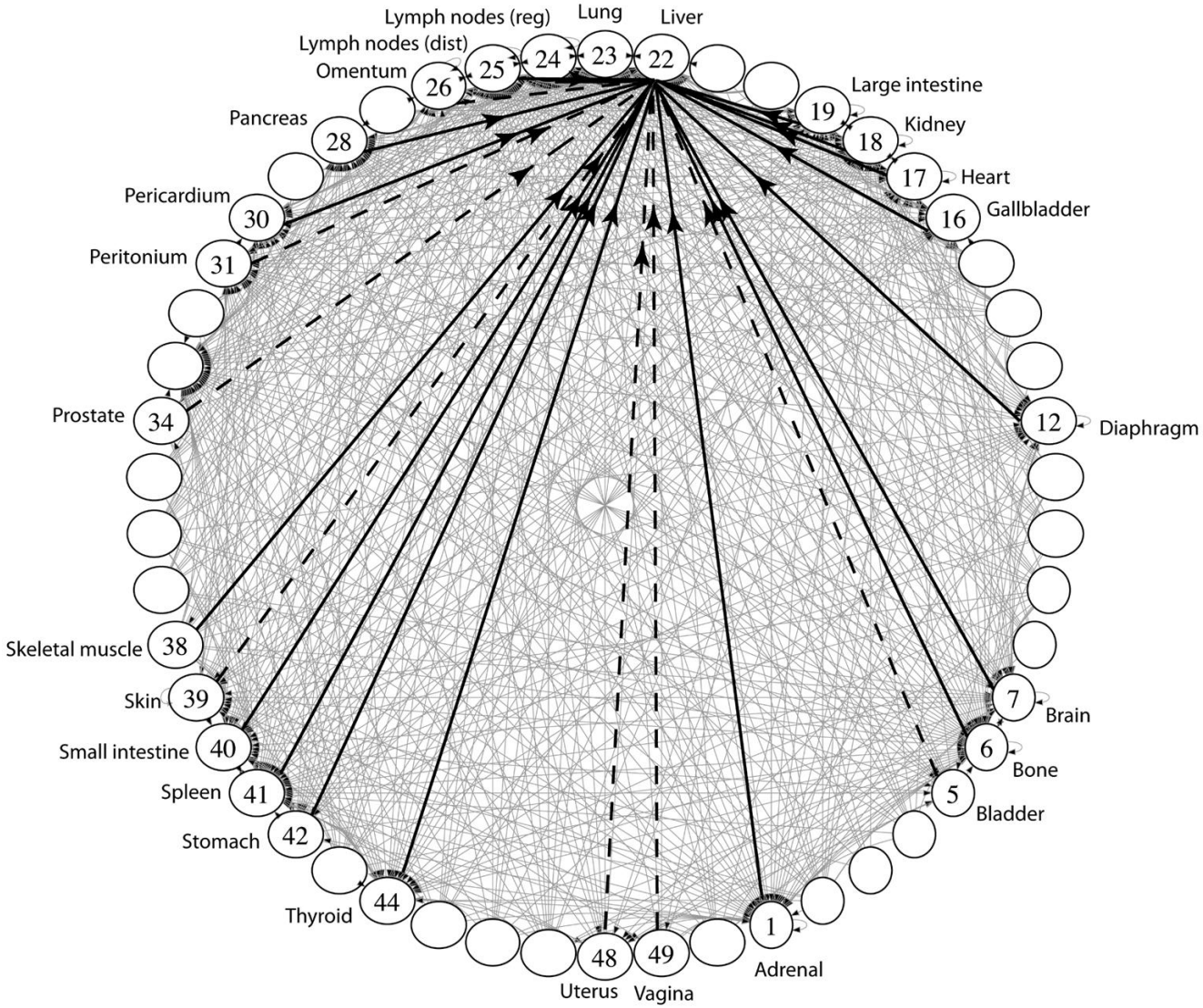
- Soziale Netzwerke
  - Knoten sind Accounts,
  - Kanten sind Follower-Beziehungen
- UML Klassendiagramme
  - Knoten sind Klassen, Kanten sind Verwendungen



Graphen Anwendungen (3/4)

Beeinflussung

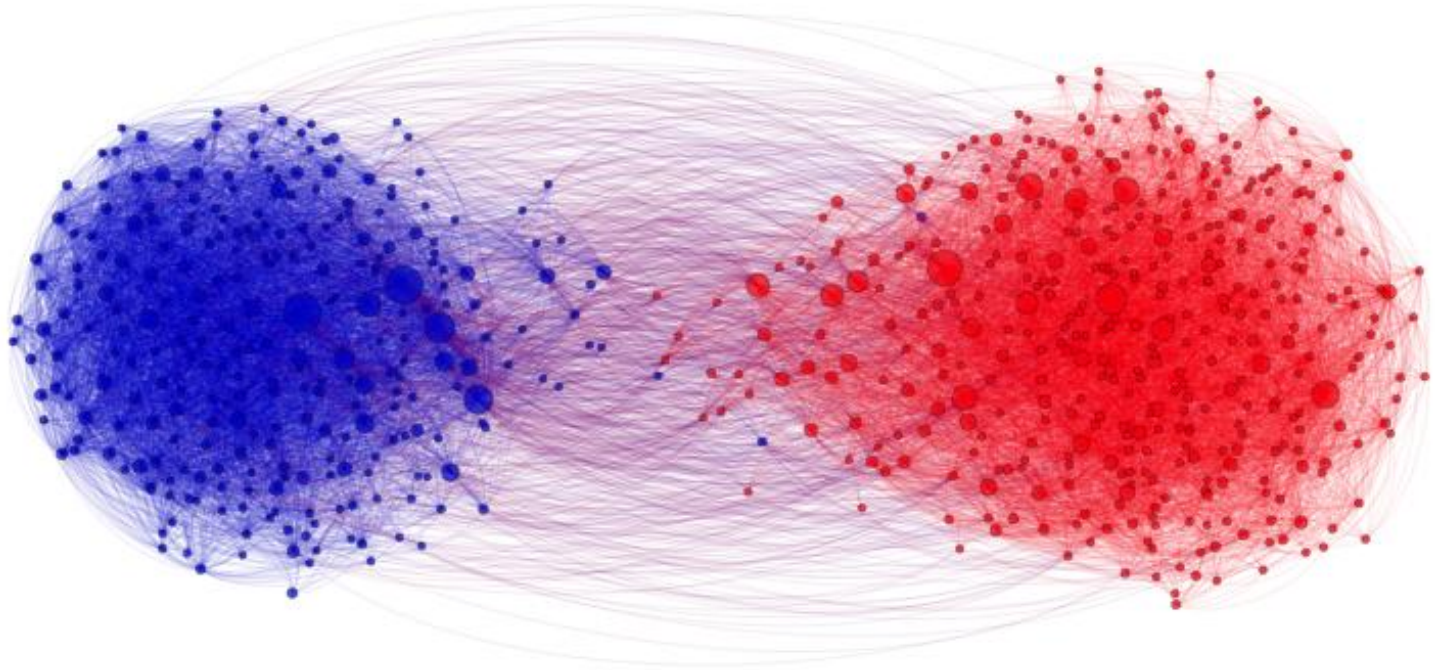
- Biologie
  - Knoten sind Ziele von Krebszellen,
  - Kanten sind Migrationspfade



Graphen Anwendungen (4/4)

Verwandte Themen

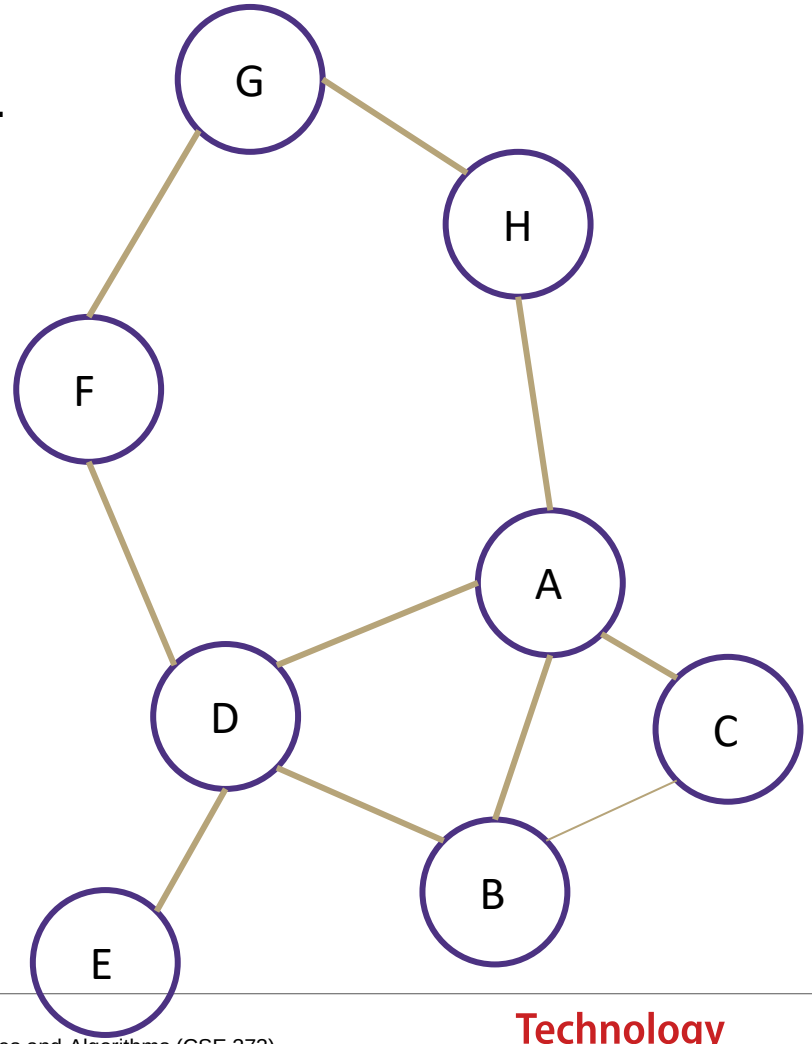
- Web Page Ranking
  - Knoten sind Webseiten, Kanten sind Hyperlinks
- Wikipedia
  - Knoten sind Artikel, Kanten sind Links



Graph: Formale Definition

Ein Graph ist definiert durch ein **Paar** von **Mengen** G = (V, E), wobei...

- V ist die Menge der Knoten
  - Ein Knoten ist eine Dateneinheit
- V = { A, B, C, D, E, F, G, H }
-
  -
  -



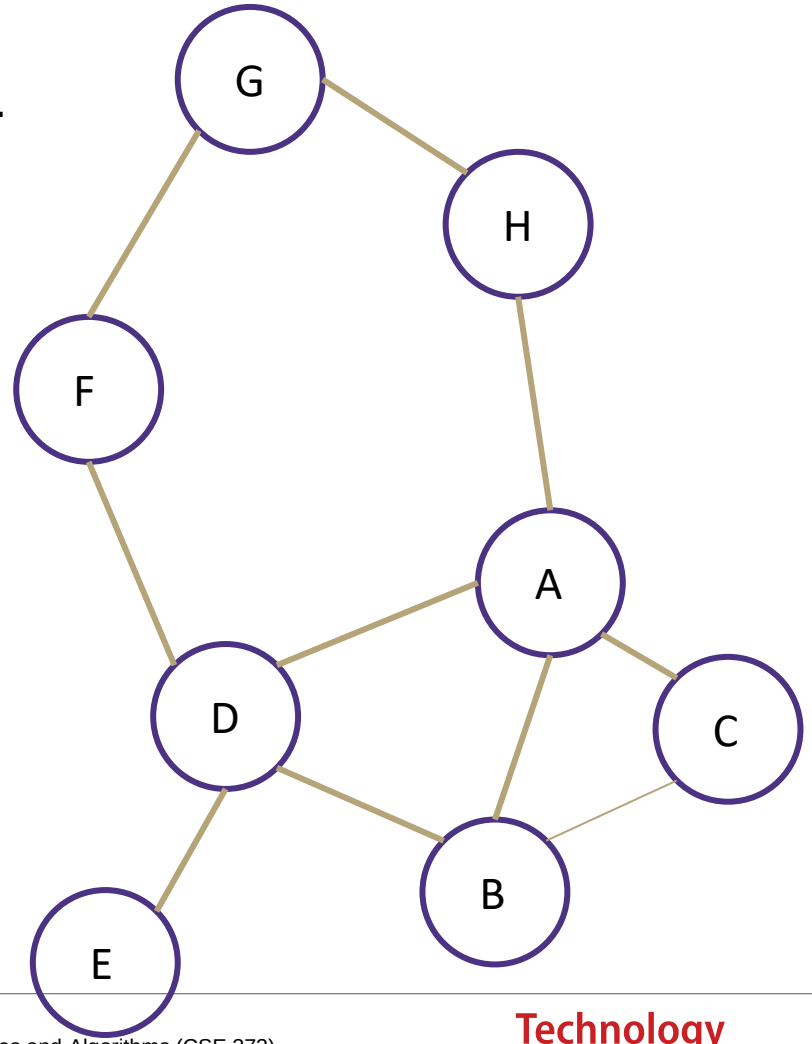
Graph: Formale Definition

Ein Graph ist definiert durch ein **Paar** von **Mengen** G = (V, E), wobei...

- V ist die Menge der Knoten
  - Ein Knoten ist eine Dateneinheit
- V = { A, B, C, D, E, F, G, H }

- E ist die Menge der Kanten
  - Eine Kante ist eine Verbindung zwischen zwei Knoten

  - E = { (A, B), (A, C), (A, D), (A, H), (C, B),
    (B, D), (D, E), (D, F), (F, G), (G, H)}



Graphen Terminologie (1/4)

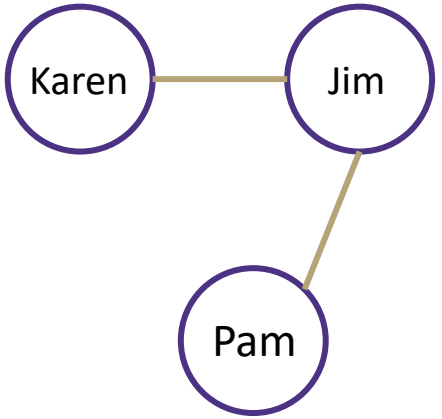
Richtung des Graphen

- **Ungerichtete Graphen** - Kanten haben keine Richtung und sind bidirektional
  -
  -
-
  -
  -

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022
CSE 373 20 SP - KASEY CHAMPION

Graphen Terminologie (1/4)

Ungerichteter Graph:



Richtung des Graphen

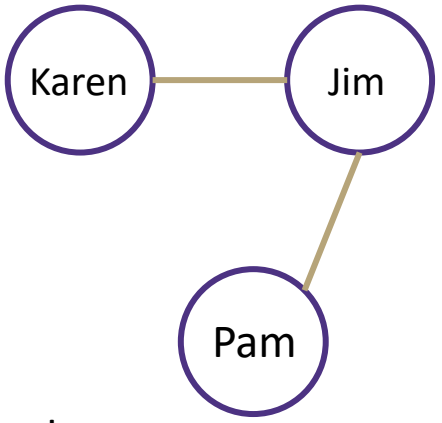
- **Ungerichtete Graphen** - Kanten haben keine Richtung und sind bidirektional
  - V = { Karen, Jim, Pam }
  - E = { (Jim, Pam), (Jim, Karen) } daraus folgt (Karen, Jim) und (Pam, Jim)
-
  -
  -

Graphen Terminologie (1/4)

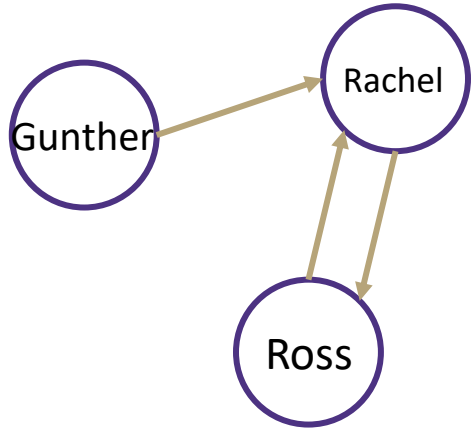
Richtung des Graphen

- **Ungerichtete Graphen** - Kanten haben keine Richtung und sind bidirektional
  - V = { Karen, Jim, Pam }
  - E = { (Jim, Pam), (Jim, Karen) } daraus folgt (Karen, Jim) und (Pam, Jim)
- **Gerichtete Graphen** - Kanten haben eine Richtung und sind daher nur in einer Richtung gültig
  - V = { Gunther, Rachel, Ross }
  - E = { (Gunther, Rachel), (Rachel, Ross), (Ross, Rachel) }

Ungerichteter Graph:



Gerichteter Graph:



Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022
CSE 373 20 SP - KASEY CHAMPION

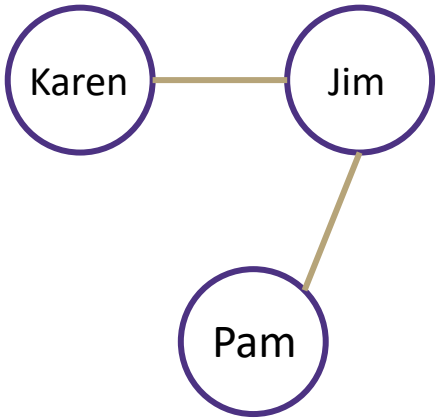
Graphen Terminologie (2/4)

Grad eines Knotens

- **Grad** - die Anzahl der Kanten, die mit diesem Knoten verbunden sind
  -
-
  -
-
  -

Graphen Terminologie (2/4)

Ungerichteter Graph:



Grad eines Knotens

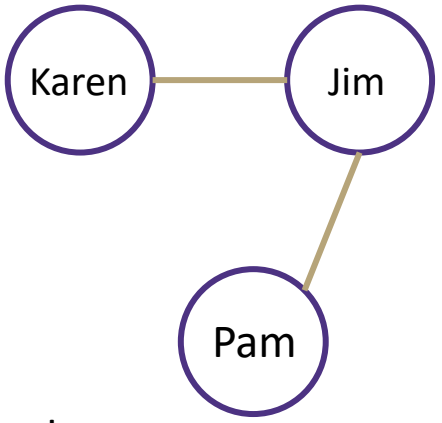
- **Grad** - die Anzahl der Kanten, die mit diesem Knoten verbunden sind
  - Karen: 1, Jim: 2, Pam: 1
-
  -
-
  -

Graphen Terminologie (2/4)

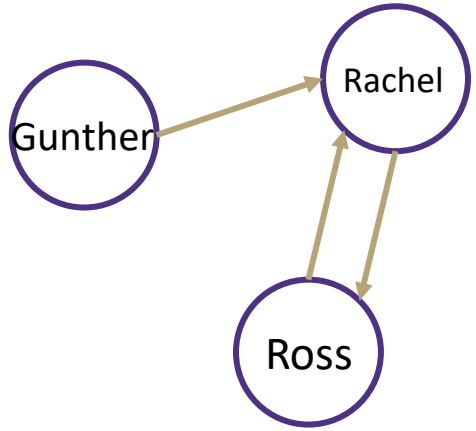
Grad eines Knotens

- **Grad** - die Anzahl der Kanten, die mit diesem Knoten verbunden sind
  - Karen: 1, Jim: 2, Pam: 1
- **Eingangsgrad** - die Anzahl der gerichteten Kanten, die in einen Knoten eintreten
  - Gunther: 0, Rachel: 2, Ross: 1
-
  -

Ungerichteter Graph:



Gerichteter Graph:

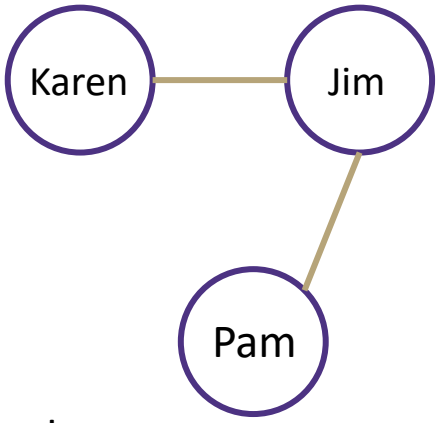


Graphen Terminologie (2/4)

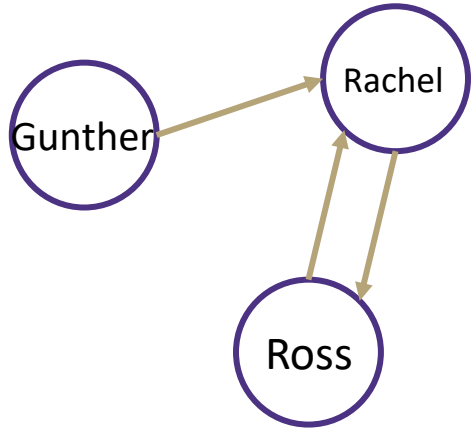
Grad eines Knotens

- **Grad** - die Anzahl der Kanten, die mit diesem Knoten verbunden sind
  - Karen: 1, Jim: 2, Pam: 1
- **Eingangsgrad** - die Anzahl der gerichteten Kanten, die in einen Knoten eintreten
  - Gunther: 0, Rachel: 2, Ross: 1
- **Ausgangsgrad** - die Anzahl der gerichteten Kanten, die aus einem Knoten austreten
  - Gunther: 1, Rachel: 1, Ross: 1

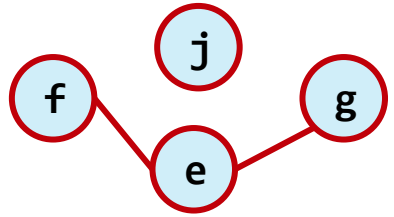
Ungerichteter Graph:



Gerichteter Graph:

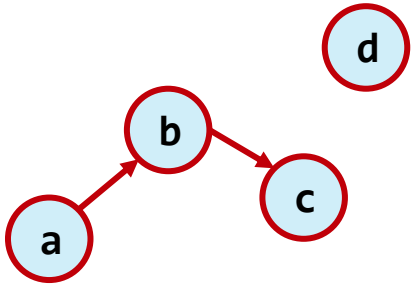


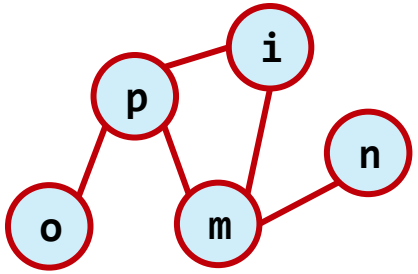
Graphen Terminologie (3/4)

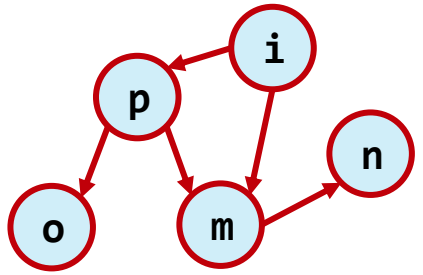


Ein **Pfad (Weg)** ist eine Folge von Knoten, die durch Kanten verbunden sind.

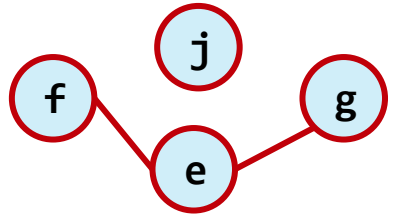
- Ein **einfacher Pfad** ist ein Pfad ohne sich wiederholende Knoten.
-
  -
-





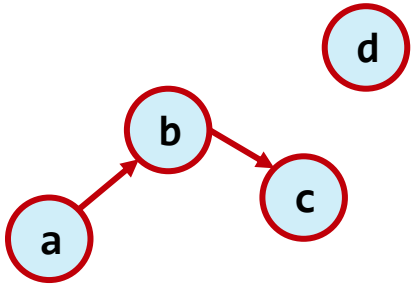


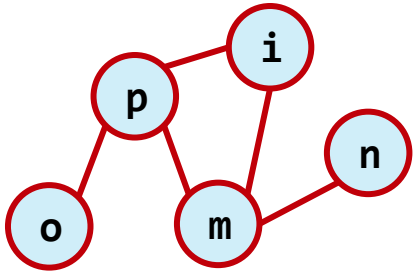
Graphen Terminologie (3/4)

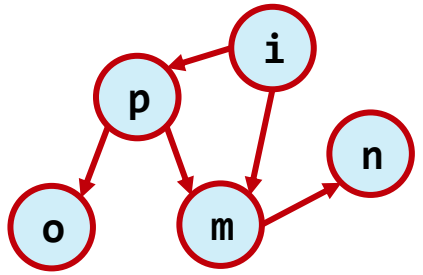


Ein **Pfad (Weg)** ist eine Folge von Knoten, die durch Kanten verbunden sind.

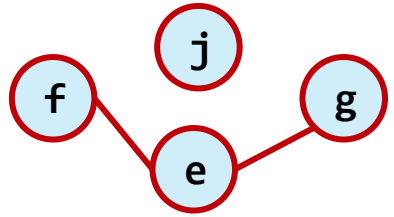
- Ein **einfacher Pfad** ist ein Pfad ohne sich wiederholende Knoten.
- Ein **Zyklus** ist ein Pfad, dessen erster und letzter Knoten derselbe ist.
  - Ein Graph mit einem Zyklus ist **zyklisch**.





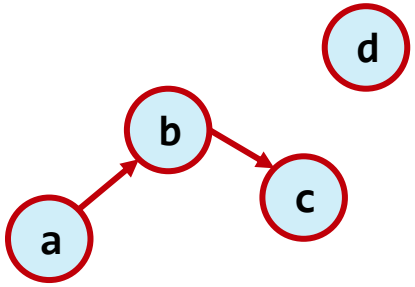


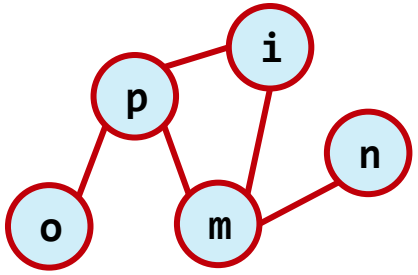
Graphen Terminologie (3/4)



Ein **Pfad (Weg)** ist eine Folge von Knoten, die durch Kanten verbunden sind.

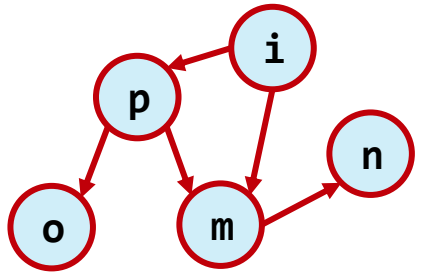
- Ein **einfacher Pfad** ist ein Pfad ohne sich wiederholende Knoten.
- Ein **Zyklus** ist ein Pfad, dessen erster und letzter Knoten derselbe ist.
  - Ein Graph mit einem Zyklus ist **zyklisch**.



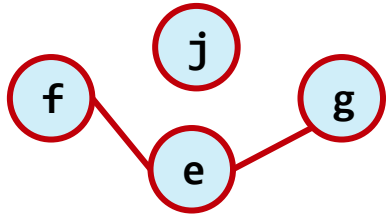


Länge eines Pfades:

- Anzahl an Kanten im Pfad

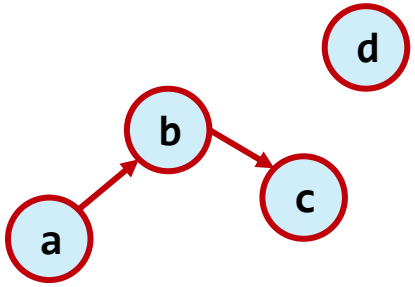


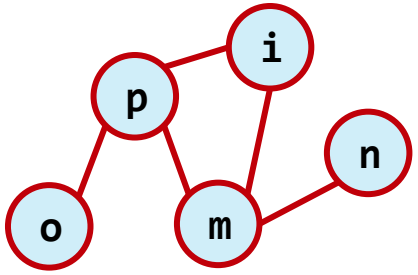
Graphen Terminologie (4/4)

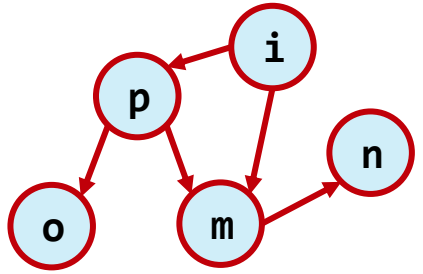


Zwei Knoten sind miteinander **verbunden**, wenn es einen **Pfad** zwischen ihnen gibt.

- Wenn alle Knoten miteinander verbunden sind, ist der Graph **zusammenhängend**.
- Bsp.: Stromnetz: Sind alle Haushalte angeschlossen (wenn ein Kabel durchtrennt wird)?

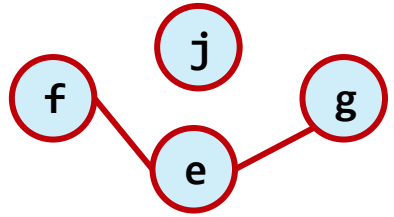






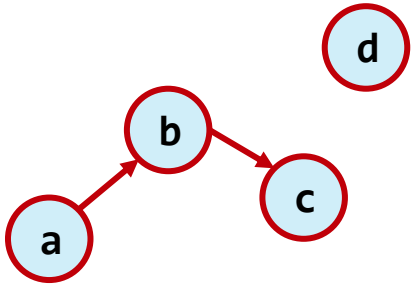
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Graphen Terminologie (4/4)



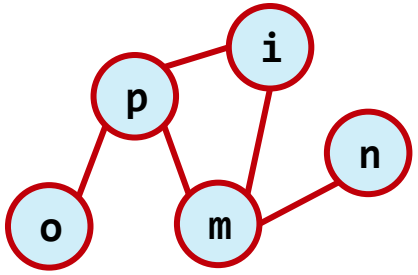
Zwei Knoten sind miteinander **verbunden**, wenn es einen **Pfad** zwischen ihnen gibt.

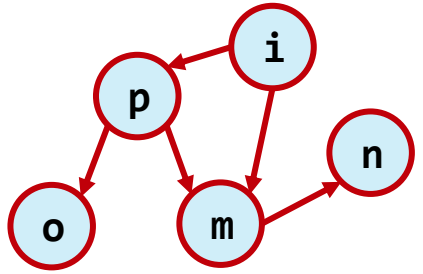
- Wenn alle Knoten miteinander verbunden sind, ist der Graph **zusammenhängend**.
- Bsp.: Stromnetz: Sind alle Haushalte angeschlossen (wenn ein Kabel durchtrennt wird)?



Zusammenhangskomponente:

- Ein **maximal zusammenhängender Teilgraph** eines ungerichteten Graphen
- Ein **Teilgraph** ist ein Teil eines Graphens (Knoten und Kanten)





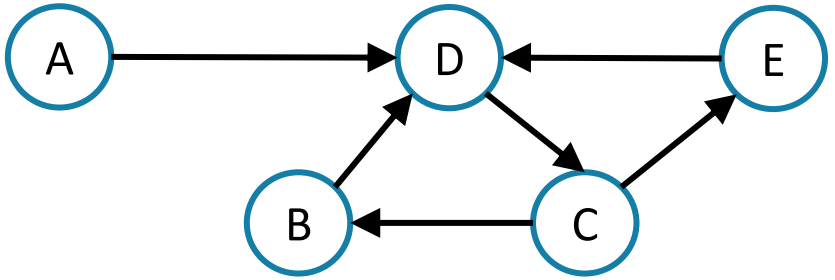
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Starke Zusammenhangskomponente

Starke Zusammenhangskomponente

Ein Teilgraph F, bei dem jedes Paar von Knoten in F über einen Pfad in **beide** Richtungen verbunden ist und es keinen anderen Knoten gibt, der mit jedem Knoten von F in beide Richtungen verbunden ist.

Ein **maximal stark zusammenhängender Teilgraph** eines gerichteten Graphen



Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2023

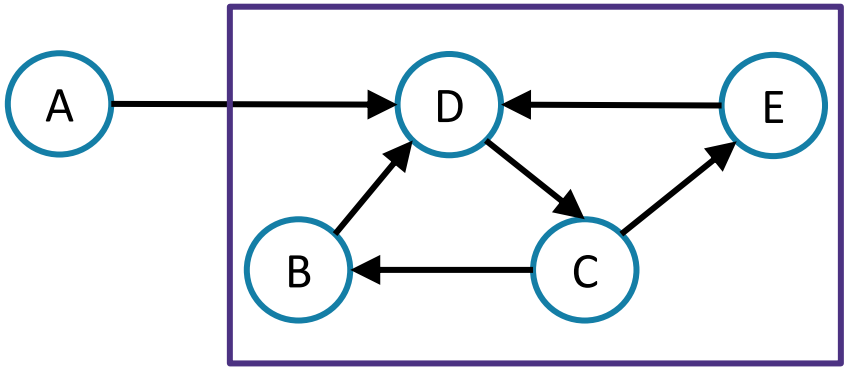
Starke Zusammenhangskomponente

Starke Zusammenhangskomponente

Ein Teilgraph F, bei dem jedes Paar von Knoten in F über einen Pfad in **beide** Richtungen verbunden ist und es keinen anderen Knoten gibt, der mit jedem Knoten von F in beide Richtungen verbunden ist.

Teilgraph F

Ein **maximal stark zusammenhängender Teilgraph** eines gerichteten Graphen



Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2023

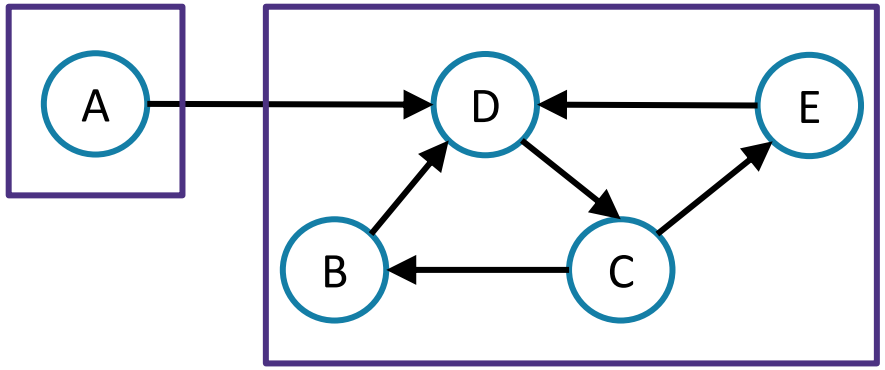
Starke Zusammenhangskomponente

Starke Zusammenhangskomponente

Ein Teilgraph F, bei dem jedes Paar von Knoten in F über einen Pfad in **beide** Richtungen verbunden ist und es keinen anderen Knoten gibt, der mit jedem Knoten von F in beide Richtungen verbunden ist.

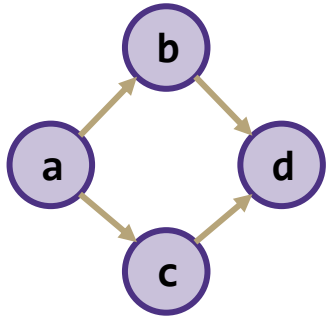
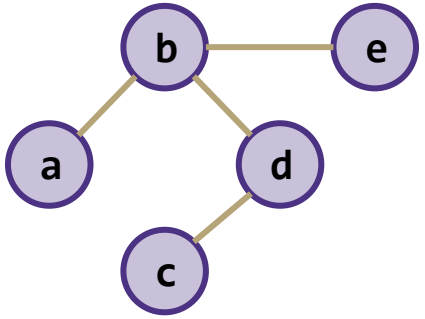
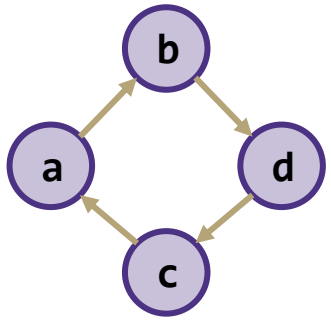
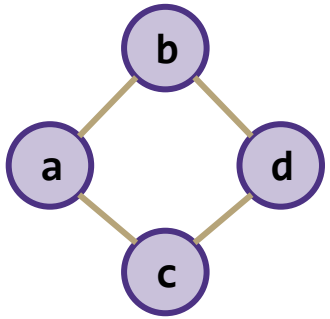
Teilgraph F

Ein **maximal stark zusammenhängender Teilgraph** eines gerichteten Graphen



Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2023

Gerichtet vs. Ungerichtet; Azyklisch vs. Zyklisch

	Gerichtet:	Ungerichtet:
Azyklisch:		
Zyklisch:		

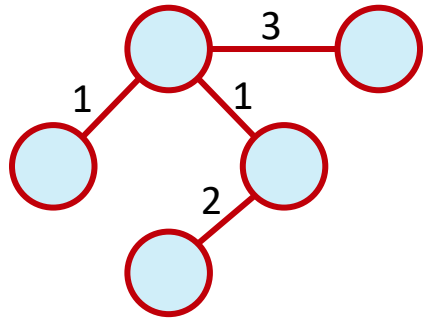
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Gewichteter Graph

An den Kanten stehen Kantengewichte

Bspw. Modellierung von:

- Kosten
- Distanzen
- Zeit
- …



Fragen?

Relevante Ideen bis hierhin

- Knoten, Kanten, Definitionen

Übungsblatt 7: Aufgabe 1, 2

Graphen: Einige Beispiele

Überlegen Sie sich für jede der folgenden Anwendungen, was Sie als Knoten und Kanten wählen sollten.

Das World Wide Web

-
-
-
-
-
-
-

Graphen: Einige Beispiele

Überlegen Sie sich für jede der folgenden Anwendungen, was Sie als Knoten und Kanten wählen sollten.

Das World Wide Web

- Knoten: Webseiten. Kanten von a nach b, wenn a einen Hyperlink zu b hat.
- Gerichtet, da Hyperlinks nur in eine Richtung gehen

Graphen: Einige Beispiele

Überlegen Sie sich für jede der folgenden Anwendungen, was Sie als Knoten und Kanten wählen sollten.

Das World Wide Web

- Knoten: Webseiten. Kanten von a nach b, wenn a einen Hyperlink zu b hat.
- Gerichtet, da Hyperlinks nur in eine Richtung gehen

Stammbaum

-
-
-
-
-

Graphen: Einige Beispiele

Überlegen Sie sich für jede der folgenden Anwendungen, was Sie als Knoten und Kanten wählen sollten.

Das World Wide Web

- Knoten: Webseiten. Kanten von a nach b, wenn a einen Hyperlink zu b hat.
- Gerichtet, da Hyperlinks nur in eine Richtung gehen

Stammbaum

- Knoten: Personen. Kanten: Beziehungen
- (Un-)gerichtet, bidirektionale Beziehungen

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Graphen: Einige Beispiele

Überlegen Sie sich für jede der folgenden Anwendungen, was Sie als Knoten und Kanten wählen sollten.

Das World Wide Web

- Knoten: Webseiten. Kanten von a nach b, wenn a einen Hyperlink zu b hat.
- Gerichtet, da Hyperlinks nur in eine Richtung gehen

Stammbaum

- Knoten: Personen. Kanten: Beziehungen
- (Un-)gerichtet, bidirektionale Beziehungen

Das „Six Degrees of Kevin Bacon"-Spiel (en.wikipedia.org/wiki/Six_Degrees_of_Kevin_Bacon)

-
-
-

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022
CSE 373 19 SU - ROBBIE WEBER

Graphen: Einige Beispiele

Überlegen Sie sich für jede der folgenden Anwendungen, was Sie als Knoten und Kanten wählen sollten.

Das World Wide Web

- Knoten: Webseiten. Kanten von a nach b, wenn a einen Hyperlink zu b hat.
- Gerichtet, da Hyperlinks nur in eine Richtung gehen

Stammbaum

- Knoten: Personen. Kanten: Beziehungen
- (Un-)gerichtet, bidirektionale Beziehungen

Das „Six Degrees of Kevin Bacon"-Spiel (en.wikipedia.org/wiki/Six_Degrees_of_Kevin_Bacon)

- Knoten: Schauspieler. Kanten: Filme
- Ungerichtet, beide Schauspieler müssen in dem Film vorkommen, damit die Kante hinzugefügt wird
-

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Graphen: Einige Beispiele

Überlegen Sie sich für jede der folgenden Anwendungen, was Sie als Knoten und Kanten wählen sollten.

Das World Wide Web

- Knoten: Webseiten. Kanten von a nach b, wenn a einen Hyperlink zu b hat.
- Gerichtet, da Hyperlinks nur in eine Richtung gehen

Stammbaum

- Knoten: Personen. Kanten: Beziehungen
- (Un-)gerichtet, bidirektionale Beziehungen

Das „Six Degrees of Kevin Bacon"-Spiel (en.wikipedia.org/wiki/Six_Degrees_of_Kevin_Bacon)

- Knoten: Schauspieler. Kanten: Filme
- Ungerichtet, beide Schauspieler müssen in dem Film vorkommen, damit die Kante hinzugefügt wird
- S. auch: Erdös-Zahl (Erdös number)

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Graphen: Einige Beispiele

Überlegen Sie sich für jede der folgenden Anwendungen, was Sie als Knoten und Kanten wählen sollten.

Modulvoraussetzungen

-
-
-
-

Graphen: Einige Beispiele

Überlegen Sie sich für jede der folgenden Anwendungen, was Sie als Knoten und Kanten wählen sollten.

Modulvoraussetzungen

- Knoten: Module. Kante: von a nach b, wenn a eine Voraussetzung für b ist.
- Gerichtet, da ein Kurs vor dem anderen kommt

Graphen: Einige Beispiele

Überlegen Sie sich für jede der folgenden Anwendungen, was Sie als Knoten und Kanten wählen sollten.

Modulvoraussetzungen

- Knoten: Module. Kante: von a nach b, wenn a eine Voraussetzung für b ist.
- Gerichtet, da ein Kurs vor dem anderen kommt

Wege zwischen Hochschulgebäuden

- Knoten: Gebäude. Kanten: Ein Straßenname oder ein Fußweg, der 2 Gebäude miteinander verbindet
- Ungerichtet, da jeder Weg in beide Richtungen gegangen werden kann

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022
CSE 373 19 SU - ROBBIE WEBER

Graphen: Übung

Wie würden Sie das folgende Szenario in ein Graphen umwandeln?

Sie erstellen ein Graph, der ein brandneues Social Media-Netzwerk darstellt. Jedes Profil hat die Möglichkeit, sich mit einem anderen Nutzer zu „befreunden“ oder ihm zu „folgen“. Wenn „Freund“ ausgewählt wird, wird das andere Profil um Erlaubnis gefragt, und wenn diese erteilt wird, werden die beiden Profile miteinander verknüpft. Wenn „folgen“ ausgewählt wird, wird keine Erlaubnis eingeholt, aber der Empfänger wird sich nicht mit dem Follower verbinden.

Beantworten Sie die folgenden Fragen zur Gestaltung des Graphen:

- Was sind die Knoten?
- Was sind die Kanten?
- Ungerichtet oder gerichtet?
- Gewichtet oder ungewichtet?

Graphen: Übung

Wie würden Sie das folgende Szenario in ein Graphen umwandeln?

Sie erstellen ein Graph, der ein brandneues Social Media-Netzwerk darstellt. Jedes Profil hat die Möglichkeit, sich mit einem anderen Nutzer zu „befreunden“ oder ihm zu „folgen“. Wenn „Freund“ ausgewählt wird, wird das andere Profil um Erlaubnis gefragt, und wenn diese erteilt wird, werden die beiden Profile miteinander verknüpft. Wenn „folgen“ ausgewählt wird, wird keine Erlaubnis eingeholt, aber der Empfänger wird sich nicht mit dem Follower verbinden.

Beantworten Sie die folgenden Fragen zur Gestaltung des Graphen:

- Was sind die Knoten? **Profile**
- Was sind die Kanten?
- Ungerichtet oder gerichtet?
- Gewichtet oder ungewichtet?

Graphen: Übung

Wie würden Sie das folgende Szenario in ein Graphen umwandeln?

Sie erstellen ein Graph, der ein brandneues Social Media-Netzwerk darstellt. Jedes Profil hat die Möglichkeit, sich mit einem anderen Nutzer zu „befreunden“ oder ihm zu „folgen“. Wenn „Freund“ ausgewählt wird, wird das andere Profil um Erlaubnis gefragt, und wenn diese erteilt wird, werden die beiden Profile miteinander verknüpft. Wenn „folgen“ ausgewählt wird, wird keine Erlaubnis eingeholt, aber der Empfänger wird sich nicht mit dem Follower verbinden.

Beantworten Sie die folgenden Fragen zur Gestaltung des Graphen:

- Was sind die Knoten?
- Was sind die Kanten?
- Ungerichtet oder gerichtet?
- Gewichtet oder ungewichtet?

Profile

Follower und Freundschaften

Graphen: Übung

Wie würden Sie das folgende Szenario in ein Graphen umwandeln?

Sie erstellen ein Graph, der ein brandneues Social Media-Netzwerk darstellt. Jedes Profil hat die Möglichkeit, sich mit einem anderen Nutzer zu „befreunden“ oder ihm zu „folgen“. Wenn „Freund“ ausgewählt wird, wird das andere Profil um Erlaubnis gefragt, und wenn diese erteilt wird, werden die beiden Profile miteinander verknüpft. Wenn „folgen“ ausgewählt wird, wird keine Erlaubnis eingeholt, aber der Empfänger wird sich nicht mit dem Follower verbinden.

Beantworten Sie die folgenden Fragen zur Gestaltung des Graphen:

- Was sind die Knoten? **Profile**
- Was sind die Kanten? **Follower und Freundschaften**
- Ungerichtet oder gerichtet? **Gerichtet**
- Gewichtet oder ungewichtet?

Graphen: Übung

Wie würden Sie das folgende Szenario in ein Graphen umwandeln?

Sie erstellen ein Graph, der ein brandneues Social Media-Netzwerk darstellt. Jedes Profil hat die Möglichkeit, sich mit einem anderen Nutzer zu „befreunden“ oder ihm zu „folgen“. Wenn „Freund“ ausgewählt wird, wird das andere Profil um Erlaubnis gefragt, und wenn diese erteilt wird, werden die beiden Profile miteinander verknüpft. Wenn „folgen“ ausgewählt wird, wird keine Erlaubnis eingeholt, aber der Empfänger wird sich nicht mit dem Follower verbinden.

Beantworten Sie die folgenden Fragen zur Gestaltung des Graphen:

- Was sind die Knoten?
- Was sind die Kanten?
- Ungerichtet oder gerichtet?
- Gewichtet oder ungewichtet?

Profile
Follower und Freundschaften
Gerichtet
Ungewichtet (könnte auch gewichtet sein, doppeltes following ungleich Freundschaft)

Graphen: Übung

Wie würden Sie das folgende Szenario in ein Graph umwandeln?

Sie besuchen ein Musikfestival und versuchen, den perfekten Ablauf zu planen, damit Sie alle Ihre Lieblingskünstler sehen können. Sie wissen, wann jeder Auftritt beginnt und wie lange er dauert. Sie können einen Auftritt nur besuchen, wenn Sie vor Beginn da sind, und Sie können einen Auftritt erst verlassen, wenn er zu Ende ist.

Beantworten Sie die folgenden Fragen über den Entwurf des Graphen:

Was sind die Knoten?

Was sind die Kanten?

Ungerichtet oder gerichtet?

Gewichtet oder ungewichtet?

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2023

Graphen: Übung

Wie würden Sie das folgende Szenario in ein Graph umwandeln?

Sie besuchen ein Musikfestival und versuchen, den perfekten Ablauf zu planen, damit Sie alle Ihre Lieblingskünstler sehen können. Sie wissen, wann jeder Auftritt beginnt und wie lange er dauert. Sie können einen Auftritt nur besuchen, wenn Sie vor Beginn da sind, und Sie können einen Auftritt erst verlassen, wenn er zu Ende ist.

Beantworten Sie die folgenden Fragen über den Entwurf des Graphen:

Was sind die Knoten? **Künstler (Auftritt)**

Was sind die Kanten?

Ungerichtet oder gerichtet?

Gewichtet oder ungewichtet?

Graphen: Übung

Wie würden Sie das folgende Szenario in ein Graph umwandeln?

Sie besuchen ein Musikfestival und versuchen, den perfekten Ablauf zu planen, damit Sie alle Ihre Lieblingskünstler sehen können. Sie wissen, wann jeder Auftritt beginnt und wie lange er dauert. Sie können einen Auftritt nur besuchen, wenn Sie vor Beginn da sind, und Sie können einen Auftritt erst verlassen, wenn er zu Ende ist.

Beantworten Sie die folgenden Fragen über den Entwurf des Graphen:

Was sind die Knoten?	**Künstler (Auftritt)**
Was sind die Kanten?	**Wahlmöglichkeiten für den nächsten Auftritt**
Ungerichtet oder gerichtet?	
Gewichtet oder ungewichtet?	

Graphen: Übung

Wie würden Sie das folgende Szenario in ein Graph umwandeln?

Sie besuchen ein Musikfestival und versuchen, den perfekten Ablauf zu planen, damit Sie alle Ihre Lieblingskünstler sehen können. Sie wissen, wann jeder Auftritt beginnt und wie lange er dauert. Sie können einen Auftritt nur besuchen, wenn Sie vor Beginn da sind, und Sie können einen Auftritt erst verlassen, wenn er zu Ende ist.

Beantworten Sie die folgenden Fragen über den Entwurf des Graphen:

Was sind die Knoten?	**Künstler (Auftritt)**
Was sind die Kanten?	**Wahlmöglichkeiten für den nächsten Auftritt**
Ungerichtet oder gerichtet?	**Gerichtet**
Gewichtet oder ungewichtet?	

Graphen: Übung

Wie würden Sie das folgende Szenario in ein Graph umwandeln?

Sie besuchen ein Musikfestival und versuchen, den perfekten Ablauf zu planen, damit Sie alle Ihre Lieblingskünstler sehen können. Sie wissen, wann jeder Auftritt beginnt und wie lange er dauert. Sie können einen Auftritt nur besuchen, wenn Sie vor Beginn da sind, und Sie können einen Auftritt erst verlassen, wenn er zu Ende ist.

Beantworten Sie die folgenden Fragen über den Entwurf des Graphen:

Was sind die Knoten?	**Künstler (Auftritt)**
Was sind die Kanten?	**Wahlmöglichkeiten für den nächsten Auftritt**
Ungerichtet oder gerichtet?	**Gerichtet**
Gewichtet oder ungewichtet?	**Ungewichtet**

Multi-Variable algorithmische Analyse

Bisher haben wir nur ein Argument „n“ genutzt (manchmal ein eigener Parameter, manchmal eine Größe)

- Aber es gibt keinen Grund, warum wir nicht auch mit mehreren Eingaben argumentieren können!

Multi-Variable algorithmische Analyse

Bisher haben wir nur ein Argument „n“ genutzt (manchmal ein eigener Parameter, manchmal eine Größe)

- Aber es gibt keinen Grund, warum wir nicht auch mit mehreren Eingaben argumentieren können!

Bei Graphen führen wir unsere Überlegungen normalerweise in Form von:

- n (oder |V| (Kardinalzahl von V)): Gesamtzahl der Knoten
- m (oder |E|): Gesamtzahl der Kanten
- deg(u): Grad des Knotens u (wie viele ausgehende Kanten er hat (Ausgangsgrad))

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Multi-Variable algorithmische Analyse

Bisher haben wir nur ein Argument „n“ genutzt (manchmal ein eigener Parameter, manchmal eine Größe)

- Aber es gibt keinen Grund, warum wir nicht auch mit mehreren Eingaben argumentieren können!

Bei Graphen führen wir unsere Überlegungen normalerweise in Form von:

- n (oder |V| (Kardinalzahl von V)): Gesamtzahl der Knoten
- m (oder |E|): Gesamtzahl der Kanten
- deg(u): Grad des Knotens u (wie viele ausgehende Kanten er hat (Ausgangsgrad))

Warum multivariabel?

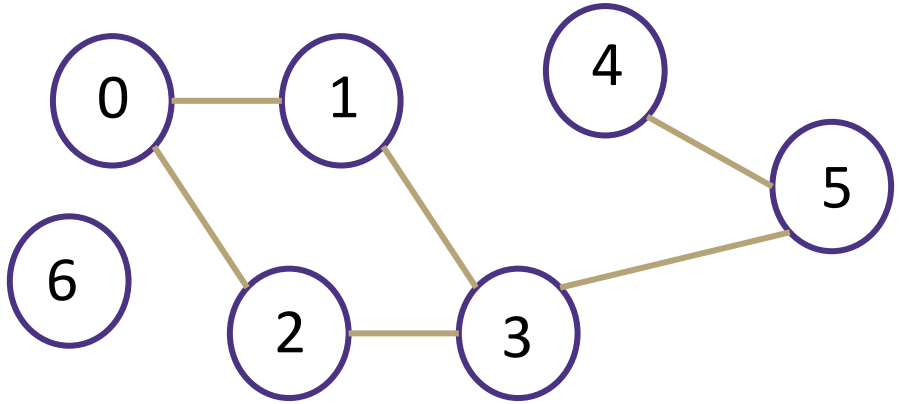
- Die algorithmische Analyse ist nur ein Werkzeug, um den Code zu verstehen.
- Manchmal ist es hilfreicher, eine O/Ω/Θ-Schranke für mehrere Variablen zu erstellen, als diese Variablen in einer Fallanalyse zu behandeln.

Mit welchen grundlegenden Datenstrukturen implementiert man Graphen?

- Adjazenzmatrix
  - Array
- Adjazenzliste
  - Array und verkettete Liste
  - Array und Hashtabelle

Adjazenzmatrix

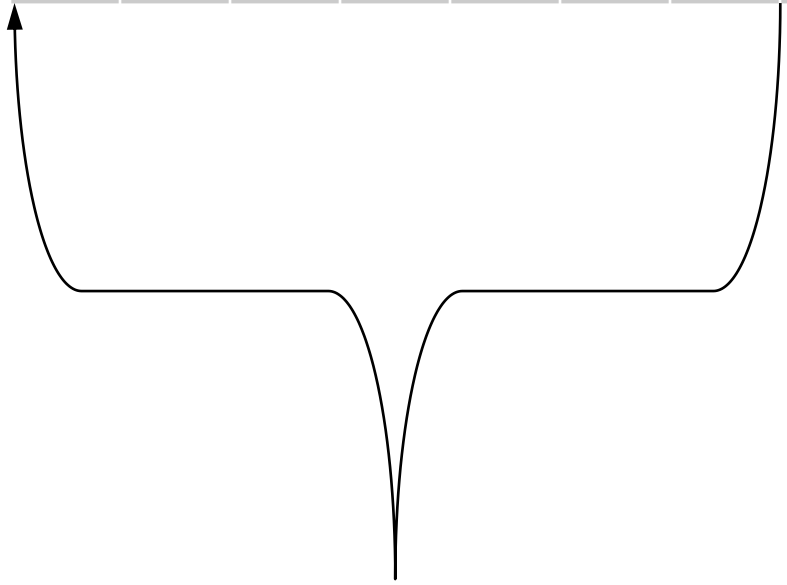
In einer Adjazenzmatrix A ist A[u][v] = 1, wenn es eine Kante (u,v) gibt, und sonst 0.



	0	1	2	3	4	5	6
0	0	**1**	**1**	0	0	0	0
1	**1**	0	0	**1**	0	0	0
2	**1**	0	0	**1**	0	0	0
3	0	**1**	**1**	0	0	**1**	0
4	0	0	0	0	0	**1**	0
5	0	0	0	**1**	**1**	0	0
6	0	0	0	0	0	0	0

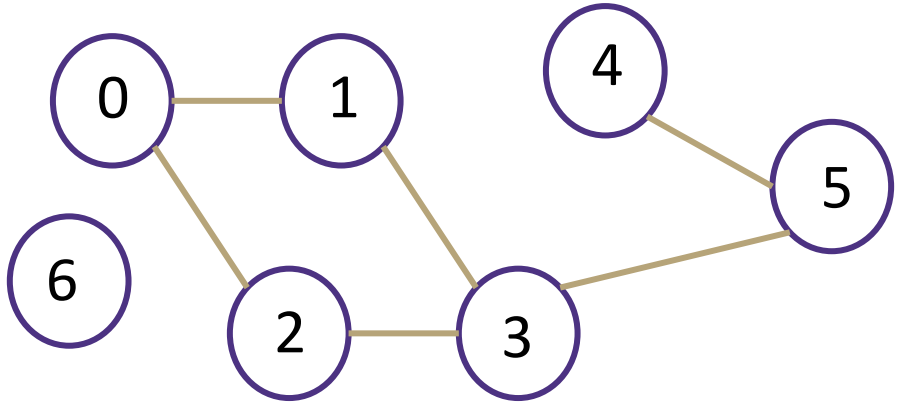
Adjazenzmatrix implementiert als Array (zeilenweise)

0	1	2	3	4	5	6	0	1	…
0	1	1	0	0	0	0	1	0	



	0	1	2	3	4	5	6
0	0	**1**	**1**	0	0	0	0
1	**1**	0	0	**1**	0	0	0
2	**1**	0	0	**1**	0	0	0
3	0	**1**	**1**	0	0	**1**	0
4	0	0	0	0	0	**1**	0
5	0	0	0	**1**	**1**	0	0
6	0	0	0	0	0	0	0

Adjazenzmatrix



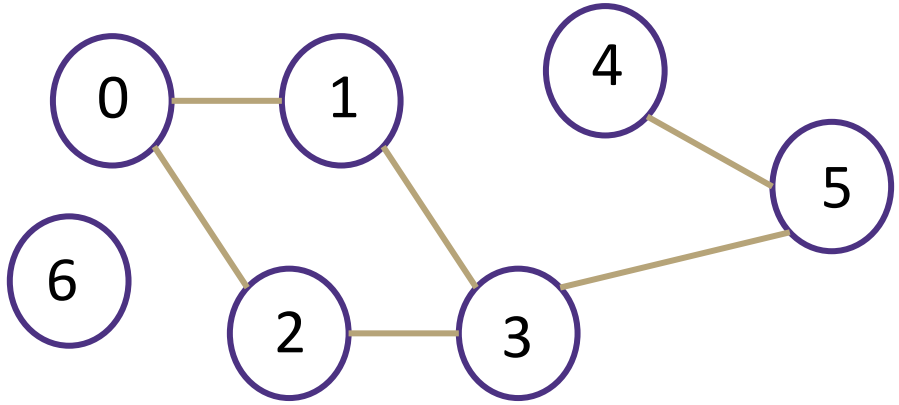
In einer Adjazenzmatrix A ist A[u][v] = 1, wenn es eine Kante (u,v) gibt, und sonst 0.

Worst Case Laufzeit (|V| = n, |E| = m):

- Kante hinzufügen:
- Kante entfernen:
- Prüfen, ob Kante (u,v) existiert:
- Erhalte ausgehende Kanten von u:
- Erhalte eingehende Kanten von u:

	0	1	2	3	4	5	6
0	0	**1**	**1**	0	0	0	0
1	**1**	0	0	**1**	0	0	0
2	**1**	0	0	**1**	0	0	0
3	0	**1**	**1**	0	0	**1**	0
4	0	0	0	0	0	**1**	0
5	0	0	0	**1**	**1**	0	0
6	0	0	0	0	0	0	0

Adjazenzmatrix



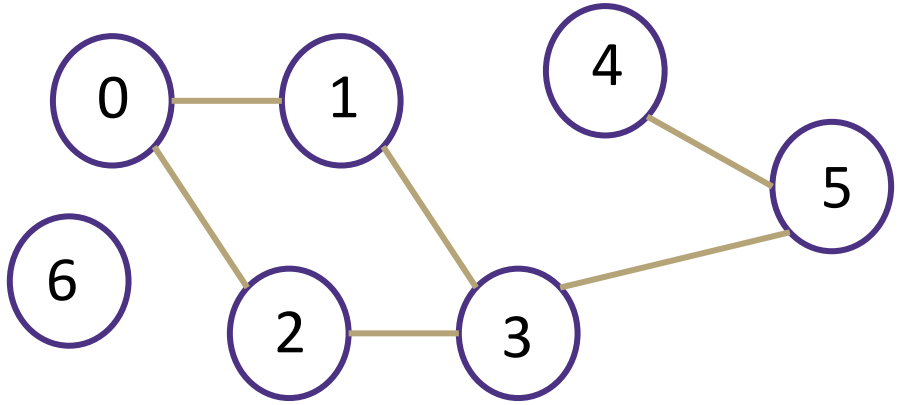
In einer Adjazenzmatrix A ist A[u][v] = 1, wenn es eine Kante (u,v) gibt, und sonst 0.

Worst Case Laufzeit (|V| = n, |E| = m):

- Kante hinzufügen: $\Theta(1)$
- Kante entfernen:
- Prüfen, ob Kante (u,v) existiert:
- Erhalte ausgehende Kanten von u:
- Erhalte eingehende Kanten von u:

	0	1	2	3	4	5	6
0	0	**1**	**1**	0	0	0	0
1	**1**	0	0	**1**	0	0	0
2	**1**	0	0	**1**	0	0	0
3	0	**1**	**1**	0	0	**1**	0
4	0	0	0	0	0	**1**	0
5	0	0	0	**1**	**1**	0	0
6	0	0	0	0	0	0	0

Adjazenzmatrix



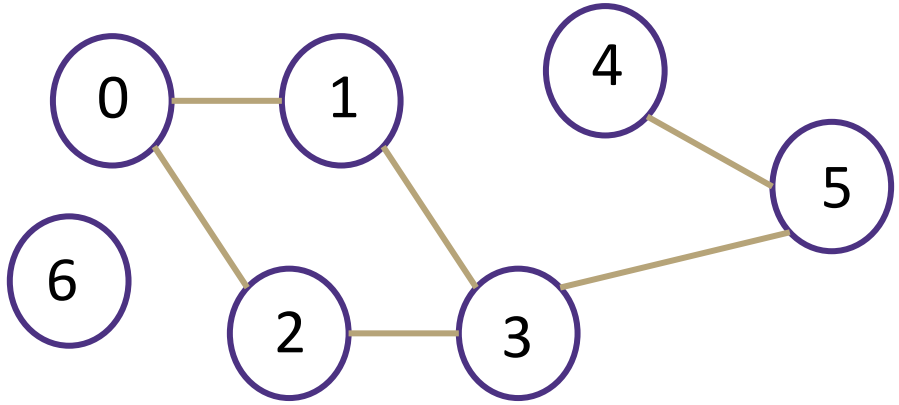
In einer Adjazenzmatrix A ist A[u][v] = 1, wenn es eine Kante (u,v) gibt, und sonst 0.

Worst Case Laufzeit (|V| = n, |E| = m):

- Kante hinzufügen: $\Theta(1)$
- Kante entfernen: $\Theta(1)$
- Prüfen, ob Kante (u,v) existiert:
- Erhalte ausgehende Kanten von u:
- Erhalte eingehende Kanten von u:

	0	1	2	3	4	5	6
0	0	**1**	**1**	0	0	0	0
1	**1**	0	0	**1**	0	0	0
2	**1**	0	0	**1**	0	0	0
3	0	**1**	**1**	0	0	**1**	0
4	0	0	0	0	0	**1**	0
5	0	0	0	**1**	**1**	0	0
6	0	0	0	0	0	0	0

Adjazenzmatrix



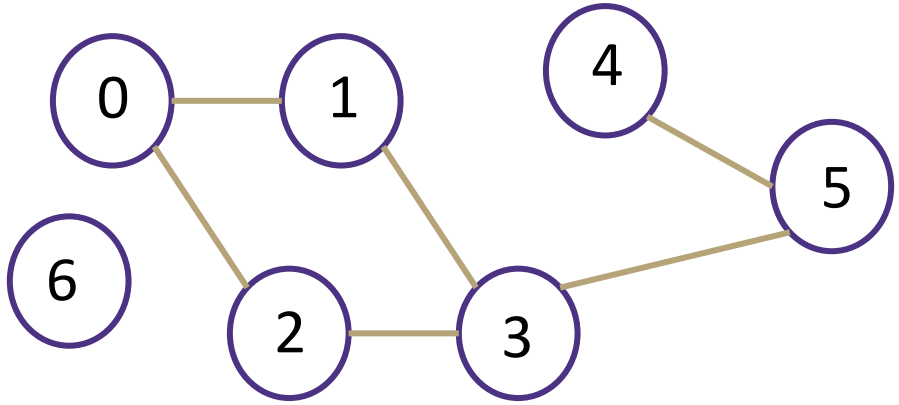
In einer Adjazenzmatrix A ist A[u][v] = 1, wenn es eine Kante (u,v) gibt, und sonst 0.

Worst Case Laufzeit (|V| = n, |E| = m):

- Kante hinzufügen: $\Theta(1)$
- Kante entfernen: $\Theta(1)$
- Prüfen, ob Kante (u,v) existiert: $\Theta(1)$
- Erhalte ausgehende Kanten von u:
- Erhalte eingehende Kanten von u:

	0	1	2	3	4	5	6
0	0	**1**	**1**	0	0	0	0
1	**1**	0	0	**1**	0	0	0
2	**1**	0	0	**1**	0	0	0
3	0	**1**	**1**	0	0	**1**	0
4	0	0	0	0	0	**1**	0
5	0	0	0	**1**	**1**	0	0
6	0	0	0	0	0	0	0

Adjazenzmatrix



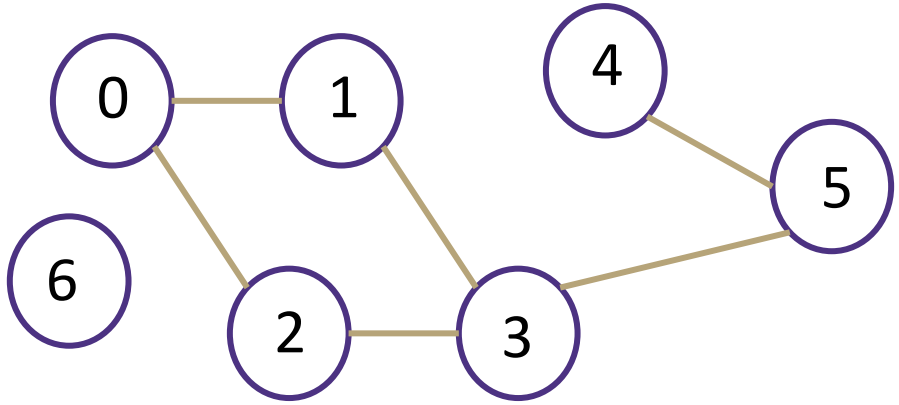
In einer Adjazenzmatrix A ist A[u][v] = 1, wenn es eine Kante (u,v) gibt, und sonst 0.

Worst Case Laufzeit (|V| = n, |E| = m):

- Kante hinzufügen: $\Theta(1)$
- Kante entfernen: $\Theta(1)$
- Prüfen, ob Kante (u,v) existiert: $\Theta(1)$
- Erhalte ausgehende Kanten von u: $\Theta(n)$
- Erhalte eingehende Kanten von u:

	0	1	2	3	4	5	6
0	0	**1**	**1**	0	0	0	0
1	**1**	0	0	**1**	0	0	0
2	**1**	0	0	**1**	0	0	0
3	0	**1**	**1**	0	0	**1**	0
4	0	0	0	0	0	**1**	0
5	0	0	0	**1**	**1**	0	0
6	0	0	0	0	0	0	0

Adjazenzmatrix



In einer Adjazenzmatrix A ist A[u][v] = 1, wenn es eine Kante (u,v) gibt, und sonst 0.

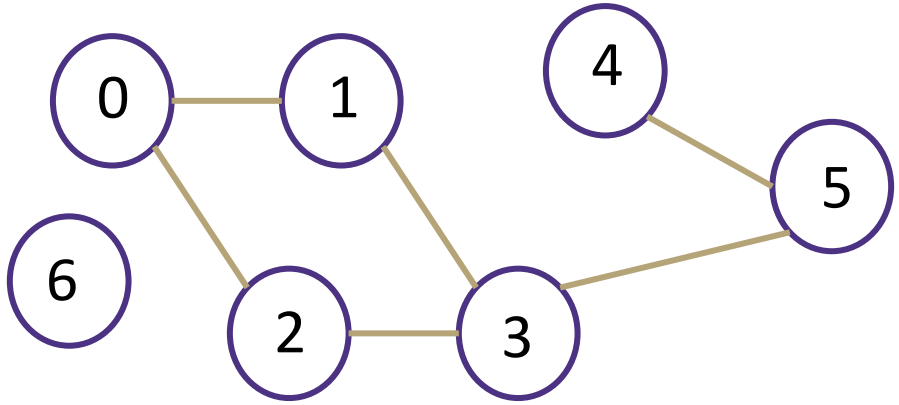
Worst Case Laufzeit (|V| = n, |E| = m):

- Kante hinzufügen: $\Theta(1)$
- Kante entfernen: $\Theta(1)$
- Prüfen, ob Kante (u,v) existiert: $\Theta(1)$
- Erhalte ausgehende Kanten von u: $\Theta(n)$
- Erhalte eingehende Kanten von u: $\Theta(n)$

	0	1	2	3	4	5	6
0	0	**1**	**1**	0	0	0	0
1	**1**	0	0	**1**	0	0	0
2	**1**	0	0	**1**	0	0	0
3	0	**1**	**1**	0	0	**1**	0
4	0	0	0	0	0	**1**	0
5	0	0	0	**1**	**1**	0	0
6	0	0	0	0	0	0	0

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022
CSE 373 19 SU - ROBBIE WEBER

Adjazenzmatrix



In einer Adjazenzmatrix A ist A[u][v] = 1, wenn es eine Kante (u,v) gibt, und sonst 0.

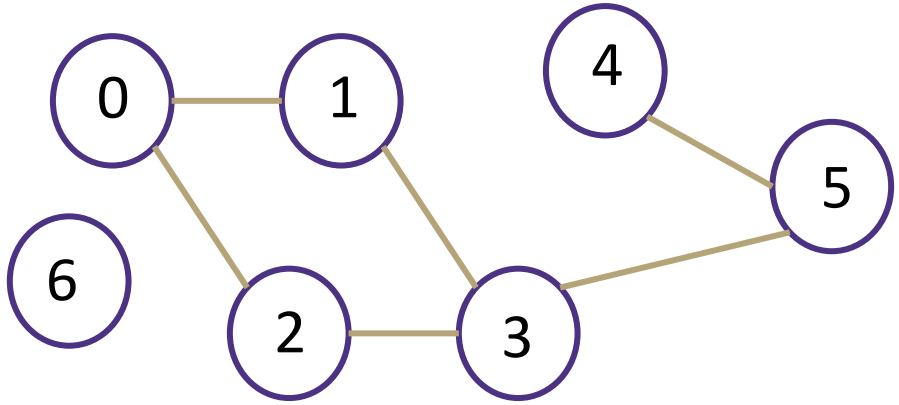
Worst Case Laufzeit (|V| = n, |E| = m):

- Kante hinzufügen: $\Theta(1)$
- Kante entfernen: $\Theta(1)$
- Prüfen, ob Kante (u,v) existiert: $\Theta(1)$
- Erhalte ausgehende Kanten von u: $\Theta(n)$
- Erhalte eingehende Kanten von u: $\Theta(n)$

Speicherplatzbedarf:

	0	1	2	3	4	5	6
0	0	**1**	**1**	0	0	0	0
1	**1**	0	0	**1**	0	0	0
2	**1**	0	0	**1**	0	0	0
3	0	**1**	**1**	0	0	**1**	0
4	0	0	0	0	0	**1**	0
5	0	0	0	**1**	**1**	0	0
6	0	0	0	0	0	0	0

Adjazenzmatrix



In einer Adjazenzmatrix A ist A[u][v] = 1, wenn es eine Kante (u,v) gibt, und sonst 0.

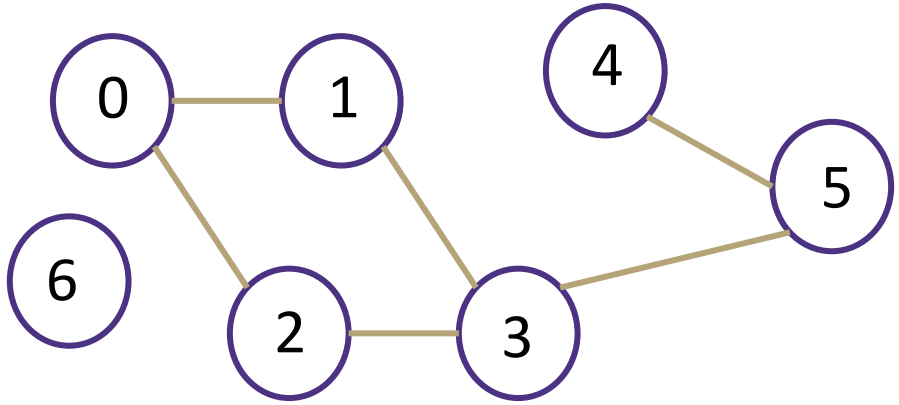
Worst Case Laufzeit (|V| = n, |E| = m):

- Kante hinzufügen: $\Theta(1)$
- Kante entfernen: $\Theta(1)$
- Prüfen, ob Kante (u,v) existiert: $\Theta(1)$
- Erhalte ausgehende Kanten von u: $\Theta(n)$
- Erhalte eingehende Kanten von u: $\Theta(n)$

Speicherplatzbedarf: $\Theta(n * n)$

	0	1	2	3	4	5	6
0	0	**1**	**1**	0	0	0	0
1	**1**	0	0	**1**	0	0	0
2	**1**	0	0	**1**	0	0	0
3	0	**1**	**1**	0	0	**1**	0
4	0	0	0	0	0	**1**	0
5	0	0	0	**1**	**1**	0	0
6	0	0	0	0	0	0	0

Adjazenzmatrix



In einer Adjazenzmatrix A ist A[u][v] = 1, wenn es eine Kante (u,v) gibt, und sonst 0.

Worst Case Laufzeit (|V| = n, |E| = m):

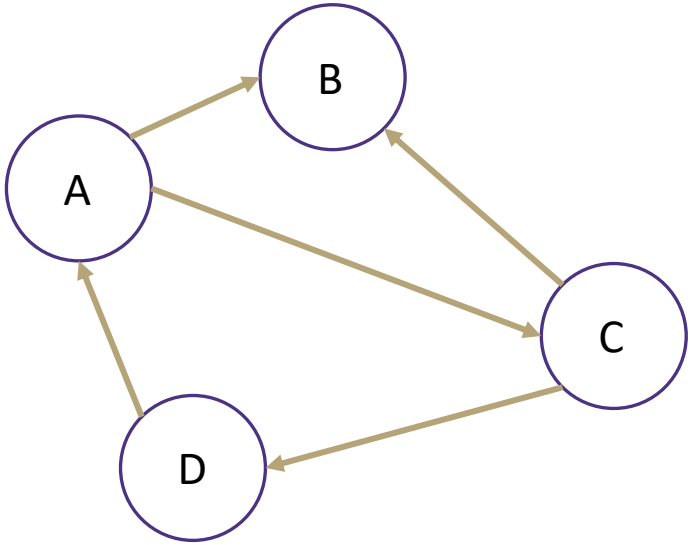
- Kante hinzufügen: $\Theta(1)$
- Kante entfernen: $\Theta(1)$
- Prüfen, ob Kante (u,v) existiert: $\Theta(1)$
- Erhalte ausgehende Kanten von u: $\Theta(n)$
- Erhalte eingehende Kanten von u: $\Theta(n)$

Speicherplatzbedarf: $\Theta(n * n)$

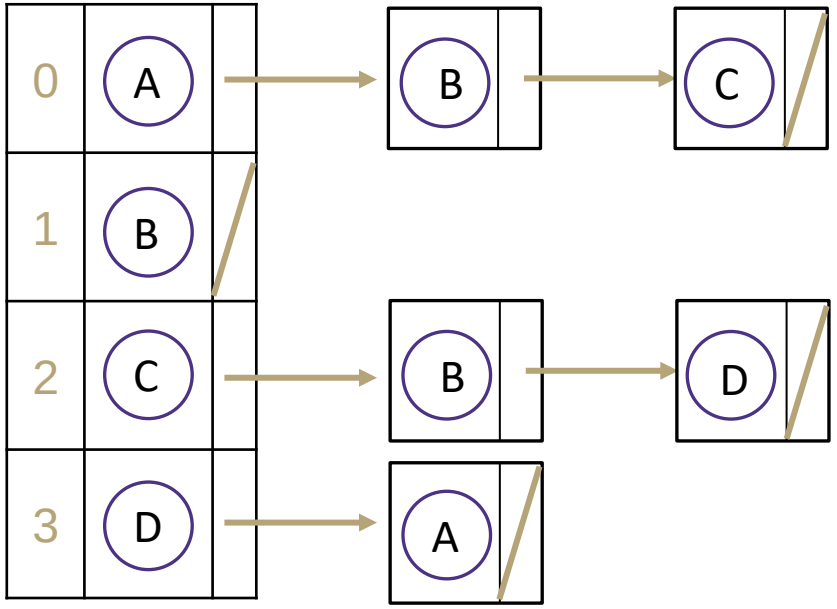
Auch geeignet für gewichtete/gerichtete Graphen

	0	1	2	3	4	5	6
0	0	**1**	**1**	0	0	0	0
1	**1**	0	0	**1**	0	0	0
2	**1**	0	0	**1**	0	0	0
3	0	**1**	**1**	0	0	**1**	0
4	0	0	0	0	0	**1**	0
5	0	0	0	**1**	**1**	0	0
6	0	0	0	0	0	0	0

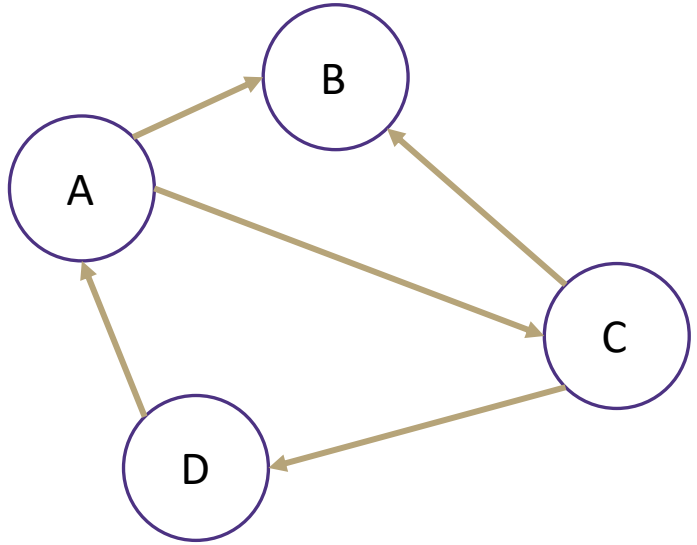
Adjazenzliste



Verkettete Liste



Adjazenzliste

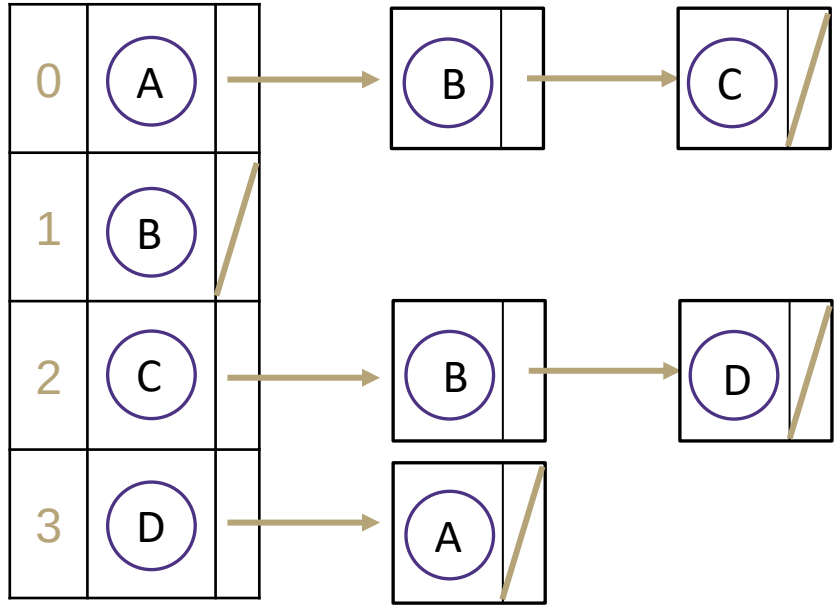


Annahme: Verkettete Liste hat keine Variable size. Sonst ausgehende Kanten in O(1) bestimmbar.

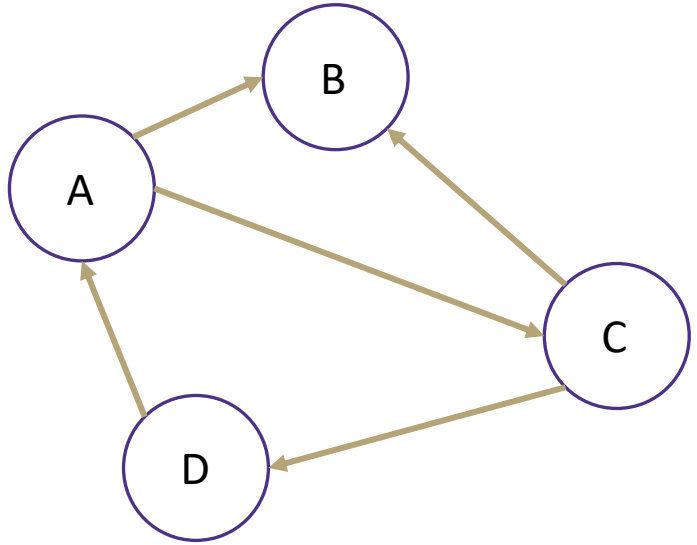
Worst Case Laufzeit (|V| = n, |E| = m):

- Kante hinzufügen:
- Kante entfernen bei Knoten u:
- Prüfen, ob Kante (u,v) existiert:
- Erhalte ausgehende Kanten von u:
- Erhalte eingehende Kanten von u:

Verkettete Liste



Adjazenzliste

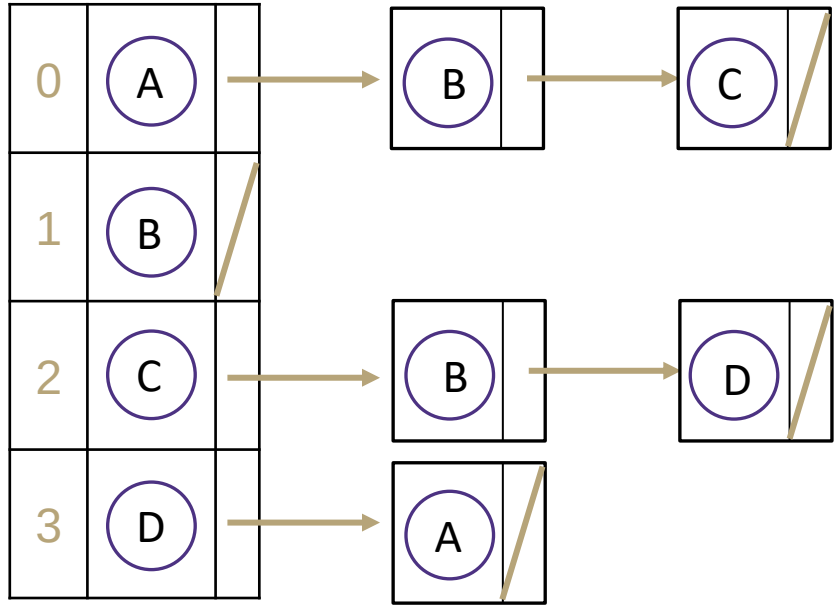


Annahme: Verkettete Liste hat keine Variable size. Sonst ausgehende Kanten in O(1) bestimmbar.

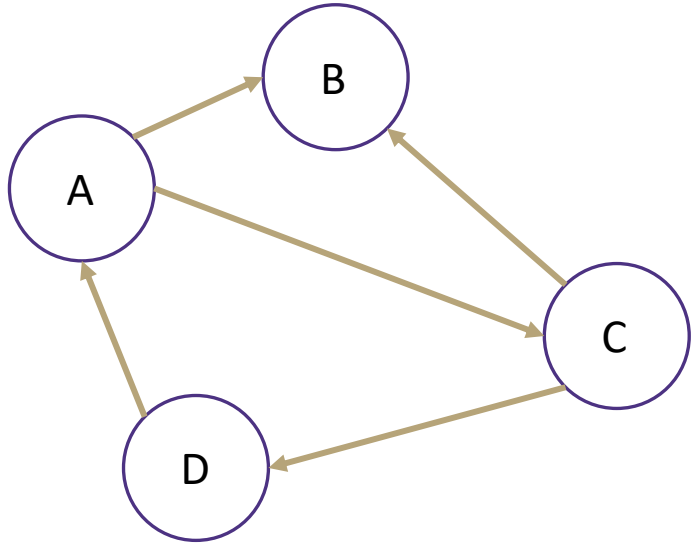
Worst Case Laufzeit (|V| = n, |E| = m):

- Kante hinzufügen: $\Theta(1)$
- Kante entfernen bei Knoten u:
- Prüfen, ob Kante (u,v) existiert:
- Erhalte ausgehende Kanten von u:
- Erhalte eingehende Kanten von u:

Verkettete Liste



Adjazenzliste

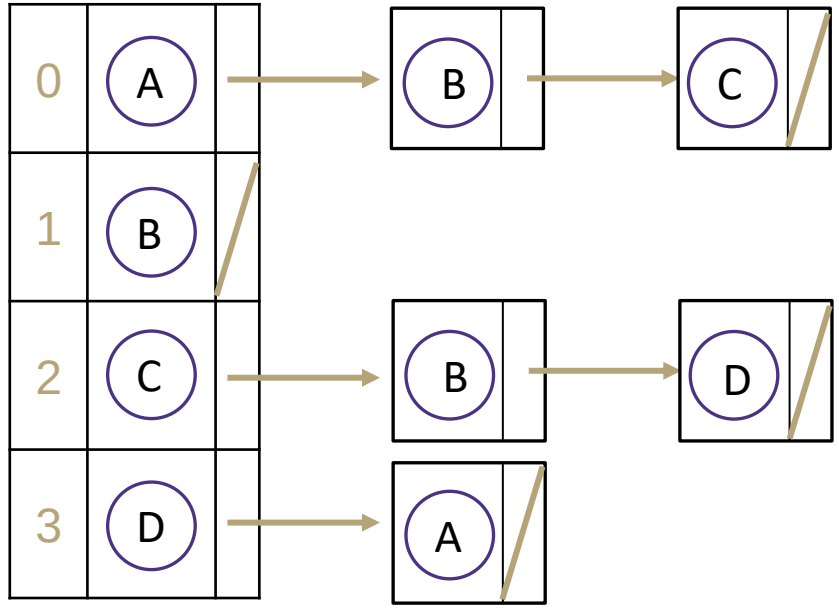


Annahme: Verkettete Liste hat keine Variable size. Sonst ausgehende Kanten in O(1) bestimmbar.

Worst Case Laufzeit (|V| = n, |E| = m):

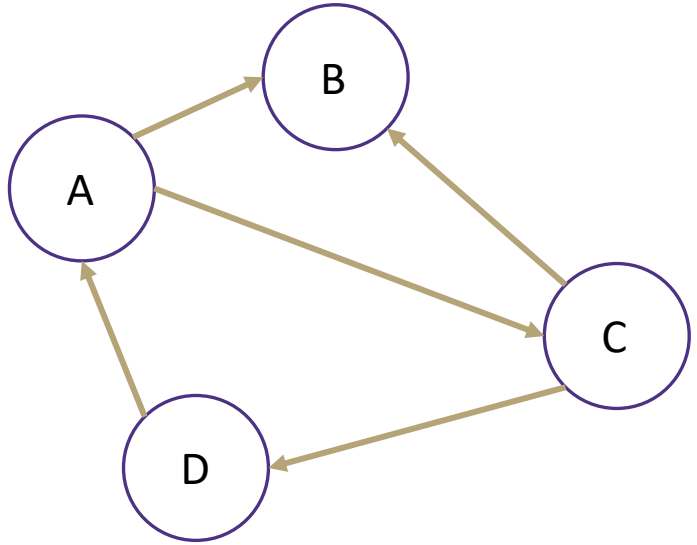
- Kante hinzufügen: $\Theta(1)$
- Kante entfernen bei Knoten u: $\Theta(\deg(u))$
- Prüfen, ob Kante (u,v) existiert:
- Erhalte ausgehende Kanten von u:
- Erhalte eingehende Kanten von u:

Verkettete Liste



Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022
CSE 373 20 SP - KASEY CHAMPION

Adjazenzliste

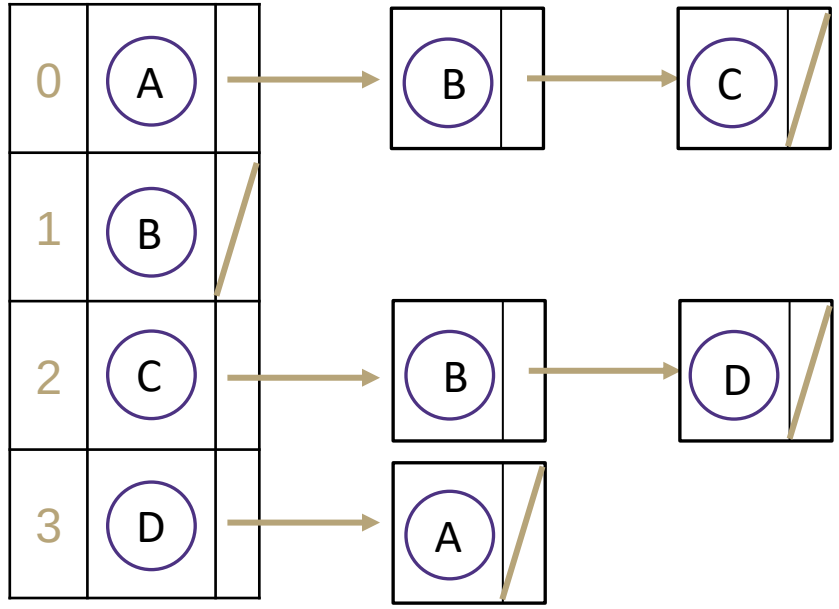


Annahme: Verkettete Liste hat keine Variable size. Sonst ausgehende Kanten in O(1) bestimmbar.

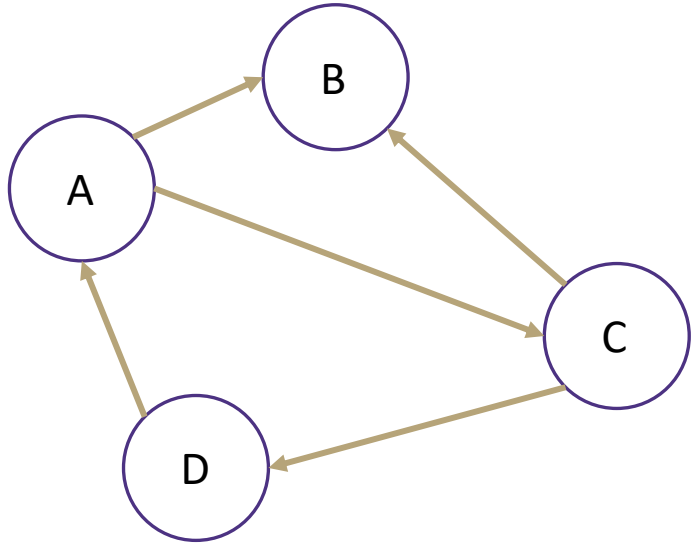
Worst Case Laufzeit (|V| = n, |E| = m):

- Kante hinzufügen: $\Theta(1)$
- Kante entfernen bei Knoten u: $\Theta(\deg(u))$
- Prüfen, ob Kante (u,v) existiert: $\Theta(\deg(u))$
- Erhalte ausgehende Kanten von u:
- Erhalte eingehende Kanten von u:

Verkettete Liste



Adjazenzliste

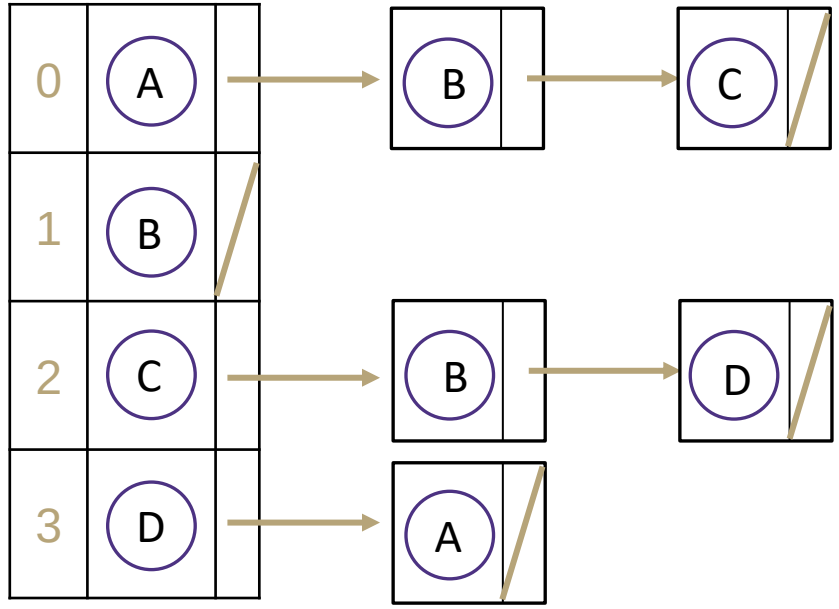


Annahme: Verkettete Liste hat keine Variable size. Sonst ausgehende Kanten in O(1) bestimmbar.

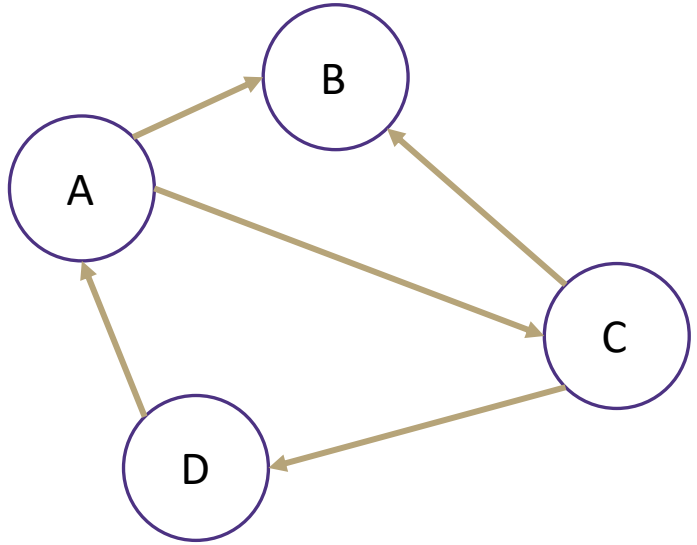
Worst Case Laufzeit (|V| = n, |E| = m):

- Kante hinzufügen: $\Theta(1)$
- Kante entfernen bei Knoten u: $\Theta(\deg(u))$
- Prüfen, ob Kante (u,v) existiert: $\Theta(\deg(u))$
- Erhalte ausgehende Kanten von u: $\Theta(\deg(u))$
- Erhalte eingehende Kanten von u:

Verkettete Liste



Adjazenzliste

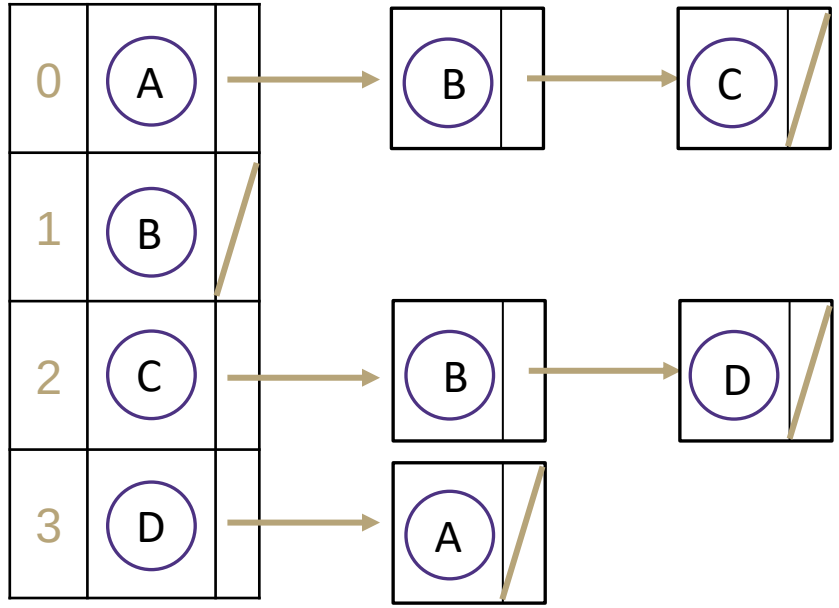


Annahme: Verkettete Liste hat keine Variable size. Sonst ausgehende Kanten in O(1) bestimmbar.

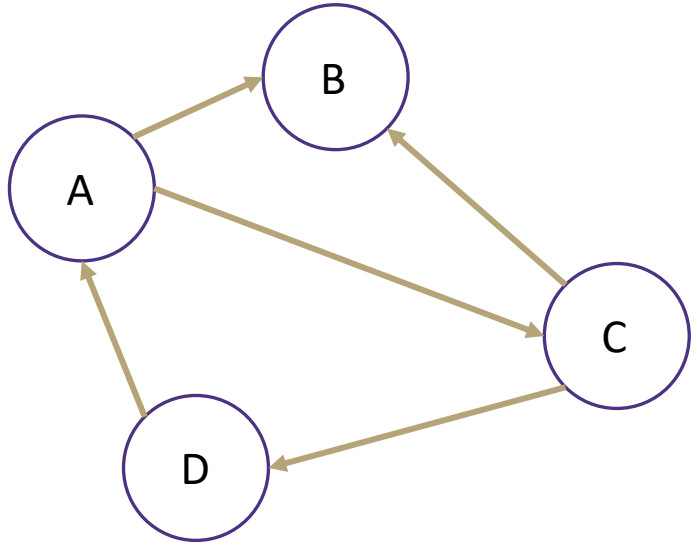
Worst Case Laufzeit (|V| = n, |E| = m):

- Kante hinzufügen: $\Theta(1)$
- Kante entfernen bei Knoten u: $\Theta(\deg(u))$
- Prüfen, ob Kante (u,v) existiert: $\Theta(\deg(u))$
- Erhalte ausgehende Kanten von u: $\Theta(\deg(u))$
- Erhalte eingehende Kanten von u: $\Theta(n + m)$

Verkettete Liste



Adjazenzliste



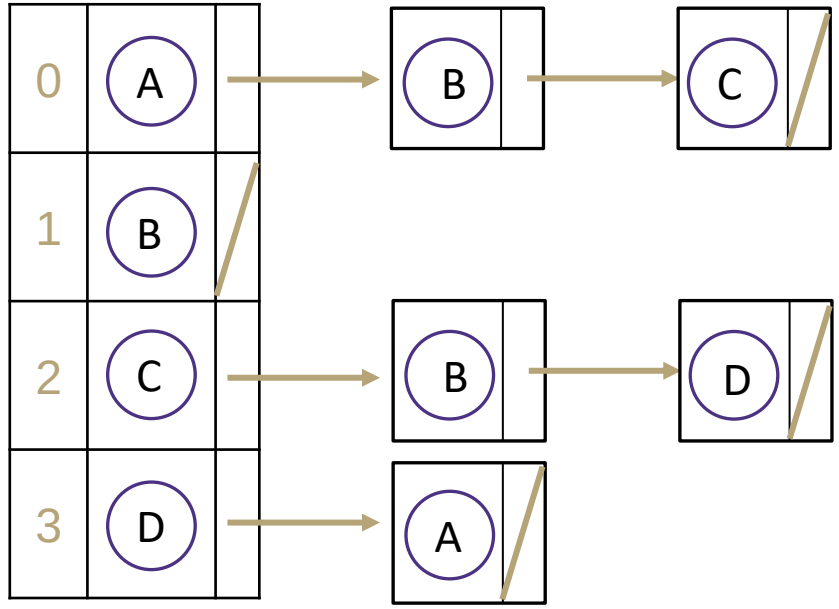
Annahme: Verkettete Liste hat keine Variable size. Sonst ausgehende Kanten in O(1) bestimmbar.

Worst Case Laufzeit (|V| = n, |E| = m):

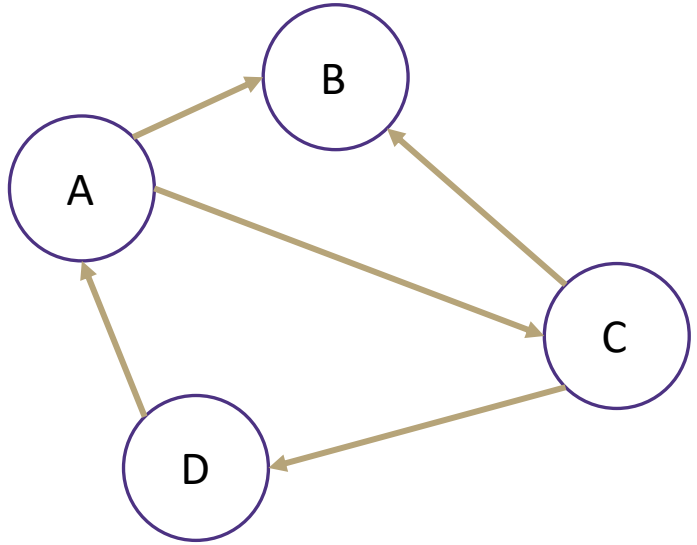
- Kante hinzufügen: $\Theta(1)$
- Kante entfernen bei Knoten u: $\Theta(\deg(u))$
- Prüfen, ob Kante (u,v) existiert: $\Theta(\deg(u))$
- Erhalte ausgehende Kanten von u: $\Theta(\deg(u))$
- Erhalte eingehende Kanten von u: $\Theta(n + m)$

Speicherplatzbedarf:

Verkettete Liste



Adjazenzliste



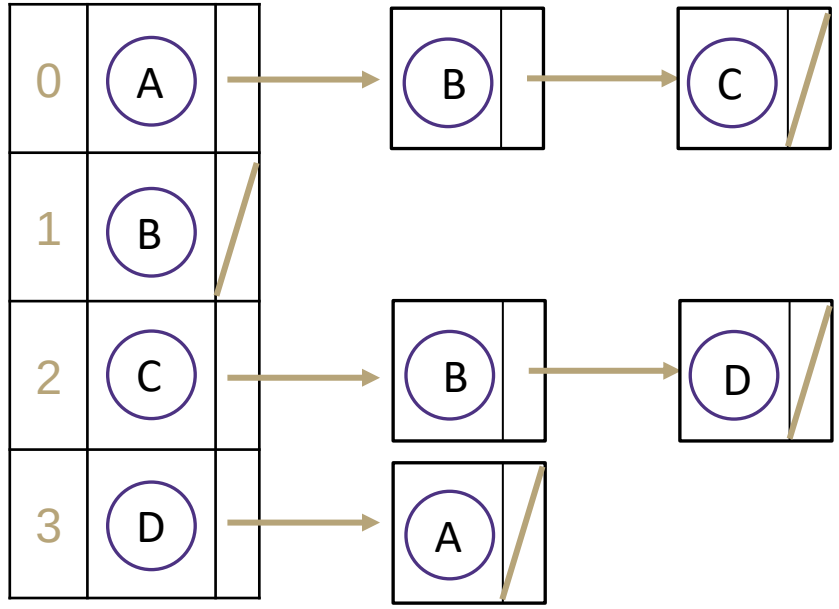
Annahme: Verkettete Liste hat keine Variable size. Sonst ausgehende Kanten in O(1) bestimmbar.

Worst Case Laufzeit (|V| = n, |E| = m):

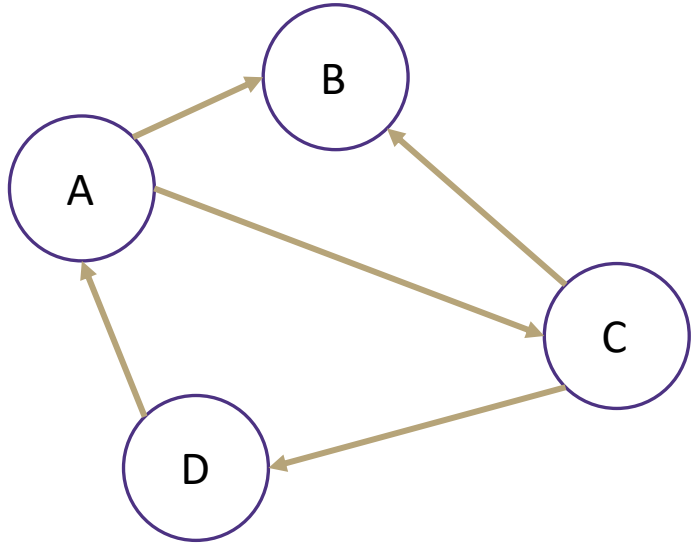
- Kante hinzufügen: $\Theta(1)$
- Kante entfernen bei Knoten u: $\Theta(\deg(u))$
- Prüfen, ob Kante (u,v) existiert: $\Theta(\deg(u))$
- Erhalte ausgehende Kanten von u: $\Theta(\deg(u))$
- Erhalte eingehende Kanten von u: $\Theta(n + m)$

Speicherplatzbedarf: $\Theta(n + m)$

Verkettete Liste



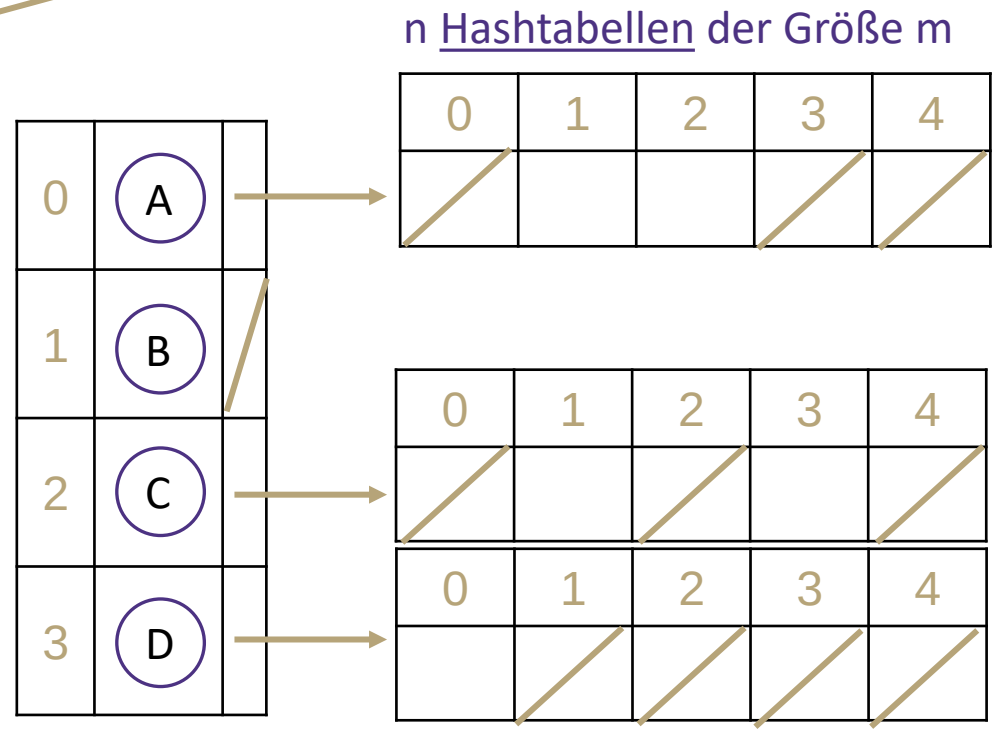
Adjazenzliste



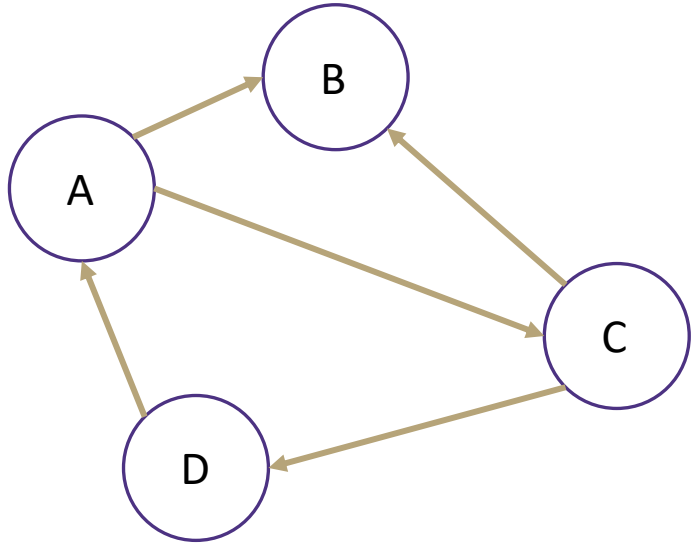
„In der Praxis“ Laufzeit (|V| = n, |E| = m):

- Kante hinzufügen:
- Kante entfernen:
- Prüfen, ob Kante (u,v) existiert:
- Erhalte ausgehende Kanten von u:
- Erhalte eingehende Kanten von u:

Speicherplatzbedarf:



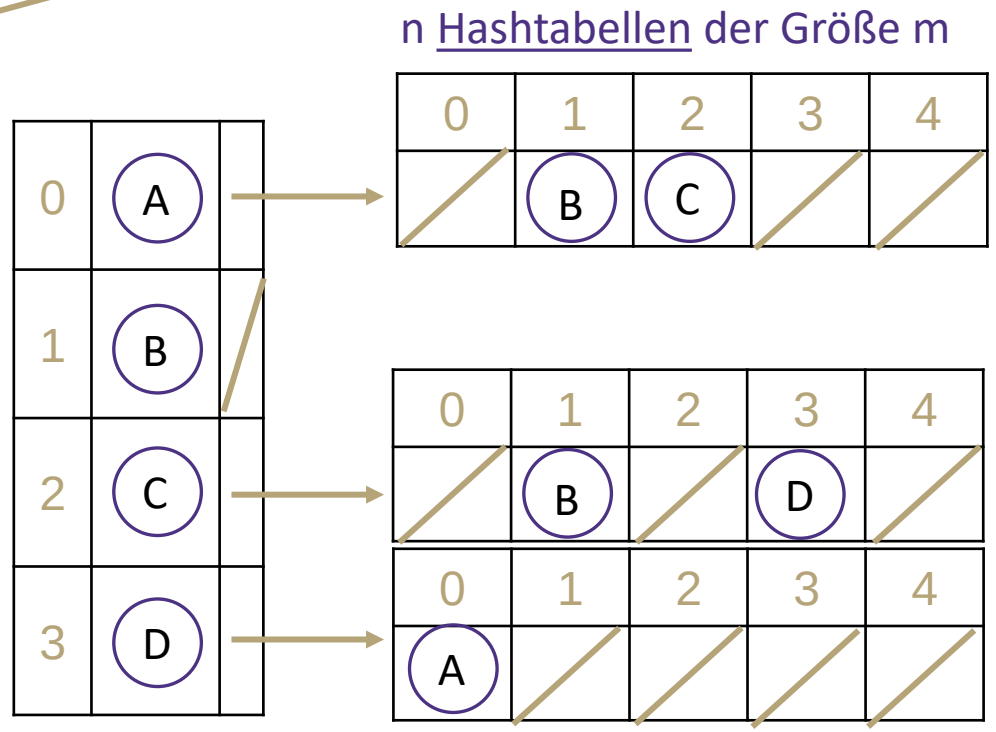
Adjazenzliste



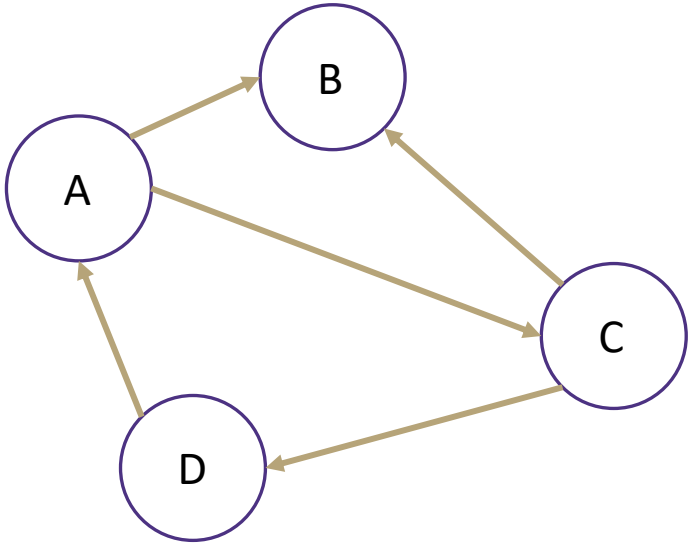
„In der Praxis“ Laufzeit (|V| = n, |E| = m):

- Kante hinzufügen:
- Kante entfernen:
- Prüfen, ob Kante (u,v) existiert:
- Erhalte ausgehende Kanten von u:
- Erhalte eingehende Kanten von u:

Speicherplatzbedarf:



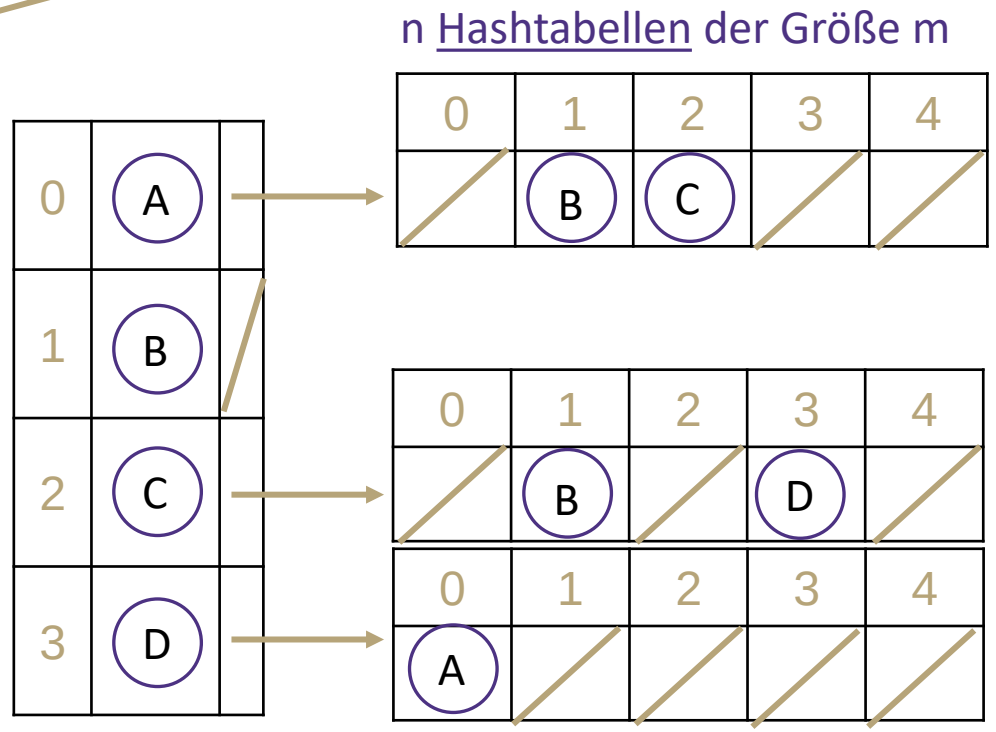
Adjazenzliste



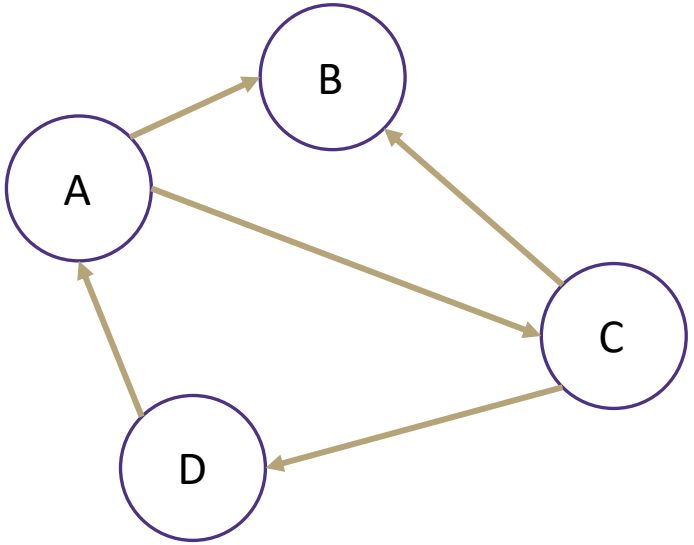
„In der Praxis“ Laufzeit (|V| = n, |E| = m):

- Kante hinzufügen: $\Theta(1+\lambda)$
- Kante entfernen:
- Prüfen, ob Kante (u,v) existiert:
- Erhalte ausgehende Kanten von u:
- Erhalte eingehende Kanten von u:

Speicherplatzbedarf:



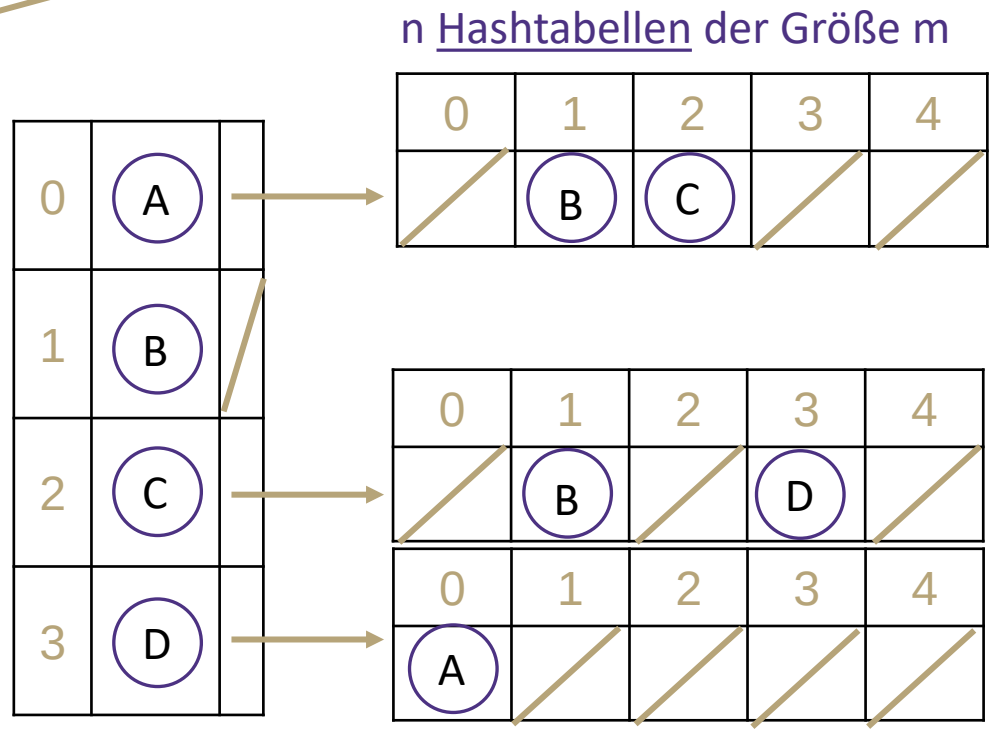
Adjazenzliste



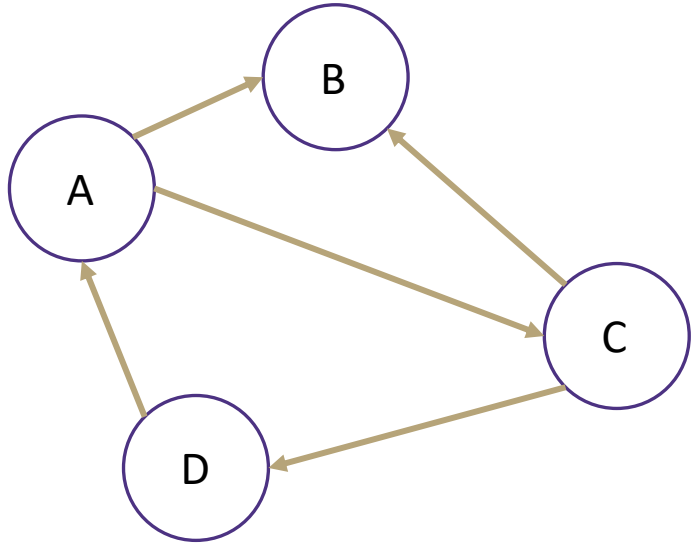
„In der Praxis“ Laufzeit (|V| = n, |E| = m):

- Kante hinzufügen: $\Theta(1+\lambda)$
- Kante entfernen: $\Theta(1+\lambda)$
- Prüfen, ob Kante (u,v) existiert:
- Erhalte ausgehende Kanten von u:
- Erhalte eingehende Kanten von u:

Speicherplatzbedarf:



Adjazenzliste



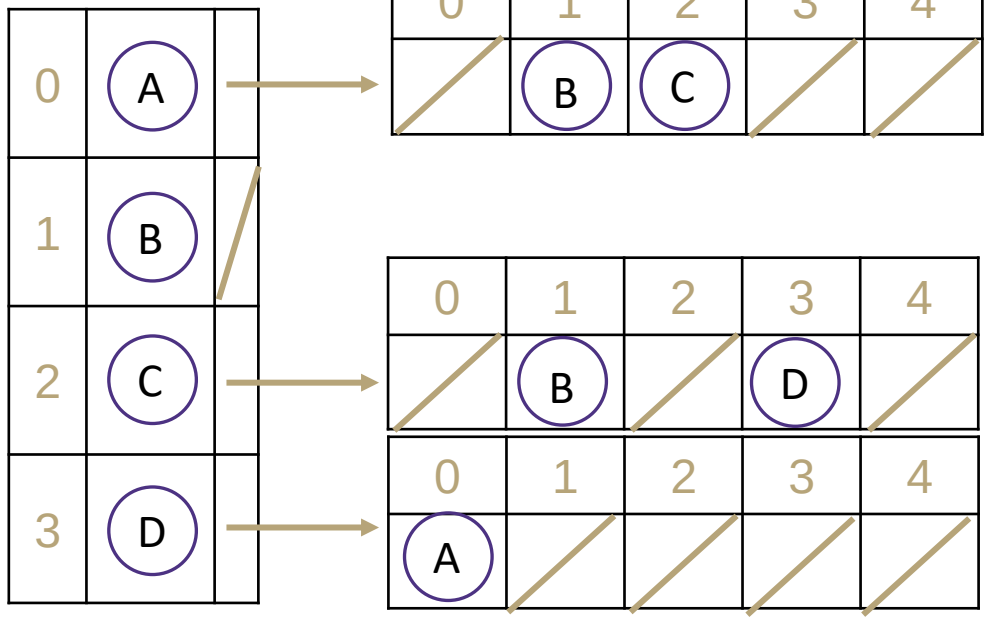
Annahme: Tabelle hat keine Variable size. Sonst ausgehende Kanten in O(1) bestimmbar.

„In der Praxis“ Laufzeit (|V| = n, |E| = m):

- Kante hinzufügen: $\Theta(1+\lambda)$
- Kante entfernen: $\Theta(1+\lambda)$
- Prüfen, ob Kante (u,v) existiert: $\Theta(1+\lambda)$
- Erhalte ausgehende Kanten von u:
- Erhalte eingehende Kanten von u:

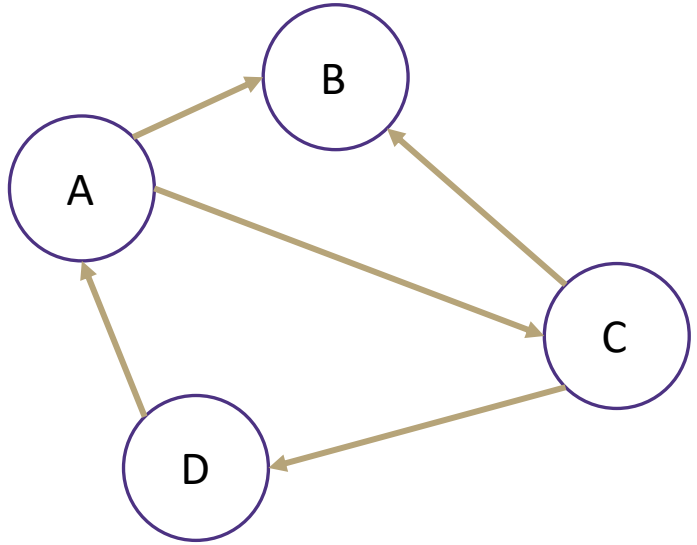
Speicherplatzbedarf:

n Hashtabellen der Größe m



Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022
CSE 373 20 SP - KASEY CHAMPION

Adjazenzliste



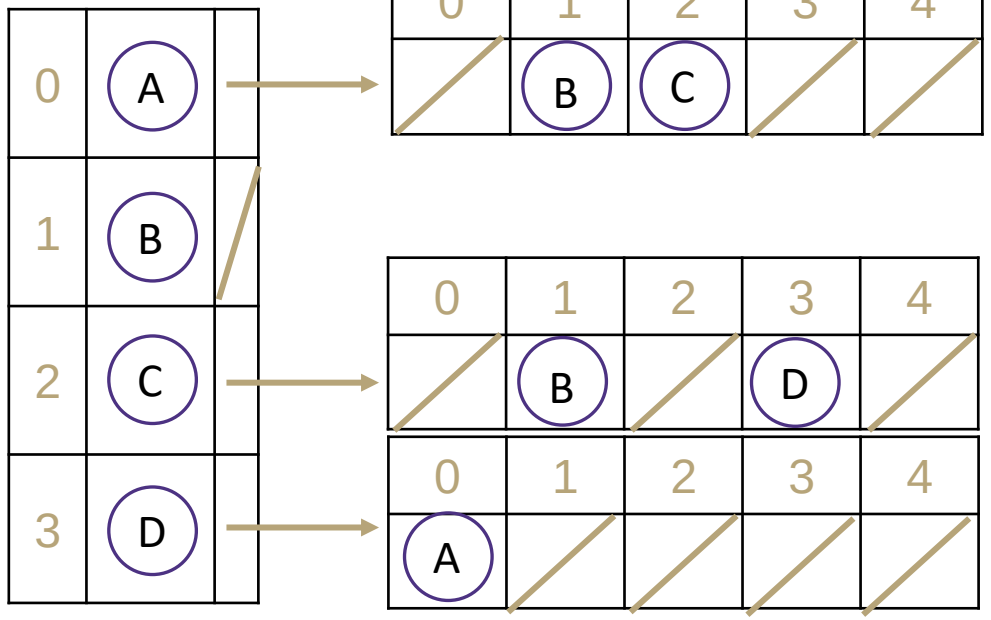
Annahme: Tabelle hat keine Variable size. Sonst ausgehende Kanten in O(1) bestimmbar.

„In der Praxis“ Laufzeit (|V| = n, |E| = m):

- Kante hinzufügen: $\Theta(1+\lambda)$
- Kante entfernen: $\Theta(1+\lambda)$
- Prüfen, ob Kante (u,v) existiert: $\Theta(1+\lambda)$
- Erhalte ausgehende Kanten von u: $\Theta(m)$
- Erhalte eingehende Kanten von u:

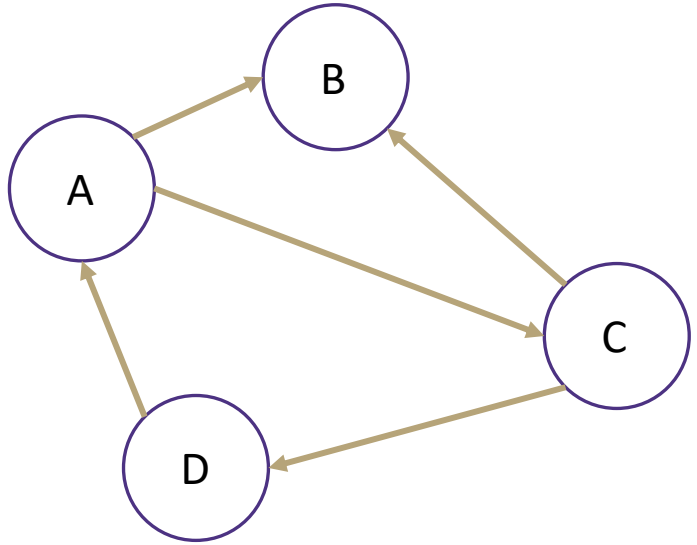
Speicherplatzbedarf:

n Hashtabellen der Größe m



Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022
CSE 373 20 SP - KASEY CHAMPION

Adjazenzliste



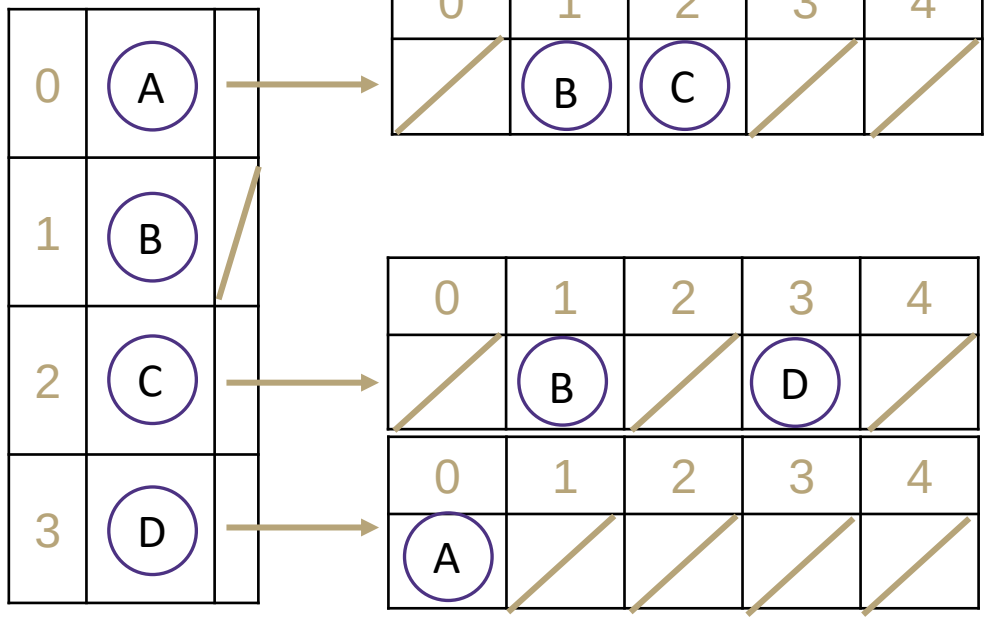
Annahme: Tabelle hat keine Variable size. Sonst ausgehende Kanten in O(1) bestimmbar.

„In der Praxis“ Laufzeit (|V| = n, |E| = m):

- Kante hinzufügen: $\Theta(1+\lambda)$
- Kante entfernen: $\Theta(1+\lambda)$
- Prüfen, ob Kante (u,v) existiert: $\Theta(1+\lambda)$
- Erhalte ausgehende Kanten von u: $\Theta(m)$
- Erhalte eingehende Kanten von u: $\Theta(n*(2+\lambda))$

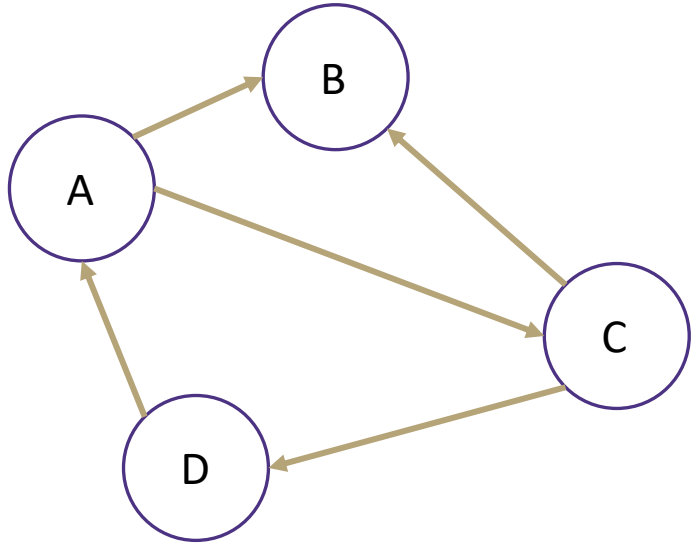
Speicherplatzbedarf:

n Hashtabellen der Größe m



Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022
CSE 373 20 SP - KASEY CHAMPION

Adjazenzliste



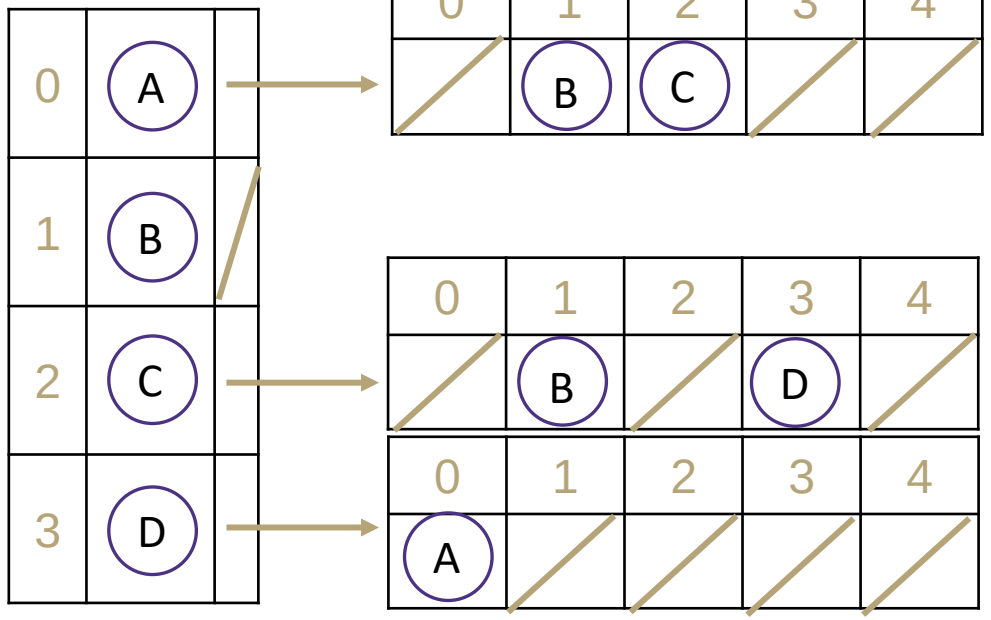
Annahme: Tabelle hat keine Variable size. Sonst ausgehende Kanten in O(1) bestimmbar.

„In der Praxis“ Laufzeit (|V| = n, |E| = m):

- Kante hinzufügen: $\Theta(1+\lambda)$
- Kante entfernen: $\Theta(1+\lambda)$
- Prüfen, ob Kante (u,v) existiert: $\Theta(1+\lambda)$
- Erhalte ausgehende Kanten von u: $\Theta(m)$
- Erhalte eingehende Kanten von u: $\Theta(n*(2+\lambda))$

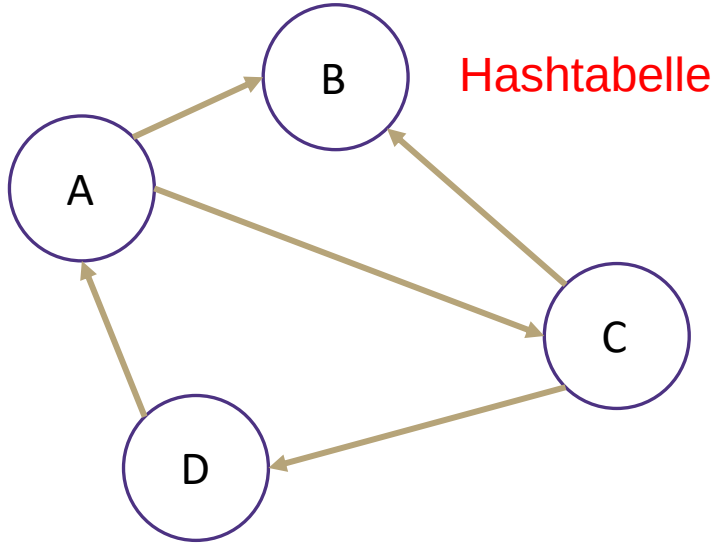
Speicherplatzbedarf: $\Theta(n * (m+1))$

n Hashtabellen der Größe m



Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022
CSE 373 20 SP - KASEY CHAMPION

Adjazenzliste



Hashtabelle dürfte auch Größe n haben

Annahme: Tabelle hat keine Variable size. Sonst ausgehende Kanten in O(1) bestimmbar.

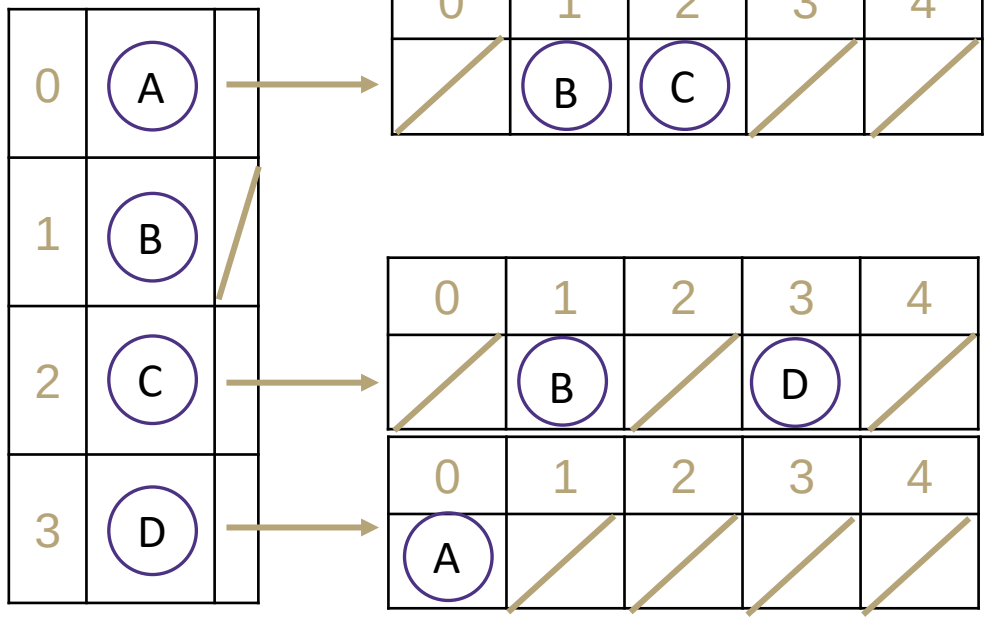
„In der Praxis“ Laufzeit (|V| = n, |E| = m):

- Kante hinzufügen: $\Theta(1+\lambda)$
- Kante entfernen: $\Theta(1+\lambda)$
- Prüfen, ob Kante (u,v) existiert: $\Theta(1+\lambda)$
- Erhalte ausgehende Kanten von u: $\Theta(m)$
- Erhalte eingehende Kanten von u: $\Theta(n*(2+\lambda))$

Speicherplatzbedarf: $\Theta(n * (m+1))$

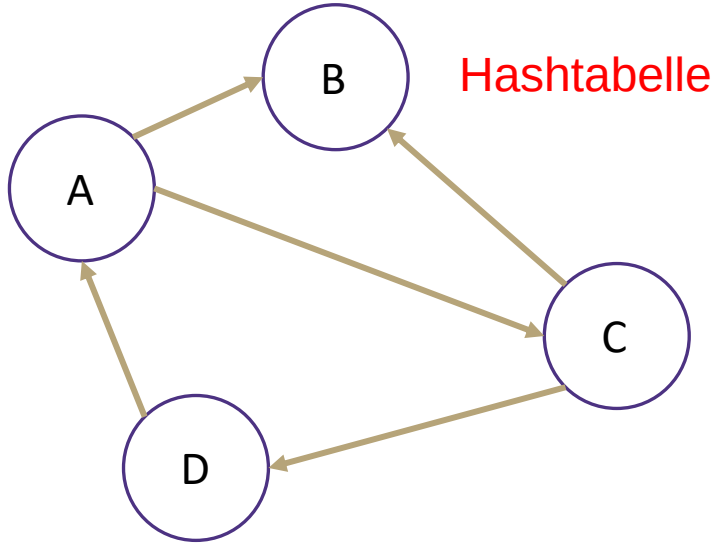
Wenn Hashtabellen nur Größe n haben:

n Hashtabellen der Größe m



Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022
CSE 373 20 SP - KASEY CHAMPION

Adjazenzliste



Hashtabelle dürfte auch Größe n haben

Annahme: Tabelle hat keine Variable size. Sonst ausgehende Kanten in O(1) bestimmbar.

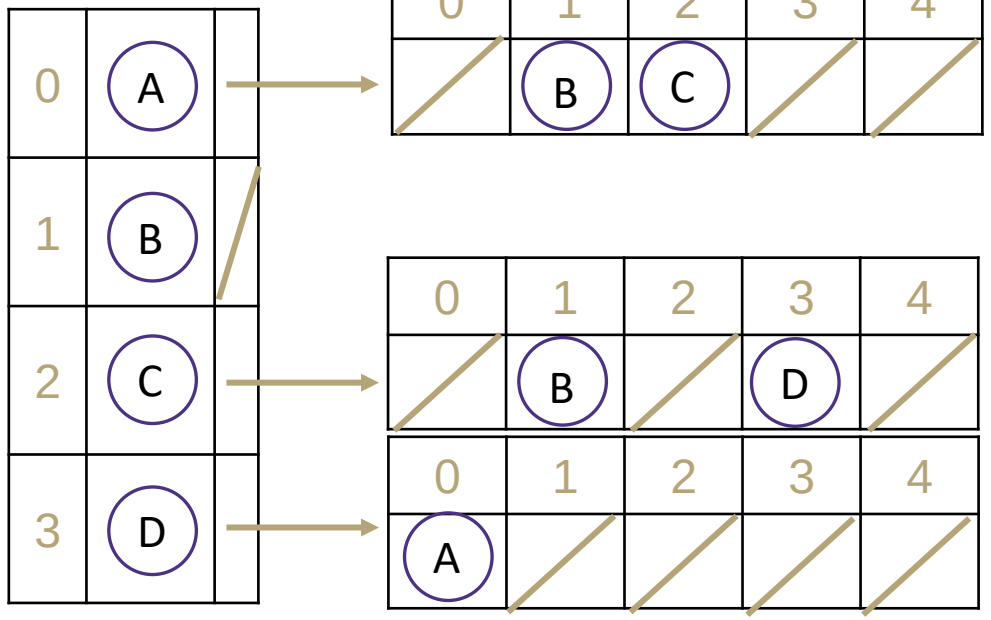
„In der Praxis“ Laufzeit (|V| = n, |E| = m):

- Kante hinzufügen: $\Theta(1+\lambda)$
- Kante entfernen: $\Theta(1+\lambda)$
- Prüfen, ob Kante (u,v) existiert: $\Theta(1+\lambda)$
- Erhalte ausgehende Kanten von u: $\Theta(m)$
- Erhalte eingehende Kanten von u: $\Theta(n*(2+\lambda))$

Speicherplatzbedarf: $\Theta(n * (m+1))$

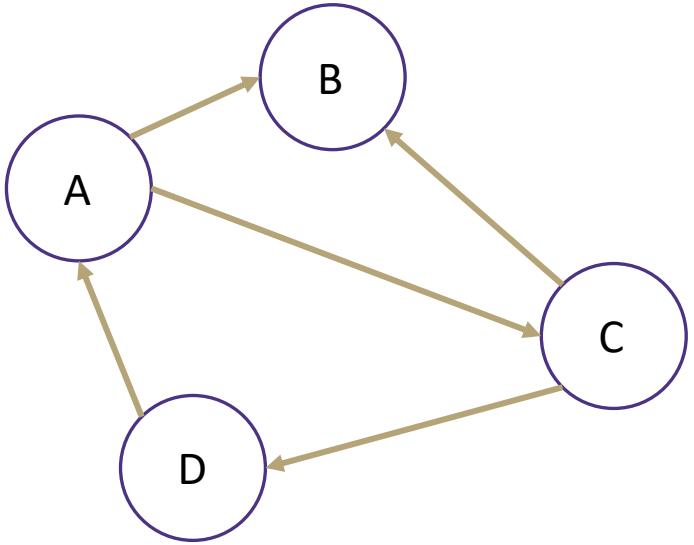
Wenn Hashtabellen nur Größe n haben: $\Theta(n + n * n)$

n Hashtabellen der Größe m



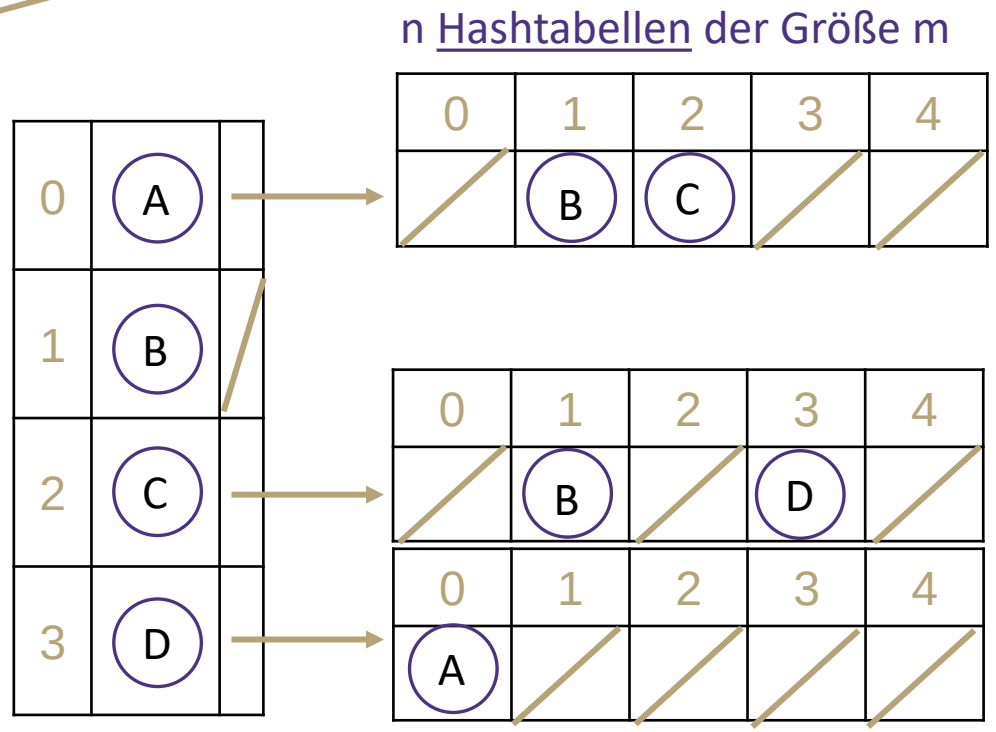
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022
CSE 373 20 SP - KASEY CHAMPION

Adjazenzliste

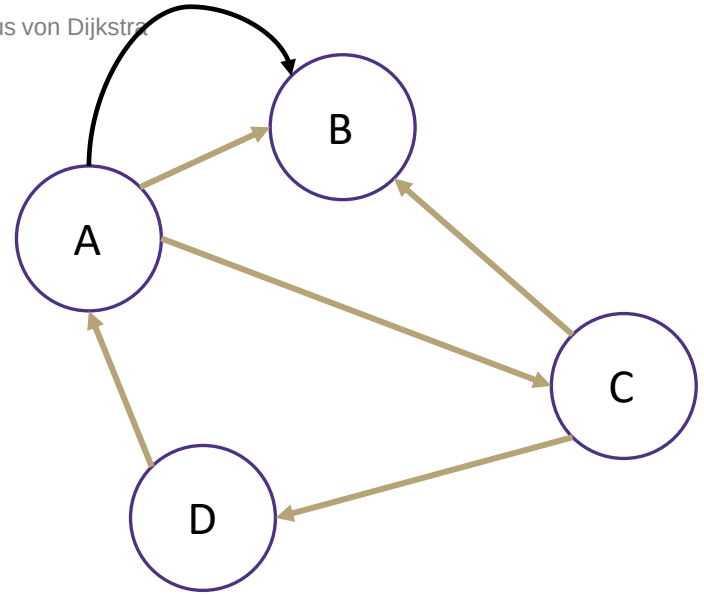


Hinzufügen einer zweiten Kante (A, B)

- Bspw. lineare Sondierung

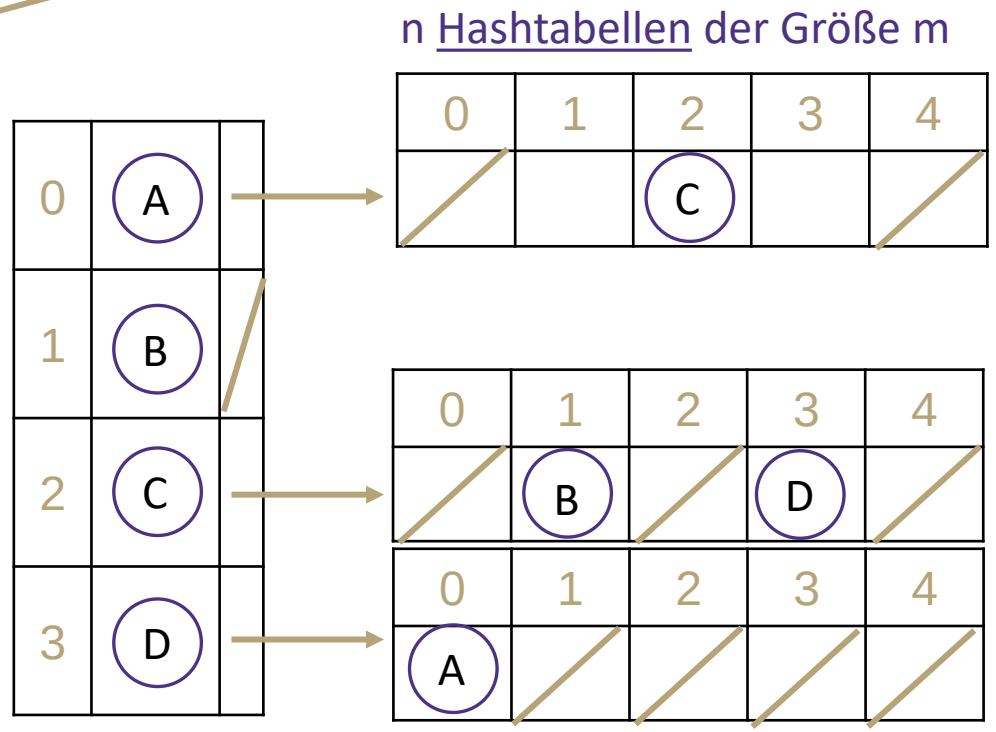


Adjazenzliste

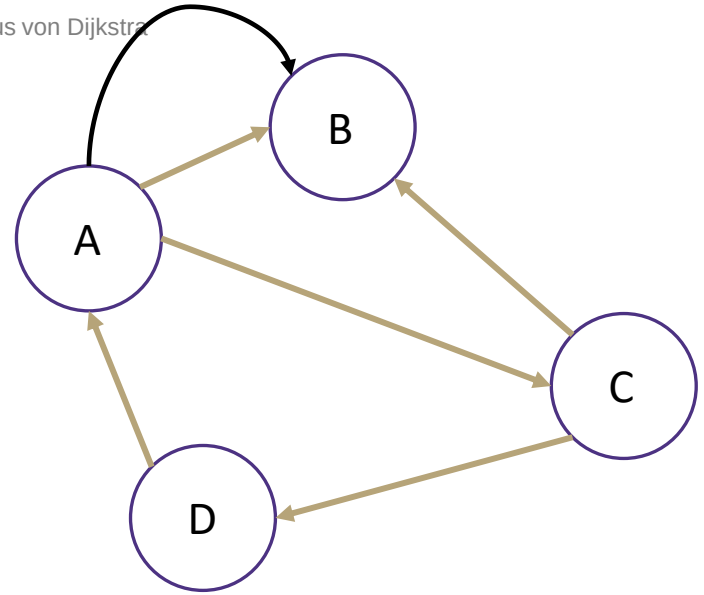


Hinzufügen einer zweiten Kante (A, B)

- Bspw. lineare Sondierung

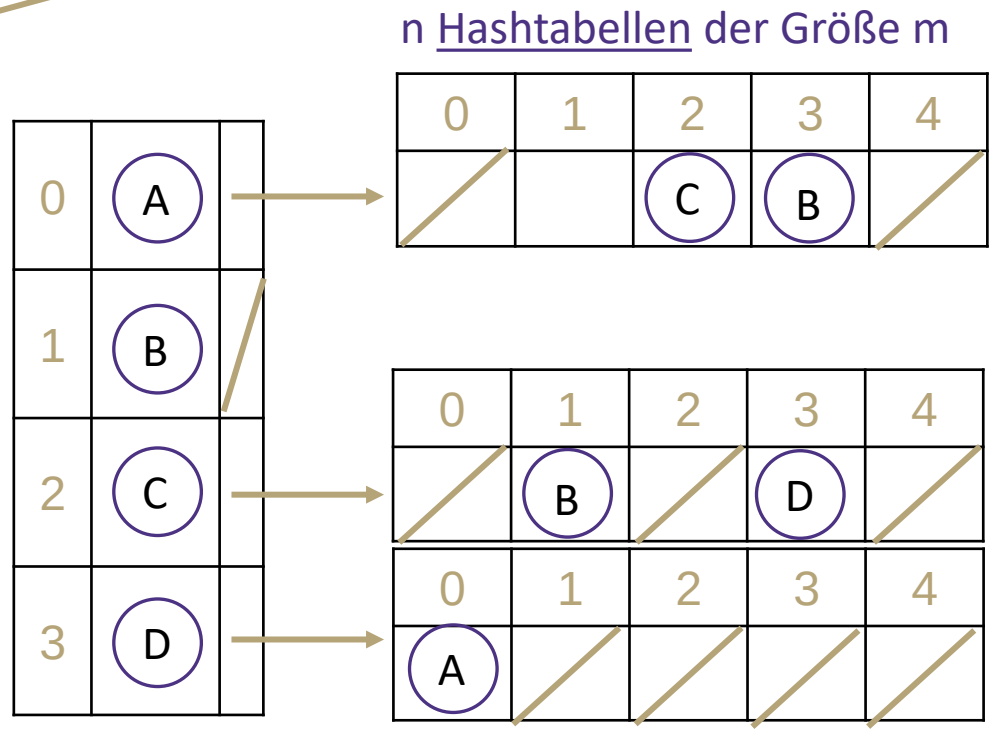


Adjazenzliste

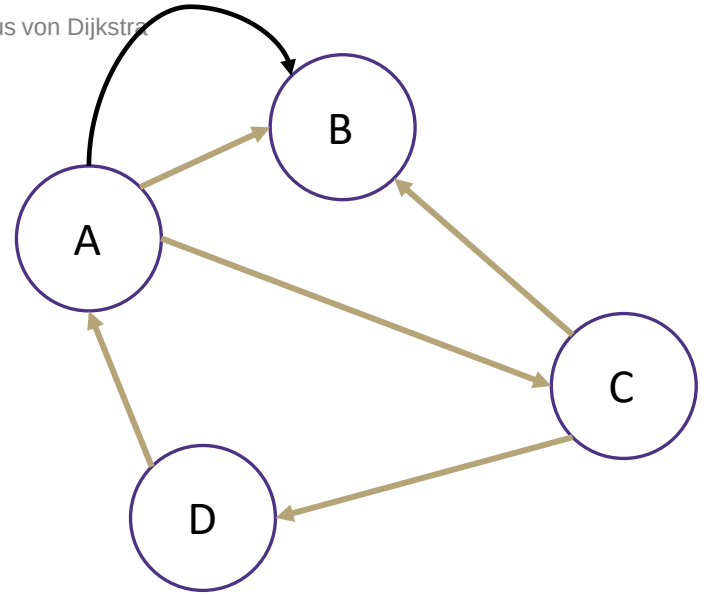


Hinzufügen einer zweiten Kante (A, B)

- Bspw. lineare Sondierung



Adjazenzliste

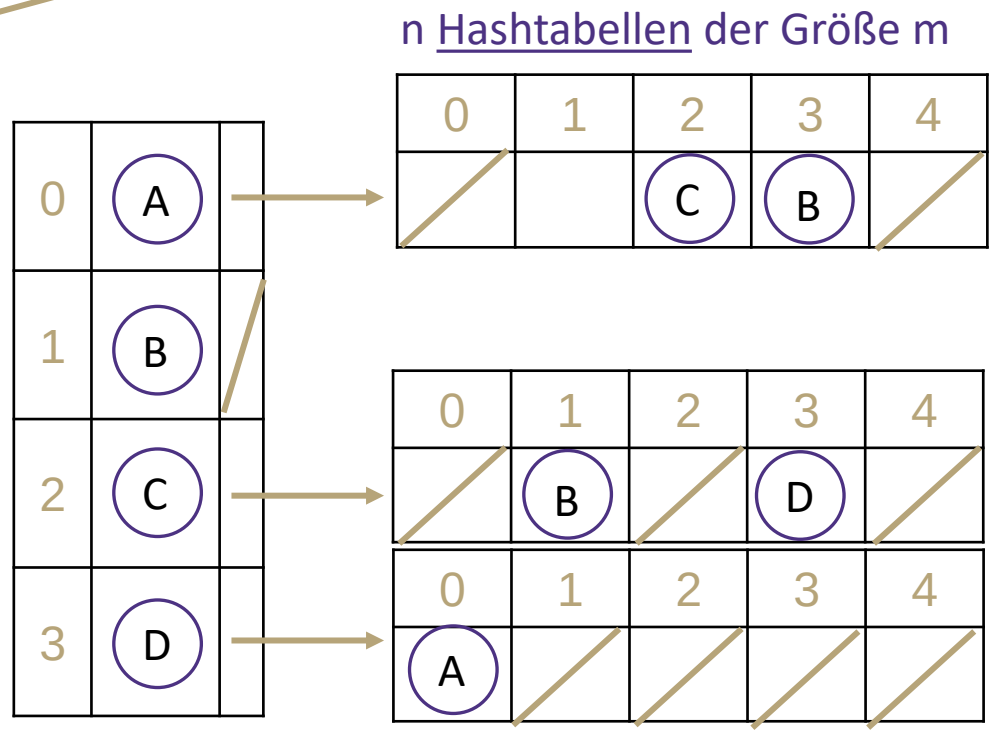


Hinzufügen einer zweiten Kante (A, B)

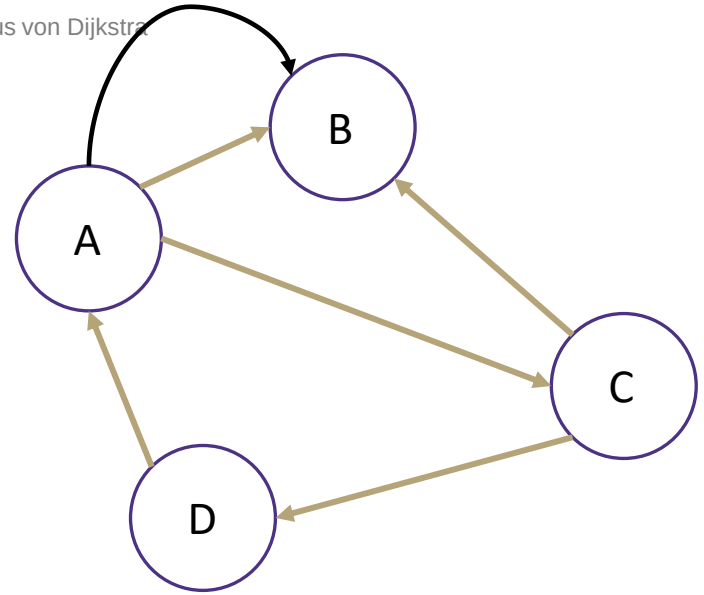
- Bspw. lineare Sondierung

Löschen der Kante (A, B)

-
  -



Adjazenzliste

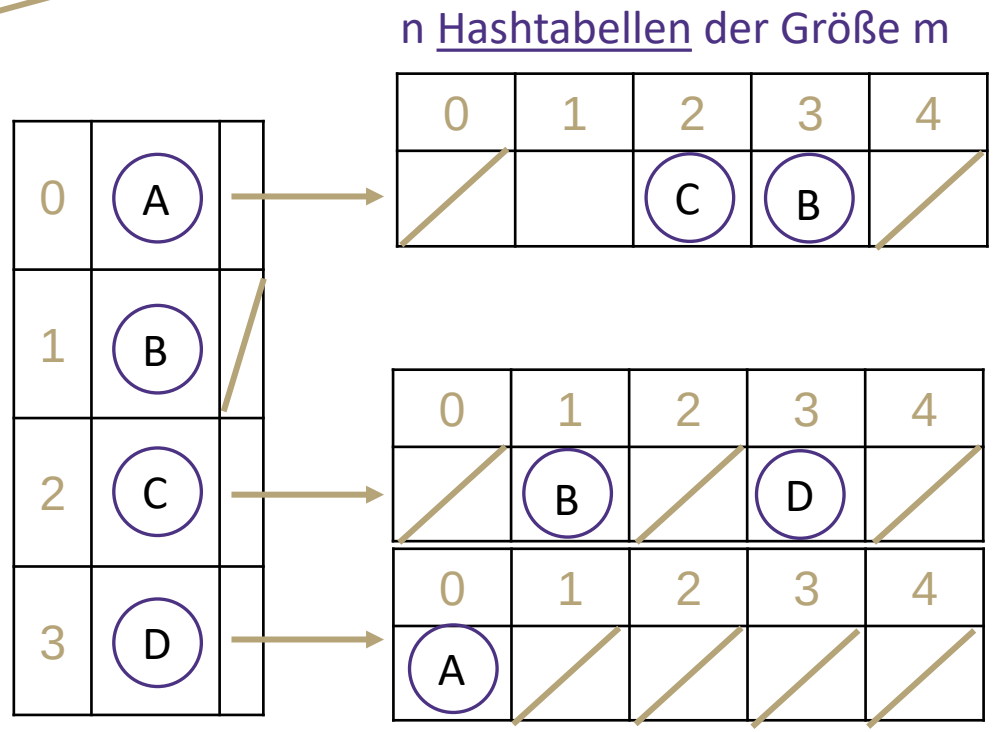


Hinzufügen einer zweiten Kante (A, B)

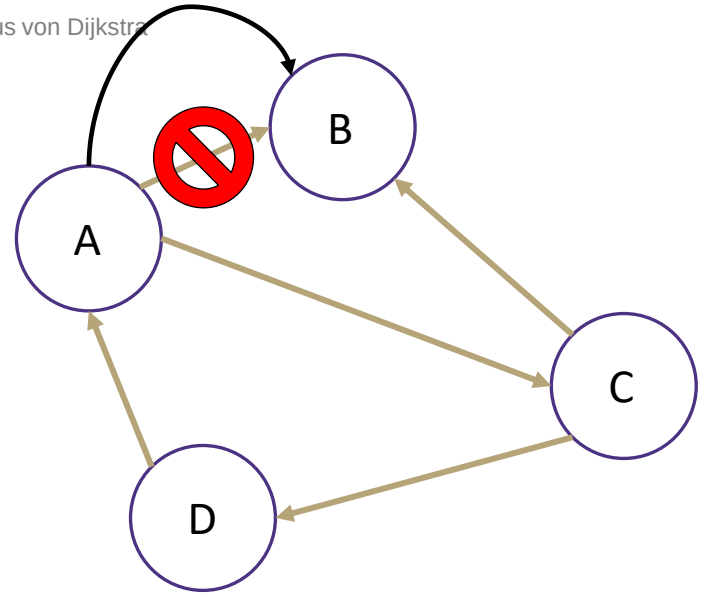
- Bspw. lineare Sondierung

Löschen der Kante (A, B)

- Lazy Deletion:
  - Löschen des ersten „B“s, das wir finden. Ersetzen durch einen tombstone.



Adjazenzliste

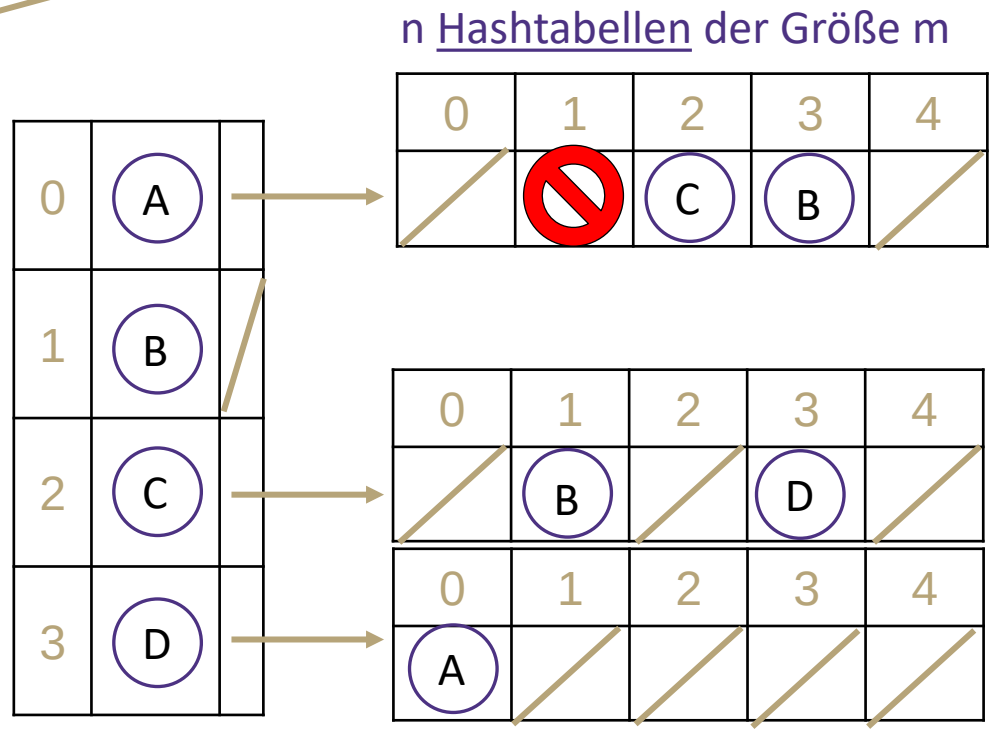


Hinzufügen einer zweiten Kante (A, B)

- Bspw. lineare Sondierung

Löschen der Kante (A, B)

- Lazy Deletion:
  - Löschen des ersten „B“s, das wir finden. Ersetzen durch einen tombstone.



Adjazenzmatrix und –liste: Abwägungen

Adjazenzmatrizen vs. Adjazenzlisten (implementiert als verkettete Liste):

- Prüfung ob Kante existiert: Adjazenzmatrix: Θ(1); Adjazenzliste: Θ(deg(u))

Adjazenzmatrix und –liste: Abwägungen

Adjazenzmatrizen vs. Adjazenzlisten (implementiert als verkettete Liste):

- Prüfung ob Kante existiert: Adjazenzmatrix: Θ(1); Adjazenzliste: Θ(deg(u))
- Ausgangsverbindung zählen: Adjazenzmatrix: Θ(n); Adjazenzliste: Θ(deg(u))
-
  -
-
-
-
-
-

Adjazenzmatrix und –liste: Abwägungen

Adjazenzmatrizen vs. Adjazenzlisten (implementiert als verkettete Liste):

- Prüfung ob Kante existiert: Adjazenzmatrix: Θ(1); Adjazenzliste: Θ(deg(u))
- Ausgangsverbindung zählen: Adjazenzmatrix: Θ(n); Adjazenzliste: Θ(deg(u))
- Speicherbedarf: Adjazenzmatrix: Θ(n²); Adjazenzliste: Θ(n + m)
  - Bei **dichten Graphen** (wo m nahe bei n^2 liegt) liegt der Speicherbedarf nahe beieinander
-
-
-
-
-

Adjazenzmatrix und –liste: Abwägungen

Adjazenzmatrizen vs. Adjazenzlisten (implementiert als verkettete Liste):

- Prüfung ob Kante existiert: Adjazenzmatrix: Θ(1); Adjazenzliste: Θ(deg(u))
- Ausgangsverbindung zählen: Adjazenzmatrix: Θ(n); Adjazenzliste: Θ(deg(u))
- Speicherbedarf: Adjazenzmatrix: Θ(n²); Adjazenzliste: Θ(n + m)
  - Bei **dichten Graphen** (wo m nahe bei n^2 liegt) liegt der Speicherbedarf nahe beieinander
- Zugriffszeiten für Arrays (Adjazenzmatrix) können viel besser sein als für verkettete Listen
-
-
-
-

Adjazenzmatrix und –liste: Abwägungen

Adjazenzmatrizen vs. Adjazenzlisten (implementiert als verkettete Liste):

- Prüfung ob Kante existiert: Adjazenzmatrix: Θ(1); Adjazenzliste: Θ(deg(u))
- Ausgangsverbindung zählen: Adjazenzmatrix: Θ(n); Adjazenzliste: Θ(deg(u))
- Speicherbedarf: Adjazenzmatrix: Θ(n²); Adjazenzliste: Θ(n + m)
  - Bei **dichten Graphen** (wo m nahe bei n^2 liegt) liegt der Speicherbedarf nahe beieinander
- Zugriffszeiten für Arrays (Adjazenzmatrix) können viel besser sein als für verkettete Listen
- Fazit: **dünn** (m ~ n): **Adjazenzliste**; **dicht** (m ~ n^2): **Adjazenzmatrix**

Adjazenzmatrix und –liste: Abwägungen

Adjazenzmatrizen vs. Adjazenzlisten (implementiert als verkettete Liste):

- Prüfung ob Kante existiert: Adjazenzmatrix: Θ(1); Adjazenzliste: Θ(deg(u))
- Ausgangsverbindung zählen: Adjazenzmatrix: Θ(n); Adjazenzliste: Θ(deg(u))
- Speicherbedarf: Adjazenzmatrix: Θ(n²); Adjazenzliste: Θ(n + m)
  - Bei **dichten Graphen** (wo m nahe bei n^2 liegt) liegt der Speicherbedarf nahe beieinander
- Zugriffszeiten für Arrays (Adjazenzmatrix) können viel besser sein als für verkettete Listen
- Fazit: **dünn** (m ~ n): **Adjazenzliste**; **dicht** (m ~ n^2): **Adjazenzmatrix**

Adjazenzliste: Implementierung als verkettete Listen vs. Hashtabellen:

- Existiert Kante und Kante löschen: Hashtabelle: Θ(1+λ); verkettete Liste: Θ(deg(u))
- Hashtabellen benötigen mehr Speicherplatz: Θ(n * (m+1)) anstatt Θ(n + m)
-

Adjazenzmatrix und –liste: Abwägungen

Adjazenzmatrizen vs. Adjazenzlisten (implementiert als verkettete Liste):

- Prüfung ob Kante existiert: Adjazenzmatrix: Θ(1); Adjazenzliste: Θ(deg(u))
- Ausgangsverbindung zählen: Adjazenzmatrix: Θ(n); Adjazenzliste: Θ(deg(u))
- Speicherbedarf: Adjazenzmatrix: Θ(n²); Adjazenzliste: Θ(n + m)
  - Bei **dichten Graphen** (wo m nahe bei n^2 liegt) liegt der Speicherbedarf nahe beieinander
- Zugriffszeiten für Arrays (Adjazenzmatrix) können viel besser sein als für verkettete Listen
- Fazit: **dünn** (m ~ n): **Adjazenzliste**; **dicht** (m ~ n^2): **Adjazenzmatrix**

Adjazenzliste: Implementierung als verkettete Listen vs. Hashtabellen:

- Existiert Kante und Kante löschen: Hashtabelle: Θ(1+λ); verkettete Liste: Θ(deg(u))
- Hashtabellen benötigen mehr Speicherplatz: Θ(n * (m+1)) anstatt Θ(n + m)
- Was ist wichtiger? Speicherbedarf oder Laufzeit?

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Fragen?

Relevante Ideen bis hierhin

- Graphen modellieren Beziehungen zwischen realen Daten (Sie können Ihre Knoten und Kanten wählen)
- Es gibt verschiedene Implementierungen von Graphen
  - Adjazenzmatrix
  - Adjazenzliste

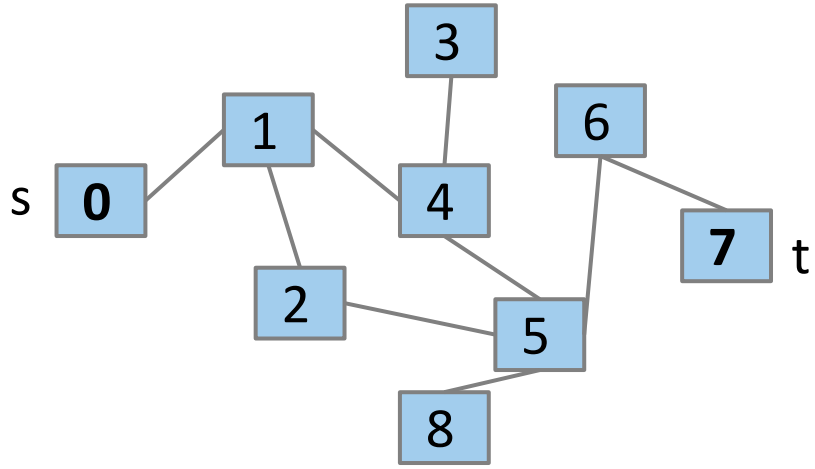
Übersicht über heute

- Graphen
  - Einführung/Anwendungen/Begriffe
- Traversierung von Graphen
  - **Erreichbarkeitsproblem**
  - **Tiefensuche**
  - **Breitensuche**
- Gerichtete Graphen
  - Topologische Sortierung
- Kürzeste Pfade
  - Ungewichteter Graph: Breitensuche
  - Gewichteter Graph: Algorithmus von Dijkstra

Erreichbarkeitsproblem

Erreichbarkeitsproblem

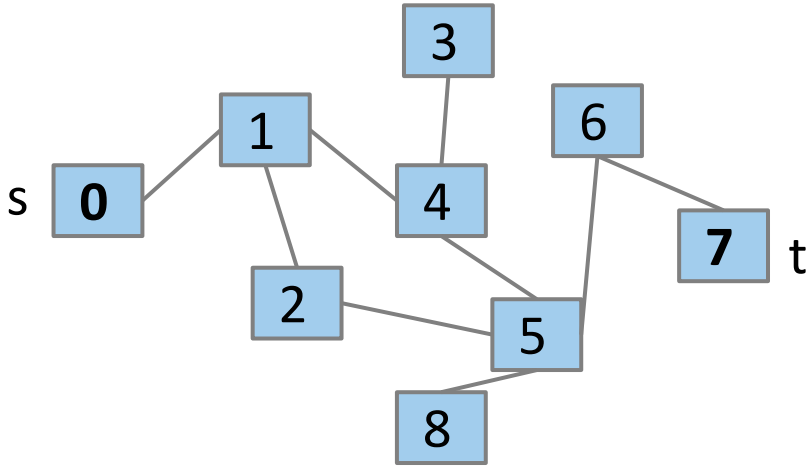
Gibt es bei einem Startknoten s und einem Zielknoten t einen Pfad zwischen s und t?



Erreichbarkeitsproblem

Erreichbarkeitsproblem

Gibt es bei einem Startknoten s und einem Zielknoten t einen Pfad zwischen s und t?



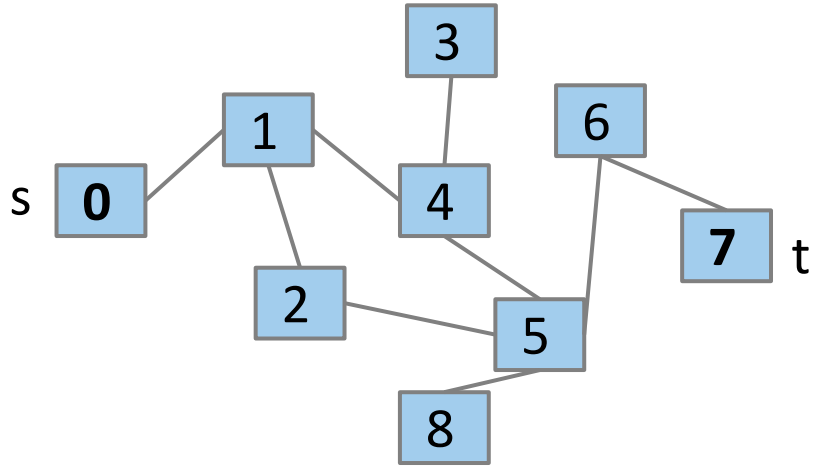
Mögliche Anwendungen:

- Reiseplanung - kann man von einem Ort (Knoten) zu einem anderen Ort (Knoten) gelangen, wenn man die verfügbaren Straßen (Kanten) berücksichtigt?
-
-

Erreichbarkeitsproblem

Erreichbarkeitsproblem

Gibt es bei einem Startknoten s und einem Zielknoten t einen Pfad zwischen s und t?



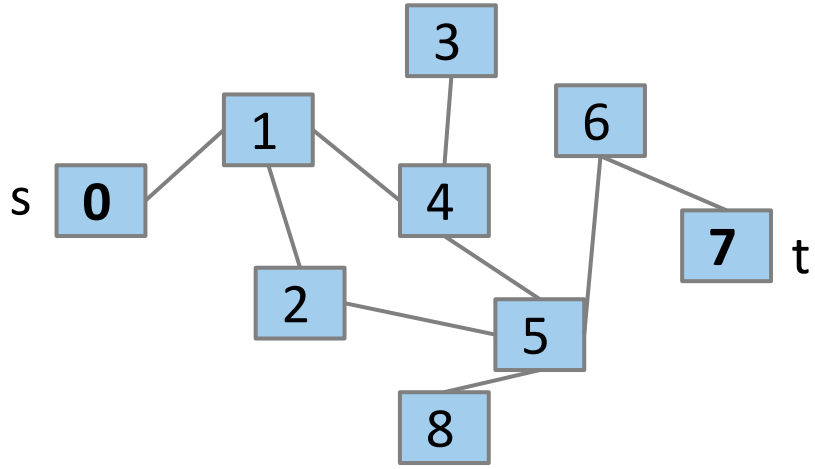
Mögliche Anwendungen:

- Reiseplanung - kann man von einem Ort (Knoten) zu einem anderen Ort (Knoten) gelangen, wenn man die verfügbaren Straßen (Kanten) berücksichtigt?
- Stammbäume und Überprüfung der Abstammung - sind zwei Personen (Knoten) durch einen gemeinsamen Vorfahren miteinander verwandt (gerichtete Kanten für direkte Eltern/Kind-Beziehungen)
-

Erreichbarkeitsproblem

Erreichbarkeitsproblem

Gibt es bei einem Startknoten s und einem Zielknoten t einen Pfad zwischen s und t?



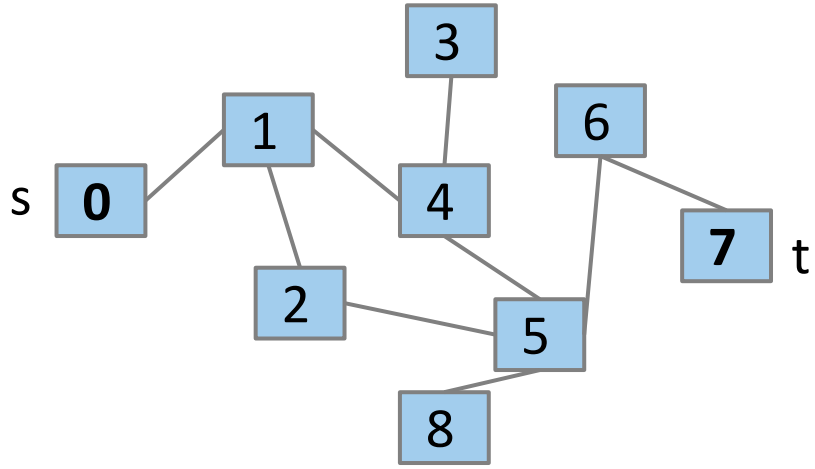
Mögliche Anwendungen:

- Reiseplanung - kann man von einem Ort (Knoten) zu einem anderen Ort (Knoten) gelangen, wenn man die verfügbaren Straßen (Kanten) berücksichtigt?
- Stammbäume und Überprüfung der Abstammung - sind zwei Personen (Knoten) durch einen gemeinsamen Vorfahren miteinander verwandt (gerichtete Kanten für direkte Eltern/Kind-Beziehungen)
- Spielzustände (KI) - kann man das Spiel vom aktuellen Knoten aus gewinnen (man denke an die aktuelle Brettposition)? Gibt es Züge (Kanten), die man machen kann, um zu dem Knoten zu gelangen, der ein bereits gewonnenes Spiel darstellt?

Erreichbarkeitsproblem

Erreichbarkeitsproblem

Gibt es bei einem Startknoten s und einem Zielknoten t einen Pfad zwischen s und t?

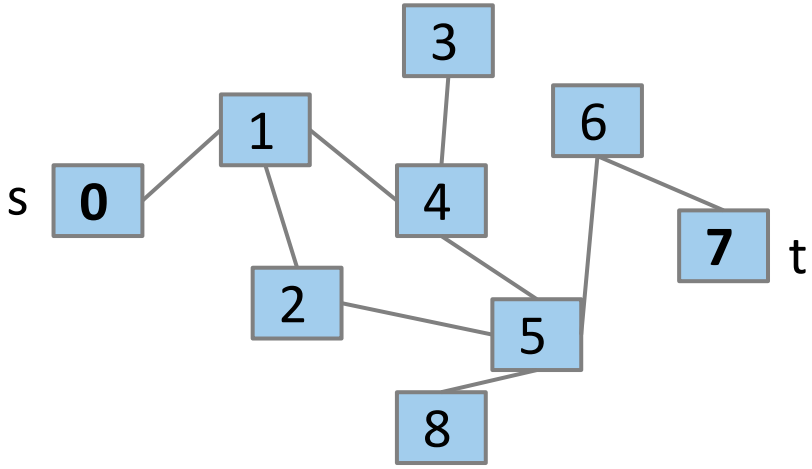


Versuchen Sie, einen Algorithmus für connected(s, t) zu finden.

Erreichbarkeitsproblem

Erreichbarkeitsproblem

Gibt es bei einem Startknoten s und einem Zielknoten t einen Pfad zwischen s und t?



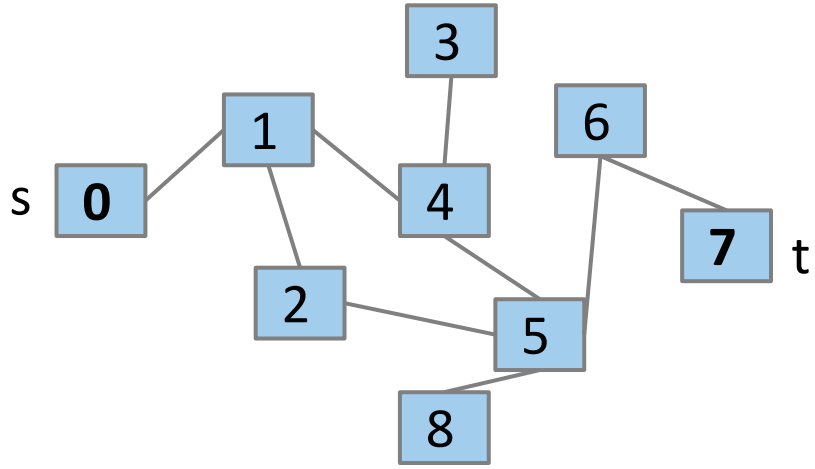
Versuchen Sie, einen Algorithmus für connected(s, t) zu finden.

Der Grundgedanke ist: Ausgehend von dem angegebenen s wird versucht, jeden möglichen Pfad zu durchlaufen, der nicht redundant ist, um zu sehen, ob er zu t führen könnte.

Erreichbarkeitsproblem

Erreichbarkeitsproblem

Gibt es bei einem Startknoten s und einem Zielknoten t einen Pfad zwischen s und t?



Versuchen Sie, einen Algorithmus für connected(s, t) zu finden.

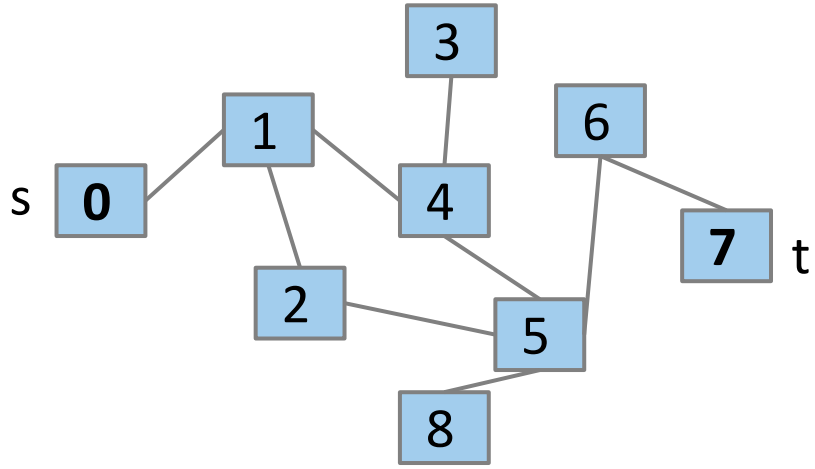
Der Grundgedanke ist: Ausgehend von dem angegebenen s wird versucht, jeden möglichen Pfad zu durchlaufen, der nicht redundant ist, um zu sehen, ob er zu t führen könnte.

Wir können eine Rekursion verwenden:

- Wenn ein Nachbar von s mit t verbunden ist, bedeutet das, dass s auch mit t verbunden ist!

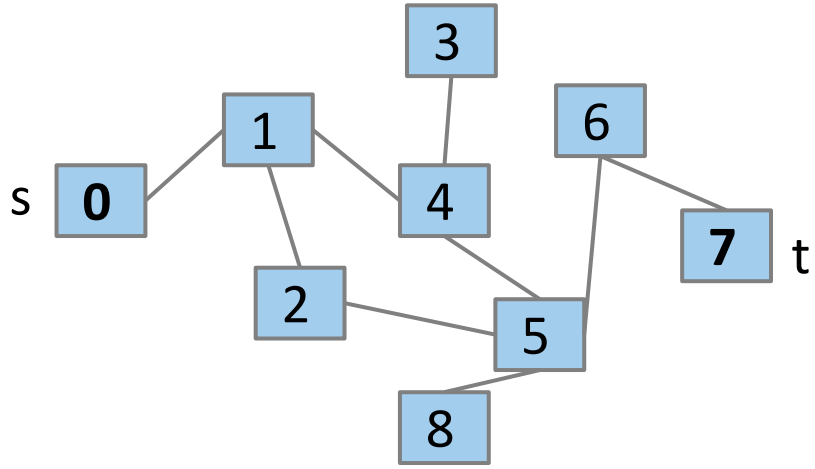
Erreichbarkeitsproblem: Vorgeschlagene Lösung

```
connected(Vertex s, Vertex t) {
  if (s == t) {
    return true;
  } else {
    for (Vertex n : s.neighbors) {
      if (connected(n, t)) {
        return true;
      }
    }
    return false;
  }
}
```



Erreichbarkeitsproblem: Was ist falsch an dieser Lösung?

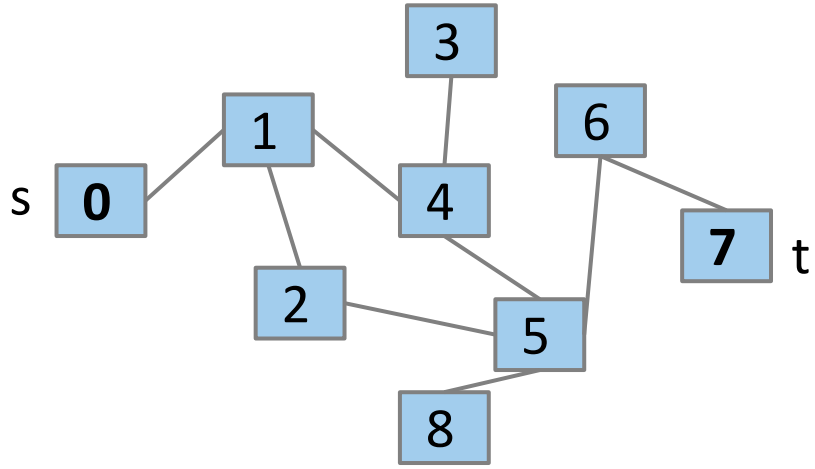
```
connected(Vertex s, Vertex t) {
  if (s == t) {
    return true;
  } else {
    for (Vertex n : s.neighbors) {
      if (connected(n, t)) {
        return true;
      }
    }
    return false;
  }
}
```



- Ist 0 == 7?
  -
-
  -
-

Erreichbarkeitsproblem: Was ist falsch an dieser Lösung?

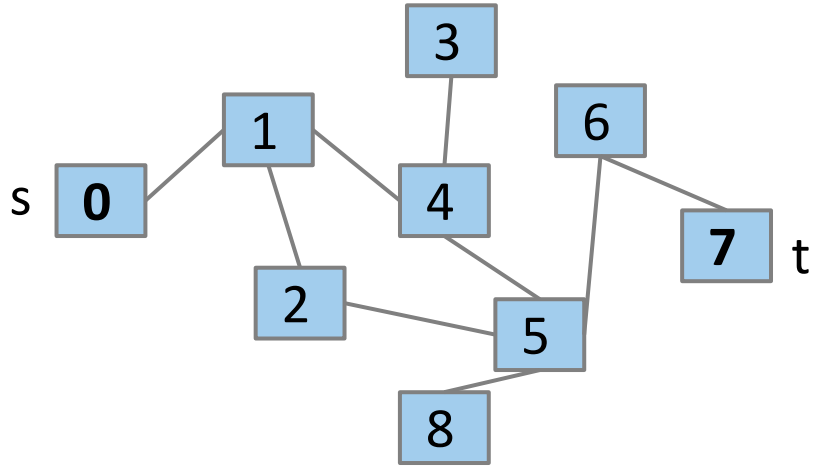
```
connected(Vertex s, Vertex t) {
  if (s == t) {
    return true;
  } else {
    for (Vertex n : s.neighbors) {
      if (connected(n, t)) {
        return true;
      }
    }
    return false;
  }
}
```



- Ist 0 == 7?
  - Nein; if(connected(1, 7)) return true;
-
  -
-

Erreichbarkeitsproblem: Was ist falsch an dieser Lösung?

```
connected(Vertex s, Vertex t) {
  if (s == t) {
    return true;
  } else {
    for (Vertex n : s.neighbors) {
      if (connected(n, t)) {
        return true;
      }
    }
    return false;
  }
}
```

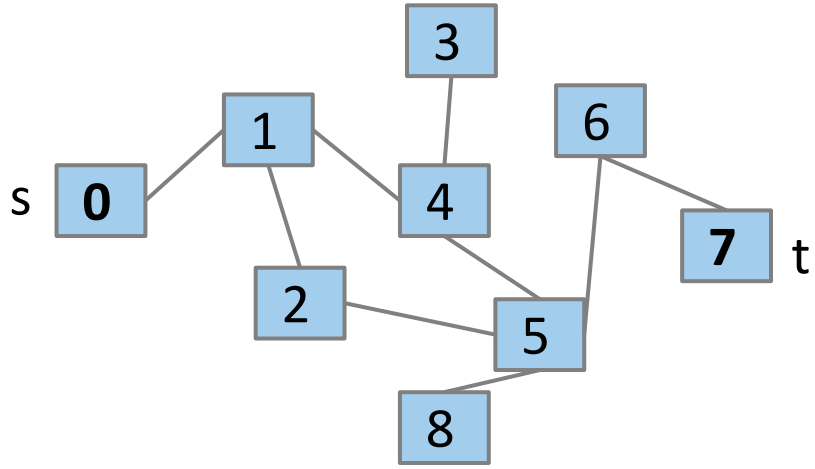


- Ist 0 == 7?
  - Nein; if(connected(1, 7)) return true;
- Ist 1 == 7?
  -
-

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Erreichbarkeitsproblem: Was ist falsch an dieser Lösung?

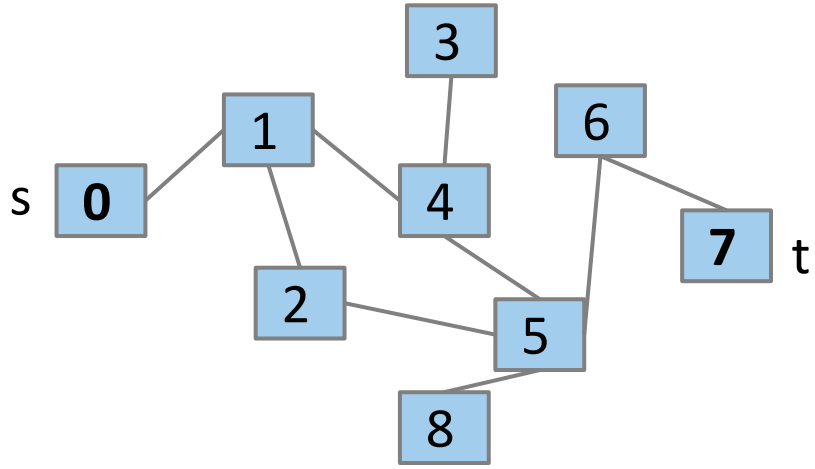
```
connected(Vertex s, Vertex t) {
  if (s == t) {
    return true;
  } else {
    for (Vertex n : s.neighbors) {
      if (connected(n, t)) {
        return true;
      }
    }
    return false;
  }
}
```



- Ist 0 == 7?
  - Nein; if(connected(1, 7)) return true;
- Ist 1 == 7?
  - Nein; if(connected(0, 7)) return true;
-

Erreichbarkeitsproblem: Was ist falsch an dieser Lösung?

```
connected(Vertex s, Vertex t) {
  if (s == t) {
    return true;
  } else {
    for (Vertex n : s.neighbors) {
      if (connected(n, t)) {
        return true;
      }
    }
    return false;
  }
}
```

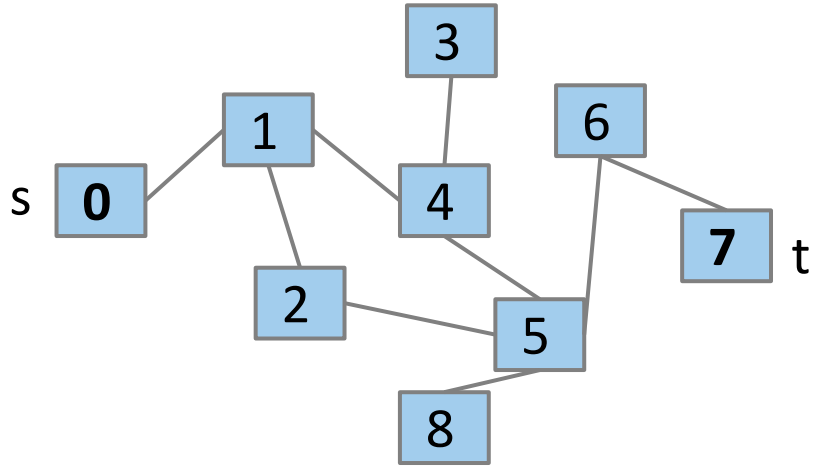


- Ist 0 == 7?
  - Nein; if(connected(1, 7)) return true;
- Ist 1 == 7?
  - Nein; if(connected(0, 7)) return true;
- Ist 0 == 7?

Erreichbarkeitsproblem: Bessere Lösung

Lösung: Markieren Sie jeden besuchten Knoten!

```
Set<Vertex> visited;  // assume global
connected(Vertex s, Vertex t) {
  if (s == t) {
    return true;
  } else {
    visited.add(s);
    for (Vertex n : s.neighbors) {
      if (!visited.contains(n)) {
        if (connected(n, t)) {
          return true;
        }
      }
    }
    return false;
  }
}
```

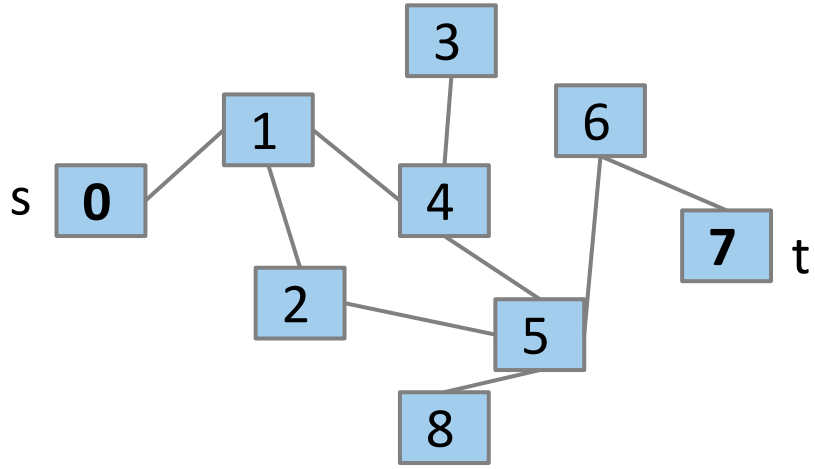


Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Erreichbarkeitsproblem: Bessere Lösung

Lösung: Markieren Sie jeden besuchten Knoten!

```
Set<Vertex> visited;  // assume global
connected(Vertex s, Vertex t) {
  if (s == t) {
    return true;
  } else {
    visited.add(s);
    for (Vertex n : s.neighbors) {
      if (!visited.contains(n)) {
        if (connected(n, t)) {
          return true;
        }
      }
    }
    return false;
  }
}
```



Dieser allgemeine Ansatz für das Durchsuchen eines Graphen wird die Grundlage für eine Vielzahl von Algorithmen sein!

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

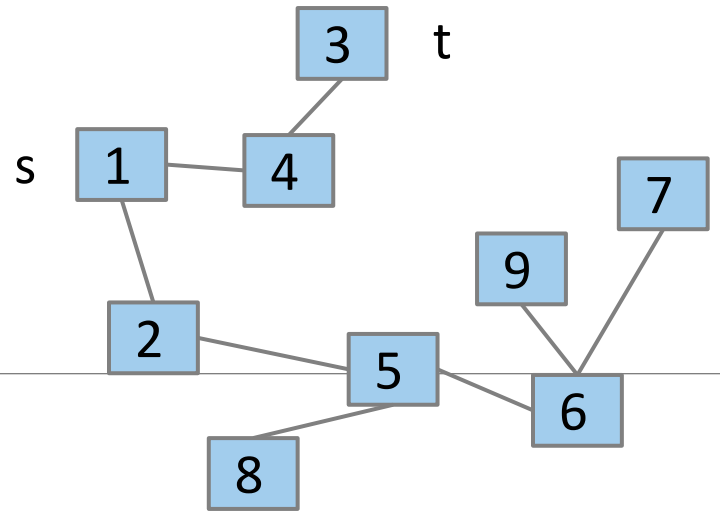
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Rekursive Tiefensuche (Depth-First Search DFS)

Welche Reihenfolge verwendet dieser Algorithmus, um die Knoten zu besuchen?

- Angenommen, die Reihenfolge der s.neighbors ist beliebig!

```
Set<Vertex> visited; // assume global
connected(Vertex s, Vertex t) {
  if (s == t) {
    return true;
  } else {
    visited.add(s);
    for (Vertex n : s.neighbors) {
      if (!visited.contains(n)) {
        if (connected(n, t)) {
          return true;
        }
      }
    }
    return false;
  }
}
```



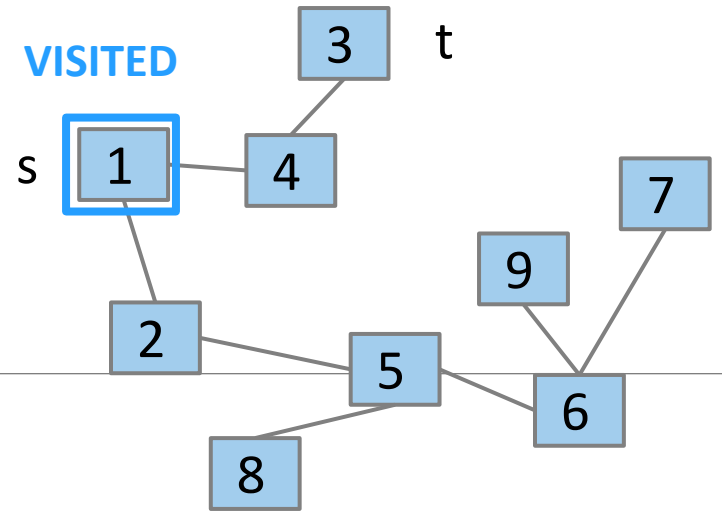
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Rekursive Tiefensuche (Depth-First Search DFS)

Welche Reihenfolge verwendet dieser Algorithmus, um die Knoten zu besuchen?

- Angenommen, die Reihenfolge der s.neighbors ist beliebig!

```
Set<Vertex> visited; // assume global
connected(Vertex s, Vertex t) {
  if (s == t) {
    return true;
  } else {
    visited.add(s);
    for (Vertex n : s.neighbors) {
      if (!visited.contains(n)) {
        if (connected(n, t)) {
          return true;
        }
      }
    }
    return false;
  }
}
```



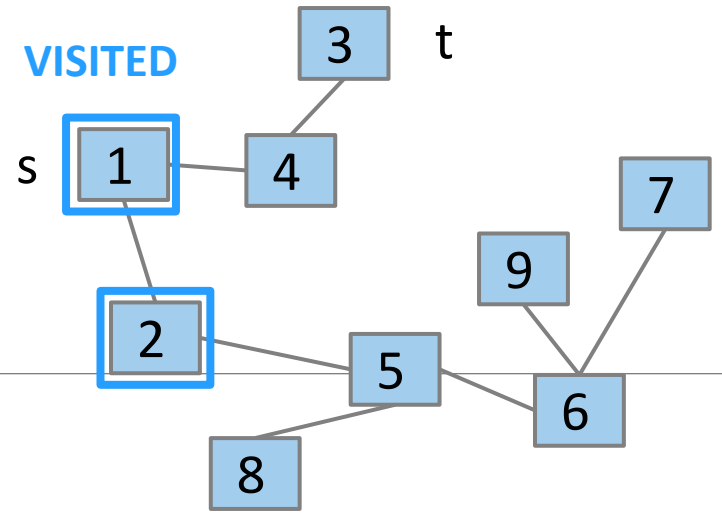
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Rekursive Tiefensuche (Depth-First Search DFS)

Welche Reihenfolge verwendet dieser Algorithmus, um die Knoten zu besuchen?

- Angenommen, die Reihenfolge der s.neighbors ist beliebig!

```
Set<Vertex> visited; // assume global
connected(Vertex s, Vertex t) {
  if (s == t) {
    return true;
  } else {
    visited.add(s);
    for (Vertex n : s.neighbors) {
      if (!visited.contains(n)) {
        if (connected(n, t)) {
          return true;
        }
      }
    }
    return false;
  }
}
```



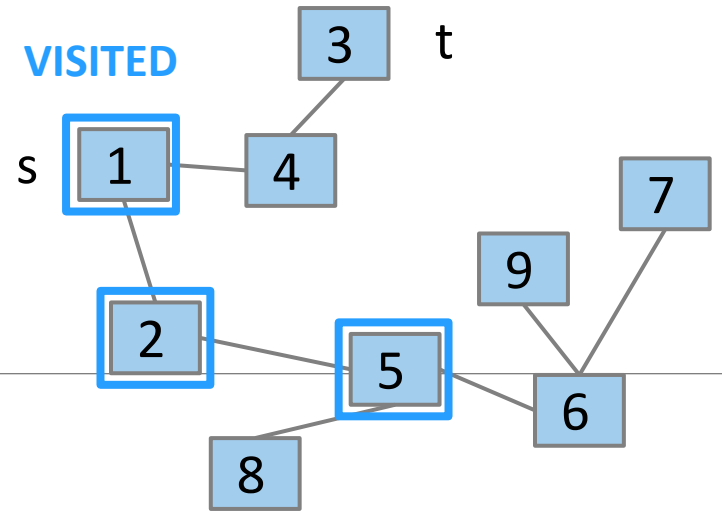
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Rekursive Tiefensuche (Depth-First Search DFS)

Welche Reihenfolge verwendet dieser Algorithmus, um die Knoten zu besuchen?

- Angenommen, die Reihenfolge der s.neighbors ist beliebig!

```
Set<Vertex> visited; // assume global
connected(Vertex s, Vertex t) {
  if (s == t) {
    return true;
  } else {
    visited.add(s);
    for (Vertex n : s.neighbors) {
      if (!visited.contains(n)) {
        if (connected(n, t)) {
          return true;
        }
      }
    }
    return false;
  }
}
```



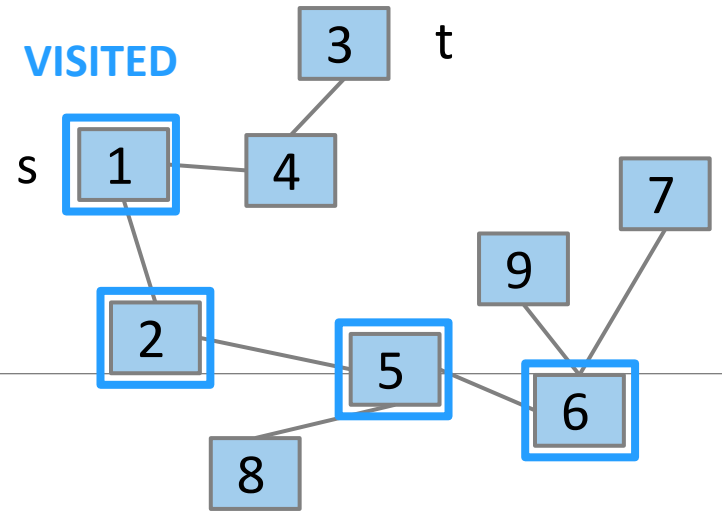
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Rekursive Tiefensuche (Depth-First Search DFS)

Welche Reihenfolge verwendet dieser Algorithmus, um die Knoten zu besuchen?

- Angenommen, die Reihenfolge der s.neighbors ist beliebig!

```
Set<Vertex> visited; // assume global
connected(Vertex s, Vertex t) {
  if (s == t) {
    return true;
  } else {
    visited.add(s);
    for (Vertex n : s.neighbors) {
      if (!visited.contains(n)) {
        if (connected(n, t)) {
          return true;
        }
      }
    }
    return false;
  }
}
```



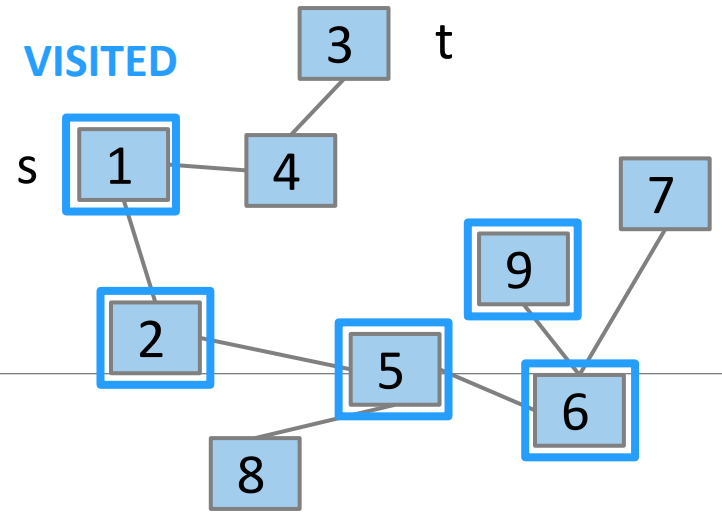
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Rekursive Tiefensuche (Depth-First Search DFS)

Welche Reihenfolge verwendet dieser Algorithmus, um die Knoten zu besuchen?

- Angenommen, die Reihenfolge der s.neighbors ist beliebig!

```
Set<Vertex> visited; // assume global
connected(Vertex s, Vertex t) {
  if (s == t) {
    return true;
  } else {
    visited.add(s);
    for (Vertex n : s.neighbors) {
      if (!visited.contains(n)) {
        if (connected(n, t)) {
          return true;
        }
      }
    }
    return false;
  }
}
```



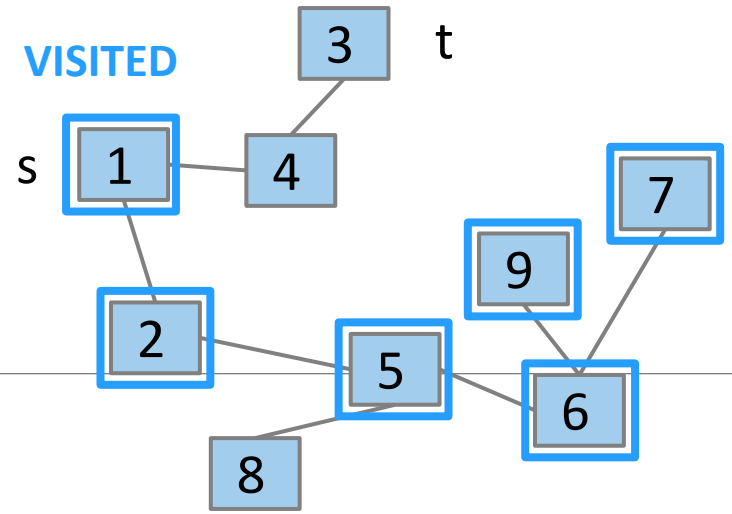
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Rekursive Tiefensuche (Depth-First Search DFS)

Welche Reihenfolge verwendet dieser Algorithmus, um die Knoten zu besuchen?

- Angenommen, die Reihenfolge der s.neighbors ist beliebig!

```
Set<Vertex> visited; // assume global
connected(Vertex s, Vertex t) {
  if (s == t) {
    return true;
  } else {
    visited.add(s);
    for (Vertex n : s.neighbors) {
      if (!visited.contains(n)) {
        if (connected(n, t)) {
          return true;
        }
      }
    }
    return false;
  }
}
```



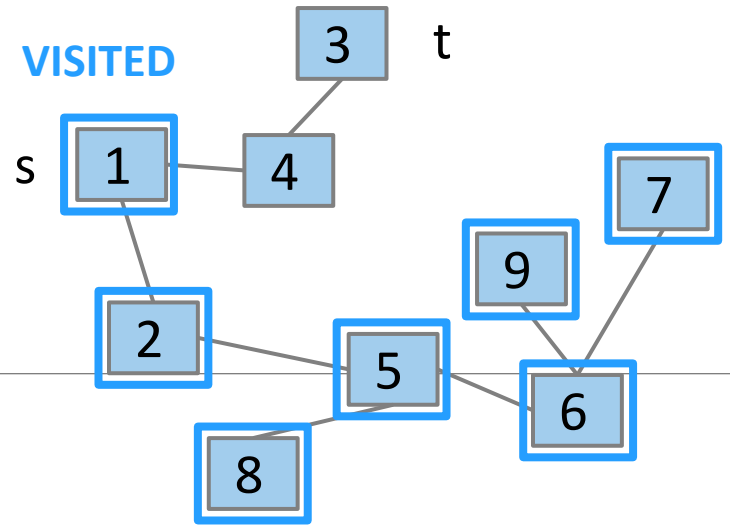
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Rekursive Tiefensuche (Depth-First Search DFS)

Welche Reihenfolge verwendet dieser Algorithmus, um die Knoten zu besuchen?

- Angenommen, die Reihenfolge der s.neighbors ist beliebig!

```
Set<Vertex> visited; // assume global
connected(Vertex s, Vertex t) {
  if (s == t) {
    return true;
  } else {
    visited.add(s);
    for (Vertex n : s.neighbors) {
      if (!visited.contains(n)) {
        if (connected(n, t)) {
          return true;
        }
      }
    }
    return false;
  }
}
```



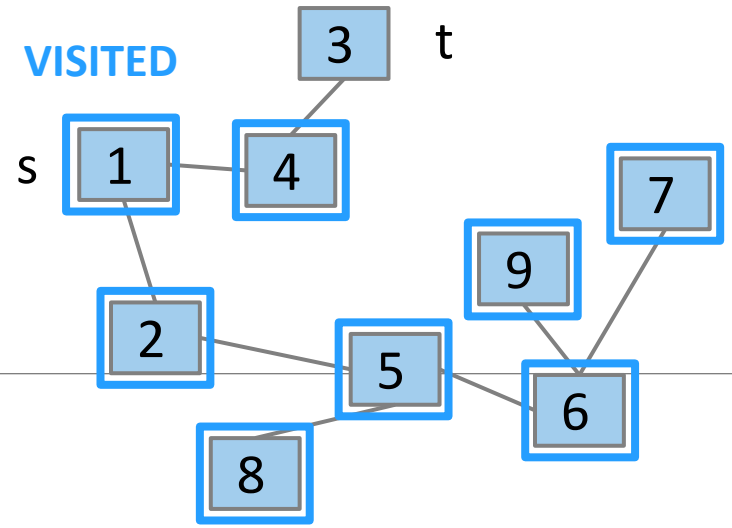
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Rekursive Tiefensuche (Depth-First Search DFS)

Welche Reihenfolge verwendet dieser Algorithmus, um die Knoten zu besuchen?

- Angenommen, die Reihenfolge der s.neighbors ist beliebig!

```
Set<Vertex> visited; // assume global
connected(Vertex s, Vertex t) {
  if (s == t) {
    return true;
  } else {
    visited.add(s);
    for (Vertex n : s.neighbors) {
      if (!visited.contains(n)) {
        if (connected(n, t)) {
          return true;
        }
      }
    }
    return false;
  }
}
```



Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Rekursive Tiefensuche (Depth-First Search DFS)

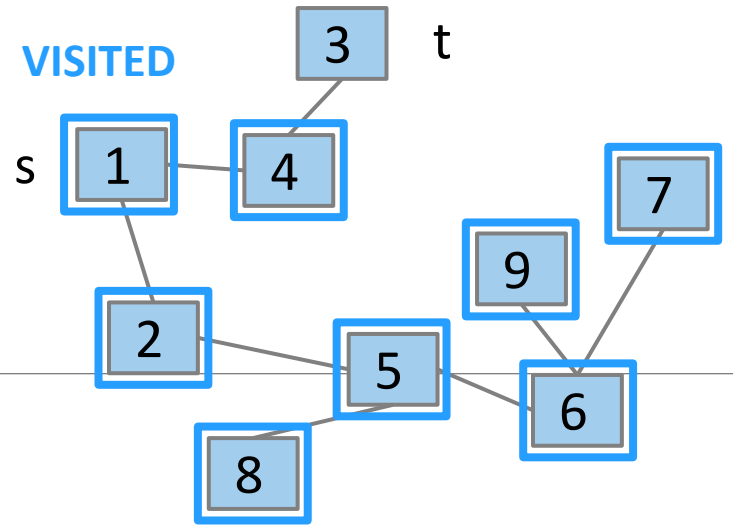
Welche Reihenfolge verwendet dieser Algorithmus, um die Knoten zu besuchen?

- Angenommen, die Reihenfolge der s.neighbors ist beliebig!

```
Set<Vertex> visited; // assume global
connected(Vertex s, Vertex t) {
  if (s == t) {
    return true;
  } else {
    visited.add(s);
    for (Vertex n : s.neighbors) {
      if (!visited.contains(n)) {
        if (connected(n, t)) {
          return true;
        }
      }
    }
    return false;
  }
}
```

Algorithmus erkundet einen Pfad „bis ganz nach unten“, bevor er zurückkommt, um andere Pfade auszuprobieren.

- Viele mögliche Reihenfolgen: {1, 2, 5, 6, 9, 7, 8, 4, 3} oder {1, 4, 3, 2, 5, 8, 6, 7, 9} sind möglich



Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Rekursive Tiefensuche (Depth-First Search DFS)

Welche Reihenfolge verwendet dieser Algorithmus, um die Knoten zu besuchen?

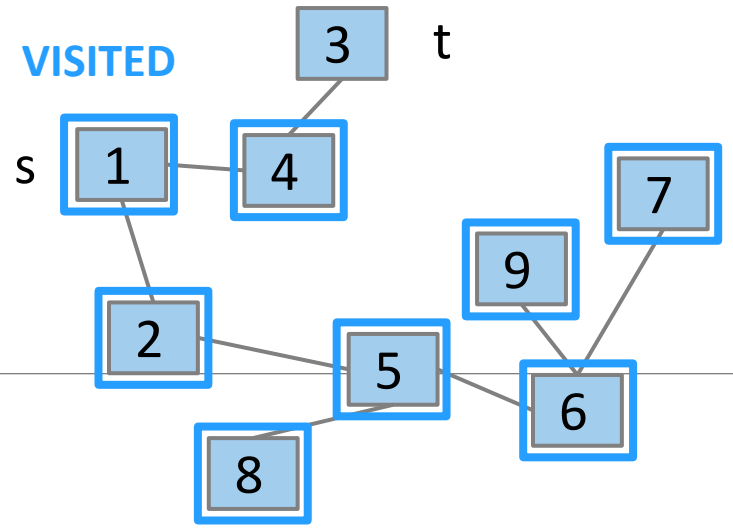
- Angenommen, die Reihenfolge der s.neighbors ist beliebig!

```
Set<Vertex> visited; // assume global
connected(Vertex s, Vertex t) {
  if (s == t) {
    return true;
  } else {
    visited.add(s);
    for (Vertex n : s.neighbors) {
      if (!visited.contains(n)) {
        if (connected(n, t)) {
          return true;
        }
      }
    }
    return false;
  }
}
```

Algorithmus erkundet einen Pfad „bis ganz nach unten“, bevor er zurückkommt, um andere Pfade auszuprobieren.

- Viele mögliche Reihenfolgen: {1, 2, 5, 6, 9, 7, 8, 4, 3} oder {1, 4, 3, 2, 5, 8, 6, 7, 9} sind möglich

Wir bezeichnen diesen Ansatz als Tiefensuche



Rekursive Tiefensuche (Depth-First Search DFS)

```
Set<Vertex> visited; // assume global

dfs(Vertex s) {
  visited.add(s);

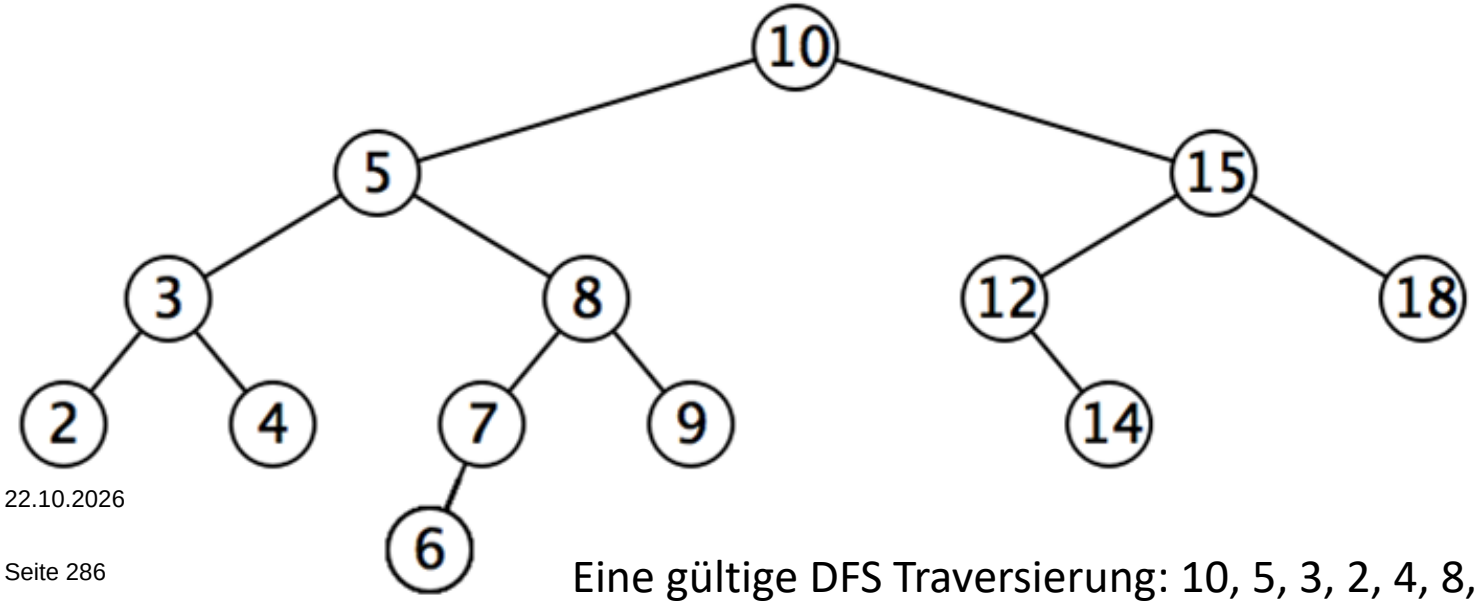
  for (Vertex n : s.neighbors) {
    if (!visited.contains(n)) {
      dfs(n)
    }
  }
}
```

Rekursive Tiefensuche bereinigt von Zielknoten

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Rekursive Tiefensuche (Depth-First Search DFS)

DFS: Man geht so weit wie möglich einen Pfad entlang, bis man in eine Sackgasse gerät (keine Nachbarn sind noch unentdeckt oder man hat keine Nachbarn). Wenn man auf eine Sackgasse stößt, geht man so lange zurück, bis man Kanten findet, die man noch nicht ausprobiert hat.



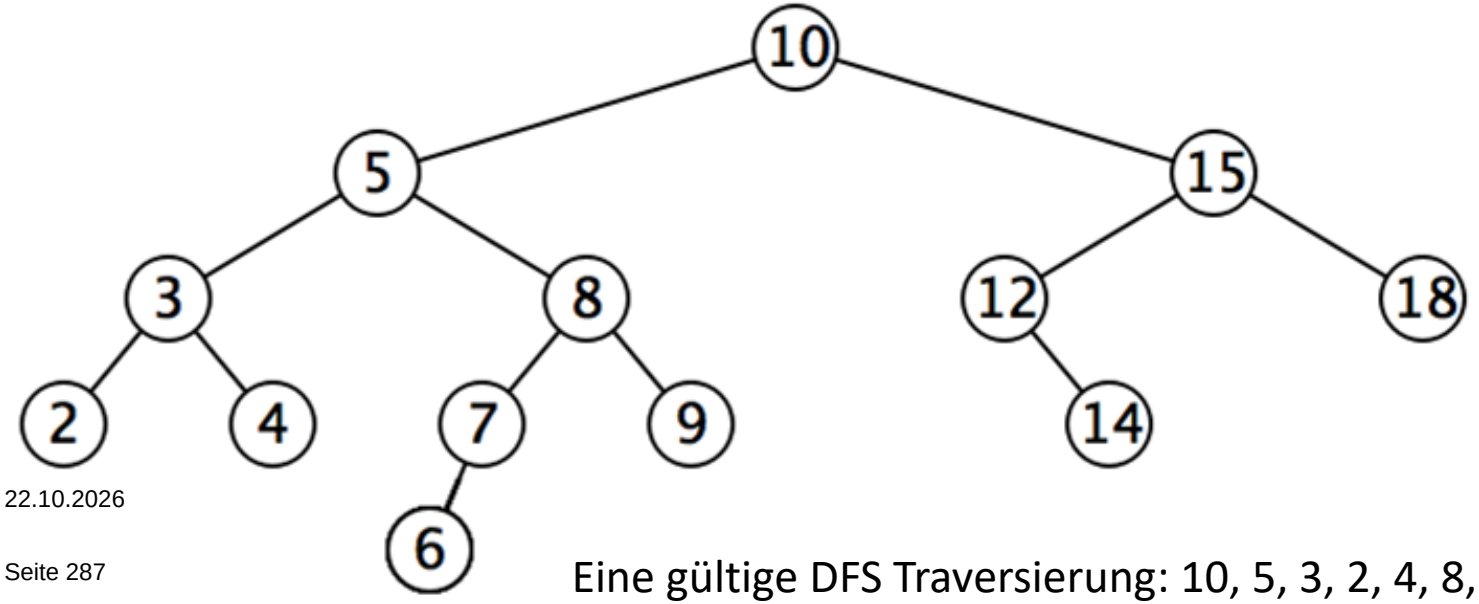
Eine gültige DFS Traversierung: 10, 5, 3, 2, 4, 8, 7, 6, 9, 15, 12, 14, 18

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Rekursive Tiefensuche (Depth-First Search DFS)

DFS: Man geht so weit wie möglich einen Pfad entlang, bis man in eine Sackgasse gerät (keine Nachbarn sind noch unentdeckt oder man hat keine Nachbarn). Wenn man auf eine Sackgasse stößt, geht man so lange zurück, bis man Kanten findet, die man noch nicht ausprobiert hat.

Es ist wie in einem Labyrinth - wenn Sie in einer Sackgasse stecken bleiben (denn Sie müssen es physisch ausprobieren, um zu wissen, dass es eine Sackgasse ist), verfolgen Sie Ihre Schritte zurück zu Ihrer letzten Entscheidung, und wenn Sie wieder dorthin gelangen, wählen Sie einen anderen Weg als zuvor.



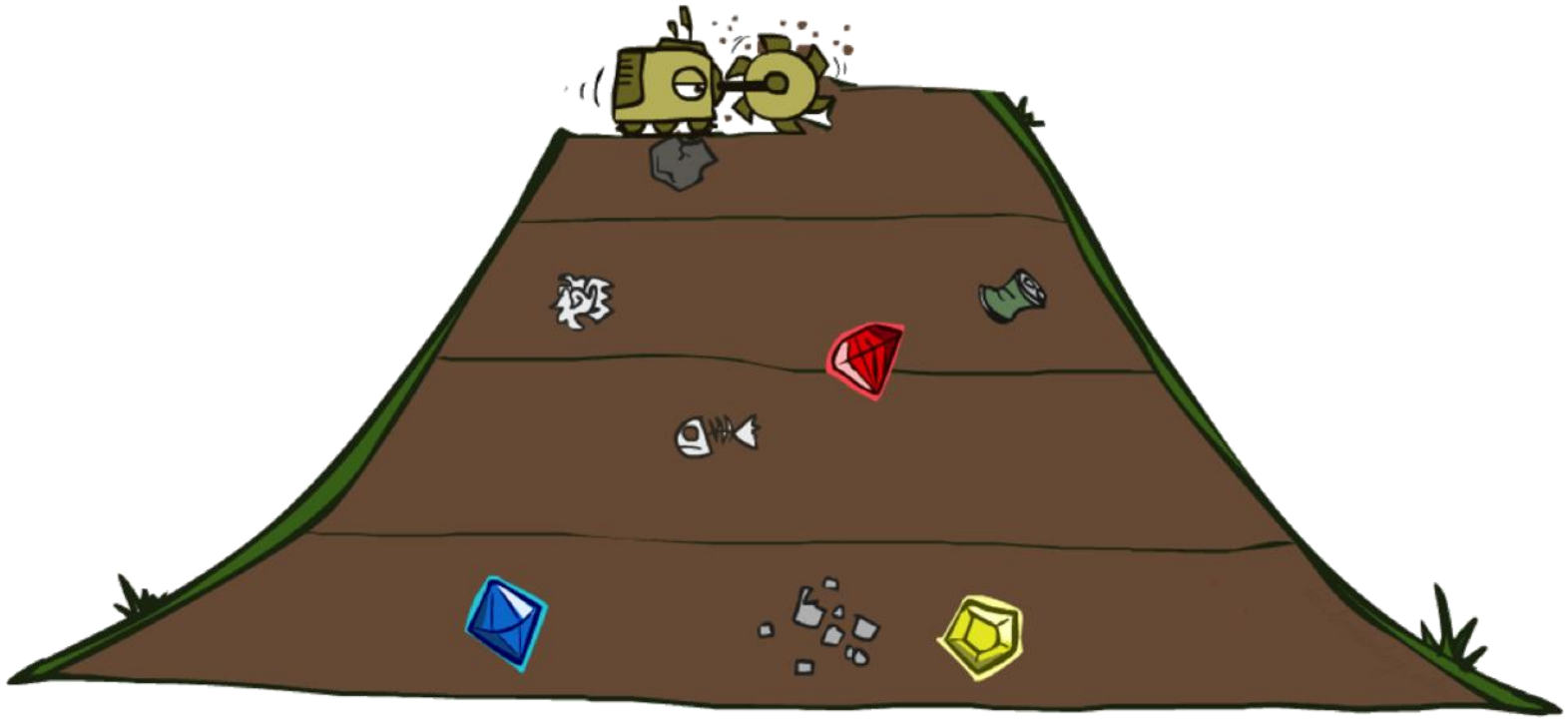
Eine gültige DFS Traversierung: 10, 5, 3, 2, 4, 8, 7, 6, 9, 15, 12, 14, 18

Rekursive Tiefensuche (Depth-First Search DFS)

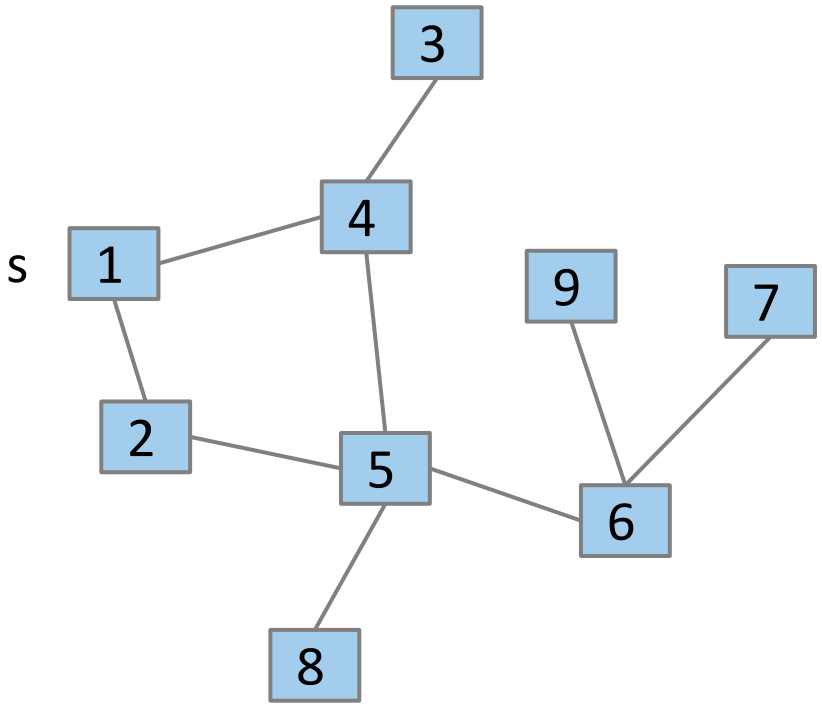


Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Artificial Intelligence (CS 188), University of California (Berkeley), Summer 2021

Breitensuche - Breadth-First Search (BFS)



Breitensuche - Breadth-First Search (BFS)

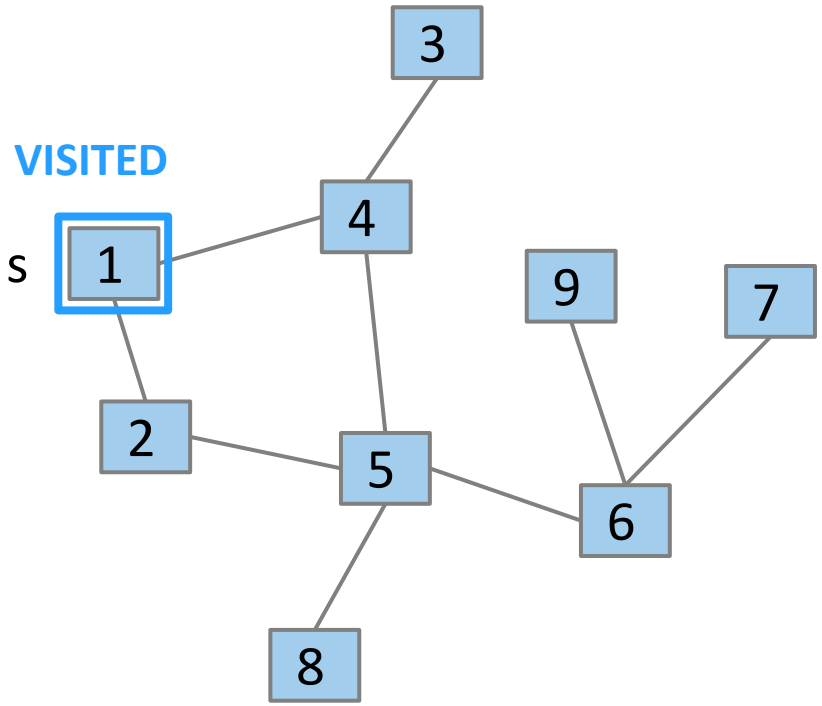


Angenommen, wir wollen zuerst die näheren Knoten besuchen, anstatt einem Pfad bis zum Ende zu folgen

- Genau wie bei der Traversierung eines Baums Ebene für Ebene, jetzt verallgemeinert auf jeden Graphen

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Breitensuche - Breadth-First Search (BFS)



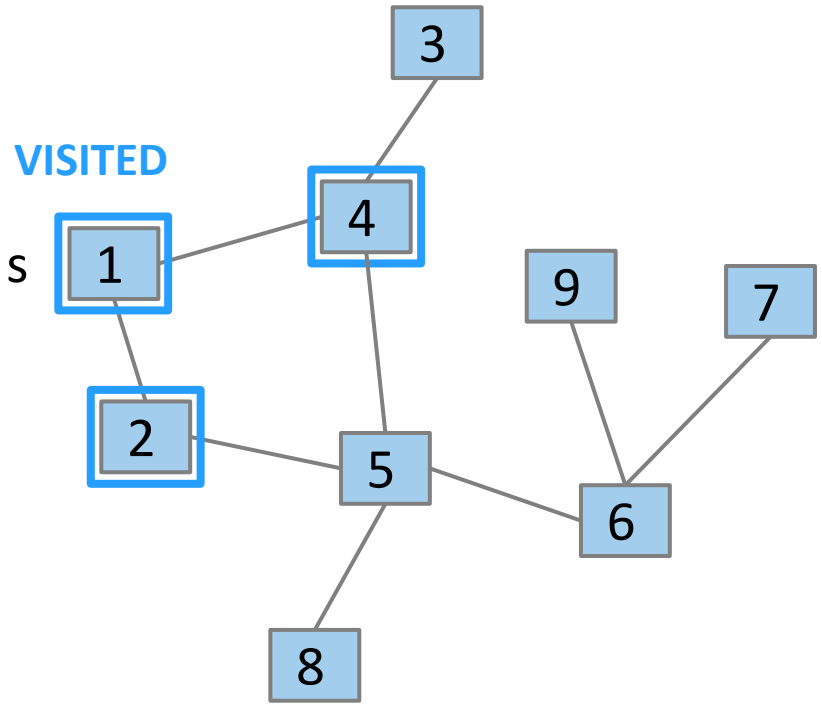
Angenommen, wir wollen zuerst die näheren Knoten besuchen, anstatt einem Pfad bis zum Ende zu folgen

- Genau wie bei der Traversierung eines Baums Ebene für Ebene, jetzt verallgemeinert auf jeden Graphen

-

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Breitensuche - Breadth-First Search (BFS)



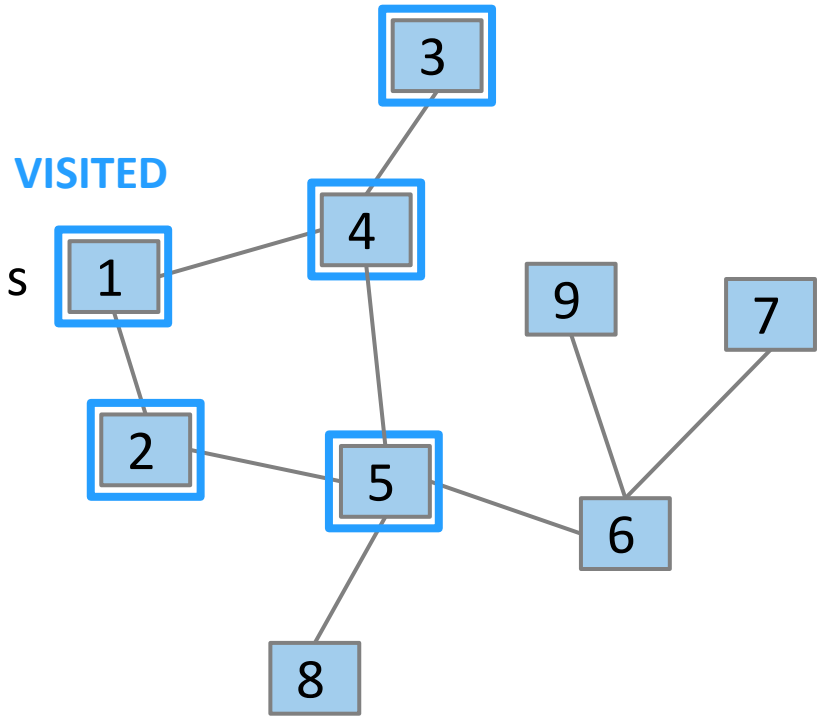
Angenommen, wir wollen zuerst die näheren Knoten besuchen, anstatt einem Pfad bis zum Ende zu folgen

- Genau wie bei der Traversierung eines Baums Ebene für Ebene, jetzt verallgemeinert auf jeden Graphen

-

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Breitensuche - Breadth-First Search (BFS)

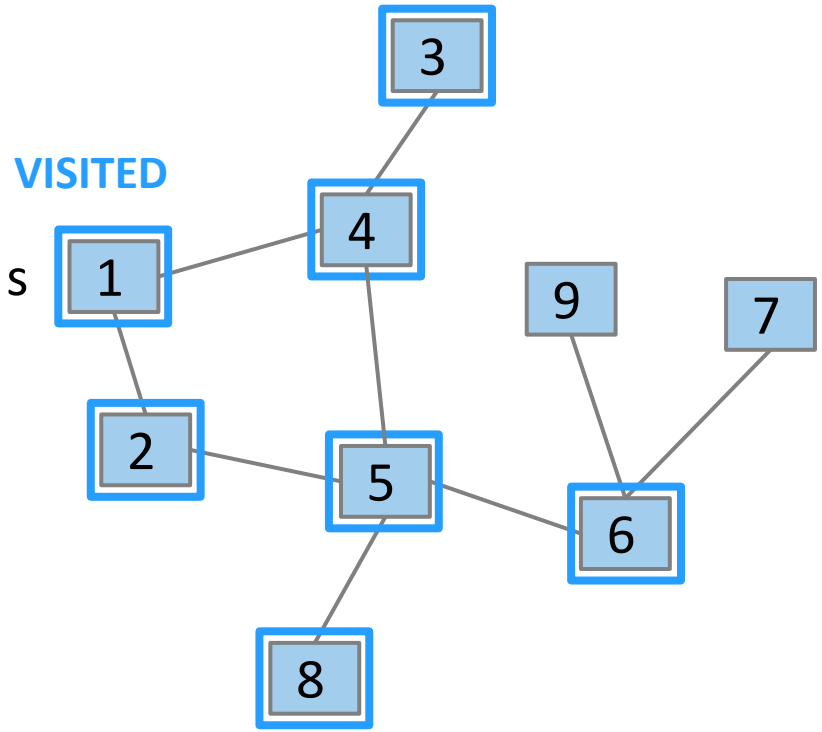


Angenommen, wir wollen zuerst die näheren Knoten besuchen, anstatt einem Pfad bis zum Ende zu folgen

- Genau wie bei der Traversierung eines Baums Ebene für Ebene, jetzt verallgemeinert auf jeden Graphen

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Breitensuche - Breadth-First Search (BFS)

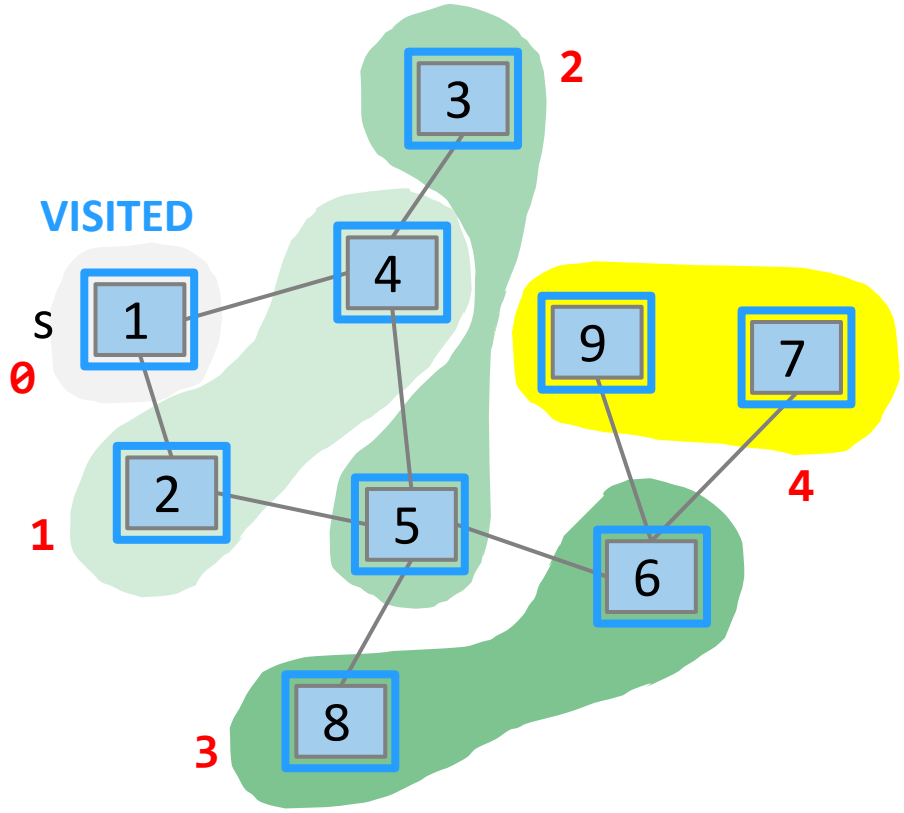


Angenommen, wir wollen zuerst die näheren Knoten besuchen, anstatt einem Pfad bis zum Ende zu folgen

- Genau wie bei der Traversierung eines Baums Ebene für Ebene, jetzt verallgemeinert auf jeden Graphen

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Breitensuche - Breadth-First Search (BFS)



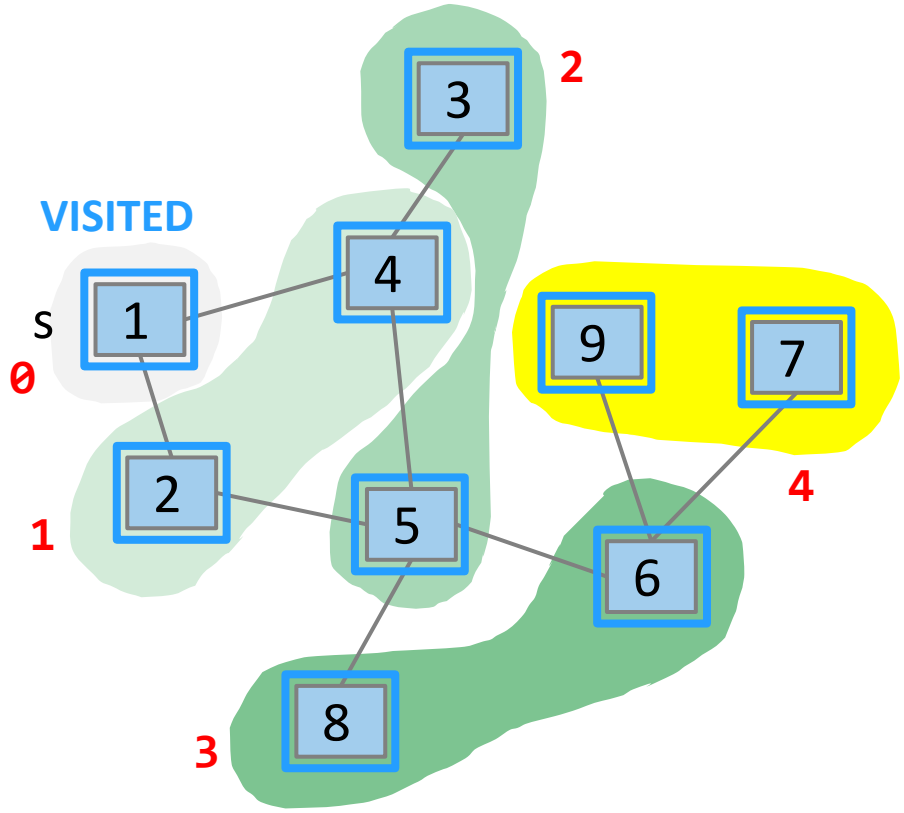
Angenommen, wir wollen zuerst die näheren Knoten besuchen, anstatt einem Pfad bis zum Ende zu folgen

- Genau wie bei der Traversierung eines Baums Ebene für Ebene, jetzt verallgemeinert auf jeden Graphen

-

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Breitensuche - Breadth-First Search (BFS)



Angenommen, wir wollen zuerst die näheren Knoten besuchen, anstatt einem Pfad bis zum Ende zu folgen

- Genau wie bei der Traversierung eines Baums Ebene für Ebene, jetzt verallgemeinert auf jeden Graphen

Wir bezeichnen diesen Ansatz als Breitensuche

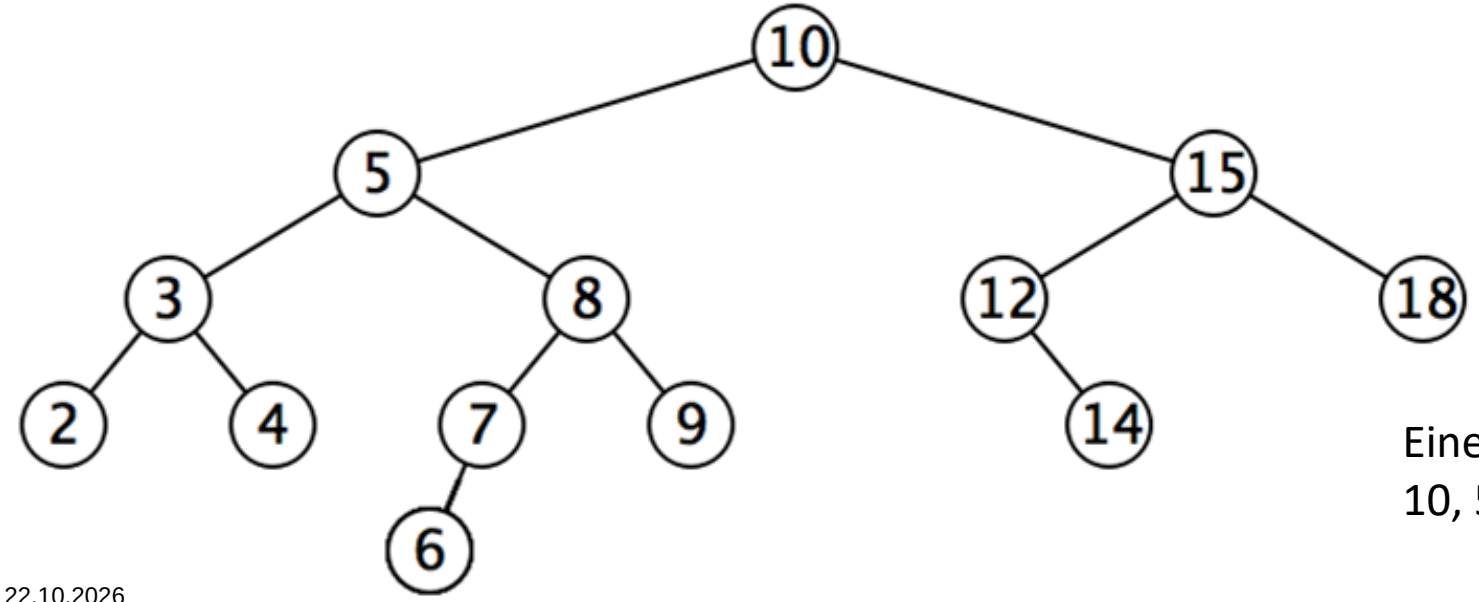
- Erkundung „Ebene für Ebene“

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Breitensuche - Breadth-First Search (BFS)

Intuitive Möglichkeiten, über BFS nachzudenken:

- entgegengesetzte Art der Ausbreitung im Vergleich zu DFS
- eine Schallwelle, die sich von einem Startpunkt aus in alle möglichen Richtungen ausbreitet.
-



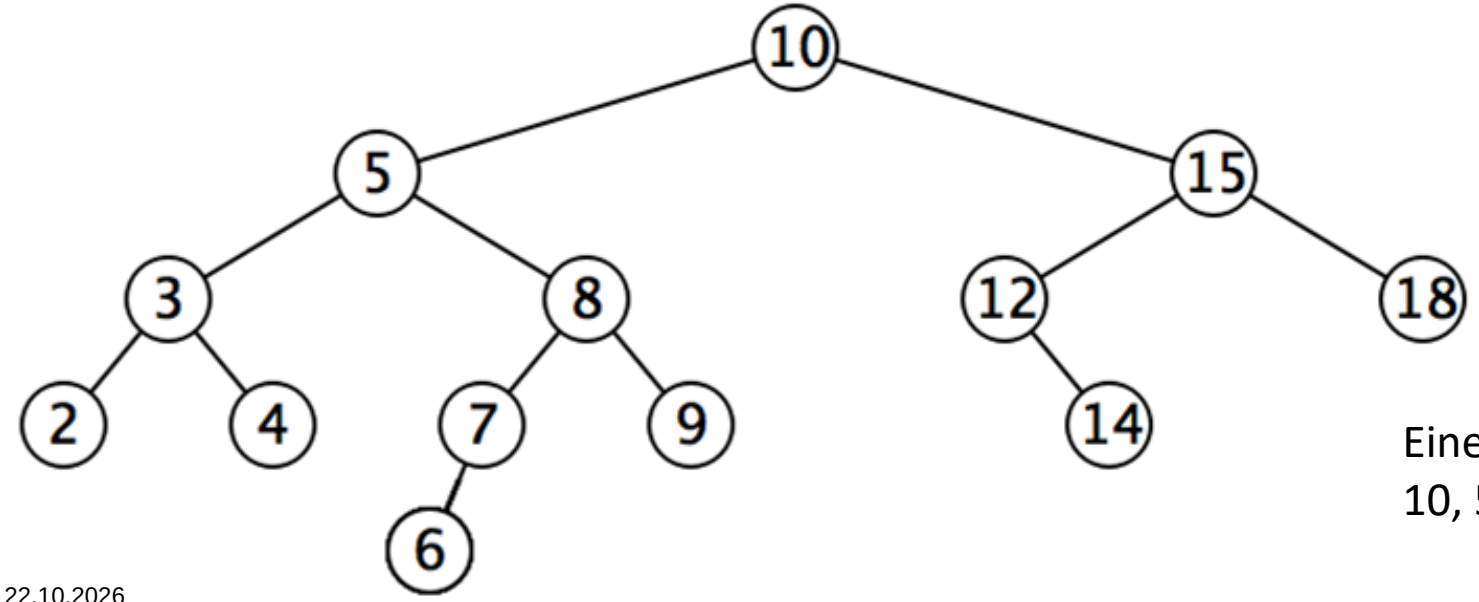
Eine gültige BFS Traversierung:
10, 5, 15, 3, 8, 12, 18, 2, 4, 7, 9, 14, 6

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Breitensuche - Breadth-First Search (BFS)

Intuitive Möglichkeiten, über BFS nachzudenken:

- entgegengesetzte Art der Ausbreitung im Vergleich zu DFS
- eine Schallwelle, die sich von einem Startpunkt aus in alle möglichen Richtungen ausbreitet.
- Bakterien auf einer Petrischale, die sich nach außen vermehren, so dass sie schließlich die gesamte Oberfläche bedecken



Eine gültige BFS Traversierung:
10, 5, 15, 3, 8, 12, 18, 2, 4, 7, 9, 14, 6

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Breitensuche - Breadth-First Search (BFS)

```
Set<Vertex> visited; // assume global

dfs(Vertex s) {
  visited.add(s);

  for (Vertex n : s.neighbors) {
    if (!visited.contains(n)) {
      dfs(n)
    }
  }
}
```

Das ist unser Ziel, aber wie lässt es sich in Code umsetzen?

- Wichtige Beobachtung: rekursive Aufrufe unterbrachen die s.neighbors-Schleife, um die Kinder sofort zu verarbeiten.
- Für BFS wollen wir stattdessen diese Schleife abschließen, bevor wir uns die Nachbarn genauer anschauen
-

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Breitensuche - Breadth-First Search (BFS)

```
Set<Vertex> visited; // assume global

dfs(Vertex s) {
  visited.add(s);

  for (Vertex n : s.neighbors) {
    if (!visited.contains(n)) {
      dfs(n)
    }
  }
}
```

Das ist unser Ziel, aber wie lässt es sich in Code umsetzen?

- Wichtige Beobachtung: rekursive Aufrufe unterbrachen die s.neighbors-Schleife, um die Kinder sofort zu verarbeiten.
- Für BFS wollen wir stattdessen diese Schleife abschließen, bevor wir uns die Nachbarn genauer anschauen
- Rekursion ist nicht die Lösung! Wir brauchen einen ADT, um die Nachbarn „in eine Warteschlange zu stellen“ ...

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Breitensuche Implementierung

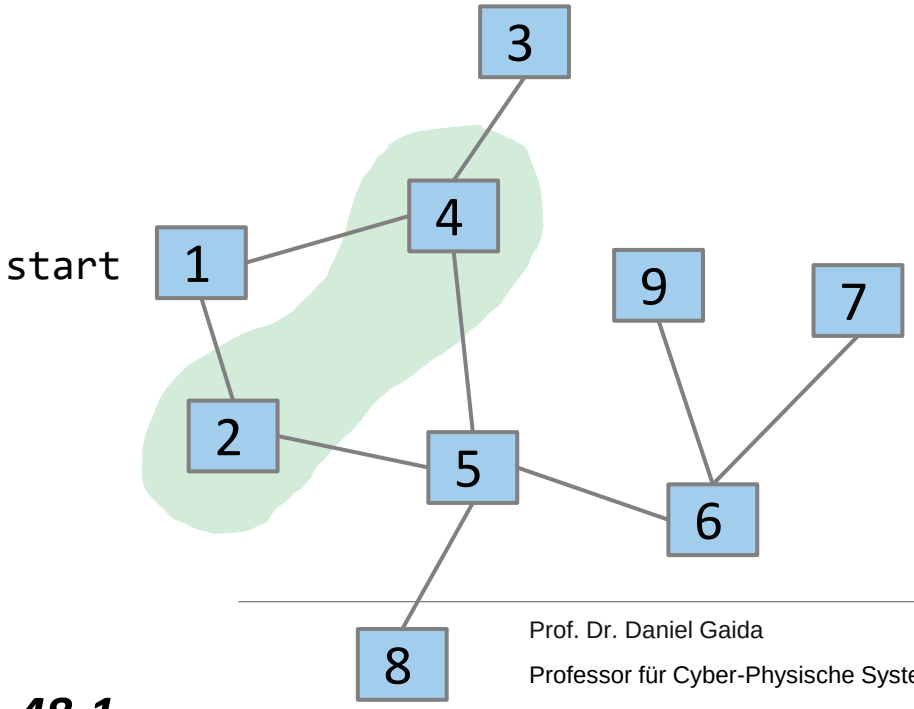
```
bfs(Graph graph, Vertex start) {
```

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Breitensuche Implementierung

Unsere zusätzliche Datenstruktur! Speichert den „äußeren Rand“ von Knoten, die noch erforscht werden müssen

```
bfs(Graph graph, Vertex start) {
    Queue<Vertex> perimeter = new Queue<>();
```



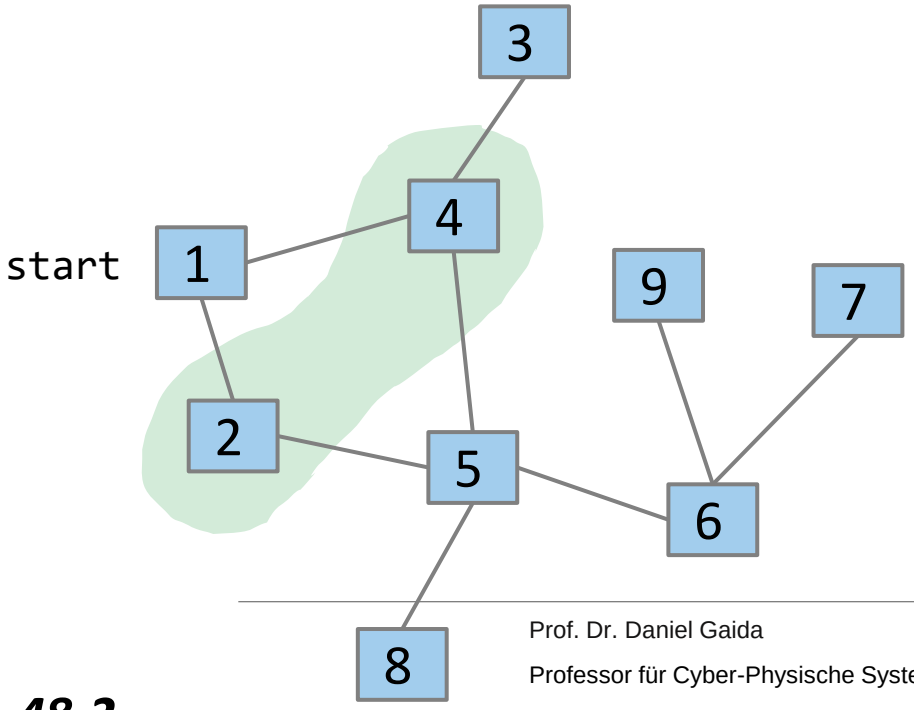
Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Breitensuche Implementierung

Unsere zusätzliche Datenstruktur! Speichert den „äußeren Rand“ von Knoten, die noch erforscht werden müssen

```
bfs(Graph graph, Vertex start) {
    Queue<Vertex> perimeter = new Queue<>();
    Set<Vertex> visited = new Set<>();
```



Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

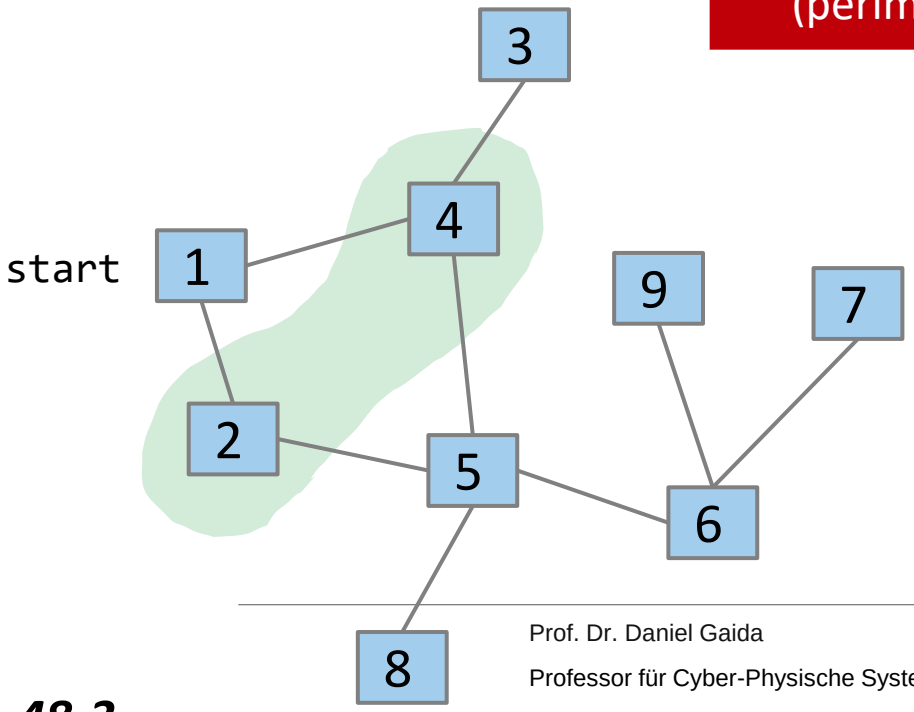
Breitensuche Implementierung

Unsere zusätzliche Datenstruktur! Speichert den „äußeren Rand“ von Knoten, die noch erforscht werden müssen

Starten des Algorithmus, indem wir start zum Umfang (perimeter) hinzufügen

```
bfs(Graph graph, Vertex start) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);
  visited.add(start);
```



Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

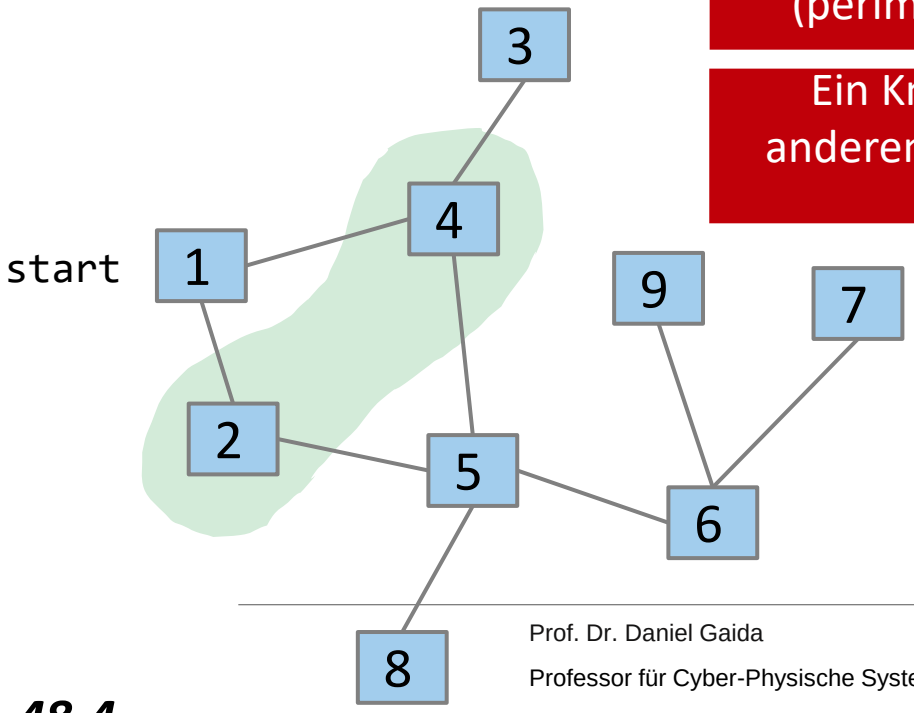
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Breitensuche Implementierung

Unsere zusätzliche Datenstruktur! Speichert den „äußeren Rand“ von Knoten, die noch erforscht werden müssen

Starten des Algorithmus, indem wir start zum Umfang (perimeter) hinzufügen

Ein Knoten nach dem anderen aus dem Umfang nehmen




```
bfs(Graph graph, Vertex start) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);
  visited.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
```

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

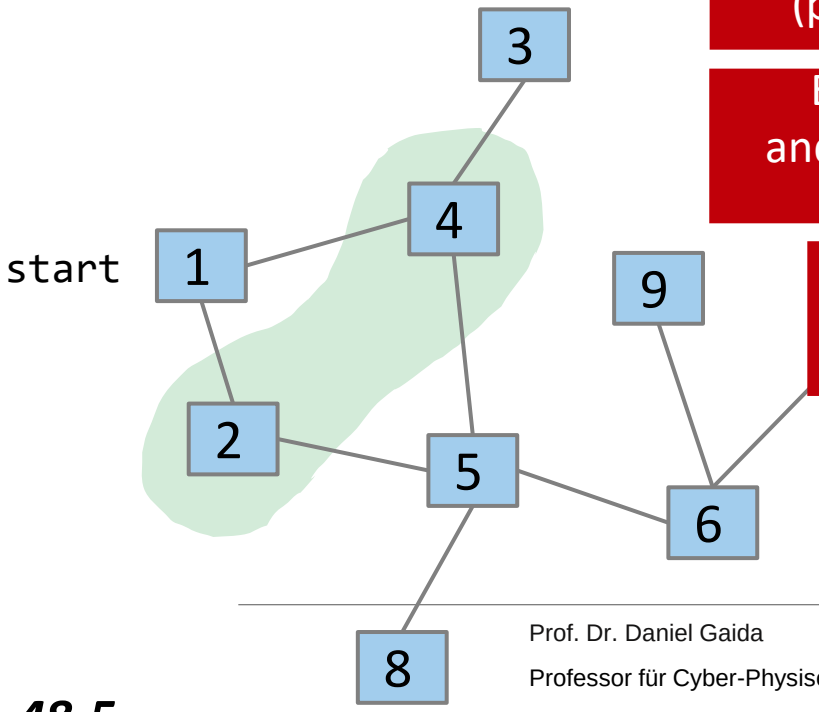
Breitensuche Implementierung

Unsere zusätzliche Datenstruktur! Speichert den „äußeren Rand“ von Knoten, die noch erforscht werden müssen

Starten des Algorithmus, indem wir start zum Umfang (perimeter) hinzufügen

Ein Knoten nach dem anderen aus dem Umfang nehmen

Alle Nachbarn dieses Knotens anschauen




```
bfs(Graph graph, Vertex start) {
   Queue<Vertex> perimeter = new Queue<>();
   Set<Vertex> visited = new Set<>();

  perimeter.add(start);
  visited.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
         Vertex to = edge.to();
```

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Breitensuche Implementierung

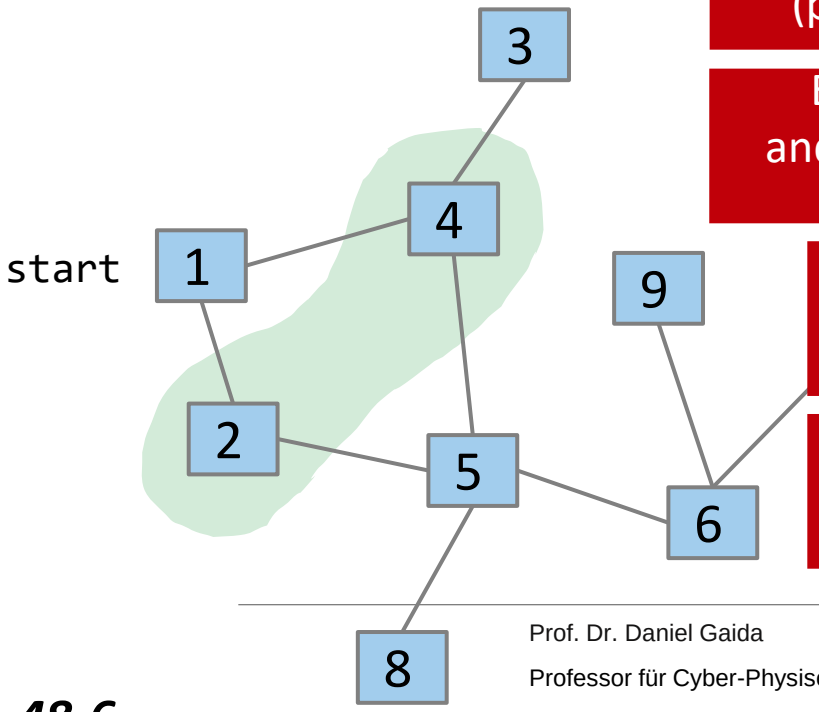
Unsere zusätzliche Datenstruktur! Speichert den „äußeren Rand“ von Knoten, die noch erforscht werden müssen

Starten des Algorithmus, indem wir start zum Umfang (perimeter) hinzufügen

Ein Knoten nach dem anderen aus dem Umfang nehmen

Alle Nachbarn dieses Knotens anschauen

Hinzufügen neuer Knoten zu perimeter



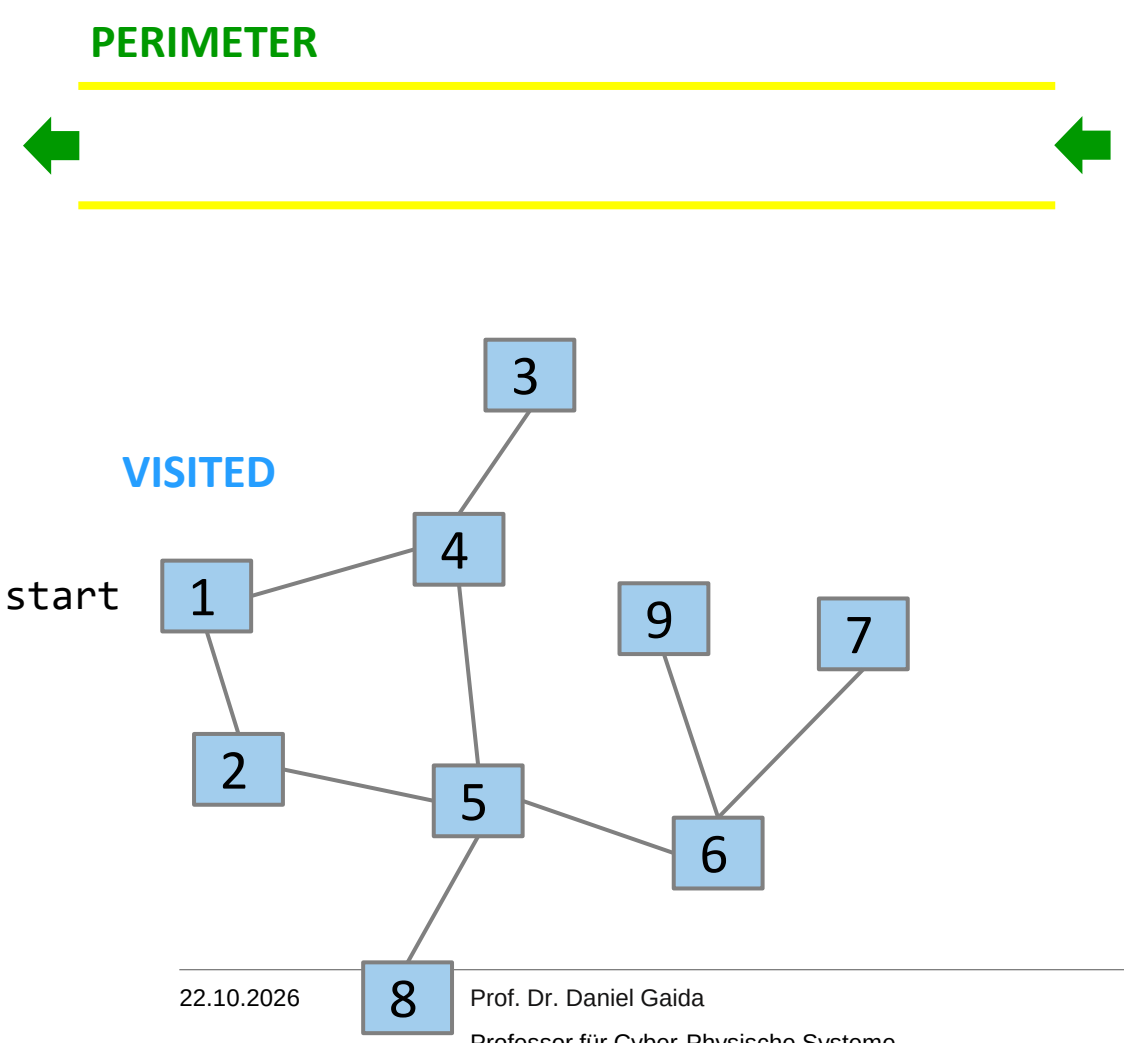

```
bfs(Graph graph, Vertex start) {
   Queue<Vertex> perimeter = new Queue<>();
   Set<Vertex> visited = new Set<>();

  perimeter.add(start);
  visited.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
         Vertex to = edge.to();
         if (!visited.contains(to)) {
           perimeter.add(to);
           visited.add(to);
         }
       }
     }
   }
```

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Breitensuche Implementierung: In Aktion



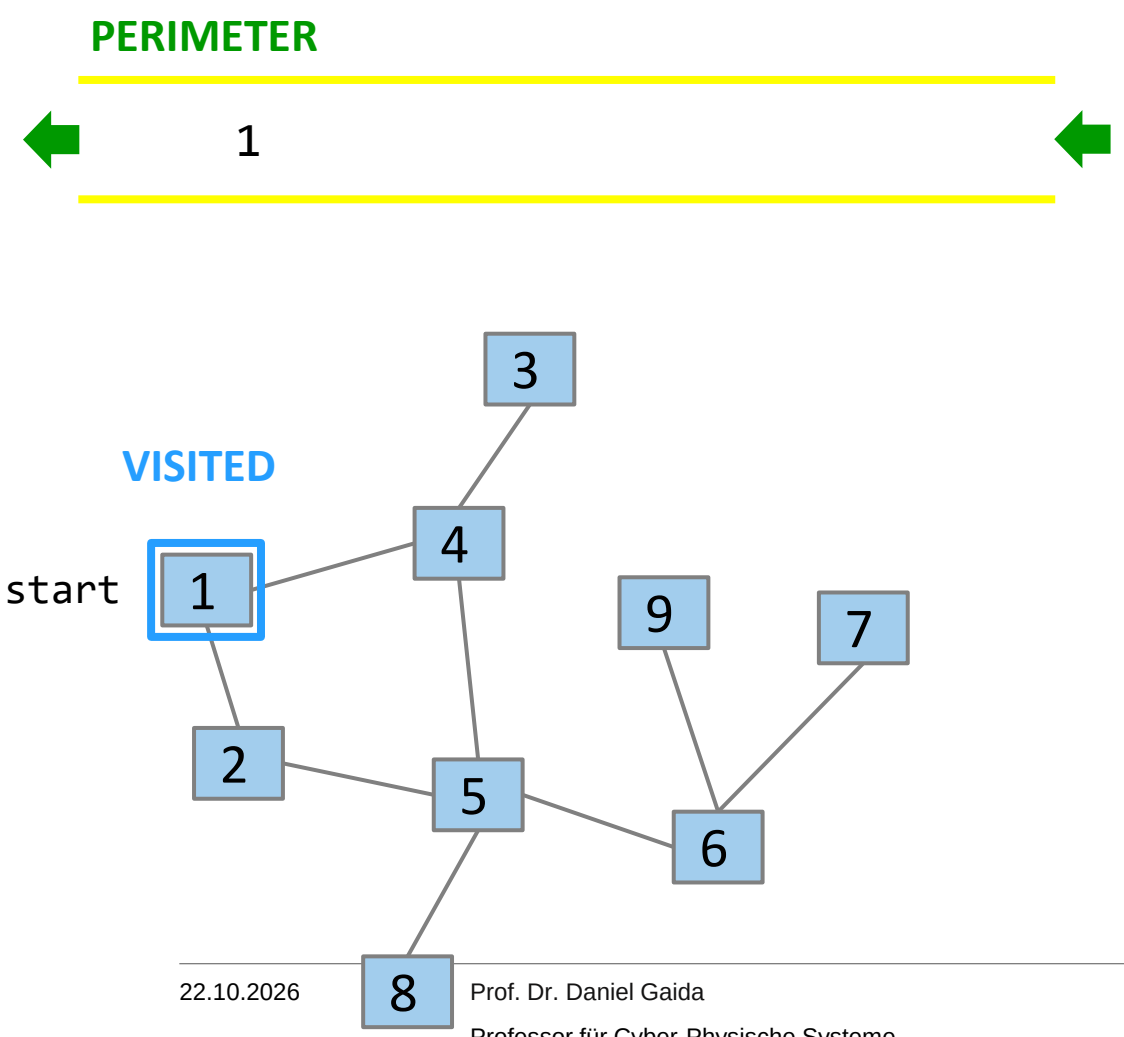

```
bfs(Graph graph, Vertex start) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);
  visited.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        perimeter.add(to);
        visited.add(to);
      }
    }
  }
}
```

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Breitensuche Implementierung: In Aktion



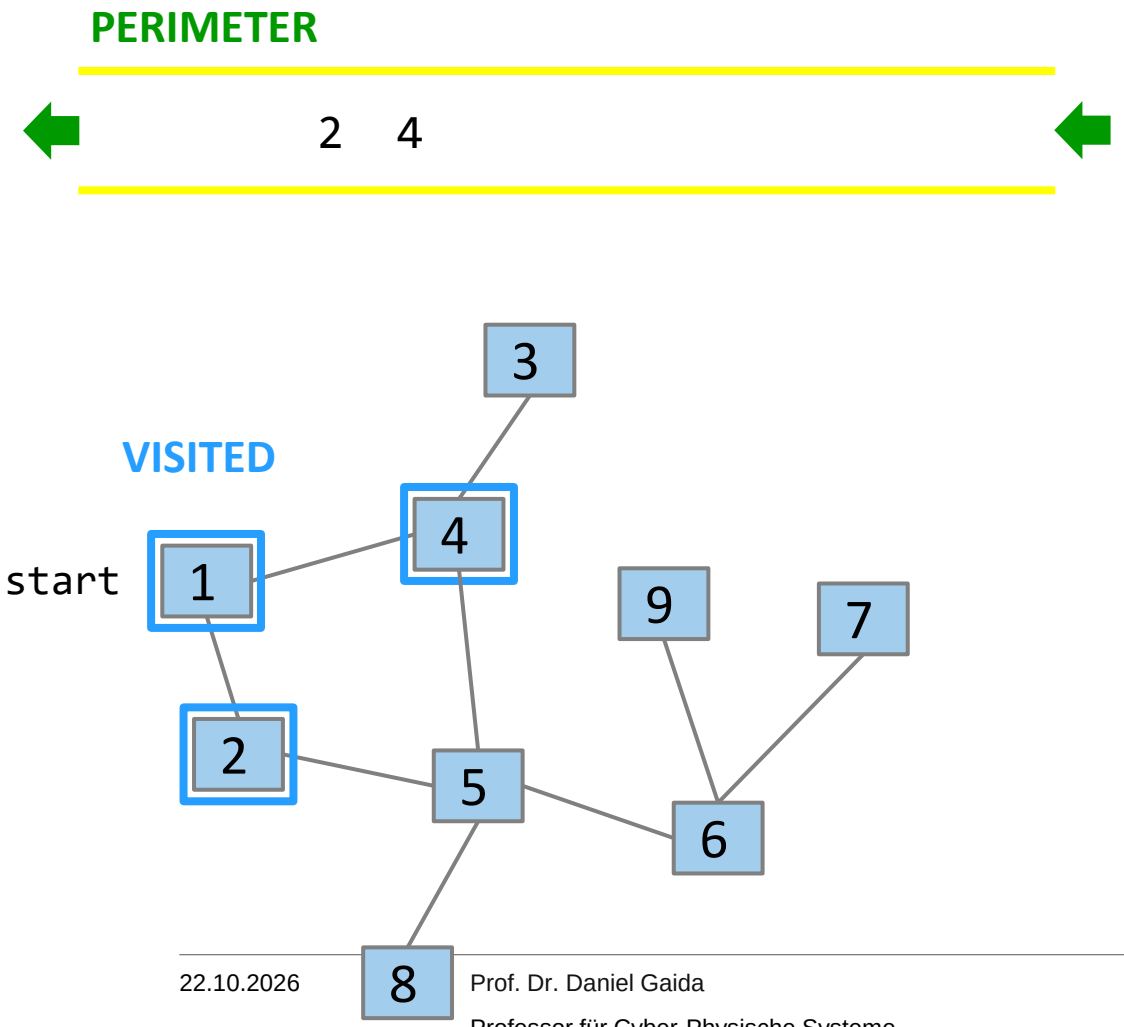

```
bfs(Graph graph, Vertex start) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);
  visited.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        perimeter.add(to);
        visited.add(to);
      }
    }
  }
}
```

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Breitensuche Implementierung: In Aktion



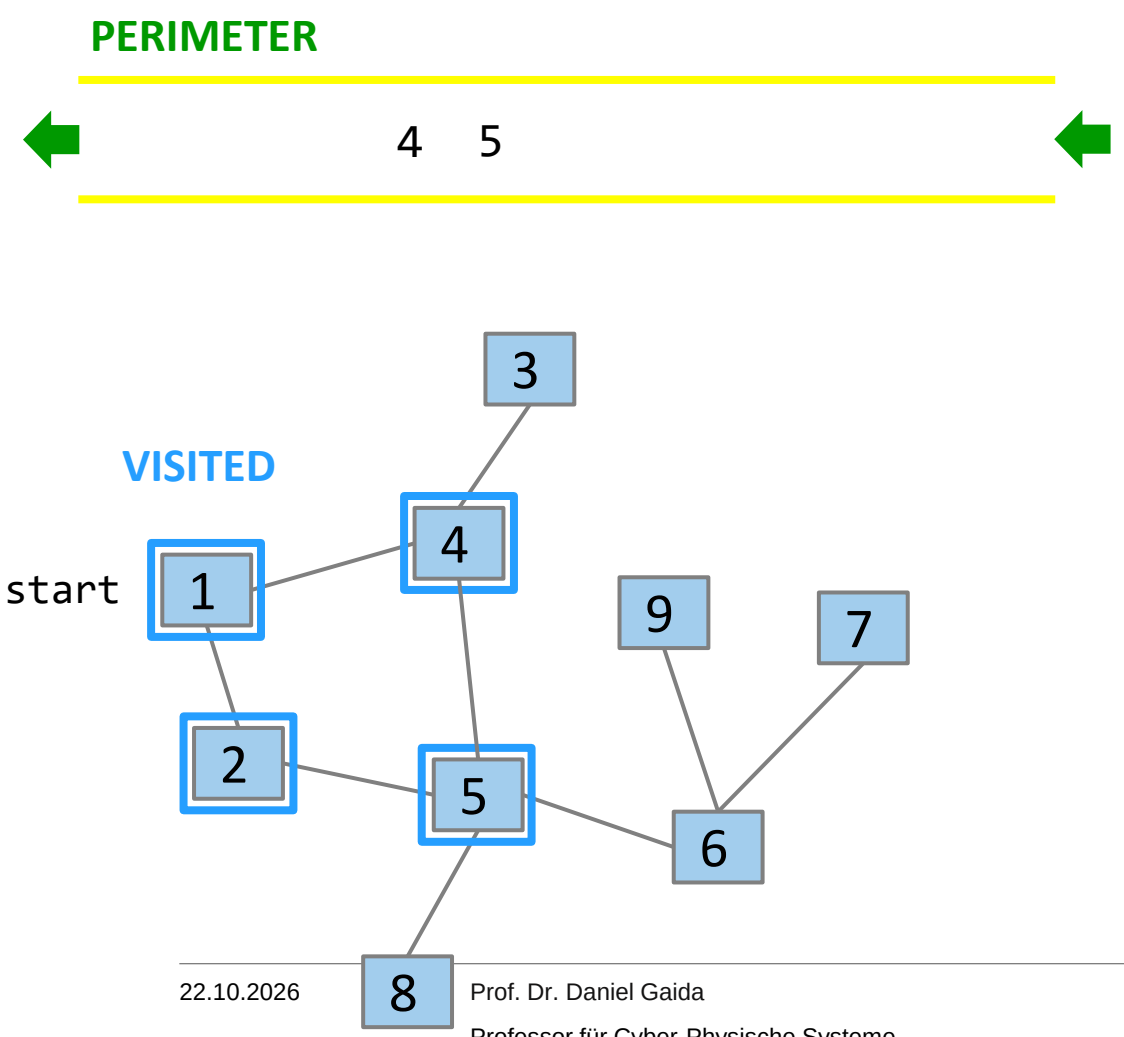

```
bfs(Graph graph, Vertex start) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);
  visited.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        perimeter.add(to);
        visited.add(to);
      }
    }
  }
}
```

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Breitensuche Implementierung: In Aktion



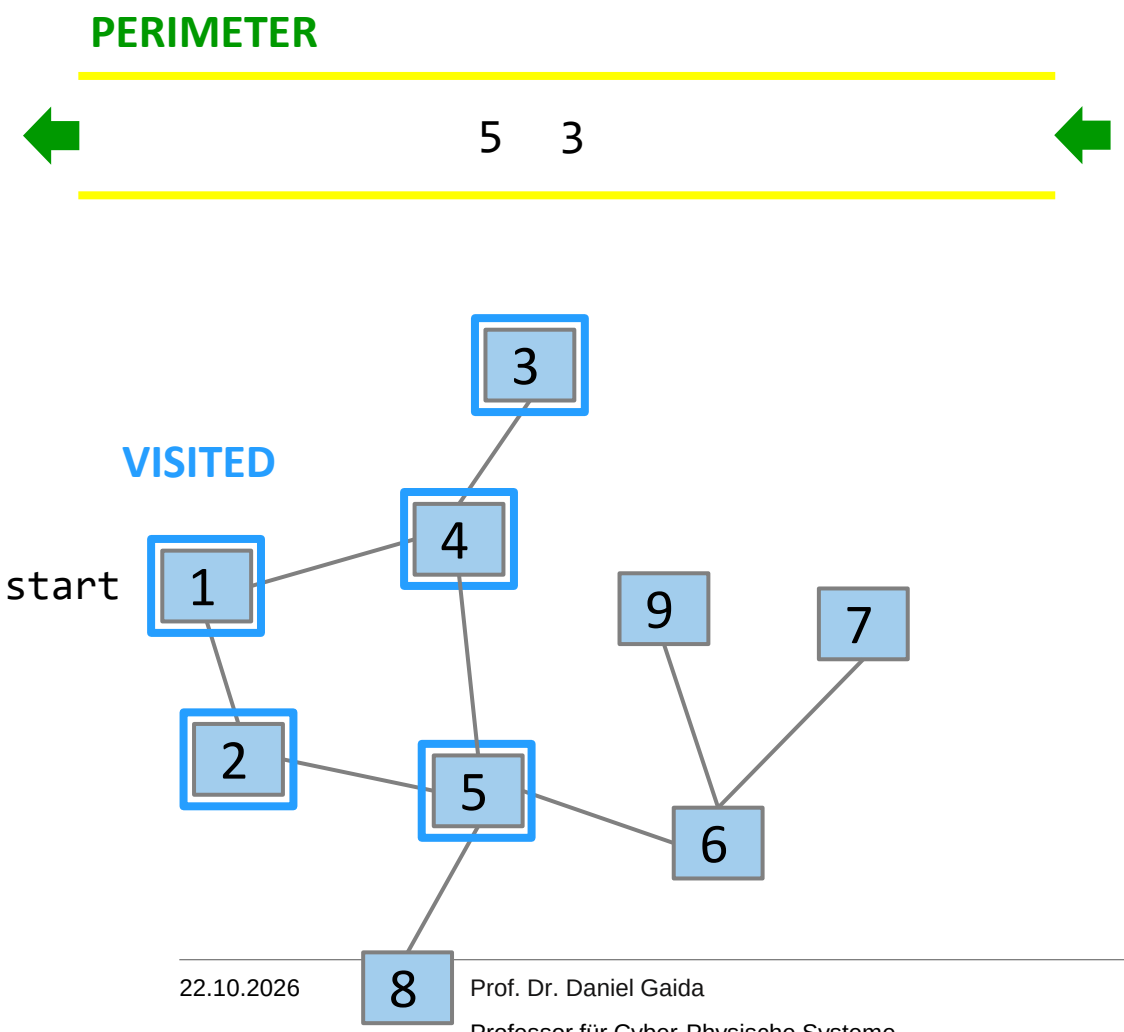

```
bfs(Graph graph, Vertex start) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);
  visited.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        perimeter.add(to);
        visited.add(to);
      }
    }
  }
}
```

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Breitensuche Implementierung: In Aktion



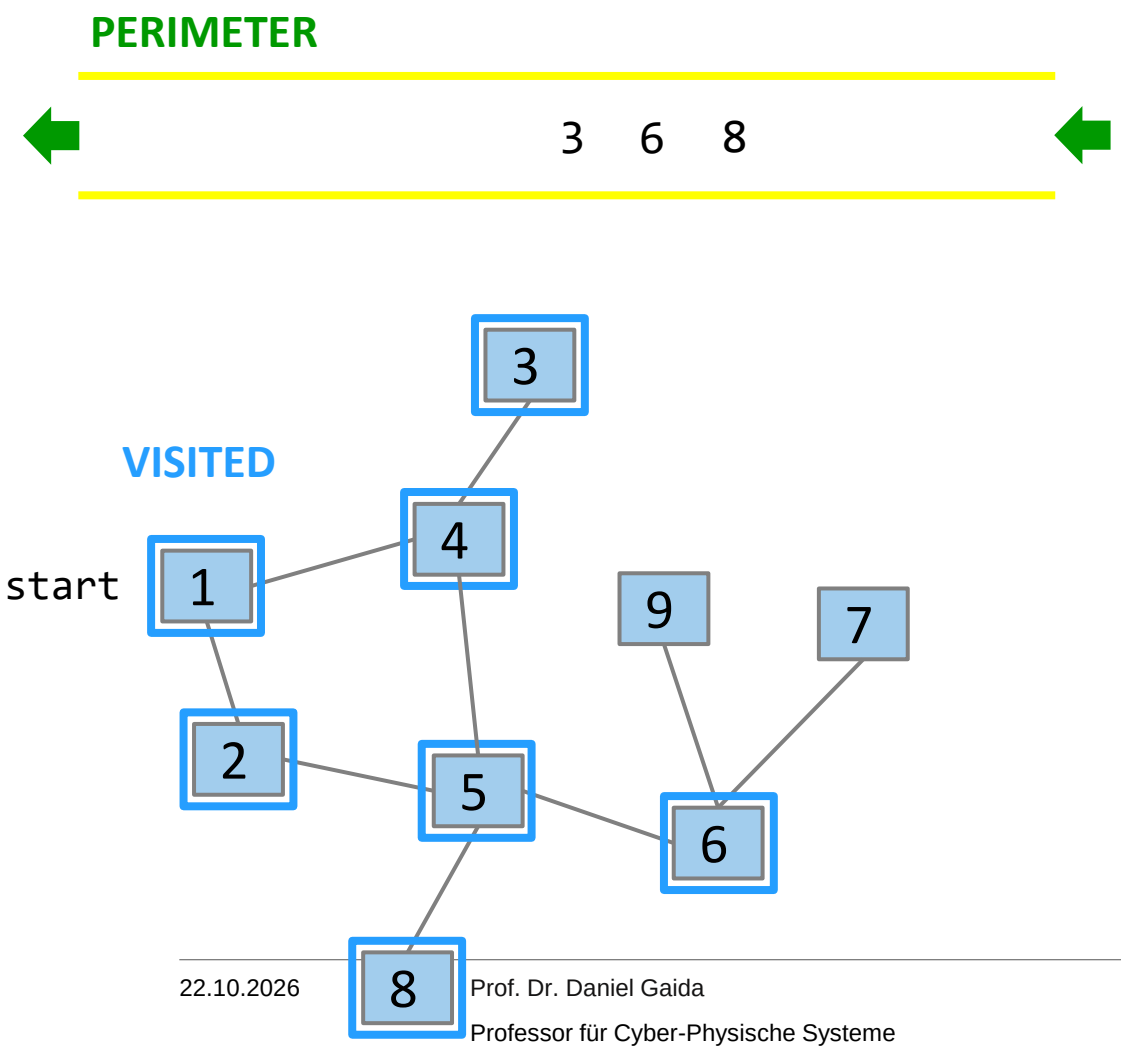

```
bfs(Graph graph, Vertex start) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);
  visited.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        perimeter.add(to);
        visited.add(to);
      }
    }
  }
}
```

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Breitensuche Implementierung: In Aktion



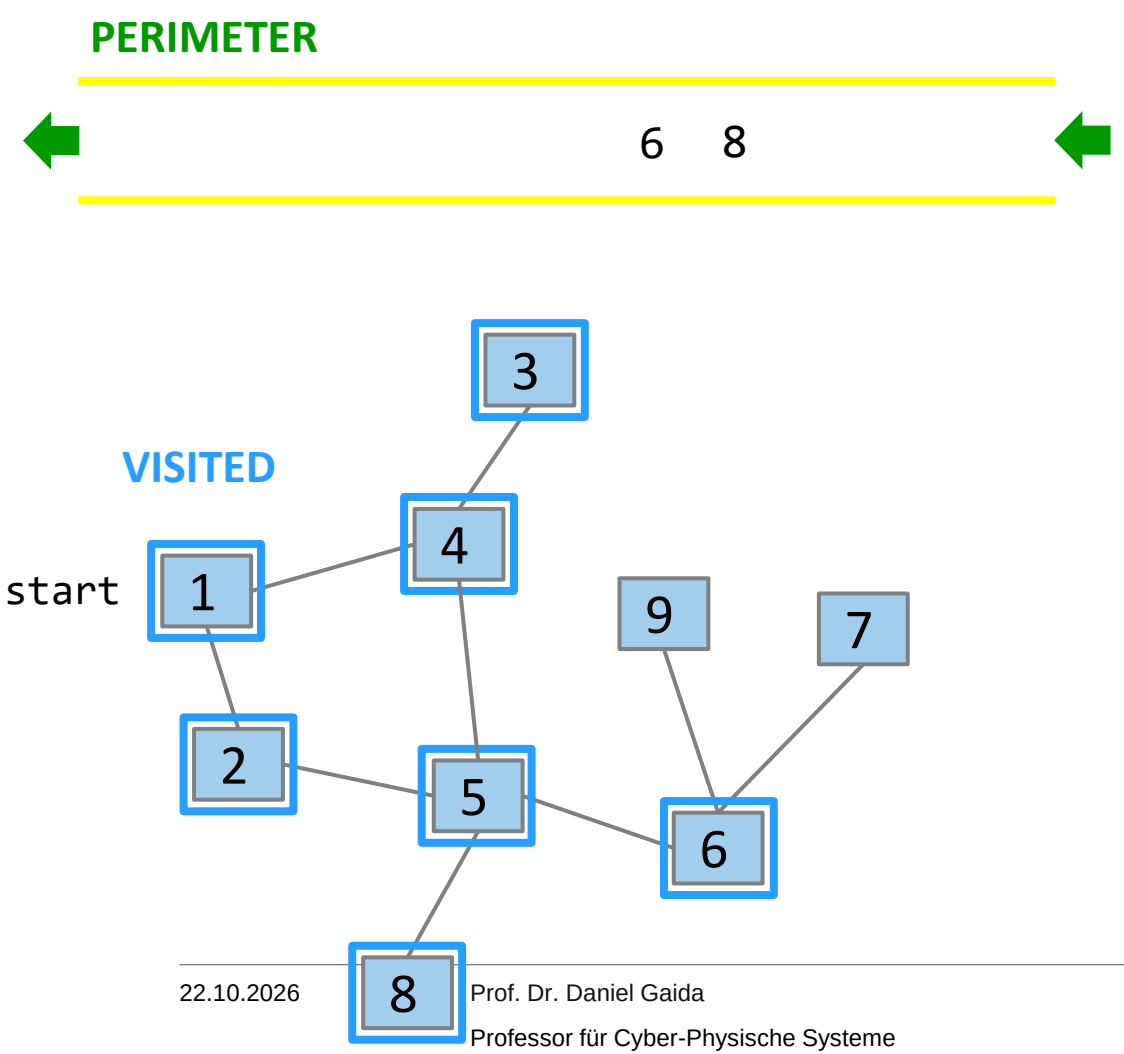

```
bfs(Graph graph, Vertex start) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);
  visited.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        perimeter.add(to);
        visited.add(to);
      }
    }
  }
}
```

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Breitensuche Implementierung: In Aktion



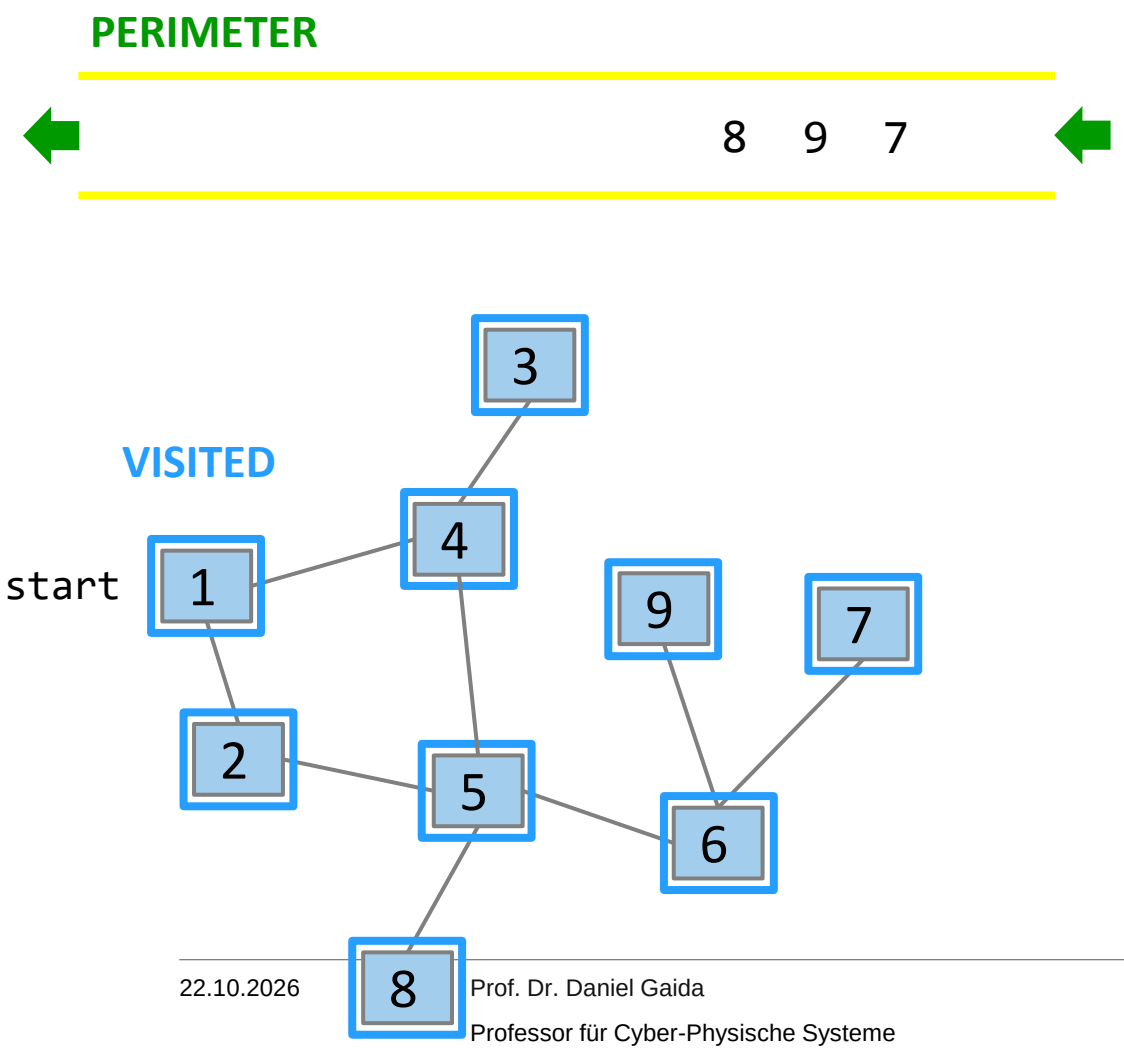

```
bfs(Graph graph, Vertex start) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);
  visited.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        perimeter.add(to);
        visited.add(to);
      }
    }
  }
}
```

Quelle: Data Structures and Algorithms (CSE 373),
University of Washington, Spring 2022

Breitensuche Implementierung: In Aktion



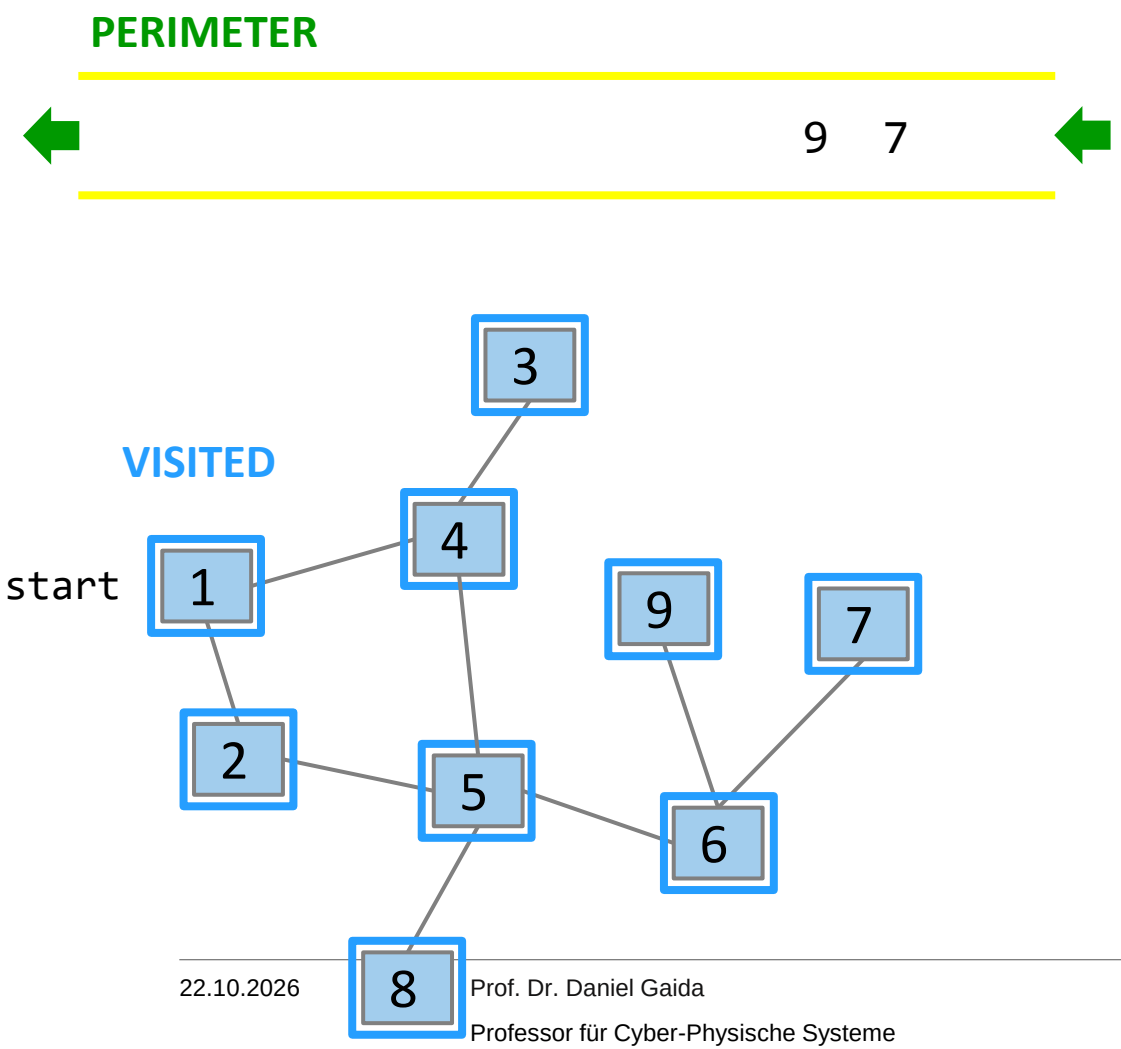

```
bfs(Graph graph, Vertex start) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);
  visited.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        perimeter.add(to);
        visited.add(to);
      }
    }
  }
}
```

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Breitensuche Implementierung: In Aktion



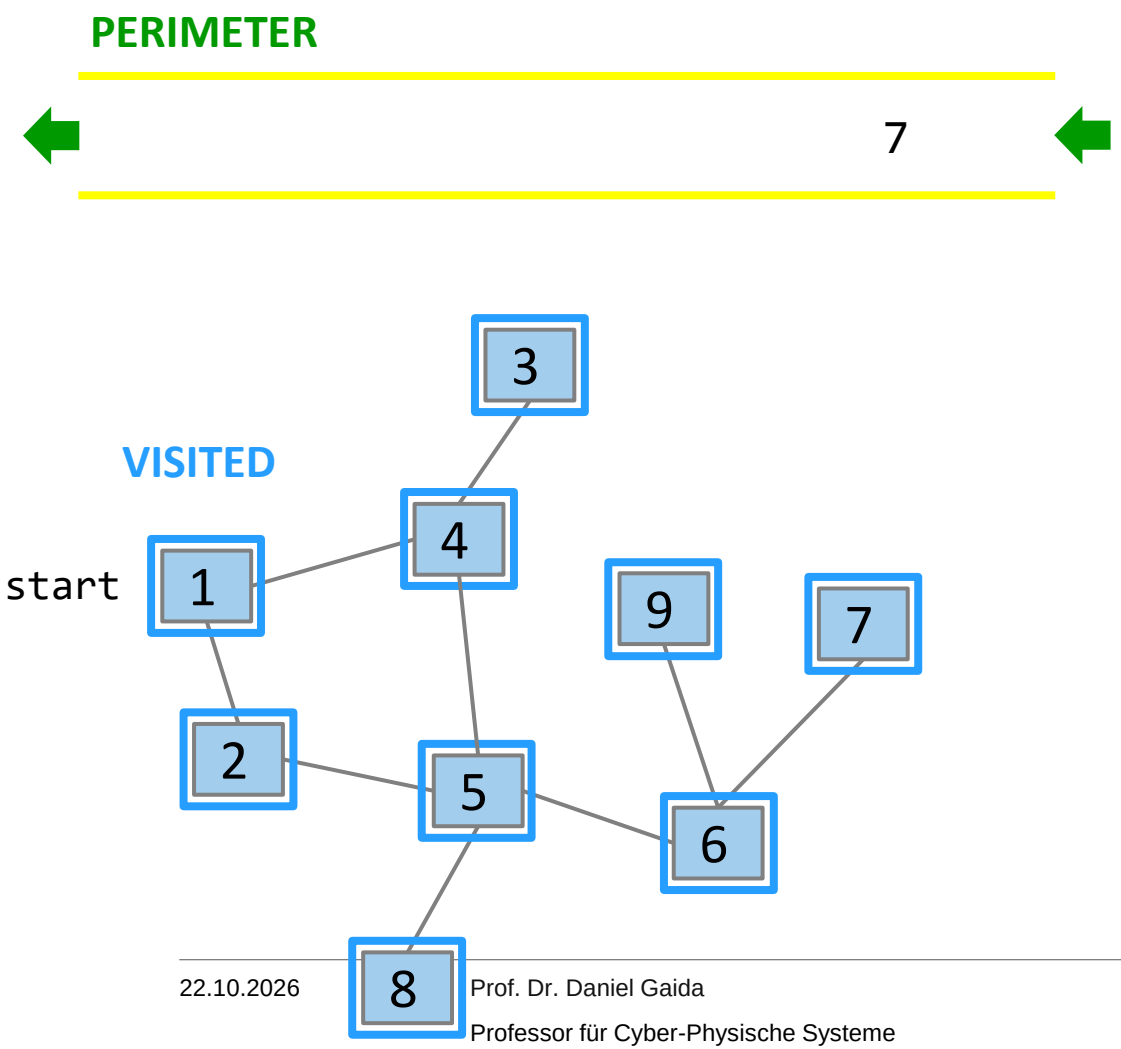

```
bfs(Graph graph, Vertex start) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);
  visited.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        perimeter.add(to);
        visited.add(to);
      }
    }
  }
}
```

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Breitensuche Implementierung: In Aktion



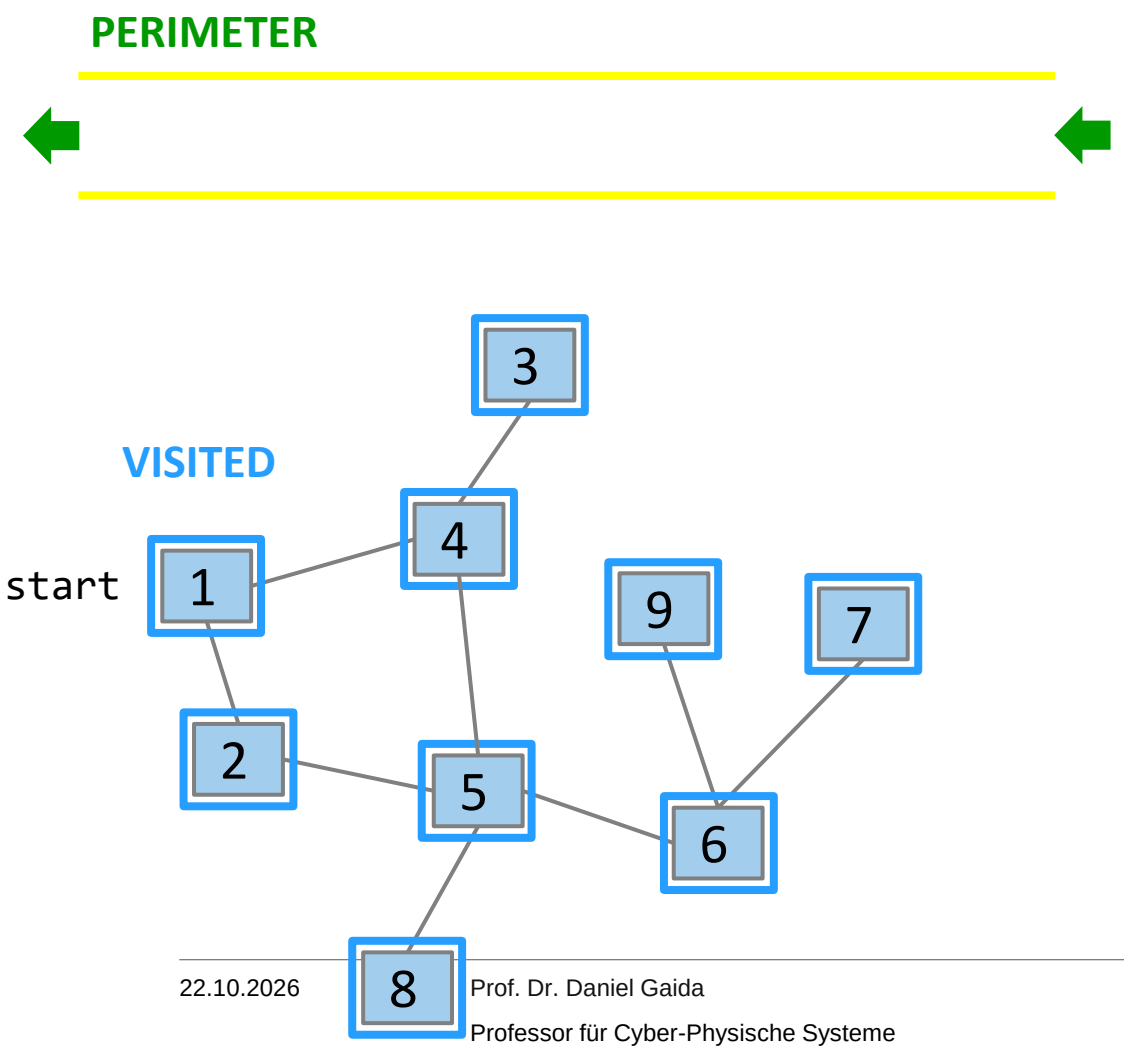

```
bfs(Graph graph, Vertex start) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);
  visited.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        perimeter.add(to);
        visited.add(to);
      }
    }
  }
}
```

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Breitensuche Implementierung: In Aktion



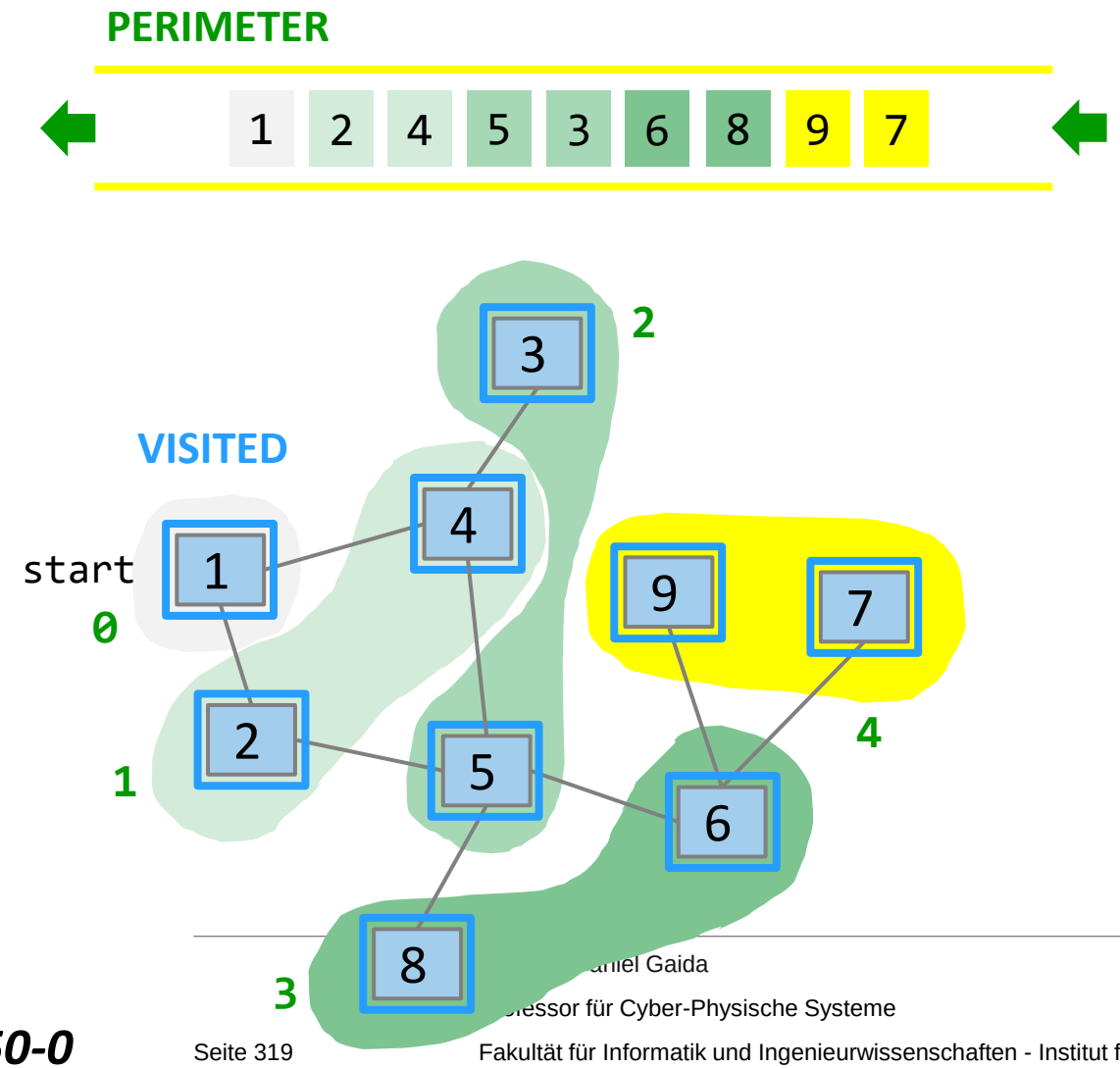

```
bfs(Graph graph, Vertex start) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);
  visited.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        perimeter.add(to);
        visited.add(to);
      }
    }
  }
}
```

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Breitensuche: Warum funktioniert sie?



- Die Eigenschaften einer Warteschlange sind genau das, was uns dieses Verhalten ermöglicht
- Solange wir eine ganze Ebene erforschen, bevor wir weitergehen (und das werden wir, mit einer Warteschlange), wird die nächste Ebene vollständig aufgebaut sein und auf uns warten, wenn wir fertig sind!
- Weitermachen, bis der Perimeter leer ist

```
while (!perimeter.isEmpty()) {
  Vertex from = perimeter.remove();
  for (Edge edge : graph.edgesFrom(from)) {
    Vertex to = edge.to();
    if (!visited.contains(to)) {
      perimeter.add(to);
      visited.add(to);
    }
  }
}
```

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Breitensuche und Tiefensuche: Iterativ

```
bfs(Graph graph, Vertex start) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);
  visited.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        perimeter.add(to);
        visited.add(to);
      }
    }
  }
}
```



Prof. Dr. Daniel Gaida

Professor für Cyber-Physische Systeme

Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Breitensuche und Tiefensuche: Iterativ

```
dfs(Graph graph, Vertex start) {
  Stack<Vertex> perimeter = new Stack<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    if (!visited.contains(from)) {
      for (Edge edge:graph.edgesFrom(from)) {
        Vertex to = edge.to();
        if (!visited.contains(to))
          perimeter.add(to)
      }
      visited.add(from);
    }
  }
}
```



```
bfs(Graph graph, Vertex start) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);
  visited.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        perimeter.add(to);
        visited.add(to);
      }
    }
  }
}
```



Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Breitensuche und Tiefensuche: Iterativ

```
dfs(Graph graph, Vertex start) {
  Stack<Vertex> perimeter = new Stack<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    if (!visited.contains(from)) {
      for (Edge edge:graph.edgesFrom(from)) {
        Vertex to = edge.to();
        if (!visited.contains(to))
          perimeter.add(to)
      }
      visited.add(from);
    }
  }
}
```

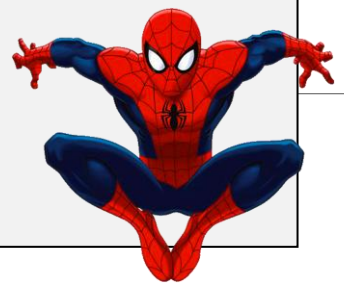
Ändern der Warteschlange für die Bearbeitungs-reihenfolge von Nachbarn in einen Stapel



```
bfs(Graph graph, Vertex start) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);
  visited.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        perimeter.add(to);
        visited.add(to);
      }
    }
  }
}
```



Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Breitensuche und Tiefensuche: Iterativ

```
dfs(Graph graph, Vertex start) {
  Stack<Vertex> perimeter = new Stack<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    if (!visited.contains(from)) {
      for (Edge edge:graph.edgesFrom(from)) {
        Vertex to = edge.to();
        if (!visited.contains(to))
          perimeter.add(to)
      }
      visited.add(from);
    }
  }
}
```

Ändern der Warteschlange für die Bearbeitungs-reihenfolge von Nachbarn in einen Stapel

```
bfs(Graph graph, Vertex start) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();

  ...add(start);
  ...(start);

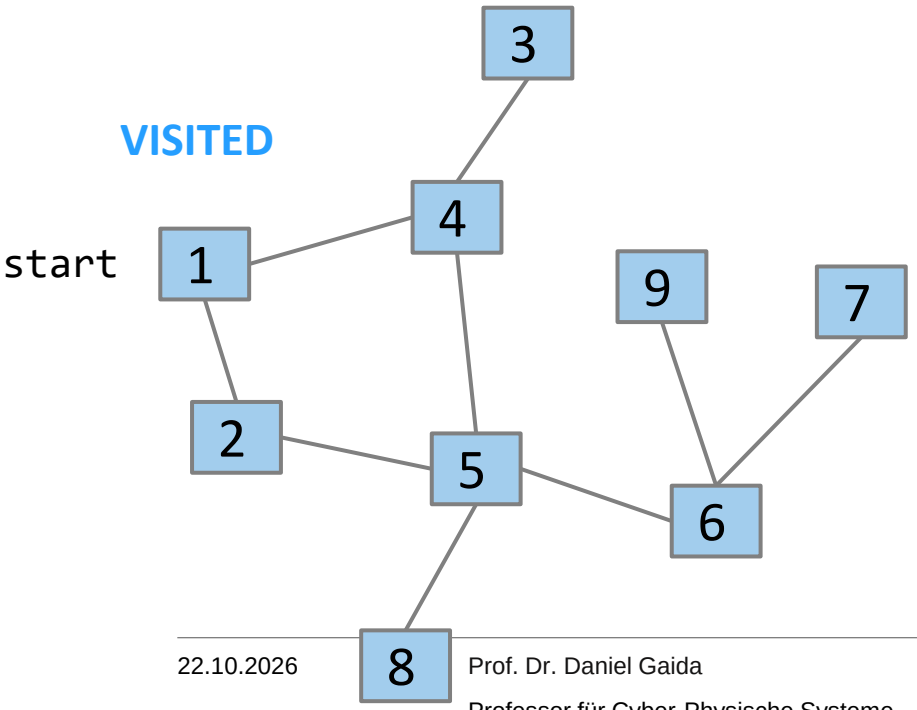
  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        perimeter.add(to);
        visited.add(to);
      }
```

Im DFS können wir einen Knoten nicht sofort als „besucht“ hinzufügen. Wir fügen ihn erst hinzu, wenn seine Nachbarn im Perimeter sind.



Tiefensuche Implementierung: In Aktion

PERIMETER




```
dfs(Graph graph, Vertex start) {
  Stack<Vertex> perimeter = new Stack<>();
  Set<Vertex> visited = new Set<>();

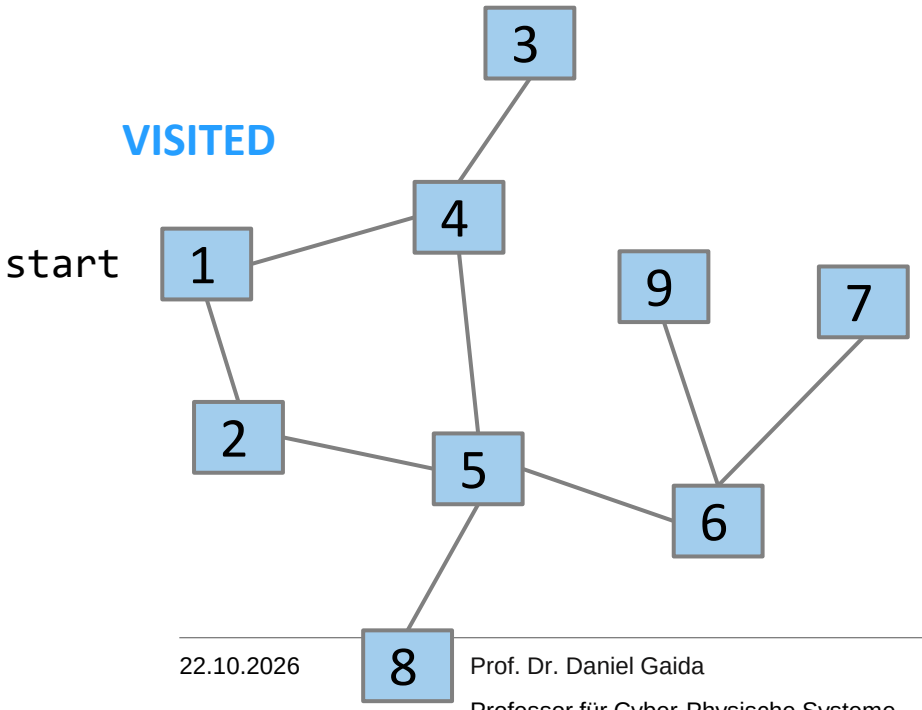
  perimeter.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    if (!visited.contains(from)) {
      for (Edge edge:graph.edgesFrom(from)) {
        Vertex to = edge.to();
        if (!visited.contains(to))
          perimeter.add(to)
      }
      visited.add(from);
    }
  }
}
```

Tiefensuche Implementierung: In Aktion

PERIMETER

1




```
dfs(Graph graph, Vertex start) {
  Stack<Vertex> perimeter = new Stack<>();
  Set<Vertex> visited = new Set<>();

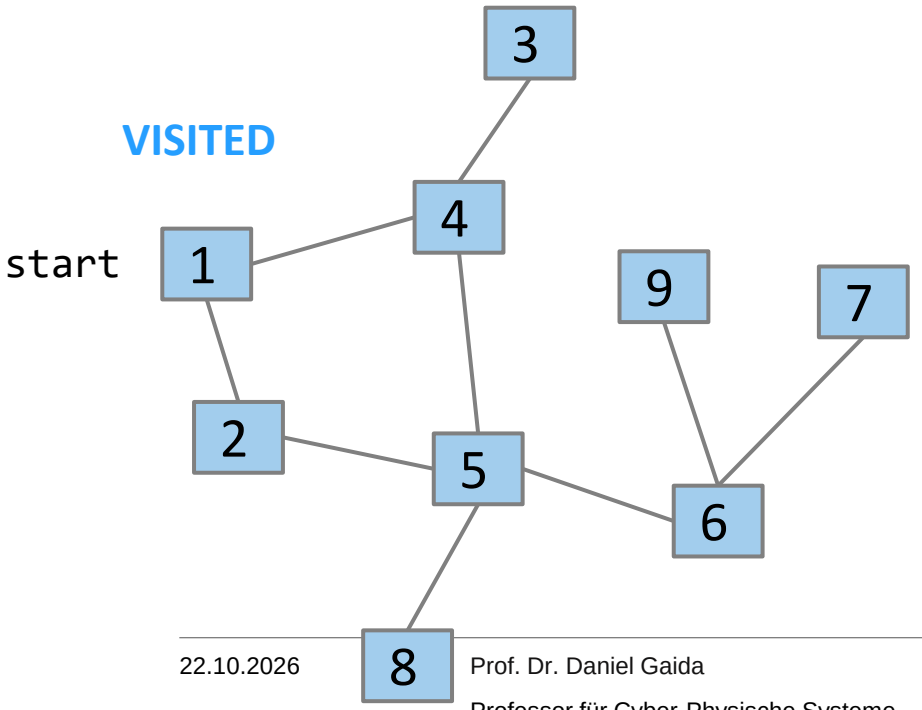
  perimeter.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    if (!visited.contains(from)) {
      for (Edge edge:graph.edgesFrom(from)) {
        Vertex to = edge.to();
        if (!visited.contains(to))
          perimeter.add(to)
      }
      visited.add(from);
    }
  }
}
```

Tiefensuche Implementierung: In Aktion

PERIMETER

4 2




```
dfs(Graph graph, Vertex start) {
  Stack<Vertex> perimeter = new Stack<>();
  Set<Vertex> visited = new Set<>();

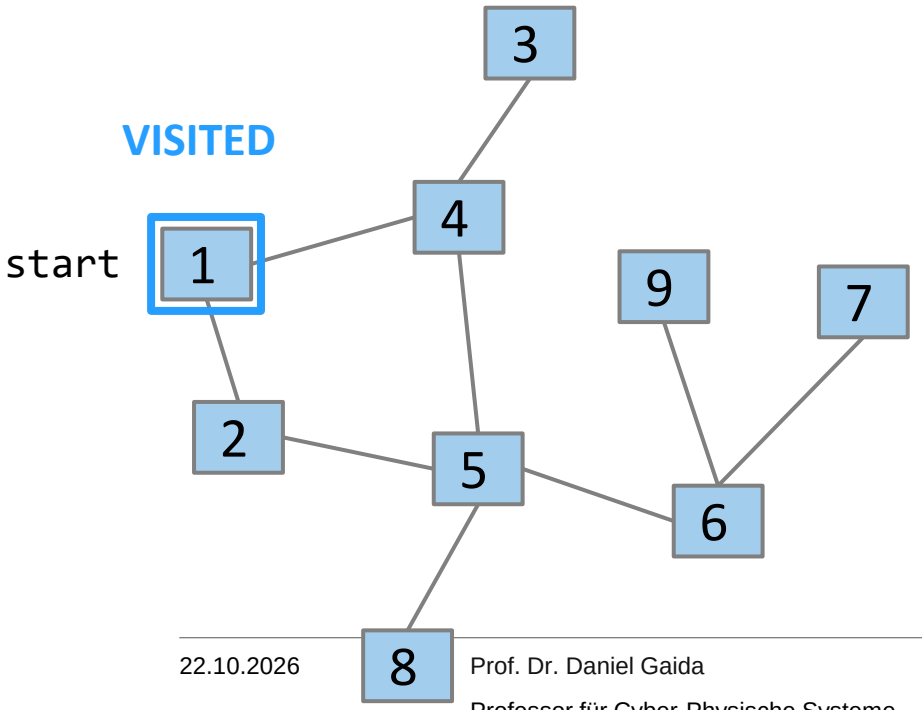
  perimeter.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    if (!visited.contains(from)) {
      for (Edge edge:graph.edgesFrom(from)) {
        Vertex to = edge.to();
        if (!visited.contains(to))
          perimeter.add(to)
      }
      visited.add(from);
    }
  }
}
```

Tiefensuche Implementierung: In Aktion

PERIMETER

4 2




```
dfs(Graph graph, Vertex start) {
  Stack<Vertex> perimeter = new Stack<>();
  Set<Vertex> visited = new Set<>();

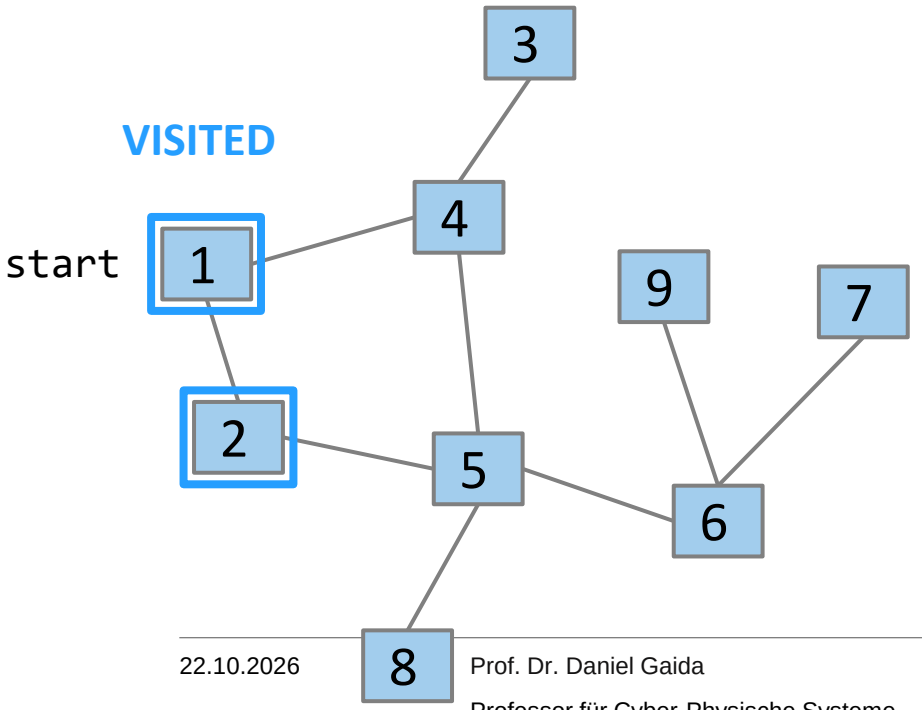
  perimeter.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    if (!visited.contains(from)) {
      for (Edge edge:graph.edgesFrom(from)) {
        Vertex to = edge.to();
        if (!visited.contains(to))
          perimeter.add(to)
      }
      visited.add(from);
    }
  }
}
```

Tiefensuche Implementierung: In Aktion

PERIMETER

4 5




```
dfs(Graph graph, Vertex start) {
  Stack<Vertex> perimeter = new Stack<>();
  Set<Vertex> visited = new Set<>();

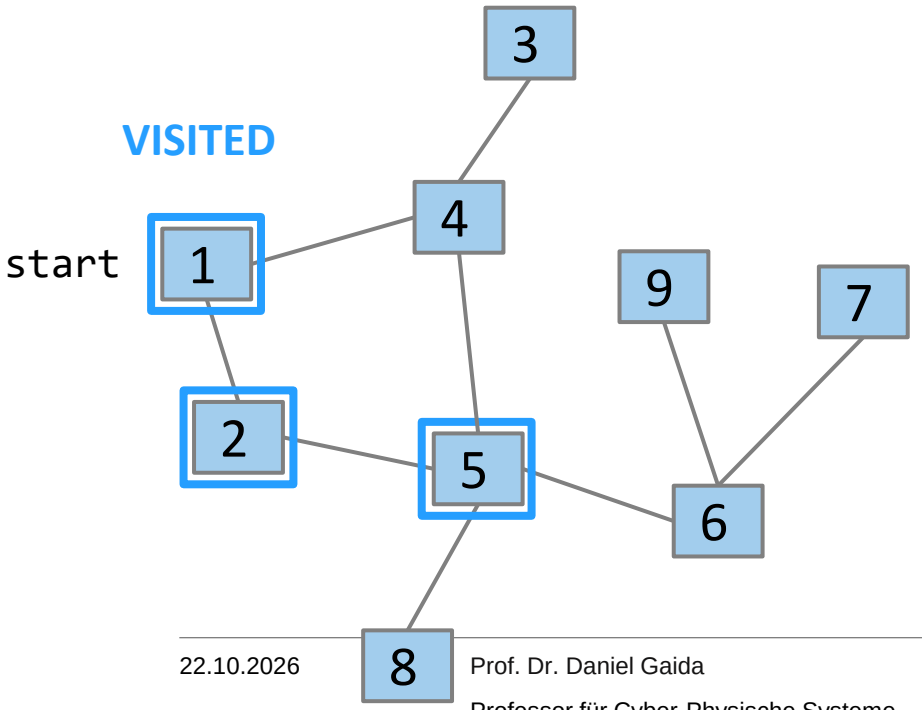
  perimeter.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    if (!visited.contains(from)) {
      for (Edge edge:graph.edgesFrom(from)) {
        Vertex to = edge.to();
        if (!visited.contains(to))
          perimeter.add(to)
      }
      visited.add(from);
    }
  }
}
```

Tiefensuche Implementierung: In Aktion

PERIMETER

4 4 6 8




```
dfs(Graph graph, Vertex start) {
  Stack<Vertex> perimeter = new Stack<>();
  Set<Vertex> visited = new Set<>();

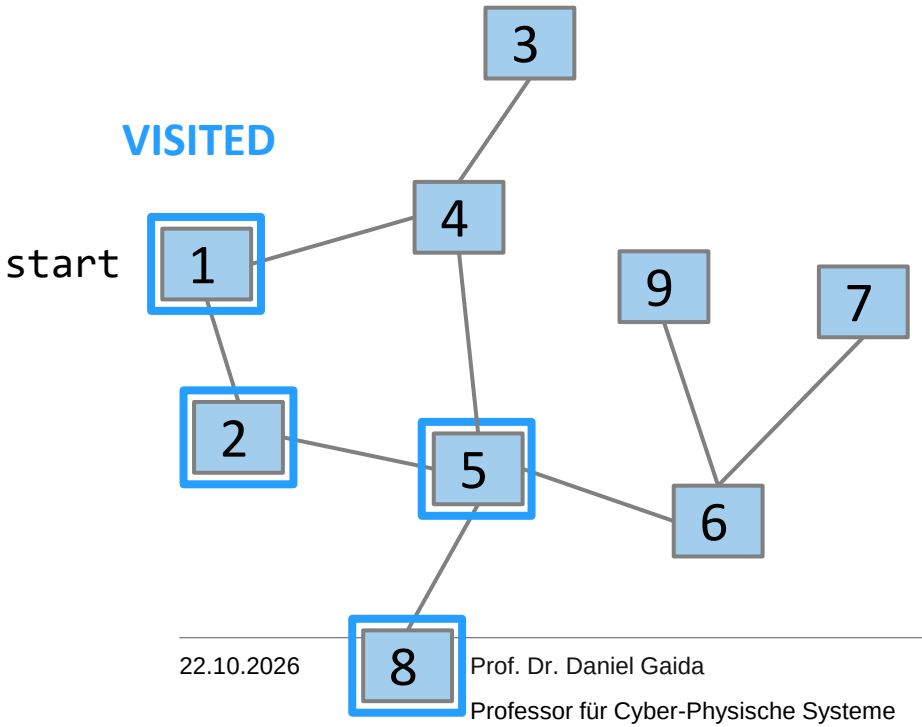
  perimeter.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    if (!visited.contains(from)) {
      for (Edge edge:graph.edgesFrom(from)) {
        Vertex to = edge.to();
        if (!visited.contains(to))
          perimeter.add(to)
      }
      visited.add(from);
    }
  }
}
```

Tiefensuche Implementierung: In Aktion

PERIMETER

4 4 6




```
dfs(Graph graph, Vertex start) {
  Stack<Vertex> perimeter = new Stack<>();
  Set<Vertex> visited = new Set<>();

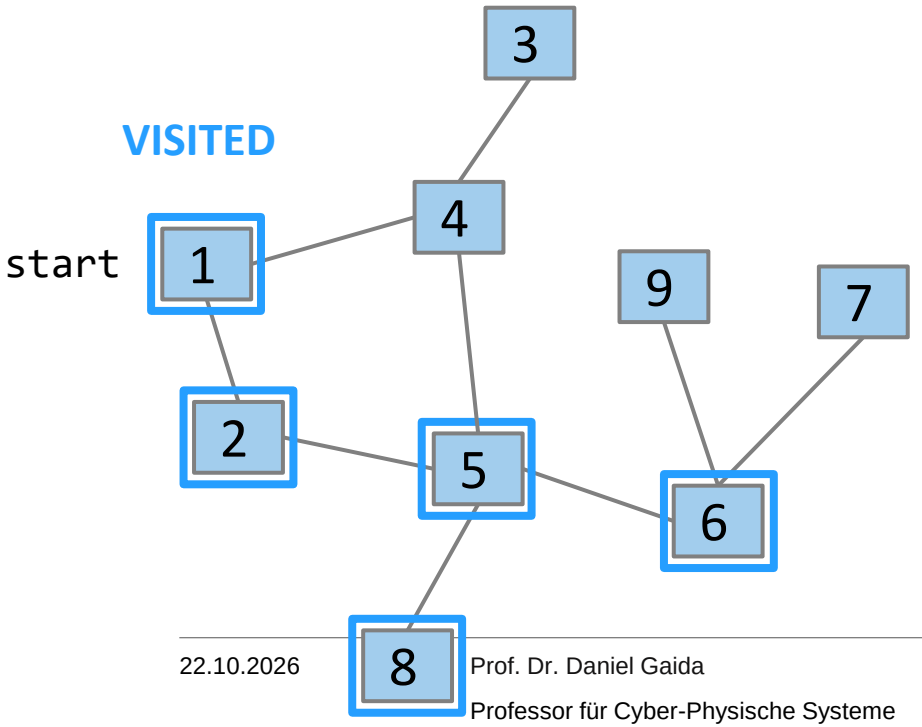
  perimeter.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    if (!visited.contains(from)) {
      for (Edge edge:graph.edgesFrom(from)) {
        Vertex to = edge.to();
        if (!visited.contains(to))
          perimeter.add(to)
      }
      visited.add(from);
    }
  }
}
```

Tiefensuche Implementierung: In Aktion

PERIMETER

4 4 9 7




```
dfs(Graph graph, Vertex start) {
  Stack<Vertex> perimeter = new Stack<>();
  Set<Vertex> visited = new Set<>();

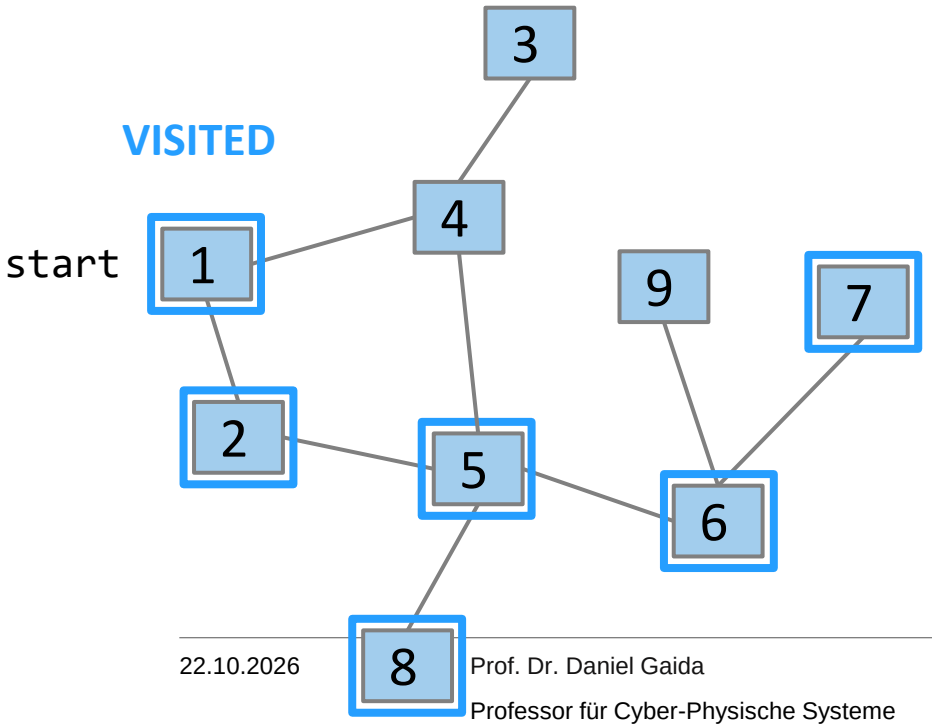
  perimeter.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    if (!visited.contains(from)) {
      for (Edge edge:graph.edgesFrom(from)) {
        Vertex to = edge.to();
        if (!visited.contains(to))
          perimeter.add(to)
      }
      visited.add(from);
    }
  }
}
```

Tiefensuche Implementierung: In Aktion

PERIMETER

4 4 9




```
dfs(Graph graph, Vertex start) {
  Stack<Vertex> perimeter = new Stack<>();
  Set<Vertex> visited = new Set<>();

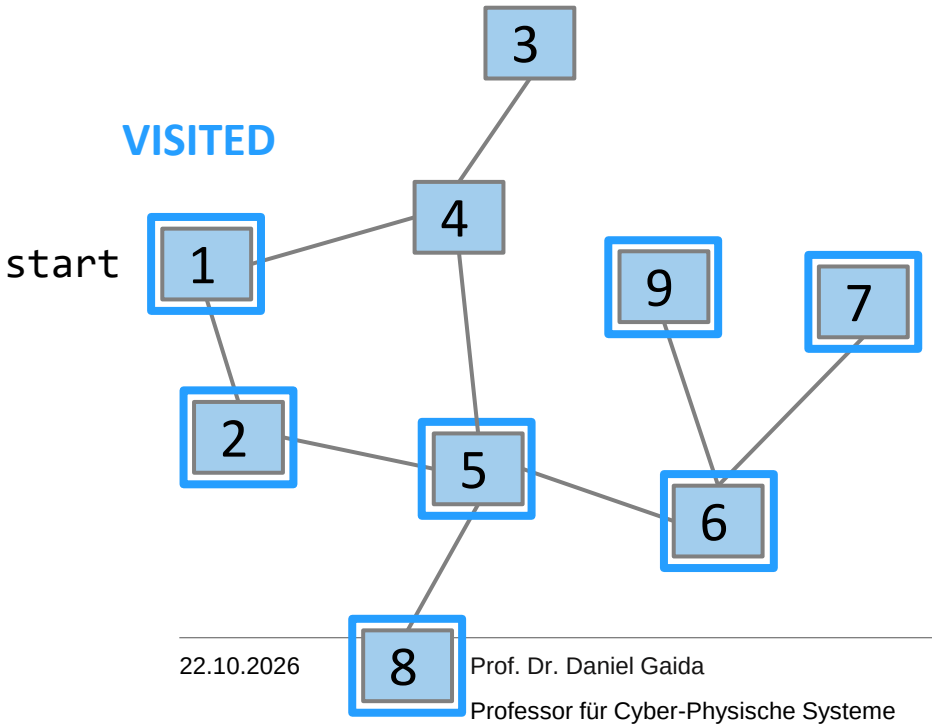
  perimeter.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    if (!visited.contains(from)) {
      for (Edge edge:graph.edgesFrom(from)) {
        Vertex to = edge.to();
        if (!visited.contains(to))
          perimeter.add(to)
      }
      visited.add(from);
    }
  }
}
```

Tiefensuche Implementierung: In Aktion

PERIMETER

4 4




```
dfs(Graph graph, Vertex start) {
  Stack<Vertex> perimeter = new Stack<>();
  Set<Vertex> visited = new Set<>();

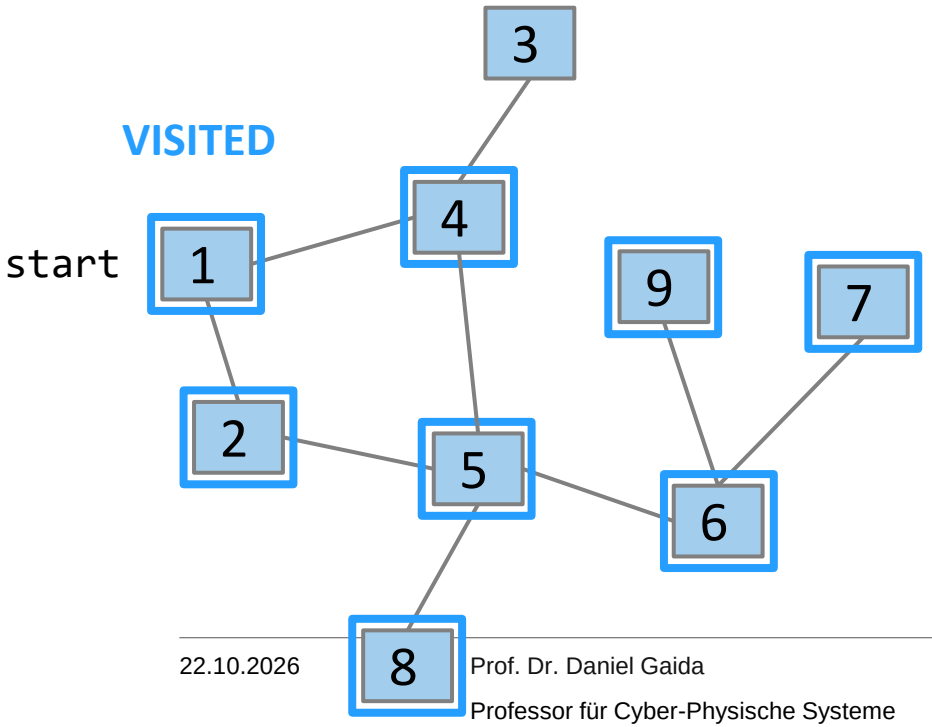
  perimeter.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    if (!visited.contains(from)) {
      for (Edge edge:graph.edgesFrom(from)) {
        Vertex to = edge.to();
        if (!visited.contains(to))
          perimeter.add(to)
      }
      visited.add(from);
    }
  }
}
```

Tiefensuche Implementierung: In Aktion

PERIMETER

4 3




```
dfs(Graph graph, Vertex start) {
  Stack<Vertex> perimeter = new Stack<>();
  Set<Vertex> visited = new Set<>();

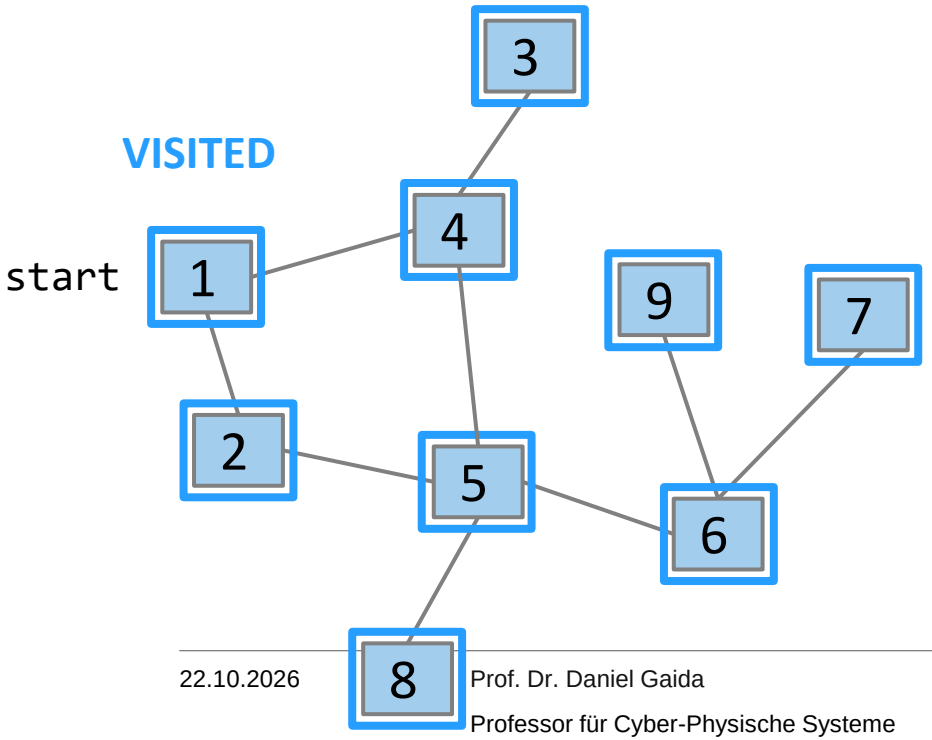
  perimeter.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    if (!visited.contains(from)) {
      for (Edge edge:graph.edgesFrom(from)) {
        Vertex to = edge.to();
        if (!visited.contains(to))
          perimeter.add(to)
      }
      visited.add(from);
    }
  }
}
```

Tiefensuche Implementierung: In Aktion

PERIMETER

4



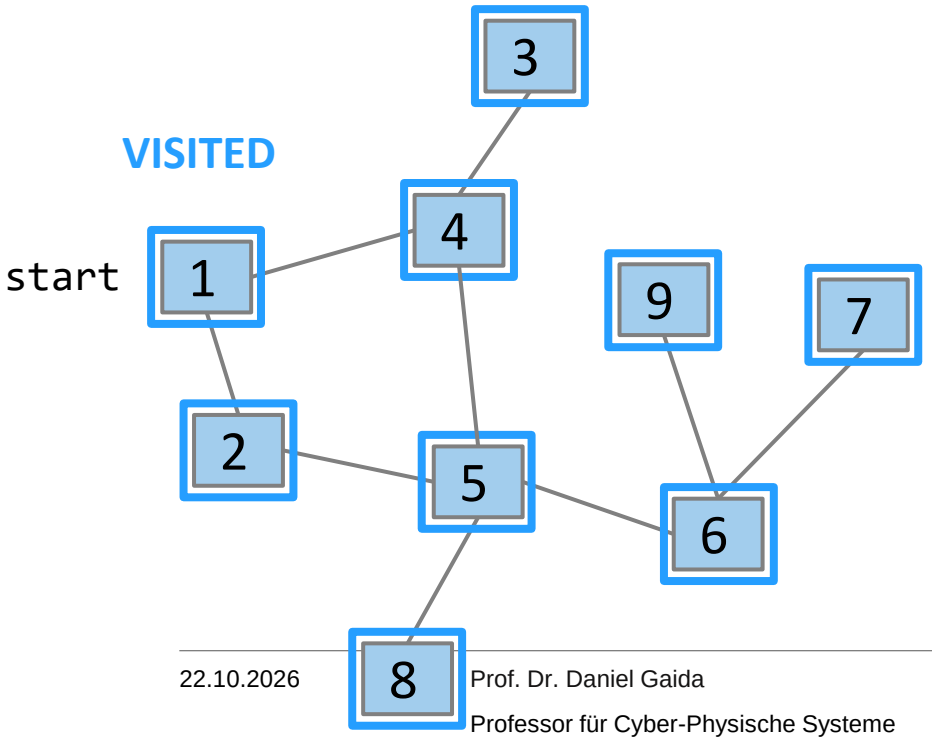

```
dfs(Graph graph, Vertex start) {
  Stack<Vertex> perimeter = new Stack<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    if (!visited.contains(from)) {
      for (Edge edge:graph.edgesFrom(from)) {
        Vertex to = edge.to();
        if (!visited.contains(to))
          perimeter.add(to)
      }
      visited.add(from);
    }
  }
}
```

Tiefensuche Implementierung: In Aktion

PERIMETER




```
dfs(Graph graph, Vertex start) {
  Stack<Vertex> perimeter = new Stack<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    if (!visited.contains(from)) {
      for (Edge edge:graph.edgesFrom(from)) {
        Vertex to = edge.to();
        if (!visited.contains(to))
          perimeter.add(to)
      }
      visited.add(from);
    }
  }
}
```

Zusammenfassung: Graphen-Traversierung

Wir haben zwei Ansätze für die Reihenfolge einer Graphen-Traversierung gesehen

- BFS und DFS sind nur Techniken zur Iteration! (Vergleiche: for-Schleife über ein Array)
- Sie müssen Code hinzufügen, der tatsächlich etwas verarbeitet, um ein Problem zu lösen.

Tiefensuche

(Iterativ)

- Folgen Sie einem Pfad bis zum Ende und kehren Sie dann zurück, um andere Pfadmöglichkeiten zu besichtigen.
- Verwendet einen Stapel!

DFS
(Rekursiv)

Zusammenfassung: Graphen-Traversierung

Wir haben zwei Ansätze für die Reihenfolge einer Graphen-Traversierung gesehen

- BFS und DFS sind nur Techniken zur Iteration! (Vergleiche: for-Schleife über ein Array)
- Sie müssen Code hinzufügen, der tatsächlich etwas verarbeitet, um ein Problem zu lösen.

Tiefensuche

(Iterativ)

- Folgen Sie einem Pfad bis zum Ende und kehren Sie dann zurück, um andere Pfadmöglichkeiten zu besichtigen.
- Verwendet einen Stapel!

DFS
(Rekursiv)

Seien Sie vorsichtig bei der Verwendung dieser Funktion - bei großen Graphen kann der Aufrufstapel überlaufen

Zusammenfassung: Graphen-Traversierung

Wir haben zwei Ansätze für die Reihenfolge einer Graphen-Traversierung gesehen

- BFS und DFS sind nur Techniken zur Iteration! (Vergleiche: for-Schleife über ein Array)
- Sie müssen Code hinzufügen, der tatsächlich etwas verarbeitet, um ein Problem zu lösen.

Tiefensuche

(Iterativ)

- Folgen Sie einem Pfad bis zum Ende und kehren Sie dann zurück, um andere Pfadmöglichkeiten zu besichtigen.
- Verwendet einen Stapel!

Breitensuche

(Iterativ)

- Erkundung Ebene für Ebene: Untersuchung jedes Knotens in einer bestimmten Entfernung vom Startknoten, dann Untersuchung von Knoten, die eine Ebene weiter entfernt sind
- Verwendet eine Warteschlange!

DFS
(Rekursiv)

Seien Sie vorsichtig bei der Verwendung dieser Funktion - bei großen Graphen kann der Aufrufstapel überlaufen

Laufzeiten für BFS und DFS

Annahme: Implementierung des Graphen als Adjazenzliste mit verketteten Listen.

BFS:

- While Schleife:
  -
  -
  -
  -
  -
  -
  -
  -

```
bfs(Graph graph, Vertex start) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);
  visited.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        perimeter.add(to);
        visited.add(to);
      }
    }
  }
}
```

Laufzeiten für BFS und DFS

Annahme: Implementierung des Graphen als Adjazenzliste mit verketteten Listen.

BFS:

- While Schleife:
  - Jeder Knoten steht einmal in der Warteschlange: **O(n)**
-
  -
-
  -
-
-

```
bfs(Graph graph, Vertex start) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);
  visited.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        perimeter.add(to);
        visited.add(to);
      }
    }
  }
}
```

Laufzeiten für BFS und DFS

Annahme: Implementierung des Graphen als Adjazenzliste mit verketteten Listen.

BFS:

- While Schleife:
  - Jeder Knoten steht einmal in der Warteschlange: **O(n)**
- For Schleife:
  - Für Knoten u: deg(u) Kanten prüfen
-
  -
-
-

```
bfs(Graph graph, Vertex start) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);
  visited.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        perimeter.add(to);
        visited.add(to);
      }
    }
  }
}
```

Laufzeiten für BFS und DFS

Annahme: Implementierung des Graphen als Adjazenzliste mit verketteten Listen.

BFS:

- While Schleife:
  - Jeder Knoten steht einmal in der Warteschlange: **O(n)**
- For Schleife:
  - Für Knoten u: deg(u) Kanten prüfen
- While und for zusammen:
  - Für alle n Knoten müssen wir deg(u) Kanten prüfen: n mal deg(u) = m → **O(m)**
-
-

```
bfs(Graph graph, Vertex start) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);
  visited.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        perimeter.add(to);
        visited.add(to);
      }
    }
  }
}
```

Laufzeiten für BFS und DFS

Annahme: Implementierung des Graphen als Adjazenzliste mit verketteten Listen.

BFS:

- While Schleife:
  - Jeder Knoten steht einmal in der Warteschlange: **O(n)**
- For Schleife:
  - Für Knoten u: deg(u) Kanten prüfen
- While und for zusammen:
  - Für alle n Knoten müssen wir deg(u) Kanten prüfen: n mal deg(u) = m → **O(m)**
-
-

Implementiert als Hashtabelle

```
bfs(Graph graph, Vertex start) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);
  visited.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        perimeter.add(to);
        visited.add(to);
      }
    }
  }
}
```

Laufzeiten für BFS und DFS

Annahme: Implementierung des Graphen als Adjazenzliste mit verketteten Listen.

BFS:

- While Schleife:
  - Jeder Knoten steht einmal in der Warteschlange: **O(n)**
- For Schleife:
  - Für Knoten u: deg(u) Kanten prüfen
- While und for zusammen:
  - Für alle n Knoten müssen wir deg(u) Kanten prüfen: n mal deg(u) = m → **O(m)**
- Add und Remove Operationen auf Warteschlange haben Laufzeit O(1)
- → **O(n + m)**

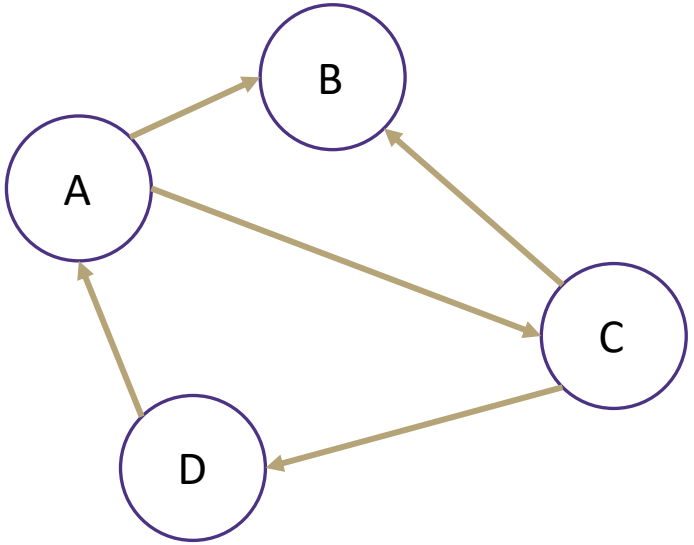
Implementiert als Hashtabelle

```
bfs(Graph graph, Vertex start) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);
  visited.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        perimeter.add(to);
        visited.add(to);
      }
    }
  }
}
```

Graph als Adjazenzliste



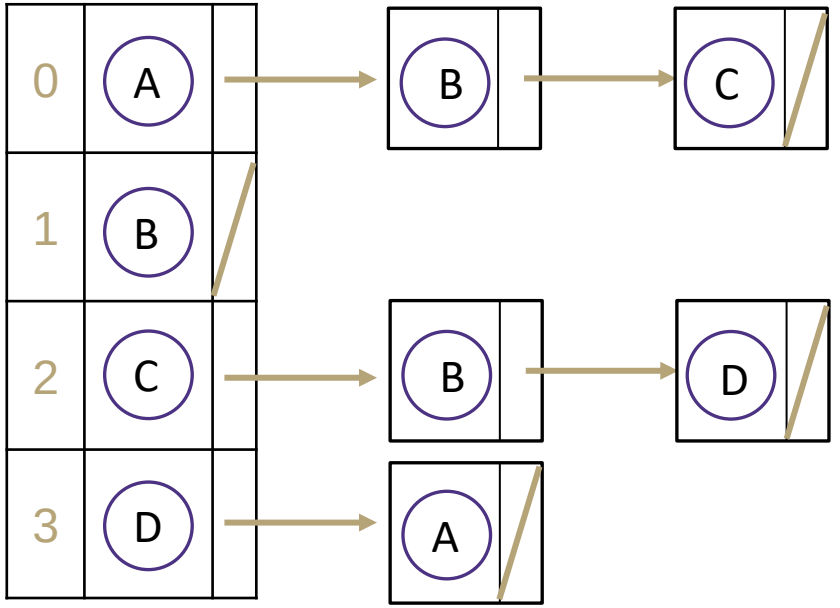
Anzahl ausgehende Kanten:

- A: 2
- B: 0
- C: 2
- D: 1
- Summe = 5

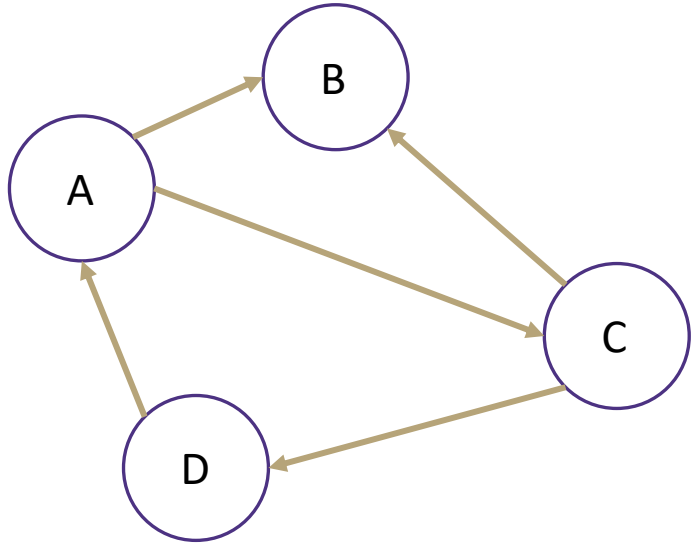
- Anzahl an Kanten im Graph: 5

-

Verkettete Liste



Graph als Adjazenzliste



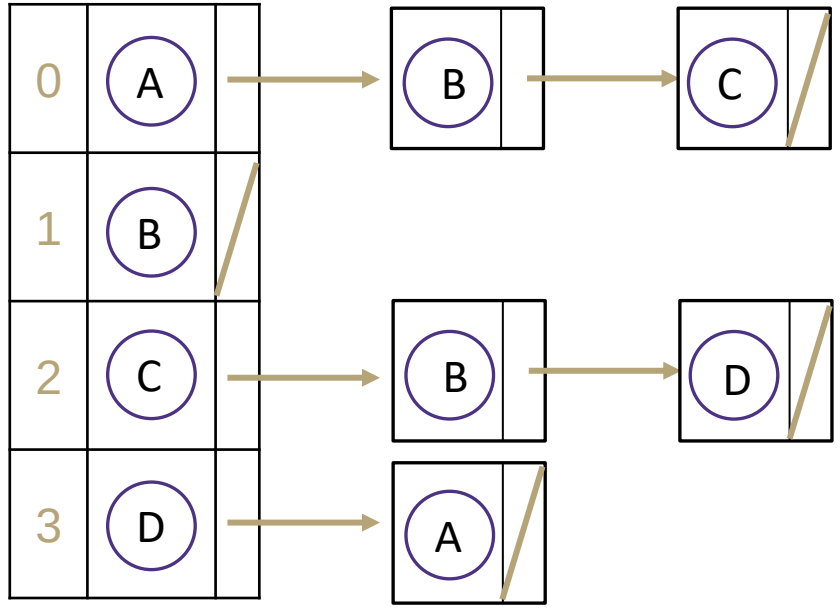
Anzahl ausgehende Kanten:

- A: 2
- B: 0
- C: 2
- D: 1
- Summe = 5

- Anzahl an Kanten im Graph: 5

- In der Funktion `graph.edgesFrom(from)` gehen wir durch die verkettete Liste des from Knoten.

Verkettete Liste



Laufzeiten für BFS und DFS

Annahme: Implementierung des Graphen als Adjazenzliste mit verketteten Listen.

DFS (iterativ):

-
-
-
-
-

```
dfs(Graph graph, Vertex start) {
  Stack<Vertex> perimeter = new Stack<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    if (!visited.contains(from)) {
      for (Edge edge:graph.edgesFrom(from)) {
        Vertex to = edge.to();
        if (!visited.contains(to))
          perimeter.add(to)
      }
      visited.add(from);
    }
  }
}
```

Laufzeiten für BFS und DFS

Annahme: Implementierung des Graphen als Adjazenzliste mit verketteten Listen.

DFS (iterativ):

- Jeder Knoten steht einmal im Stapel: O(n)
-
-
-
-

```
dfs(Graph graph, Vertex start) {
  Stack<Vertex> perimeter = new Stack<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    if (!visited.contains(from)) {
      for (Edge edge:graph.edgesFrom(from)) {
        Vertex to = edge.to();
        if (!visited.contains(to))
          perimeter.add(to)
      }
      visited.add(from);
    }
  }
}
```

Laufzeiten für BFS und DFS

Annahme: Implementierung des Graphen als Adjazenzliste mit verketteten Listen.

DFS (iterativ):

- Jeder Knoten steht einmal im Stapel: O(n)
- Für alle n Knoten müssen wir deg(u) Kanten prüfen: n mal deg(u) = m → O(m)
-
-
-

```
dfs(Graph graph, Vertex start) {
  Stack<Vertex> perimeter = new Stack<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    if (!visited.contains(from)) {
      for (Edge edge:graph.edgesFrom(from)) {
        Vertex to = edge.to();
        if (!visited.contains(to))
          perimeter.add(to)
      }
      visited.add(from);
    }
  }
}
```

Laufzeiten für BFS und DFS

Annahme: Implementierung des Graphen als Adjazenzliste mit verketteten Listen.

DFS (iterativ):

- Jeder Knoten steht einmal im Stapel: O(n)
- Für alle n Knoten müssen wir deg(u) Kanten prüfen: n mal deg(u) = m → O(m)
- → O(n + m)
- „Wie besuchen alle Knoten und Kanten“

```
dfs(Graph graph, Vertex start) {
  Stack<Vertex> perimeter = new Stack<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    if (!visited.contains(from)) {
      for (Edge edge:graph.edgesFrom(from)) {
        Vertex to = edge.to();
        if (!visited.contains(to))
          perimeter.add(to)
      }
      visited.add(from);
    }
  }
}
```

Laufzeiten für BFS und DFS

Annahme: Implementierung des Graphen als Adjazenzliste mit verketteten Listen.

DFS (iterativ):

- Jeder Knoten steht einmal im Stapel: O(n)
- Für alle n Knoten müssen wir deg(u) Kanten prüfen: n mal deg(u) = m → O(m)
- → O(n + m)
- „Wie besuchen alle Knoten und Kanten“

DFS (rekursiv)

- O(n + m) -> es werden dieselben Knoten und Kanten besucht wie im iterativen Ansatz

```
dfs(Graph graph, Vertex start) {
  Stack<Vertex> perimeter = new Stack<>();
  Set<Vertex> visited = new Set<>();

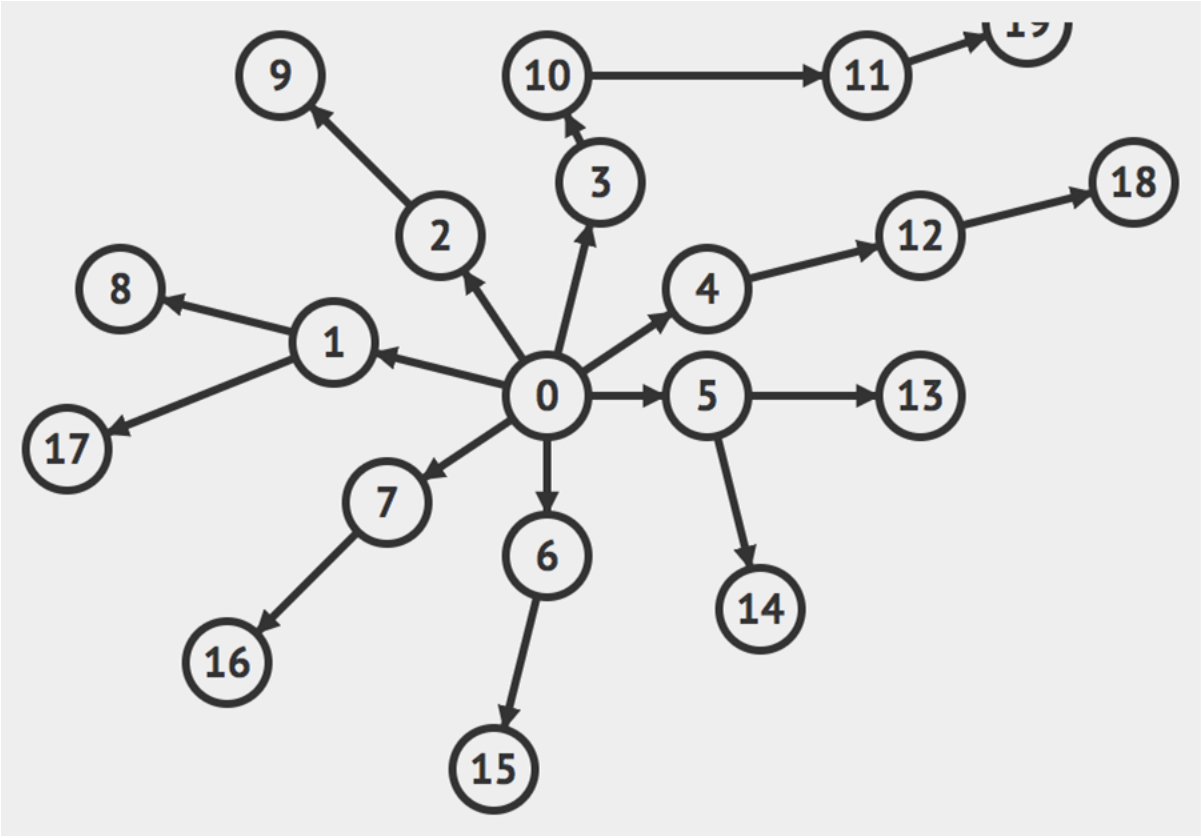
  perimeter.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    if (!visited.contains(from)) {
      for (Edge edge:graph.edgesFrom(from)) {
        Vertex to = edge.to();
        if (!visited.contains(to))
          perimeter.add(to)
      }
      visited.add(from);
    }
  }
}
```

Übung: DFS und BFS

Versuchen Sie, sich zwei mögliche Traversierungs-Reihenfolgen, beginnend mit dem Knoten 0, auszudenken:

- eine BFS-Reihenfolge (die Reihenfolge innerhalb jeder Ebene spielt keine Rolle / jede Reihenfolge ist gültig)
  -
-
  -
  -
  -

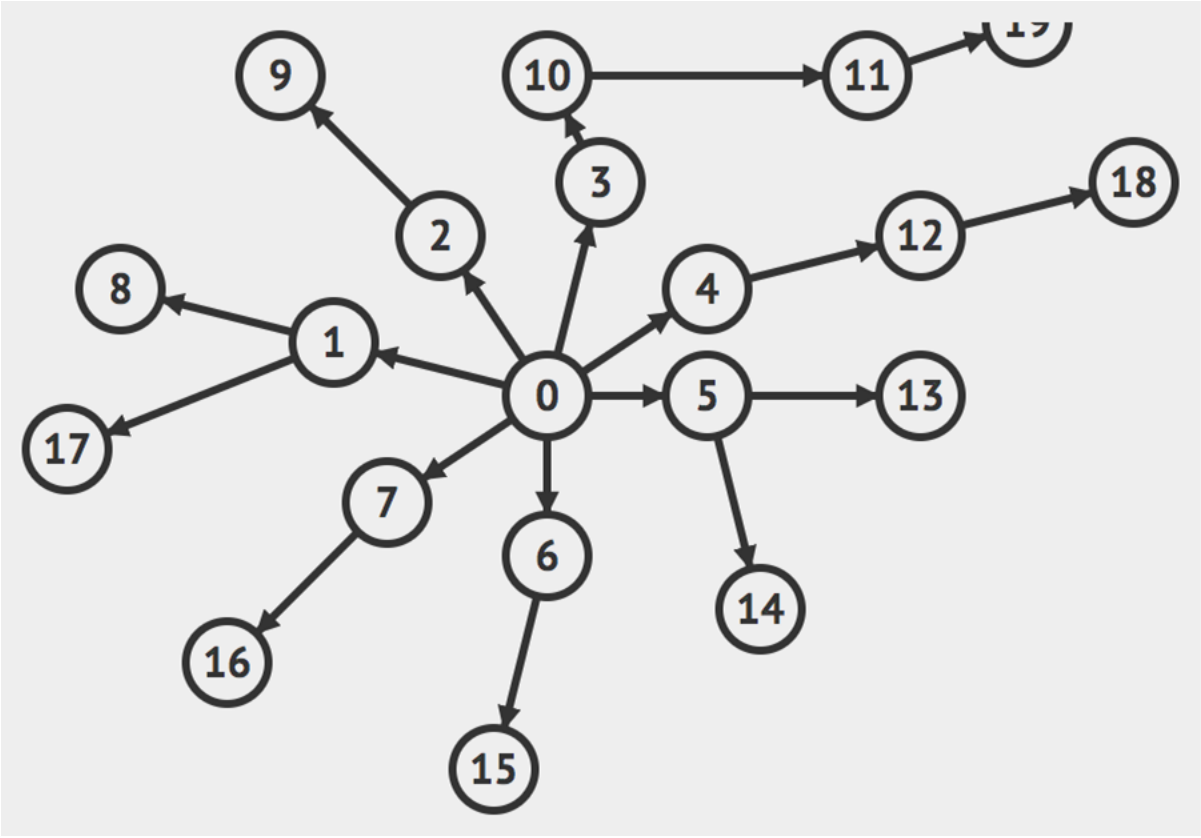


Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Übung: DFS und BFS

Versuchen Sie, sich zwei mögliche Traversierungs-Reihenfolgen, beginnend mit dem Knoten 0, auszudenken:

- eine BFS-Reihenfolge (die Reihenfolge innerhalb jeder Ebene spielt keine Rolle / jede Reihenfolge ist gültig)
  - 0, [1, 2, 3, 4, 5, 6, 7], [17, 8, 9, 10, 12, 13, 14, 15, 16], [11, 18], [19]
-

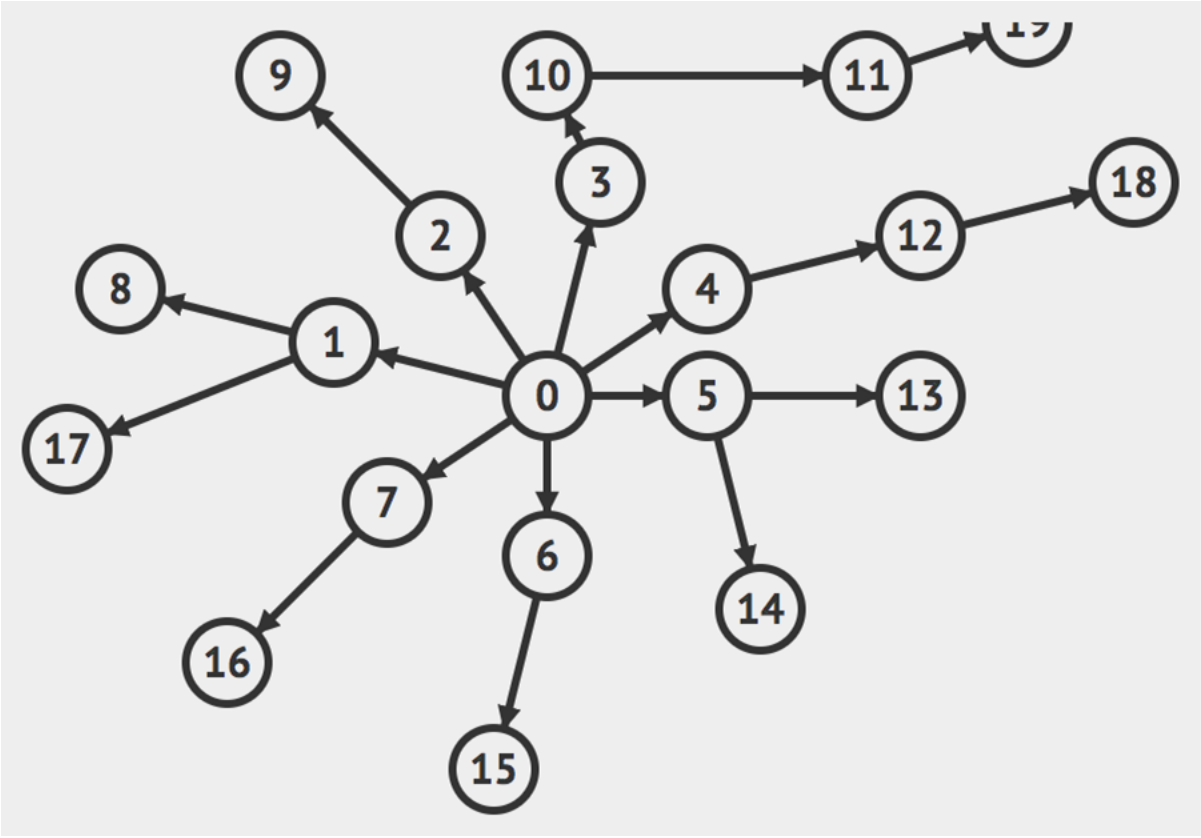


Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Übung: DFS und BFS

Versuchen Sie, sich zwei mögliche Traversierungs-Reihenfolgen, beginnend mit dem Knoten 0, auszudenken:

- eine BFS-Reihenfolge (die Reihenfolge innerhalb jeder Ebene spielt keine Rolle / jede Reihenfolge ist gültig)
  - 0, [1, 2, 3, 4, 5, 6, 7], [17, 8, 9, 10, 12, 13, 14, 15, 16], [11, 18], [19]
- eine DFS-Reihenfolge (welcher Pfad als nächster gewählt wird, spielt keine Rolle / jeder ist gültig, solange man ihn nicht vorher erkundet hat)
  -

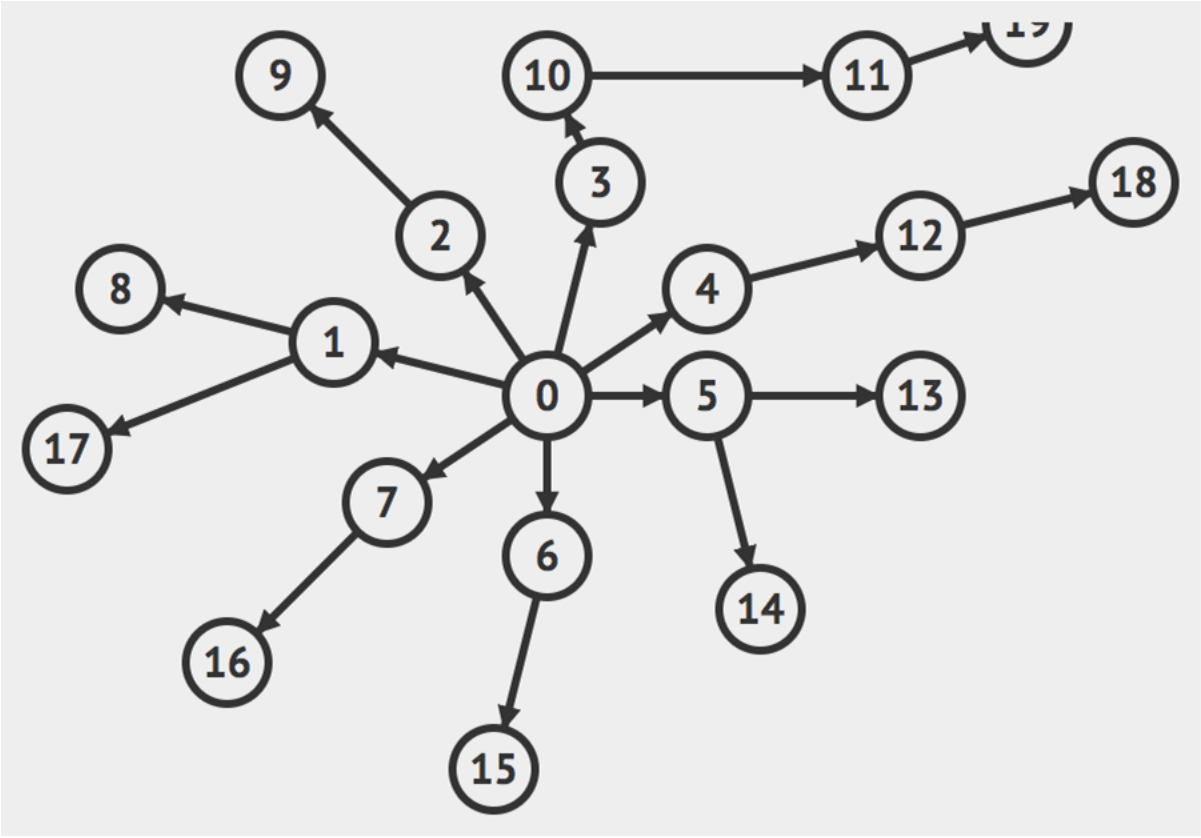


Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Übung: DFS und BFS

Versuchen Sie, sich zwei mögliche Traversierungs-Reihenfolgen, beginnend mit dem Knoten 0, auszudenken:

- eine BFS-Reihenfolge (die Reihenfolge innerhalb jeder Ebene spielt keine Rolle / jede Reihenfolge ist gültig)
  - 0, [1, 2, 3, 4, 5, 6, 7], [17, 8, 9, 10, 12, 13, 14, 15, 16], [11, 18], [19]
- eine DFS-Reihenfolge (welcher Pfad als nächster gewählt wird, spielt keine Rolle / jeder ist gültig, solange man ihn nicht vorher erkundet hat)
  - 0, 2, 9, 3, 10, 11, 19, 4, 12, 18, 5, 13, 14, 6, 15, 7, 16, 1,17, 8



Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

BFS zur Lösung des Erreichbarkeitsproblems

Erreichbarkeitsproblem

Gibt es bei einem Startknoten s und einem Zielknoten t einen Pfad zwischen s und t?

BFS ist ein großartiger Grundstein - wir müssen nur jeden Knoten überprüfen, um zu sehen, ob wir t erreicht haben!

```
stCon(Graph graph, Vertex start, Vertex t) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);
  visited.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    if (from == t) { return true; }
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        perimeter.add(to);
        visited.add(to);
      }
    }
  }
  return false;
}
```

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

BFS zur Lösung des Erreichbarkeitsproblems

Erreichbarkeitsproblem

Gibt es bei einem Startknoten s und einem Zielknoten t einen Pfad zwischen s und t?

BFS ist ein großartiger Grundstein - wir müssen nur jeden Knoten überprüfen, um zu sehen, ob wir t erreicht haben!

Hinweis: Wir verwenden hier keine spezifischen Eigenschaften von BFS, wir brauchen nur eine Traversierung. DFS würde auch funktionieren.

```
stCon(Graph graph, Vertex start, Vertex t) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();

  perimeter.add(start);
  visited.add(start);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    if (from == t) { return true; }
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        perimeter.add(to);
        visited.add(to);
      }
    }
  }
  return false;
}
```

Fragen?

- Übungsblatt 7: Aufgabe 3, 7, 11 und 13

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Technology
Arts Sciences
TH Köln

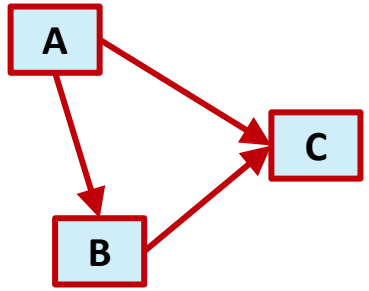
Übersicht über heute

- Graphen
  - Einführung/Anwendungen/Begriffe
- Traversierung von Graphen
  - Erreichbarkeitsproblem
  - Tiefensuche
  - Breitensuche
- Gerichtete Graphen
  - **Topologische Sortierung**
- Kürzeste Pfade
  - Ungewichteter Graph: Breitensuche
  - Gewichteter Graph: Algorithmus von Dijkstra

Topologische Sortierung eines DAG

Eine topologische Sortierung eines **gerichteten azyklischen Graphen** G ist eine Anordnung der Knoten, bei der für jede Kante im Graphen der Start- vor dem Zielknoten in der Anordnung erscheint

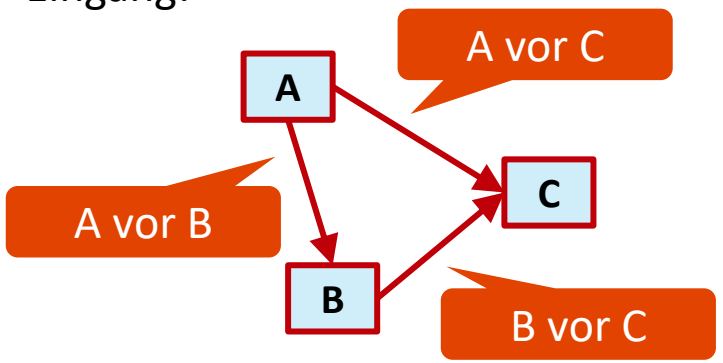
Eingang:



Topologische Sortierung eines DAG

Eine topologische Sortierung eines **gerichteten azyklischen Graphen** G ist eine Anordnung der Knoten, bei der für jede Kante im Graphen der Start- vor dem Zielknoten in der Anordnung erscheint

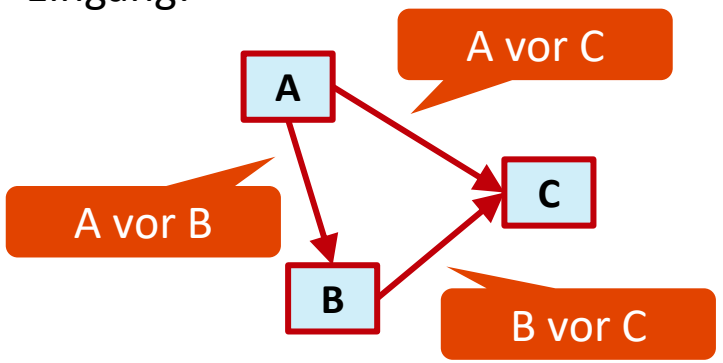
Eingang:



Topologische Sortierung eines DAG

Eine topologische Sortierung eines **gerichteten azyklischen Graphen** G ist eine Anordnung der Knoten, bei der für jede Kante im Graphen der Start- vor dem Zielknoten in der Anordnung erscheint

Eingang:



-
-
-
-
-

Topologische Sortierung:



Mit ursprünglichen Kanten zur Referenz:



Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

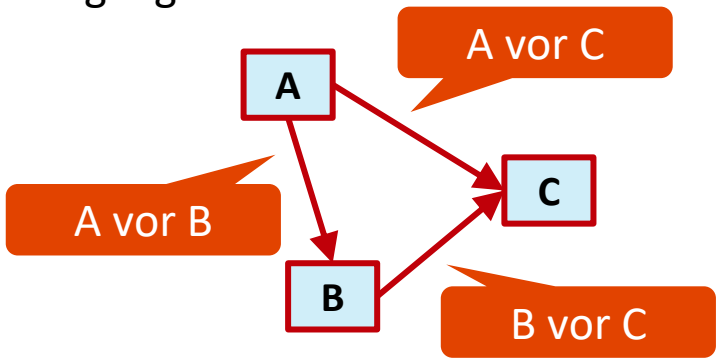
Topologische Sortierung eines DAG

Eine topologische Sortierung eines **gerichteten azyklischen Graphen** G ist eine Anordnung der Knoten, bei der für jede Kante im Graphen der Start- vor dem Zielknoten in der Anordnung erscheint

Intuition: ein „Abhängigkeitsgraph“

- Eine Kante (u, v) bedeutet, dass u vor v auftreten muss.
- Eine topologische Sortierung eines Abhängigkeitsgraphen ergibt eine Anordnung, die die Abhängigkeiten berücksichtigt
-
-
-

Eingang:



Topologische Sortierung:



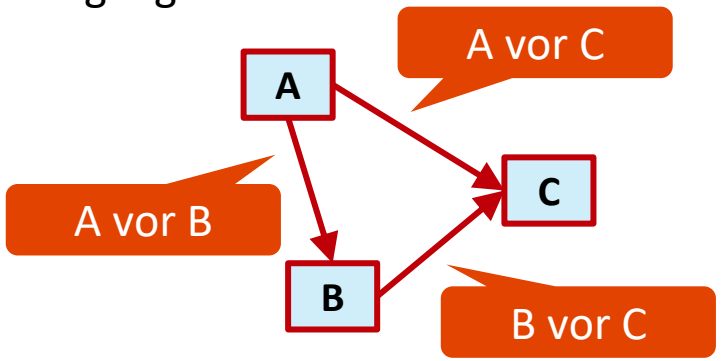
Mit ursprünglichen Kanten zur Referenz:



Topologische Sortierung eines DAG

Eine topologische Sortierung eines **gerichteten azyklischen Graphen** G ist eine Anordnung der Knoten, bei der für jede Kante im Graphen der Start- vor dem Zielknoten in der Anordnung erscheint

Eingang:



Intuition: ein „Abhängigkeitsgraph“

- Eine Kante (u, v) bedeutet, dass u vor v auftreten muss.
- Eine topologische Sortierung eines Abhängigkeitsgraphen ergibt eine Anordnung, die die Abhängigkeiten berücksichtigt

Topologische Sortierung:



Anwendungen:

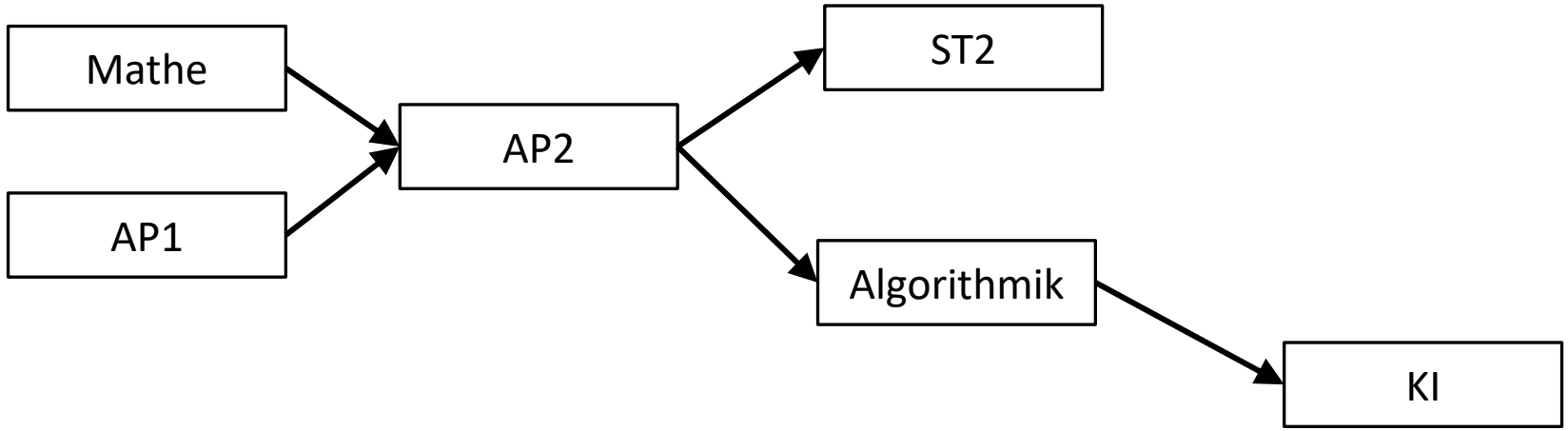
- Weg bis zum Hochschulabschluss
- Kompilieren mehrerer Java-Dateien
- Forschungsprojekt mit mehreren Arbeitspaketen

Mit ursprünglichen Kanten zur Referenz:



Übung: Topologische Sortierung

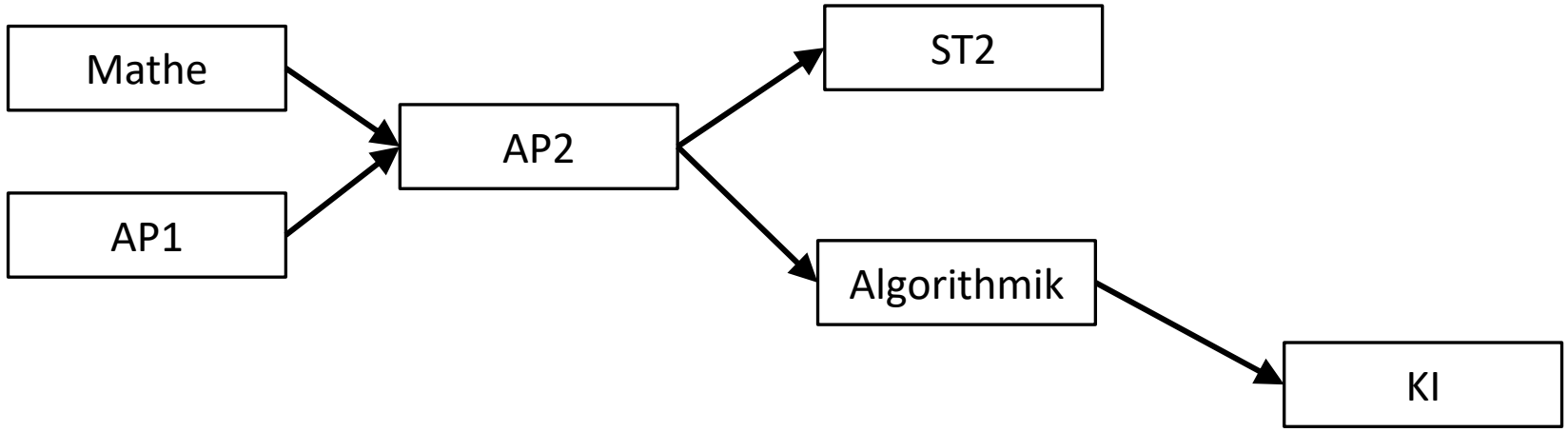
Finden Sie bei einer Menge von Modulen mit Voraussetzungen eine Reihenfolge, in der die Module zu belegen sind.



Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022
CSE 373 18 SP - KASEY CHAMPION

Topologische Sortierung

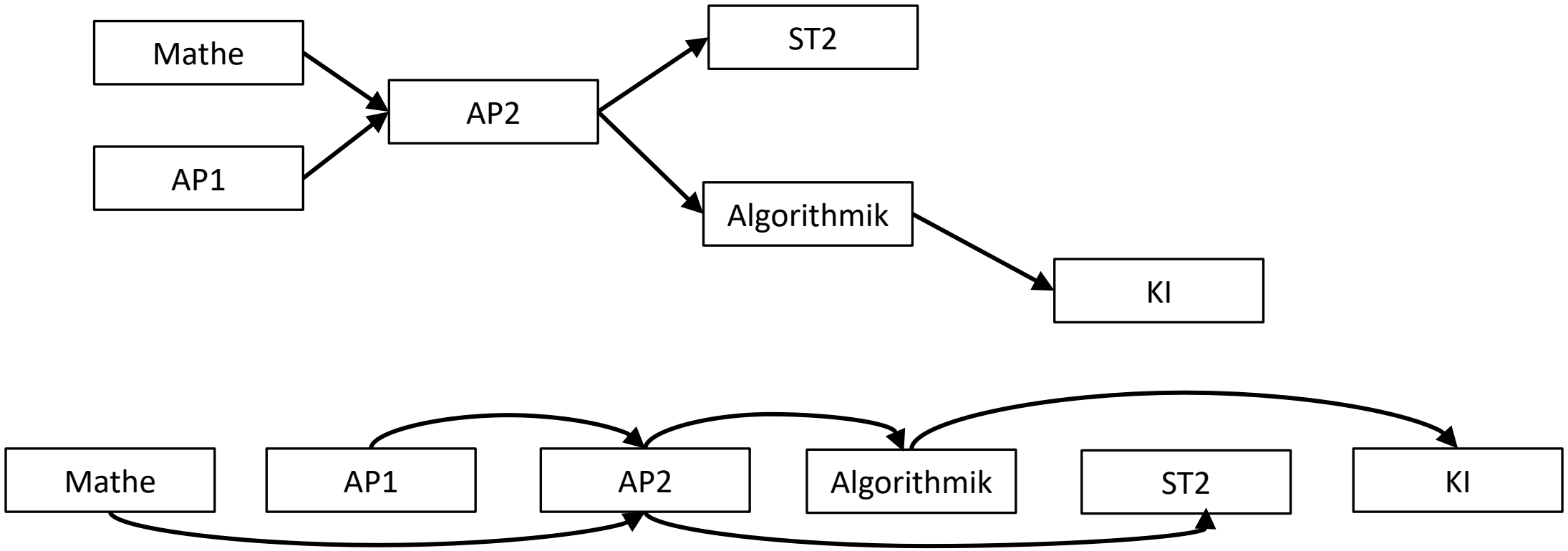
Eine grafische Darstellung der Modulvoraussetzungen und eine mögliche topologische Sortierung.



Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022
CSE 373 19 WI - KASEY CHAMPION

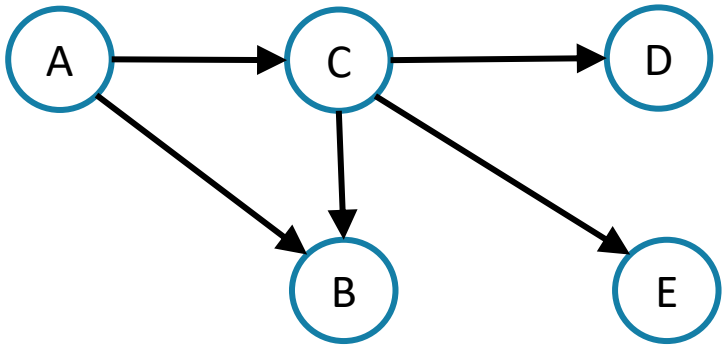
Topologische Sortierung

Eine grafische Darstellung der Modulvoraussetzungen und eine mögliche topologische Sortierung.



Topologische Sortierung

Wie finden wir für diesen Graph eine topologische Sortierung?



Topologische Sortierung

Wie finden wir für diesen Graph eine topologische Sortierung?

Wenn ein Knoten keine Kanten hat, die in ihn hineinführen, können wir ihn der Sortierung hinzufügen.

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022
CSE 373 19 WI - KASEY CHAMPION

Topologische Sortierung

Wie finden wir für diesen Graph eine topologische Sortierung?

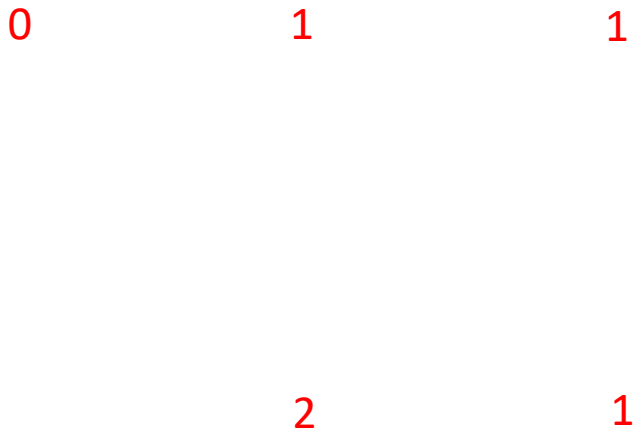
0 1 1

2 1

Wenn ein Knoten keine Kanten hat, die in ihn hineinführen, können wir ihn der Sortierung hinzufügen.

Topologische Sortierung

Wie finden wir für diesen Graph eine topologische Sortierung?



Topologische Sortierung:

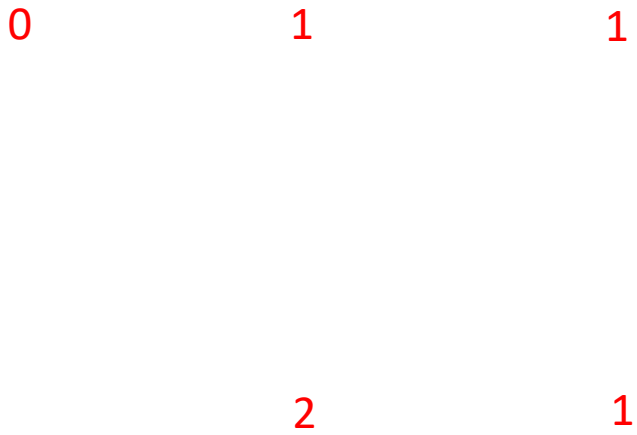
A

Wenn ein Knoten keine Kanten hat, die in ihn hineinführen, können wir ihn der Sortierung hinzufügen.

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022
CSE 373 19 WI - KASEY CHAMPION

Topologische Sortierung

Wie finden wir für diesen Graph eine topologische Sortierung?



Topologische Sortierung:

A

Wenn ein Knoten keine Kanten hat, die in ihn hineinführen, können wir ihn der Sortierung hinzufügen.

Allgemeiner ausgedrückt: Wenn die einzigen eingehenden Kanten von Knoten stammen, die bereits in der Sortierung enthalten sind, ist es sicher, den Knoten hinzuzufügen.

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022
CSE 373 19 WI - KASEY CHAMPION

Topologische Sortierung

Wie finden wir für diesen Graph eine topologische Sortierung?



Topologische Sortierung:

A

Wenn ein Knoten keine Kanten hat, die in ihn hineinführen, können wir ihn der Sortierung hinzufügen.

Allgemeiner ausgedrückt: Wenn die einzigen eingehenden Kanten von Knoten stammen, die bereits in der Sortierung enthalten sind, ist es sicher, den Knoten hinzuzufügen.

Topologische Sortierung

Wie finden wir für diesen Graph eine topologische Sortierung?



Topologische Sortierung:

A C

Wenn ein Knoten keine Kanten hat, die in ihn hineinführen, können wir ihn der Sortierung hinzufügen.

Allgemeiner ausgedrückt: Wenn die einzigen eingehenden Kanten von Knoten stammen, die bereits in der Sortierung enthalten sind, ist es sicher, den Knoten hinzuzufügen.

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022
CSE 373 19 WI - KASEY CHAMPION

Topologische Sortierung

Wie finden wir für diesen Graph eine topologische Sortierung?



Topologische Sortierung:

A C

Wenn ein Knoten keine Kanten hat, die in ihn hineinführen, können wir ihn der Sortierung hinzufügen.

Allgemeiner ausgedrückt: Wenn die einzigen eingehenden Kanten von Knoten stammen, die bereits in der Sortierung enthalten sind, ist es sicher, den Knoten hinzuzufügen.

Topologische Sortierung

Wie finden wir für diesen Graph eine topologische Sortierung?



Topologische Sortierung:

A C B



Wenn ein Knoten keine Kanten hat, die in ihn hineinführen, können wir ihn der Sortierung hinzufügen.

Allgemeiner ausgedrückt: Wenn die einzigen eingehenden Kanten von Knoten stammen, die bereits in der Sortierung enthalten sind, ist es sicher, den Knoten hinzuzufügen.

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022
CSE 373 19 WI - KASEY CHAMPION

Topologische Sortierung

Wie finden wir für diesen Graph eine topologische Sortierung?

0

Topologische Sortierung:

A C B

0

Wenn ein Knoten keine Kanten hat, die in ihn hineinführen, können wir ihn der Sortierung hinzufügen.

Allgemeiner ausgedrückt: Wenn die einzigen eingehenden Kanten von Knoten stammen, die bereits in der Sortierung enthalten sind, ist es sicher, den Knoten hinzuzufügen.

Topologische Sortierung

Wie finden wir für diesen Graph eine topologische Sortierung?

0

Topologische Sortierung:

A C B D

0

Wenn ein Knoten keine Kanten hat, die in ihn hineinführen, können wir ihn der Sortierung hinzufügen.

Allgemeiner ausgedrückt: Wenn die einzigen eingehenden Kanten von Knoten stammen, die bereits in der Sortierung enthalten sind, ist es sicher, den Knoten hinzuzufügen.

Topologische Sortierung

Wie finden wir für diesen Graph eine topologische Sortierung?

Topologische Sortierung:

A C B D

0

Wenn ein Knoten keine Kanten hat, die in ihn hineinführen, können wir ihn der Sortierung hinzufügen.

Allgemeiner ausgedrückt: Wenn die einzigen eingehenden Kanten von Knoten stammen, die bereits in der Sortierung enthalten sind, ist es sicher, den Knoten hinzuzufügen.

Topologische Sortierung

Wie finden wir für diesen Graph eine topologische Sortierung?

Topologische Sortierung:

A C B D E

0

Wenn ein Knoten keine Kanten hat, die in ihn hineinführen, können wir ihn der Sortierung hinzufügen.

Allgemeiner ausgedrückt: Wenn die einzigen eingehenden Kanten von Knoten stammen, die bereits in der Sortierung enthalten sind, ist es sicher, den Knoten hinzuzufügen.

Topologische Sortierung

Wie finden wir für diesen Graph eine topologische Sortierung?

Topologische Sortierung:

A C B D E

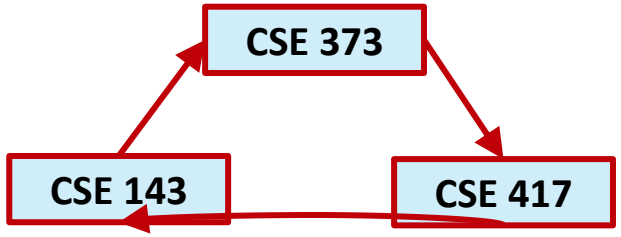
Wenn ein Knoten keine Kanten hat, die in ihn hineinführen, können wir ihn der Sortierung hinzufügen.

Allgemeiner ausgedrückt: Wenn die einzigen eingehenden Kanten von Knoten stammen, die bereits in der Sortierung enthalten sind, ist es sicher, den Knoten hinzuzufügen.

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022
CSE 373 19 WI - KASEY CHAMPION

Topologische Sortierung funktioniert nur für DAGs

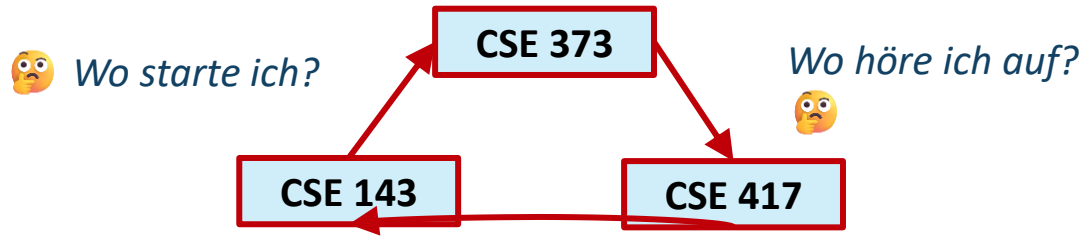
Können Sie diesen Graphen topologisch sortieren?



Topologische Sortierung funktioniert nur für DAGs

Können Sie diesen Graphen topologisch sortieren?

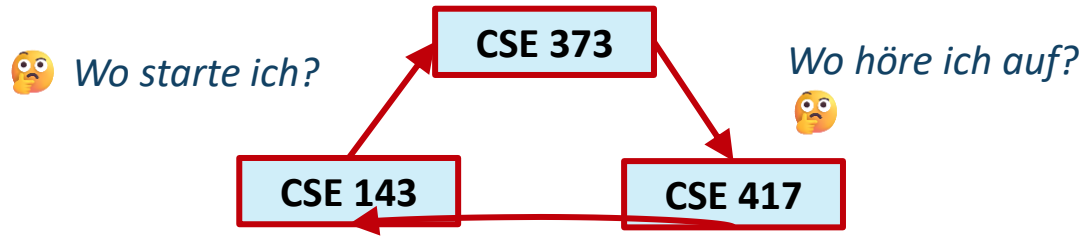
Nein 😭



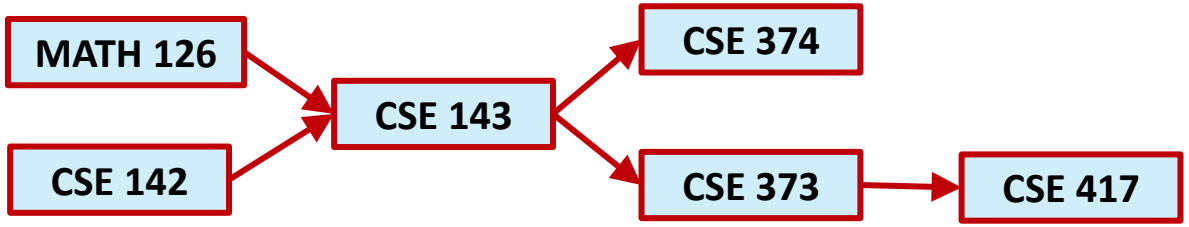
Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Topologische Sortierung funktioniert nur für DAGs

Können Sie diesen Graphen topologisch sortieren? Nein 😭



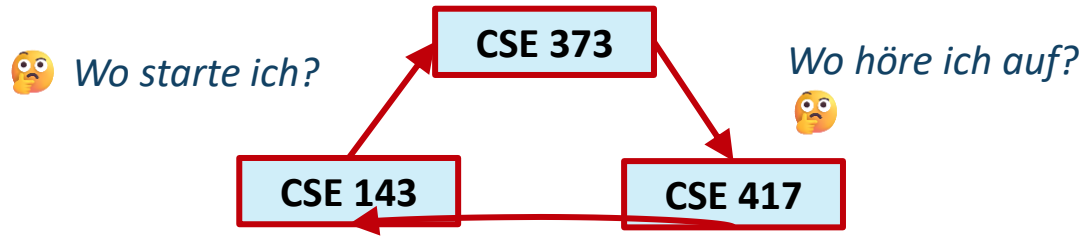
Was ist der Unterschied zwischen diesem Graphen und unserem ersten Graphen?



Topologische Sortierung funktioniert nur für DAGs

Können Sie diesen Graphen topologisch sortieren?

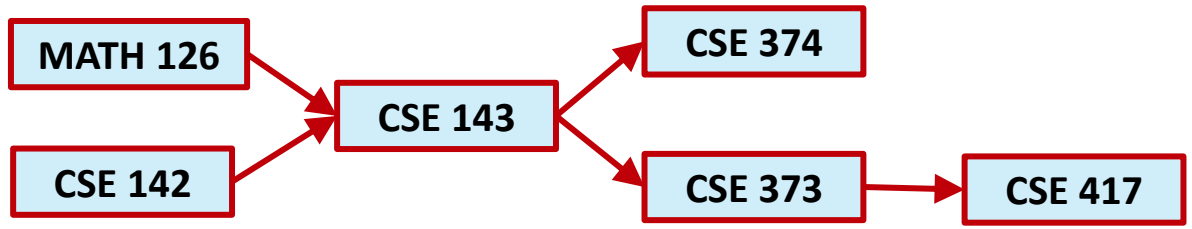
Nein 😭



GERICHTETER AZYKLISCHER GRAPH

- Ein gerichteter Graph ohne Zyklen
- Kanten können gewichtet oder ungewichtet sein

Was ist der Unterschied zwischen diesem Graphen und unserem ersten Graphen?



Ein Graph hat eine topologische Sortierung, wenn er ein DAG ist.

- Aber ein DAG (directed acyclic graph) kann mehrere topologische Sortierungen haben

Topologische Sortierung – Pseudocode – Teil 1

```
toposort(Graph graph) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();
  Map<Vertex, Integer> indegree = countInDegree(graph);

  for (Vertex v : indegree.keySet()) {
    if(indegree.get(v) == 0) {
      perimeter.add(v);
      visited.add(v);
    }
  }

continued on next slide
```

- Beginnen Sie mit dem BFS-Code
  - Warteschlange, um zu besuchende Nachbarn zu speichern
  - Menge, um besuchte Knoten zu markieren
-
-

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Topologische Sortierung – Pseudocode – Teil 1

```
toposort(Graph graph) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();
  Map<Vertex, Integer> indegree = countInDegree(graph);

  for (Vertex v : indegree.keySet()) {
    if(indegree.get(v) == 0) {
      perimeter.add(v);
      visited.add(v);
    }
  }

continued on next slide
```

- Beginnen Sie mit dem BFS-Code
  - Warteschlange, um zu besuchende Nachbarn zu speichern
  - Menge, um besuchte Knoten zu markieren
- Zählen des Eingangsgrades jedes Knotens
- Die zu besuchenden Knoten mit Eingangsgrad 0 in die Warteschlange stellen -> Startknoten

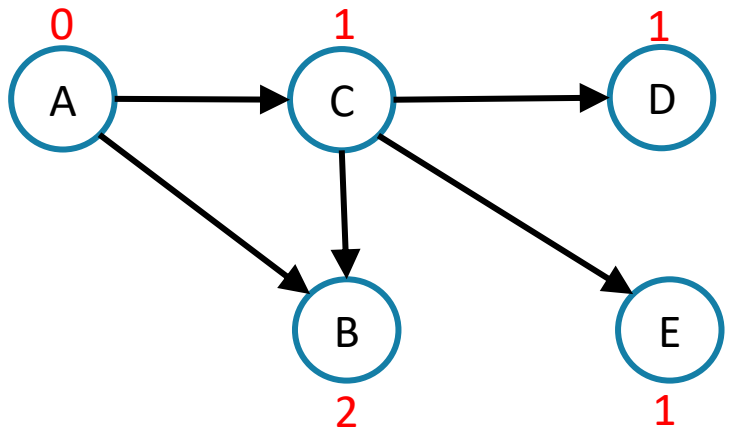
Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Topologische Sortierung – Pseudocode – Teil 1

```
toposort(Graph graph) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();
  Map<Vertex, Integer> indegree = countInDegree(graph);

  for (Vertex v : indegree.keySet()) {
    if(indegree.get(v) == 0) {
      perimeter.add(v);
      visited.add(v);
    }
  }

continued on next slide
```



perimeter:

visited:

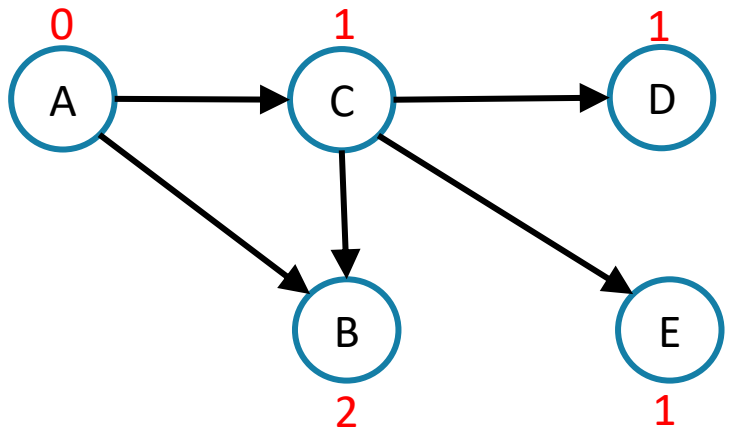
Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Topologische Sortierung – Pseudocode – Teil 1

```
toposort(Graph graph) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();
  Map<Vertex, Integer> indegree = countInDegree(graph);

  for (Vertex v : indegree.keySet()) {
    if(indegree.get(v) == 0) {
      perimeter.add(v);
      visited.add(v);
    }
  }

continued on next slide
```



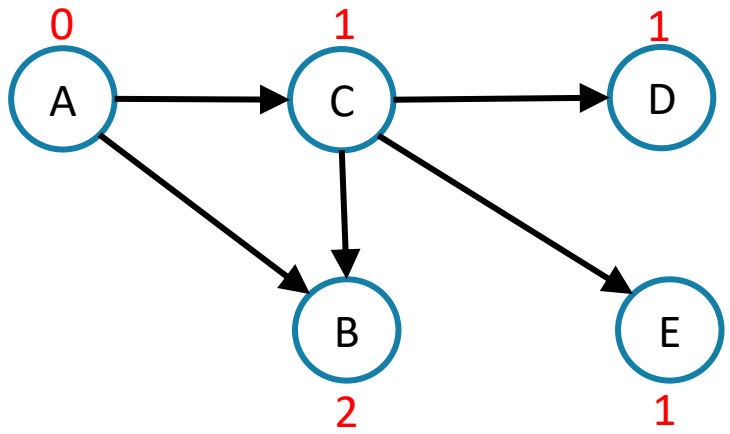
perimeter: **A**

visited: **A**

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Topologische Sortierung – Pseudocode – Teil 2

```
while (!perimeter.isEmpty()) {
  Vertex from = perimeter.remove();
  for (Edge edge : graph.edgesFrom(from)) {
    Vertex to = edge.to();
    if (!visited.contains(to)) {
      int inDeg = indegree.get(to);
      inDeg--;
      if (inDeg == 0) {
        perimeter.add(to);
        visited.add(to);
      }
      indegree.put(to, inDeg);
    }
```



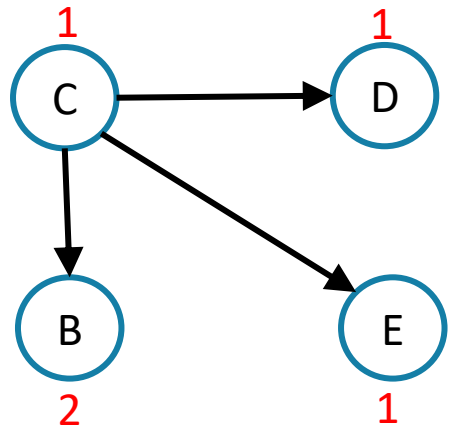
perimeter: **A**

visited: **A**

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Topologische Sortierung – Pseudocode – Teil 2

```
while (!perimeter.isEmpty()) {
  Vertex from = perimeter.remove();
  for (Edge edge : graph.edgesFrom(from)) {
    Vertex to = edge.to();
    if (!visited.contains(to)) {
      int inDeg = indegree.get(to);
      inDeg--;
      if (inDeg == 0) {
        perimeter.add(to);
        visited.add(to);
      }
      indegree.put(to, inDeg);
    }
```



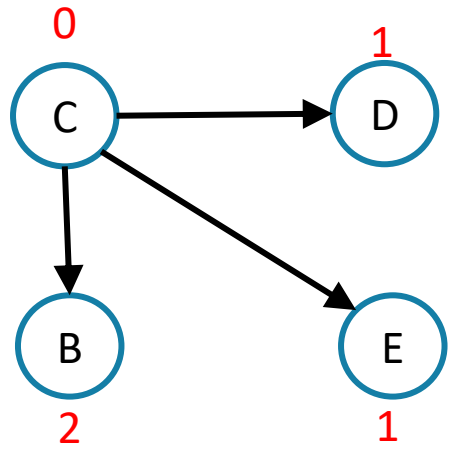
perimeter:

visited: A

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Topologische Sortierung – Pseudocode – Teil 2

```
while (!perimeter.isEmpty()) {
  Vertex from = perimeter.remove();
  for (Edge edge : graph.edgesFrom(from)) {
    Vertex to = edge.to();
    if (!visited.contains(to)) {
      int inDeg = indegree.get(to);
      inDeg--;
      if (inDeg == 0) {
        perimeter.add(to);
        visited.add(to);
      }
      indegree.put(to, inDeg);
    }
```



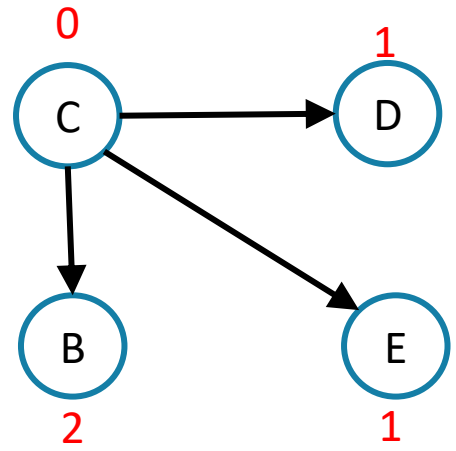
perimeter:

visited: A

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Topologische Sortierung – Pseudocode – Teil 2

```
while (!perimeter.isEmpty()) {
  Vertex from = perimeter.remove();
  for (Edge edge : graph.edgesFrom(from)) {
    Vertex to = edge.to();
    if (!visited.contains(to)) {
      int inDeg = indegree.get(to);
      inDeg--;
      if (inDeg == 0) {
        perimeter.add(to);
        visited.add(to);
      }
      indegree.put(to, inDeg);
    }
```



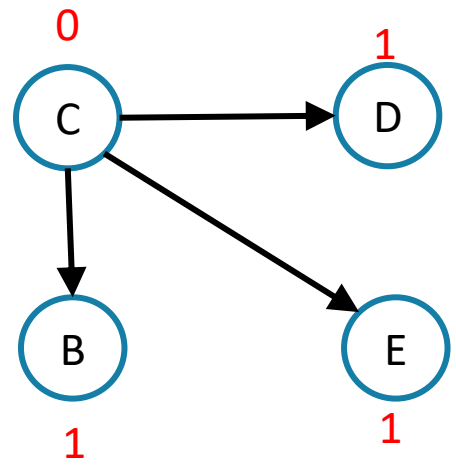
perimeter: C

visited: A C

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Topologische Sortierung – Pseudocode – Teil 2

```
while (!perimeter.isEmpty()) {
  Vertex from = perimeter.remove();
  for (Edge edge : graph.edgesFrom(from)) {
    Vertex to = edge.to();
    if (!visited.contains(to)) {
      int inDeg = indegree.get(to);
      inDeg--;
      if (inDeg == 0) {
        perimeter.add(to);
        visited.add(to);
      }
      indegree.put(to, inDeg);
    }
```



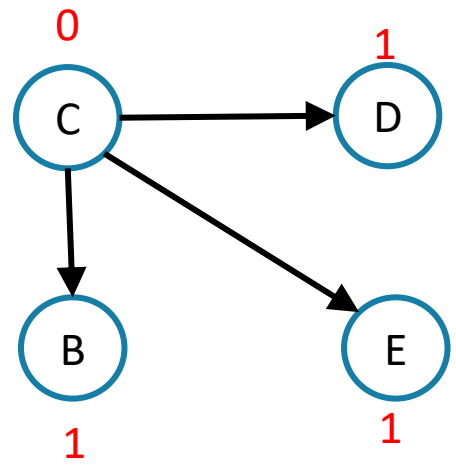
perimeter: C

visited: A C

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Topologische Sortierung – Pseudocode – Teil 2

```
while (!perimeter.isEmpty()) {
  Vertex from = perimeter.remove();
  for (Edge edge : graph.edgesFrom(from)) {
    Vertex to = edge.to();
    if (!visited.contains(to)) {
      int inDeg = indegree.get(to);
      inDeg--;
      if (inDeg == 0) {
        perimeter.add(to);
        visited.add(to);
      }
      indegree.put(to, inDeg);
    }
```



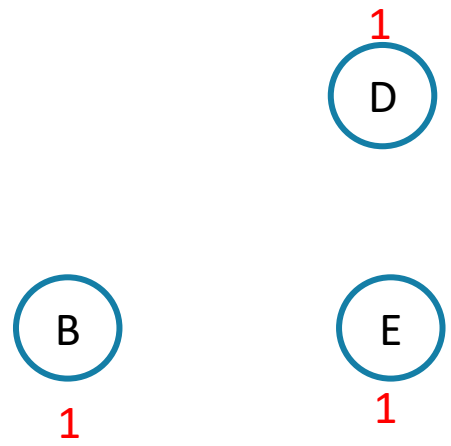
perimeter:

visited: A C

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Topologische Sortierung – Pseudocode – Teil 2

```
while (!perimeter.isEmpty()) {
  Vertex from = perimeter.remove();
  for (Edge edge : graph.edgesFrom(from)) {
    Vertex to = edge.to();
    if (!visited.contains(to)) {
      int inDeg = indegree.get(to);
      inDeg--;
      if (inDeg == 0) {
        perimeter.add(to);
        visited.add(to);
      }
      indegree.put(to, inDeg);
    }
```



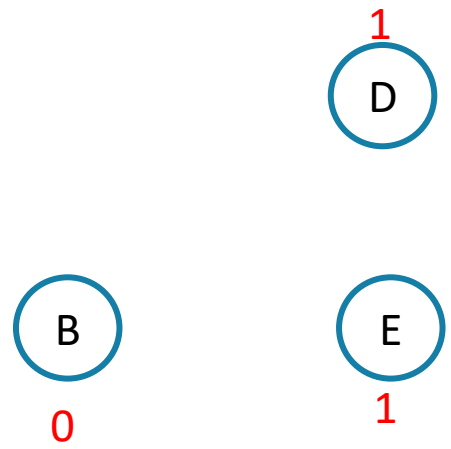
perimeter:

visited: A C

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Topologische Sortierung – Pseudocode – Teil 2

```
while (!perimeter.isEmpty()) {
  Vertex from = perimeter.remove();
  for (Edge edge : graph.edgesFrom(from)) {
    Vertex to = edge.to();
    if (!visited.contains(to)) {
      int inDeg = indegree.get(to);
      inDeg--;
      if (inDeg == 0) {
        perimeter.add(to);
        visited.add(to);
      }
      indegree.put(to, inDeg);
    }
```



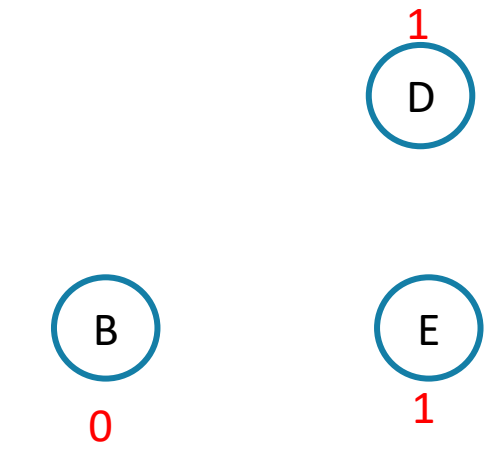
perimeter:

visited: A C

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Topologische Sortierung – Pseudocode – Teil 2

```
while (!perimeter.isEmpty()) {
  Vertex from = perimeter.remove();
  for (Edge edge : graph.edgesFrom(from)) {
    Vertex to = edge.to();
    if (!visited.contains(to)) {
      int inDeg = indegree.get(to);
      inDeg--;
      if (inDeg == 0) {
        perimeter.add(to);
        visited.add(to);
      }
      indegree.put(to, inDeg);
    }
```



perimeter: **B**

visited: **A C B**

Prof. Dr. Daniel Gaida

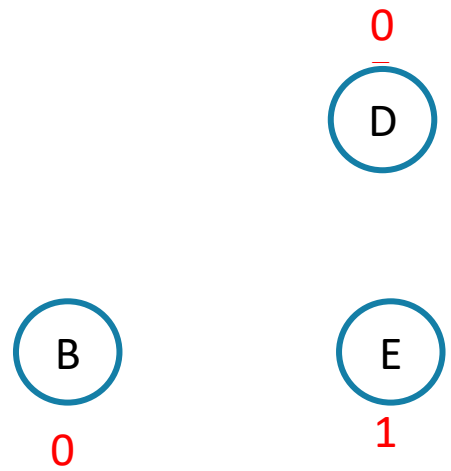
Professor für Cyber-Physische Systeme

Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Topologische Sortierung – Pseudocode – Teil 2

```
while (!perimeter.isEmpty()) {
  Vertex from = perimeter.remove();
  for (Edge edge : graph.edgesFrom(from)) {
    Vertex to = edge.to();
    if (!visited.contains(to)) {
      int inDeg = indegree.get(to);
      inDeg--;
      if (inDeg == 0) {
        perimeter.add(to);
        visited.add(to);
      }
      indegree.put(to, inDeg);
    }
```



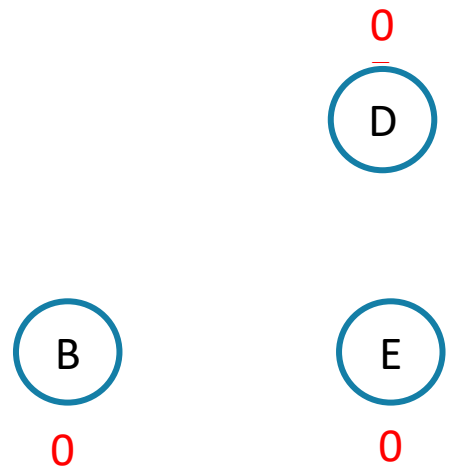
perimeter: B D

visited: A C B D

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Topologische Sortierung – Pseudocode – Teil 2

```
while (!perimeter.isEmpty()) {
  Vertex from = perimeter.remove();
  for (Edge edge : graph.edgesFrom(from)) {
    Vertex to = edge.to();
    if (!visited.contains(to)) {
      int inDeg = indegree.get(to);
      inDeg--;
      if (inDeg == 0) {
        perimeter.add(to);
        visited.add(to);
      }
      indegree.put(to, inDeg);
    }
```



perimeter: B D E

visited: A C B D E

Prof. Dr. Daniel Gaida

Professor für Cyber-Physische Systeme

Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Topologische Sortierung – Pseudocode – Teil 2

```
while (!perimeter.isEmpty()) {
  Vertex from = perimeter.remove();
  for (Edge edge : graph.edgesFrom(from)) {
    Vertex to = edge.to();
    if (!visited.contains(to)) {
      int inDeg = indegree.get(to);
      inDeg--;
      if (inDeg == 0) {
        perimeter.add(to);
        visited.add(to);
      }
      indegree.put(to, inDeg);
    }
```



perimeter: D E

visited: A C B D E

Prof. Dr. Daniel Gaida

Professor für Cyber-Physische Systeme

Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Topologische Sortierung – Pseudocode – Teil 2

```
while (!perimeter.isEmpty()) {
  Vertex from = perimeter.remove();
  for (Edge edge : graph.edgesFrom(from)) {
    Vertex to = edge.to();
    if (!visited.contains(to)) {
      int inDeg = indegree.get(to);
      inDeg--;
      if (inDeg == 0) {
        perimeter.add(to);
        visited.add(to);
      }
      indegree.put(to, inDeg);
    }
```

perimeter:

visited: A C B D E

Prof. Dr. Daniel Gaida

Professor für Cyber-Physische Systeme

Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Topologische Sortierung – Pseudocode – Teil 2

```
while (!perimeter.isEmpty()) {
  Vertex from = perimeter.remove();
  for (Edge edge : graph.edgesFrom(from)) {
    Vertex to = edge.to();
    if (!visited.contains(to)) {
      int inDeg = indegree.get(to);
      inDeg--;
      if (inDeg == 0) {
        perimeter.add(to);
        visited.add(to);
      }
      indegree.put(to, inDeg);
    }
```

perimeter:

visited: A C B D E

Prof. Dr. Daniel Gaida

Professor für Cyber-Physische Systeme

Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Topologische Sortierung – Pseudocode – Teil 2

```
while (!perimeter.isEmpty()) {
  Vertex from = perimeter.remove();
  for (Edge edge : graph.edgesFrom(from)) {
    Vertex to = edge.to();
    if (!visited.contains(to)) {
      int inDeg = indegree.get(to);
      inDeg--;
      if (inDeg == 0) {
        perimeter.add(to);
        visited.add(to);
      }
      indegree.put(to, inDeg);
    }
```

Schleife über Warteschlange

- für jeden Nachbarn eines besuchten Knotens seinen Eingangsgrad reduzieren
- Knoten, die den Eingangsgrad 0 erreichen, zur Warteschlange hinzufügen

Prof. Dr. Daniel Gaida

Professor für Cyber-Physische Systeme

Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Topologische Sortierung – Pseudocode – Teil 2

```
while (!perimeter.isEmpty()) {
  Vertex from = perimeter.remove();
  for (Edge edge : graph.edgesFrom(from)) {
    Vertex to = edge.to();
    if (!visited.contains(to)) {
      int inDeg = indegree.get(to);
      inDeg--;
      if (inDeg == 0) {
        perimeter.add(to);
        visited.add(to);
      }
      indegree.put(to, inDeg);
    }
```

Schleife über Warteschlange

- für jeden Nachbarn eines besuchten Knotens seinen Eingangsgrad reduzieren
- Knoten, die den Eingangsgrad 0 erreichen, zur Warteschlange hinzufügen

Topologische Sortierung ist die Reihenfolge, in der die Knoten „besucht“ werden

- man könnte eine separate Liste erstellen, um die Reihenfolge zu verfolgen
- man könnte sie ausdrucken, wenn man sie der Menge visited hinzufügt

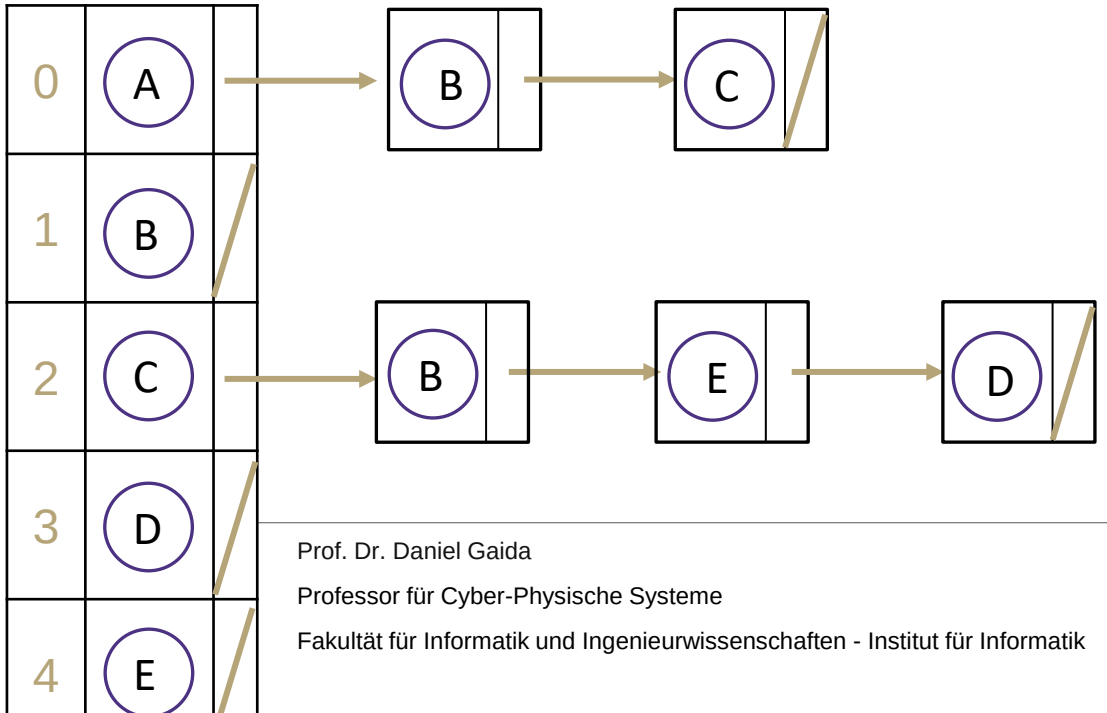
Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Topologische Sortierung – Laufzeit

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

```
toposort(Graph graph) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();
  Map<Vertex, Integer> indegree = countInDegree(graph);
```

Verkettete Liste



Laufzeit?

- Annahme: Implementierung als Adjazenzliste mit verketteten Listen
-
  -
  -
    -
  -
    -

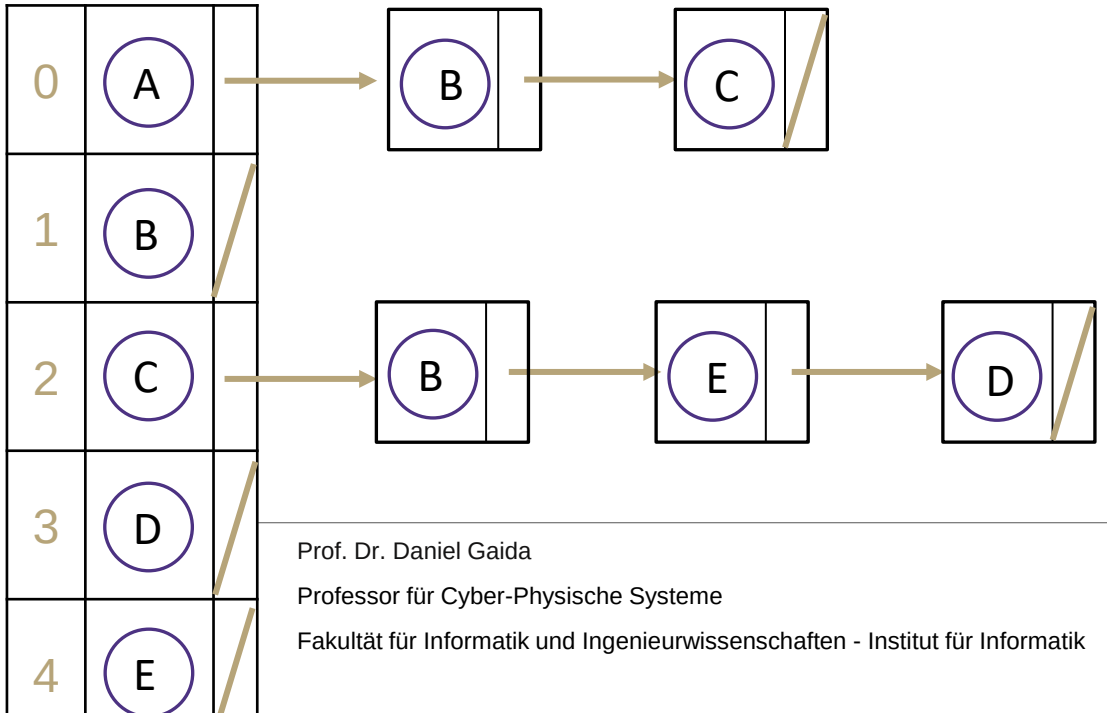
Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Topologische Sortierung – Laufzeit

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

```
toposort(Graph graph) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();
  Map<Vertex, Integer> indegree = countInDegree(graph);
```

Verkettete Liste



Laufzeit?

- Annahme: Implementierung als Adjazenzliste mit verketteten Listen
- Laufzeit von countInDegree:
  -
  -
    -
  -
  -
    -

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

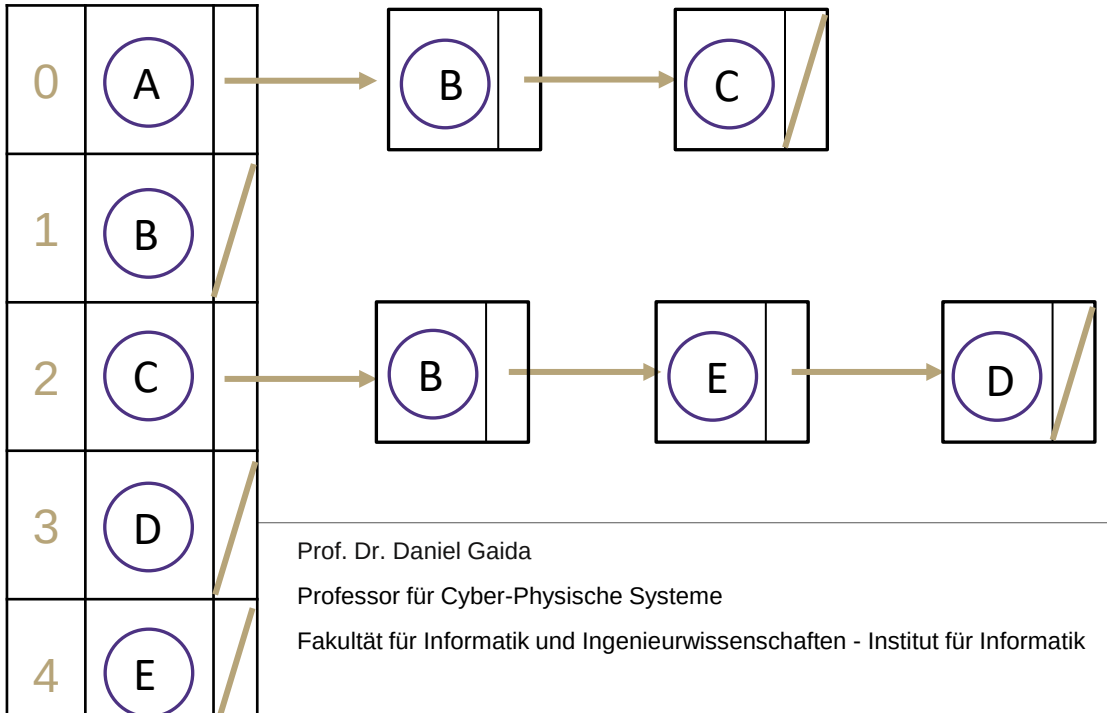
Technology
Arts Sciences
TH Köln

Topologische Sortierung – Laufzeit

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

```
toposort(Graph graph) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();
  Map<Vertex, Integer> indegree = countInDegree(graph);
```

Verkettete Liste



Laufzeit?

- Annahme: Implementierung als Adjazenzliste mit verketteten Listen
- Laufzeit von countInDegree:
  - Erstelle Hashtabelle indegree; Init: 0
  - Durchlaufen der verketteten Listen der Adjazenzliste:
    -
  -
    -

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

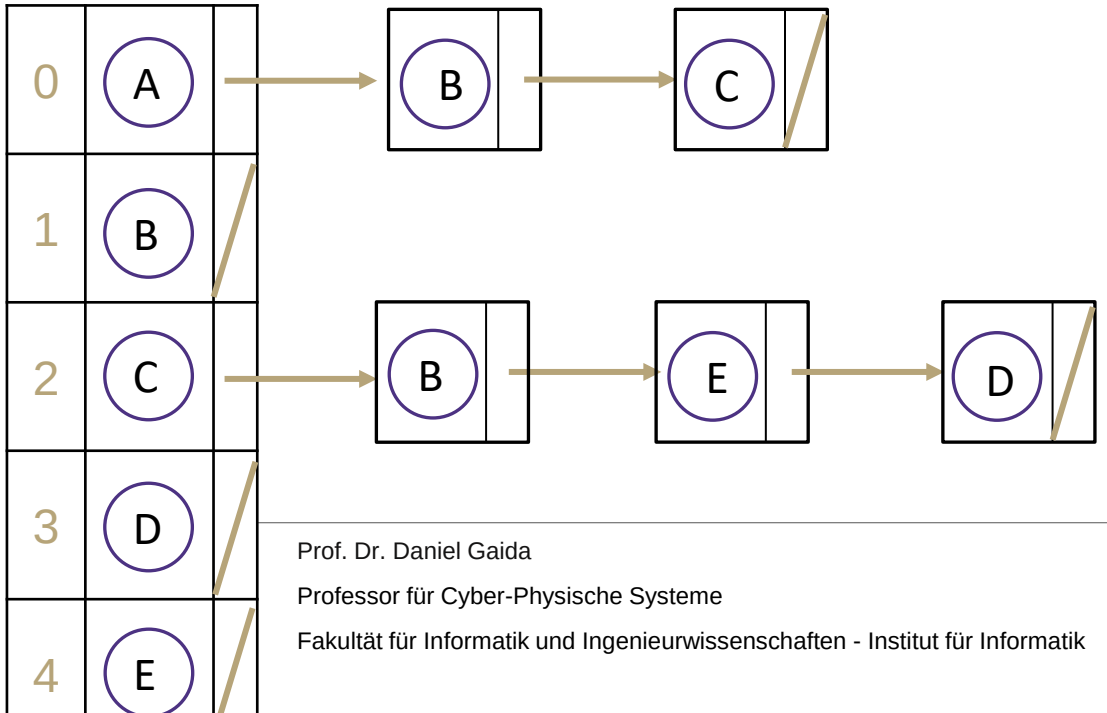
Technology
Arts Sciences
TH Köln

Topologische Sortierung – Laufzeit

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

```
toposort(Graph graph) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();
  Map<Vertex, Integer> indegree = countInDegree(graph);
```

Verkettete Liste



Laufzeit?

- Annahme: Implementierung als Adjazenzliste mit verketteten Listen
- Laufzeit von countInDegree:
  - Erstelle Hashtabelle indegree; Init: 0
  - Durchlaufen der verketteten Listen der Adjazenzliste:
    - Für jeden Knoten: Erhöhe indegree um 1, wenn Knoten bereits in indegree vorhanden, ansonsten initialisiere auf 1
  -
    -

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

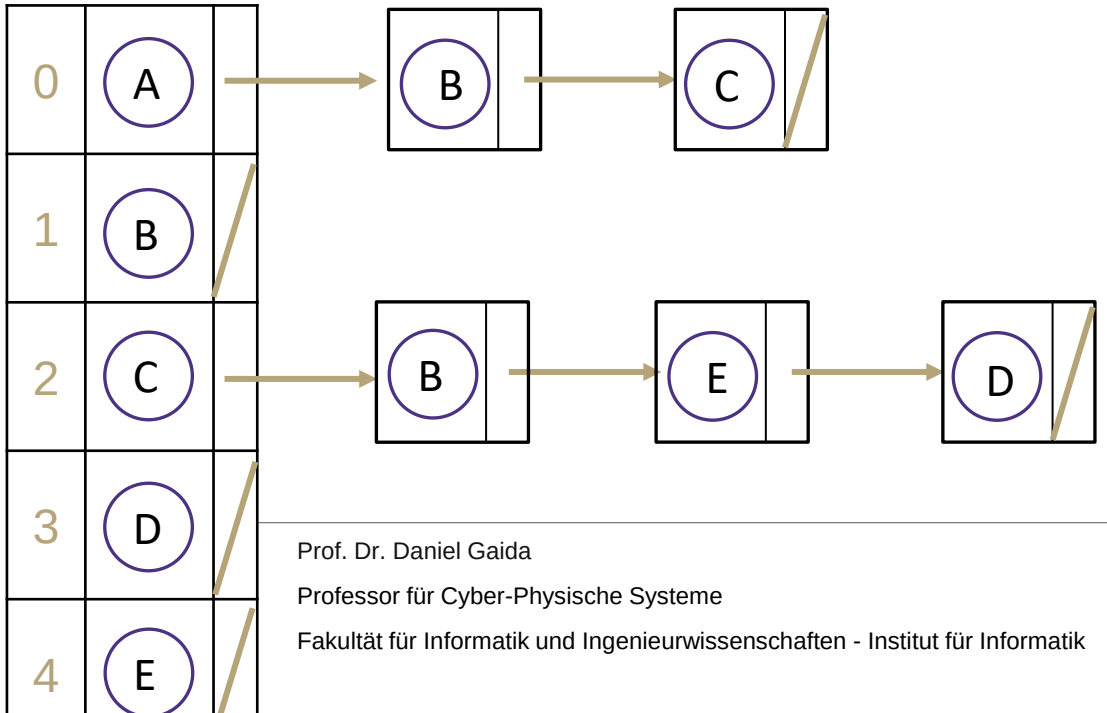
Technology
Arts Sciences
TH Köln

Topologische Sortierung – Laufzeit

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

```
toposort(Graph graph) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();
  Map<Vertex, Integer> indegree = countInDegree(graph);
```

Verkettete Liste



Laufzeit?

- Annahme: Implementierung als Adjazenzliste mit verketteten Listen
- Laufzeit von countInDegree:
  - Erstelle Hashtabelle indegree; Init: 0
  - Durchlaufen der verketteten Listen der Adjazenzliste:
    - Für jeden Knoten: Erhöhe indegree um 1, wenn Knoten bereits in indegree vorhanden, ansonsten initialisiere auf 1
  - Laufzeit: Durchlaufen des Arrays der Knoten und aller verketteten Listen:
    - O(n + m)

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Topologische Sortierung – Laufzeit

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

```
toposort(Graph graph) {
  Queue<Vertex> perimeter = new Queue<>();
  Set<Vertex> visited = new Set<>();
  Map<Vertex, Integer> indegree = countInDegree(graph);

  for (Vertex v : indegree.keySet()) {
    if(indegree.get(v) == 0) {
      perimeter.add(v);
      visited.add(v);
    }
  }

continued on next slide
```

Laufzeit?

- Annahme: Implementierung als Adjazenzliste mit verketteten Listen
- Laufzeit von countInDegree:
  - Erstelle Hashtabelle indegree
  - Durchlaufen der verketteten Listen der Adjazenzliste:
    - Für jeden Knoten: Erhöhe indegree um 1, wenn Knoten bereits in indegree vorhanden, ansonsten initialisiere auf 1
  - Laufzeit: Durchlaufen des Arrays der Knoten und aller verketteten Listen:
    - O(n + m)
  - Startknoten bestimmen: O(n)

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Technology Arts Sciences TH Köln

Topologische Sortierung – Laufzeit

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

```
while (!perimeter.isEmpty()) {
  Vertex from = perimeter.remove();
  for (Edge edge : graph.edgesFrom(from)) {
    Vertex to = edge.to();
    if (!visited.contains(to)) {
      int inDeg = indegree.get(to);
      inDeg--;
      if (inDeg == 0) {
        perimeter.add(to);
        visited.add(to);
      }
      indegree.put(to, inDeg);
    }
```

Laufzeit?

- Annahme: Implementierung als Adjazenzliste mit verketteten Listen
- Laufzeit von countInDegree und Startknoten bestimmen:
  - O(m + n) + O(n) → O(n + m)
-
  -
  -
  -
  -
-
-

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Technology Arts Sciences TH Köln

Topologische Sortierung – Laufzeit

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

```
while (!perimeter.isEmpty()) {
  Vertex from = perimeter.remove();
  for (Edge edge : graph.edgesFrom(from)) {
    Vertex to = edge.to();
    if (!visited.contains(to)) {
      int inDeg = indegree.get(to);
      inDeg--;
      if (inDeg == 0) {
        perimeter.add(to);
        visited.add(to);
      }
      indegree.put(to, inDeg);
    }
```

Laufzeit?

- Annahme: Implementierung als Adjazenzliste mit verketteten Listen
- Laufzeit von countInDegree und Startknoten bestimmen:
  - O(m + n) + O(n) → O(n + m)
- While Schleife läuft über n Knoten:
  - Element aus Warteschlange entfernen: O(1)
  - visited.contains: O(1) (Hashtabelle)
  -
  -
-
-

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Technology Arts Sciences TH Köln

Topologische Sortierung – Laufzeit

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

```
while (!perimeter.isEmpty()) {
  Vertex from = perimeter.remove();
  for (Edge edge : graph.edgesFrom(from)) {
    Vertex to = edge.to();
    if (!visited.contains(to)) {
      int inDeg = indegree.get(to);
      inDeg--;
      if (inDeg == 0) {
        perimeter.add(to);
        visited.add(to);
      }
      indegree.put(to, inDeg);
    }
```

Laufzeit?

- Annahme: Implementierung als Adjazenzliste mit verketteten Listen
- Laufzeit von countInDegree und Startknoten bestimmen:
  - O(m + n) + O(n) → O(n + m)
- While Schleife läuft über n Knoten:
  - Element aus Warteschlange entfernen: O(1)
  - visited.contains: O(1) (Hashtabelle)
  - indegree.get: O(1)
  - Add und put Operationen: O(1)
-
-

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Technology Arts Sciences TH Köln

Topologische Sortierung – Laufzeit

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

```
while (!perimeter.isEmpty()) {
  Vertex from = perimeter.remove();
  for (Edge edge : graph.edgesFrom(from)) {
    Vertex to = edge.to();
    if (!visited.contains(to)) {
      int inDeg = indegree.get(to);
      inDeg--;
      if (inDeg == 0) {
        perimeter.add(to);
        visited.add(to);
      }
      indegree.put(to, inDeg);
    }
```

Laufzeit?

- Annahme: Implementierung als Adjazenzliste mit verketteten Listen
- Laufzeit von countInDegree und Startknoten bestimmen:
  - O(m + n) + O(n) → O(n + m)
- While Schleife läuft über n Knoten:
  - Element aus Warteschlange entfernen: O(1)
  - visited.contains: O(1) (Hashtabelle)
  - indegree.get: O(1)
  - Add und put Operationen: O(1)
- → n mal O(deg(u)) = O(m + n)
- → Gesamt: O(m + n)

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Technology Arts Sciences TH Köln

Starke Zusammenhangskomponente und DAG

Anwendung in Logistik



Starke Zusammenhangskomponente und DAG

Anwendung in Logistik

- Stark zusammenhängende Komponenten (SCCs): Innerhalb einer Komponente ist jeder Knoten von jedem anderen erreichbar.
- Kondensation: Jede SCC wird zu einem Super-Knoten zusammengefasst.
- Zentrale Eigenschaft: Der Kondensationsgraph ist immer ein DAG (Directed Acyclic Graph).
- Effiziente Algorithmen: Topologische Sortierung und kürzeste Wege im DAG sind in (O(|V|+|E|)) berechenbar.
- Hierarchische Routenplanung: Zuerst wird die Route zwischen Komponenten bestimmt, anschließend der konkrete Weg innerhalb der Komponenten.



Fragen?

Relevante Ideen bis hierhin

- Topologische Sortierung
  - Gibt es nur für einen gerichteten azyklischen Graphen
  - Eine Anordnung, die Abhängigkeiten von Knoten berücksichtigt

- Übungsblatt 7: Aufgabe 12

Übersicht über heute

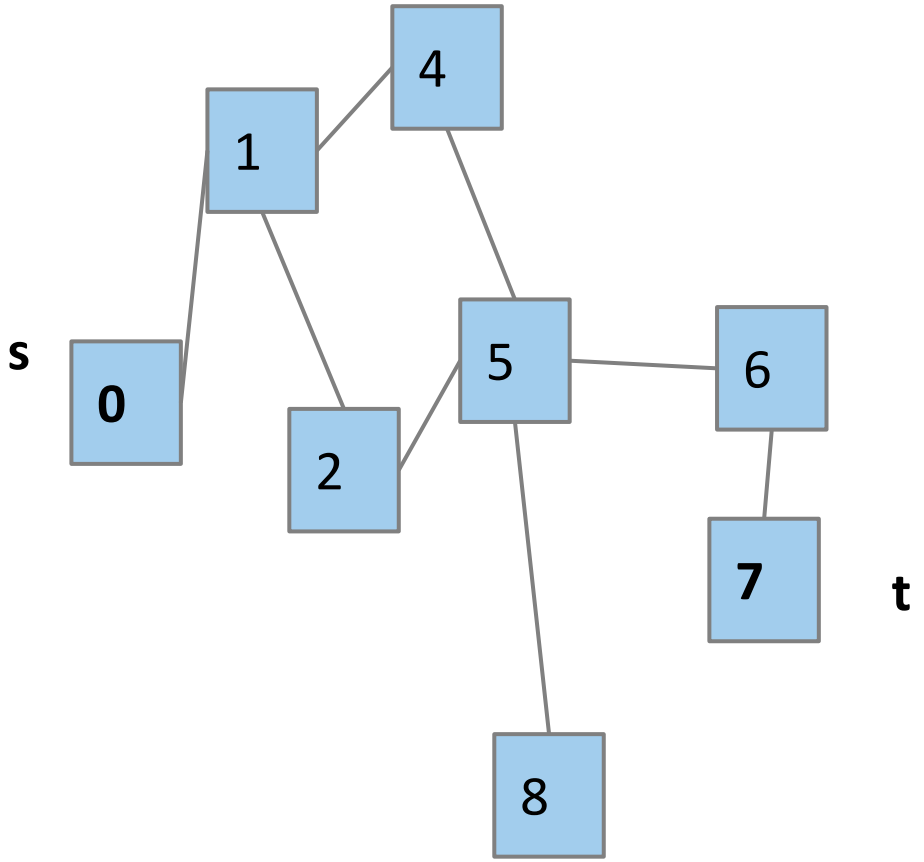
- Graphen
  - Einführung/Anwendungen/Begriffe
- Traversierung von Graphen
  - Erreichbarkeitsproblem
  - Tiefensuche
  - Breitensuche
- Gerichtete Graphen
  - Topologische Sortierung
- **Kürzeste Pfade**
  - Ungewichteter Graph: Breitensuche
  - Gewichteter Graph: Algorithmus von Dijkstra

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Technology Arts Sciences TH Köln

Ungewichteter kürzester Pfad

Ungewichteter kürzester Pfad

Wie lang ist der kürzeste Pfad von einem Start- s zu einem Zielknoten t? Aus welchen Kanten setzt sich dieser Pfad zusammen?



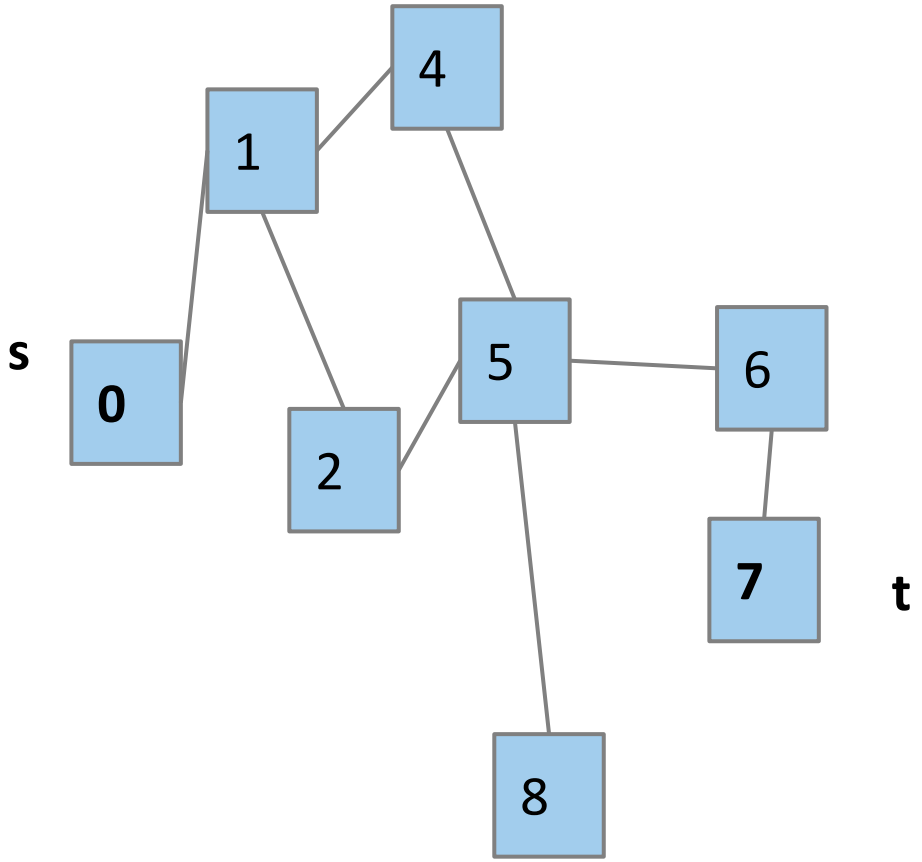
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Ungewichteter kürzester Pfad

Ungewichteter kürzester Pfad

Wie lang ist der kürzeste Pfad von einem Start- s zu einem Zielknoten t? Aus welchen Kanten setzt sich dieser Pfad zusammen?

Mit welchem Algorithmus können wir bestimmen wie weit jeder Knoten vom Startknoten entfernt ist?

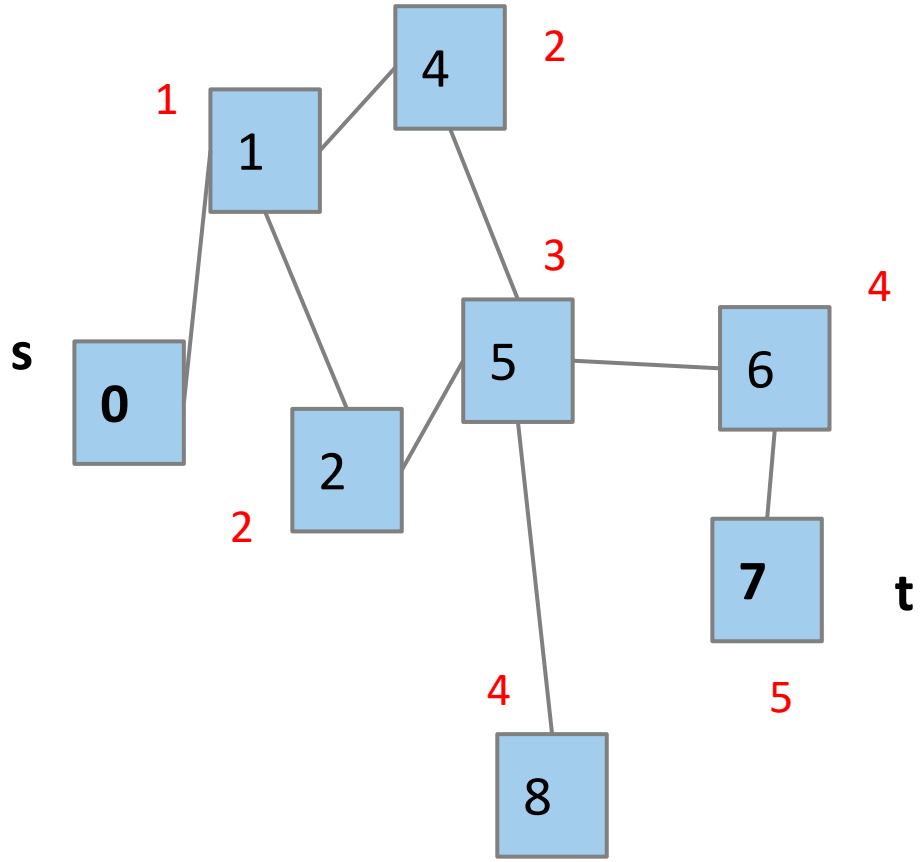


Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

BFS zur Suche des ungewichteten kürzesten Pfads

Ungewichteter kürzester Pfad

Wie lang ist der kürzeste Pfad von einem Start- s zu einem Zielknoten t? Aus welchen Kanten setzt sich dieser Pfad zusammen?



Mit welchem Algorithmus können wir bestimmen wie weit jeder Knoten vom Startknoten entfernt ist?

- Breitensuche (BFS)!

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

BFS zur Suche des ungewichteten kürzesten Pfads

Ungewichteter kürzester Pfad

Wie lang ist der kürzeste Pfad von einem Start- s zu einem Zielknoten t? Aus welchen Kanten setzt sich dieser Pfad zusammen?

Mit welchem Algorithmus können wir bestimmen wie weit jeder Knoten vom Startknoten entfernt ist?

- Breitensuche (BFS)!

```
perimeter.add(start);
visited.add(start);
```

```
...
Map<Vertex, Edge> edgeTo = ...
Map<Vertex, Integer> distTo = ...

edgeTo.put(start, null);
distTo.put(start, 0);

while (!perimeter.isEmpty()) {
  Vertex from = perimeter.remove();
  for (Edge edge : graph.edgesFrom(from)) {
    Vertex to = edge.to();
    if (!visited.contains(to)) {
      edgeTo.put(to, edge);
      distTo.put(to, distTo.get(from) + 1);
      perimeter.add(to);
      visited.add(to);
    }
  }
}
return edgeTo, distTo;
}
```

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

BFS zur Suche des ungewichteten kürzesten Pfads

Ungewichteter kürzester Pfad

Wie lang ist der kürzeste Pfad von einem Start- s zu einem Zielknoten t? Aus welchen Kanten setzt sich dieser Pfad zusammen?

Mit welchem Algorithmus können wir bestimmen wie weit jeder Knoten vom Startknoten entfernt ist?

- Breitensuche (BFS)!

```
perimeter.add(start);
visited.add(start);
```

```
...
Map<Vertex, Edge> edgeTo = ...
Map<Vertex, Integer> distTo = ...

edgeTo.put(start, null);
distTo.put(start, 0);

while (!perimeter.isEmpty()) {
  Vertex from = perimeter.remove();
  for (Edge edge : graph.edgesFrom(from)) {
    Vertex to = edge.to();
    if (!visited.contains(to)) {
      edgeTo.put(to, edge);
      distTo.put(to, distTo.get(from) + 1);
      perimeter.add(to);
      visited.add(to);
    }
  }
}
return edgeTo, distTo;
}
```

Keine Kante benötigt, um zu start Knoten zu gelangen. Er liegt auf Ebene 0

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

BFS zur Suche des ungewichteten kürzesten Pfads

Ungewichteter kürzester Pfad

Wie lang ist der kürzeste Pfad von einem Start- s zu einem Zielknoten t? Aus welchen Kanten setzt sich dieser Pfad zusammen?

Mit welchem Algorithmus können wir bestimmen wie weit jeder Knoten vom Startknoten entfernt ist?

- Breitensuche (BFS)!

```
perimeter.add(start);
visited.add(start);
```

```
...
Map<Vertex, Edge> edgeTo = ...
Map<Vertex, Integer> distTo = ...

edgeTo.put(start, null);
distTo.put(start, 0);

while (!perimeter.isEmpty()) {
  Vertex from = perimeter.remove();
  for (Edge edge : graph.edgesFrom(from)) {
    Vertex to = edge.to();
    if (!visited.contains(to)) {
      edgeTo.put(to, edge);
      distTo.put(to, distTo.get(from) + 1);
      perimeter.add(to);
      visited.add(to);
    }
  }
}
return edgeTo, distTo;
}
```

Keine Kante benötigt, um zu start Knoten zu gelangen. Er liegt auf Ebene 0

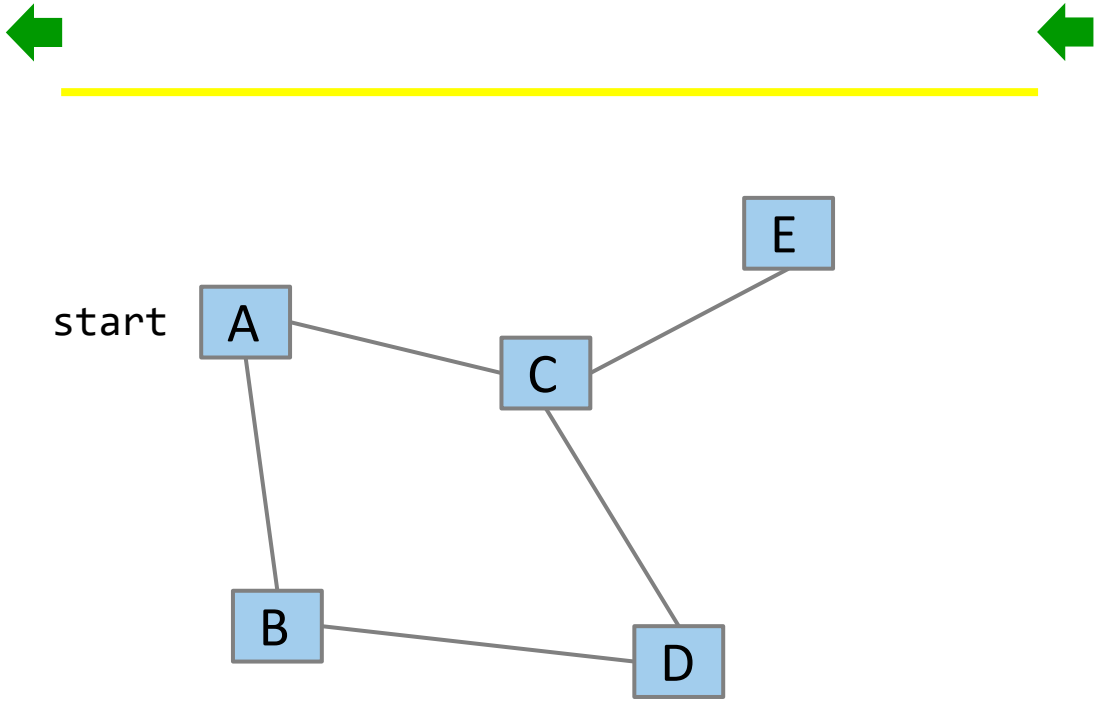
Speichern, wie wir zu diesem Knoten gekommen sind und zu welcher Ebene dieser gehört

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

BFS zur Suche des ungewichteten kürzesten Pfads

Beispiel

PERIMETER




```
perimeter.add(start);
visited.add(start);
```

```
  ...
  Map<Vertex, Edge> edgeTo = ...
  Map<Vertex, Integer> distTo = ...

  edgeTo.put(start, null);
  distTo.put(start, 0);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        edgeTo.put(to, edge);
        distTo.put(to, distTo.get(from) + 1);
        perimeter.add(to);
        visited.add(to);
      }
    }
  }
  return edgeTo, distTo;
}
```


Prof. Dr. Daniel Gaida

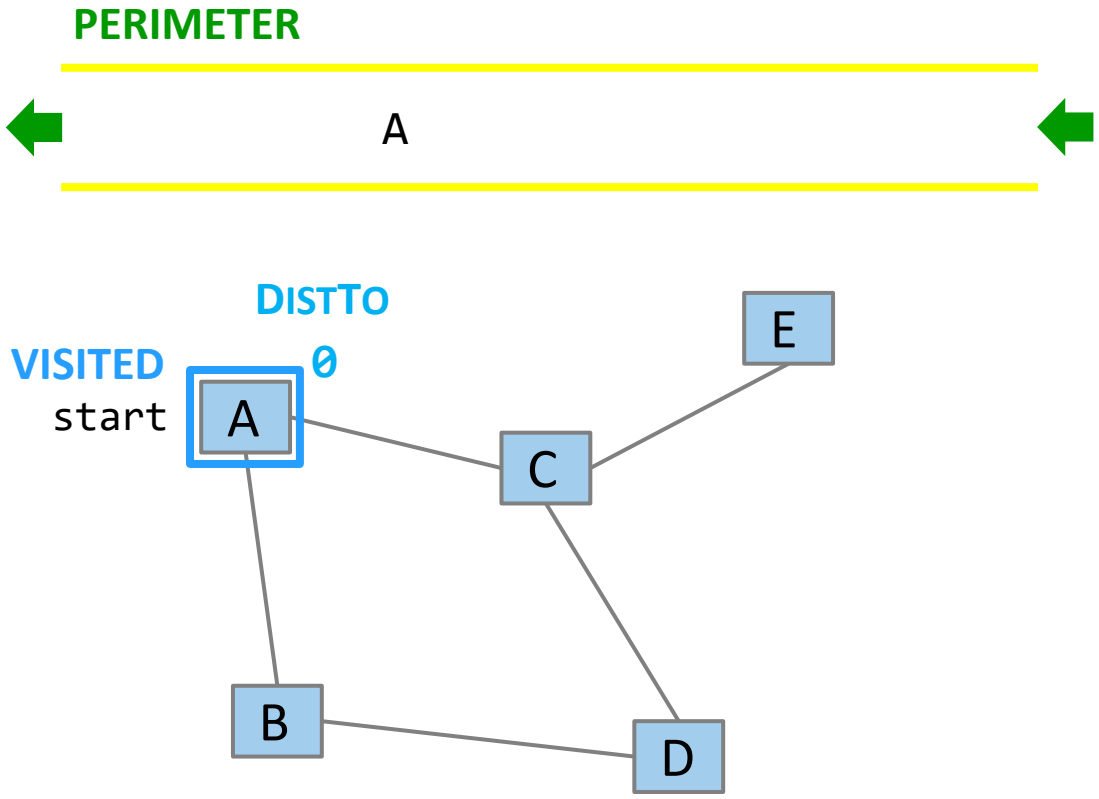
Professor für Cyber-Physische Systeme

Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

BFS zur Suche des ungewichteten kürzesten Pfads

Beispiel




```
perimeter.add(start);
visited.add(start);
```

```
  ...
  Map<Vertex, Edge> edgeTo = ...
  Map<Vertex, Integer> distTo = ...

  edgeTo.put(start, null);
  distTo.put(start, 0);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        edgeTo.put(to, edge);
        distTo.put(to, distTo.get(from) + 1);
        perimeter.add(to);
        visited.add(to);
      }
    }
  }
  return edgeTo, distTo;
}
```


Prof. Dr. Daniel Gaida

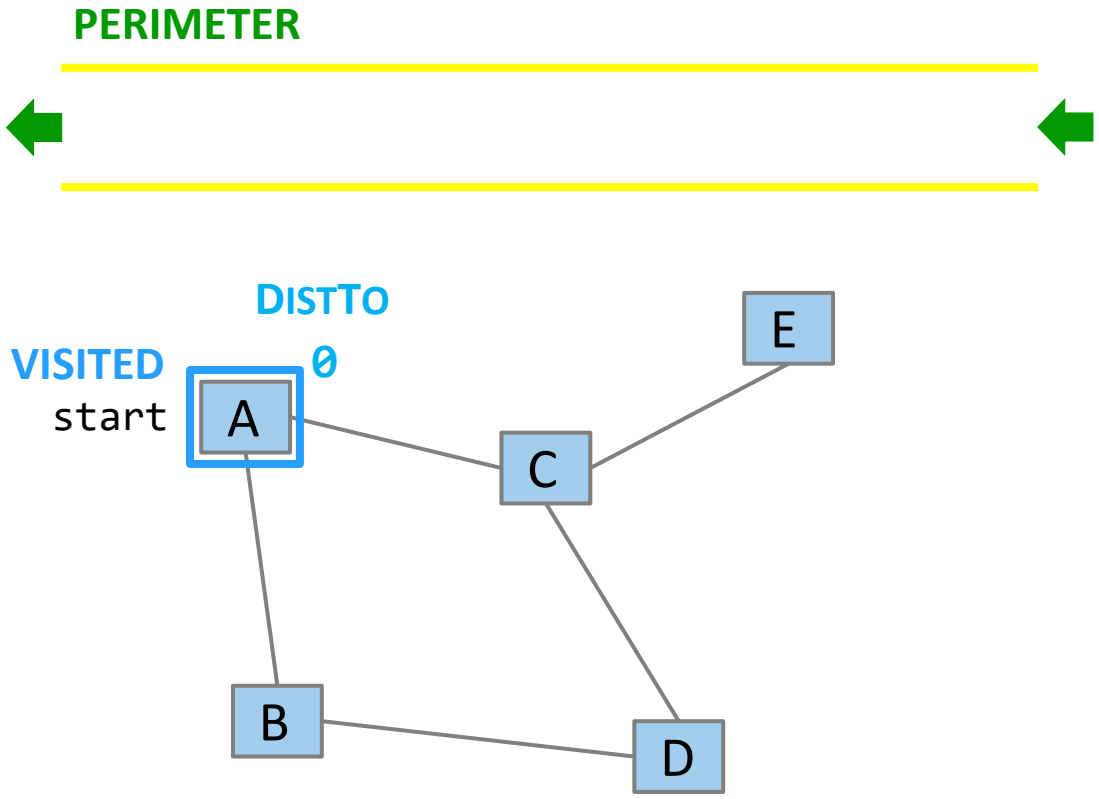
Professor für Cyber-Physische Systeme

Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

BFS zur Suche des ungewichteten kürzesten Pfads

Beispiel




```
perimeter.add(start);
visited.add(start);
```

```
  ...
  Map<Vertex, Edge> edgeTo = ...
  Map<Vertex, Integer> distTo = ...

  edgeTo.put(start, null);
  distTo.put(start, 0);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        edgeTo.put(to, edge);
        distTo.put(to, distTo.get(from) + 1);
        perimeter.add(to);
        visited.add(to);
      }
    }
  }
  return edgeTo, distTo;
}
```


Prof. Dr. Daniel Gaida

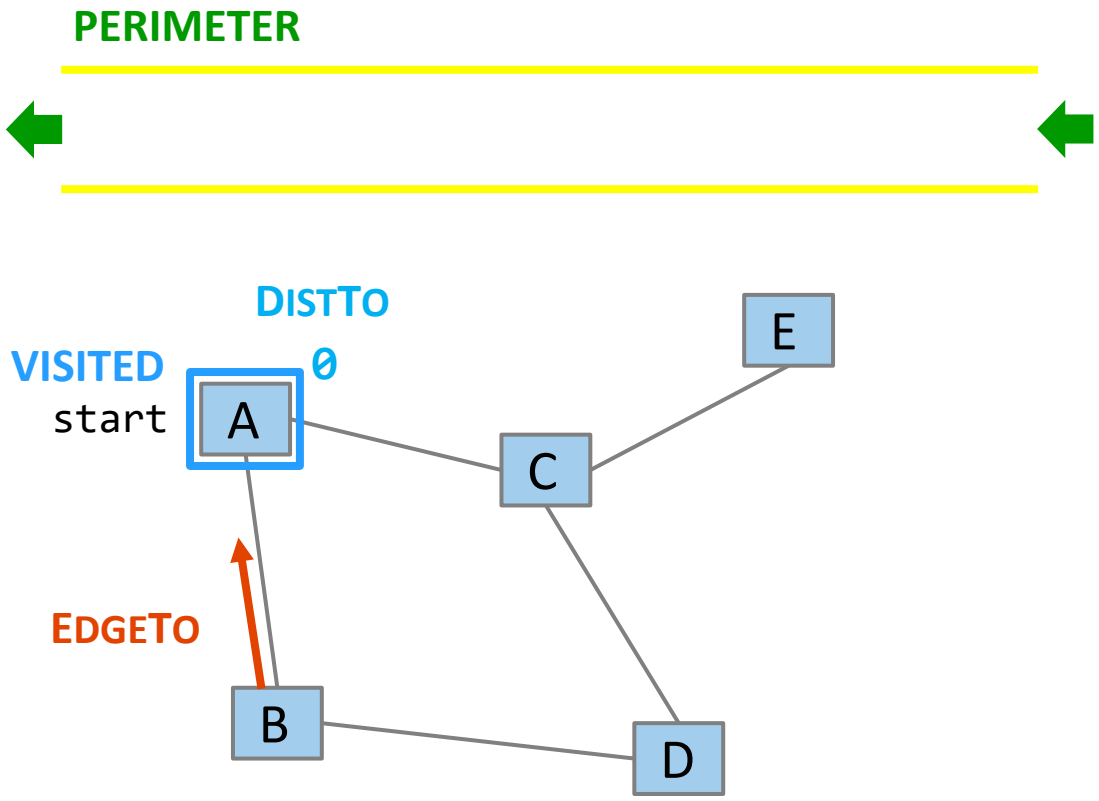
Professor für Cyber-Physische Systeme

Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

BFS zur Suche des ungewichteten kürzesten Pfads

Beispiel



Die edgeTo-Map speichert Backpointer: Jeder Knoten merkt sich, welcher Knoten benutzt wurde, um ihn zu erreichen!

```
perimeter.add(start);
visited.add(start);
```

```
...
Map<Vertex, Edge> edgeTo = ...
Map<Vertex, Integer> distTo = ...

edgeTo.put(start, null);
distTo.put(start, 0);

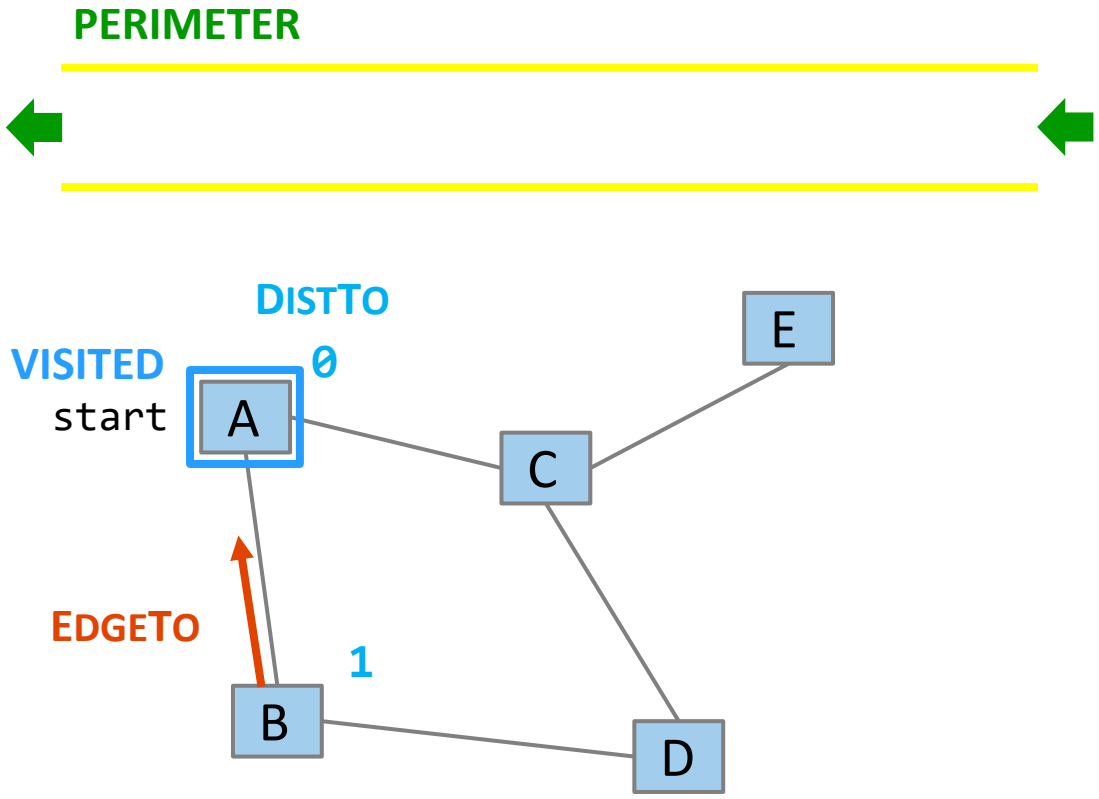
while (!perimeter.isEmpty()) {
  Vertex from = perimeter.remove();
  for (Edge edge : graph.edgesFrom(from)) {
    Vertex to = edge.to();
    if (!visited.contains(to)) {
      edgeTo.put(to, edge);
      distTo.put(to, distTo.get(from) + 1);
      perimeter.add(to);
      visited.add(to);
    }
  }
}
return edgeTo, distTo;
}
```


Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

BFS zur Suche des ungewichteten kürzesten Pfads

Beispiel



Die edgeTo-Map speichert Backpointer: Jeder Knoten merkt sich, welcher Knoten benutzt wurde, um ihn zu erreichen!

```
perimeter.add(start);
visited.add(start);
```

```
...
Map<Vertex, Edge> edgeTo = ...
Map<Vertex, Integer> distTo = ...

edgeTo.put(start, null);
distTo.put(start, 0);

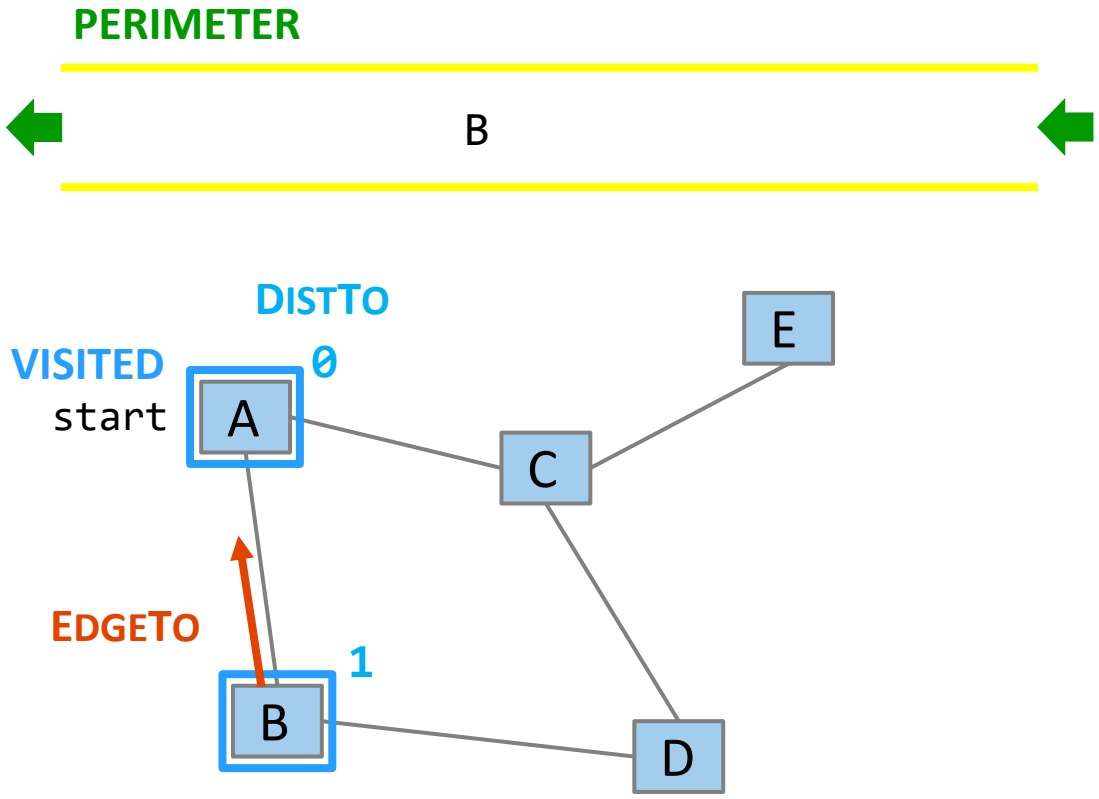
while (!perimeter.isEmpty()) {
  Vertex from = perimeter.remove();
  for (Edge edge : graph.edgesFrom(from)) {
    Vertex to = edge.to();
    if (!visited.contains(to)) {
      edgeTo.put(to, edge);
      distTo.put(to, distTo.get(from) + 1);
      perimeter.add(to);
      visited.add(to);
    }
  }
}
return edgeTo, distTo;
}
```

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

BFS zur Suche des ungewichteten kürzesten Pfads

Beispiel



Die edgeTo-Map speichert Backpointer: Jeder Knoten merkt sich, welcher Knoten benutzt wurde, um ihn zu erreichen!

```
perimeter.add(start);
visited.add(start);
```

```
...
Map<Vertex, Edge> edgeTo = ...
Map<Vertex, Integer> distTo = ...

edgeTo.put(start, null);
distTo.put(start, 0);

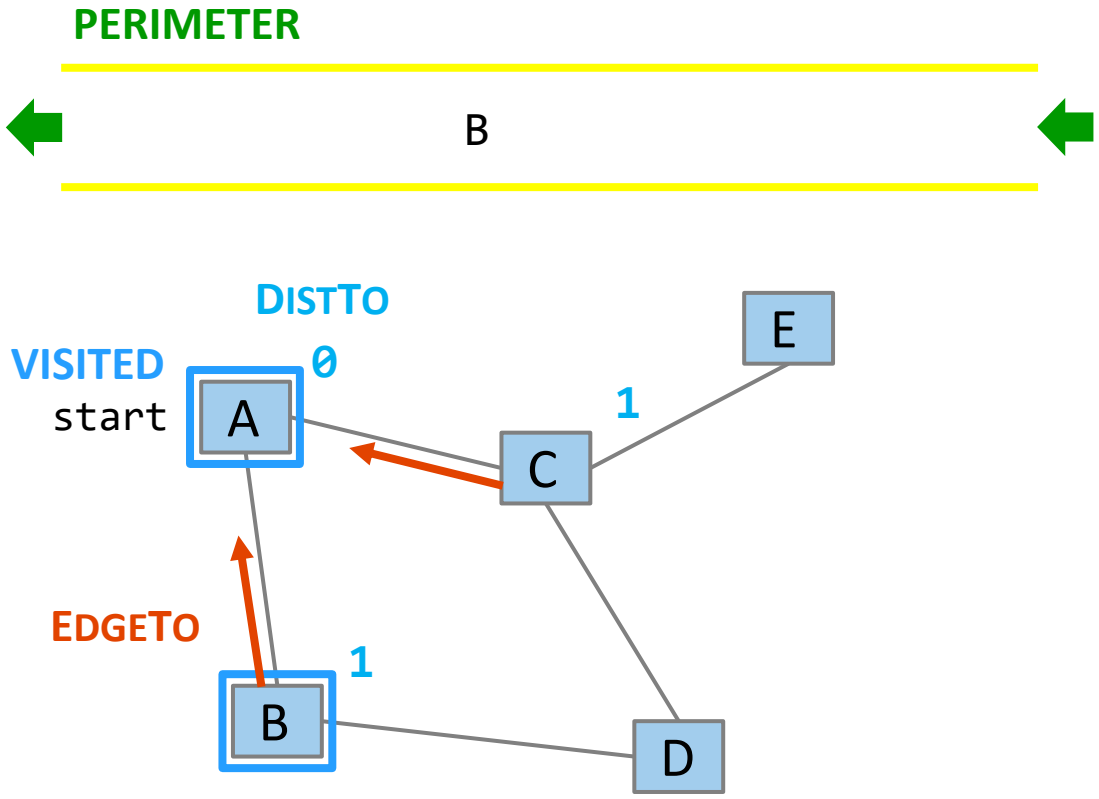
while (!perimeter.isEmpty()) {
  Vertex from = perimeter.remove();
  for (Edge edge : graph.edgesFrom(from)) {
    Vertex to = edge.to();
    if (!visited.contains(to)) {
      edgeTo.put(to, edge);
      distTo.put(to, distTo.get(from) + 1);
      perimeter.add(to);
      visited.add(to);
    }
  }
}
return edgeTo, distTo;
}
```

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

BFS zur Suche des ungewichteten kürzesten Pfads

Beispiel



Die edgeTo-Map speichert Backpointer: Jeder Knoten merkt sich, welcher Knoten benutzt wurde, um ihn zu erreichen!

```
perimeter.add(start);
visited.add(start);
```

```
...
Map<Vertex, Edge> edgeTo = ...
Map<Vertex, Integer> distTo = ...

edgeTo.put(start, null);
distTo.put(start, 0);

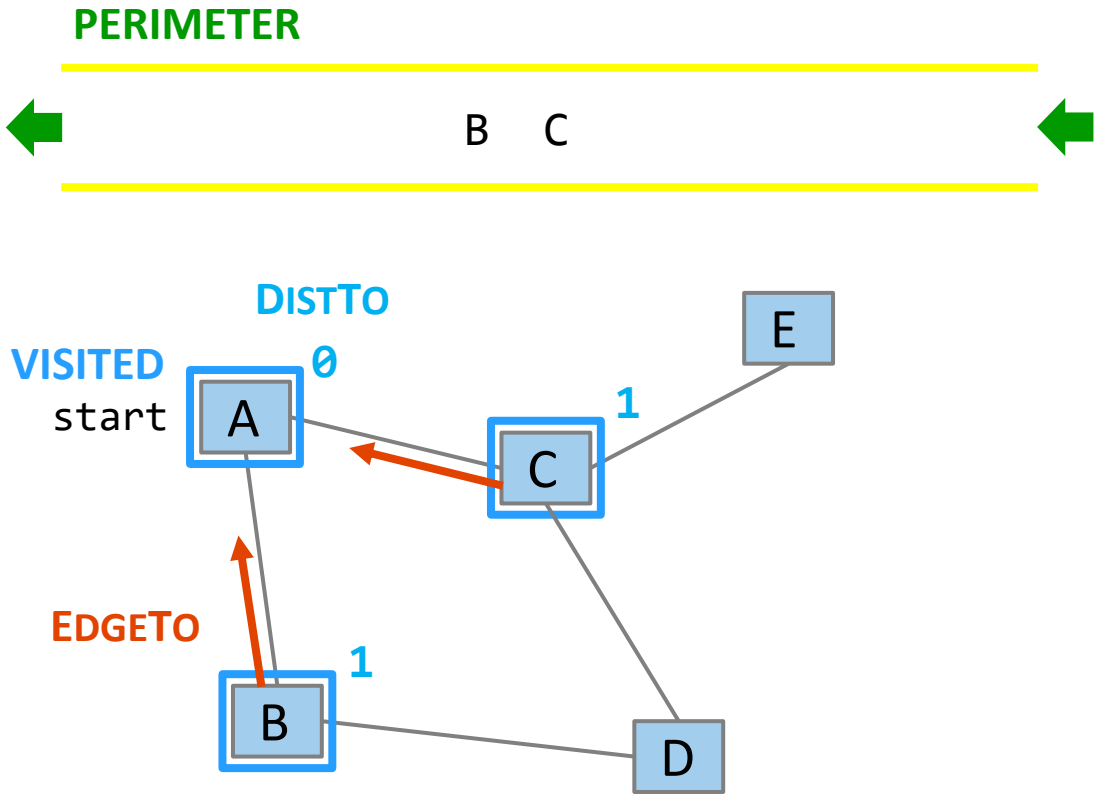
while (!perimeter.isEmpty()) {
  Vertex from = perimeter.remove();
  for (Edge edge : graph.edgesFrom(from)) {
    Vertex to = edge.to();
    if (!visited.contains(to)) {
      edgeTo.put(to, edge);
      distTo.put(to, distTo.get(from) + 1);
      perimeter.add(to);
      visited.add(to);
    }
  }
}
return edgeTo, distTo;
}
```

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

BFS zur Suche des ungewichteten kürzesten Pfads

Beispiel



Die edgeTo-Map speichert Backpointer: Jeder Knoten merkt sich, welcher Knoten benutzt wurde, um ihn zu erreichen!

```
perimeter.add(start);
visited.add(start);
```

```
...
Map<Vertex, Edge> edgeTo = ...
Map<Vertex, Integer> distTo = ...

edgeTo.put(start, null);
distTo.put(start, 0);

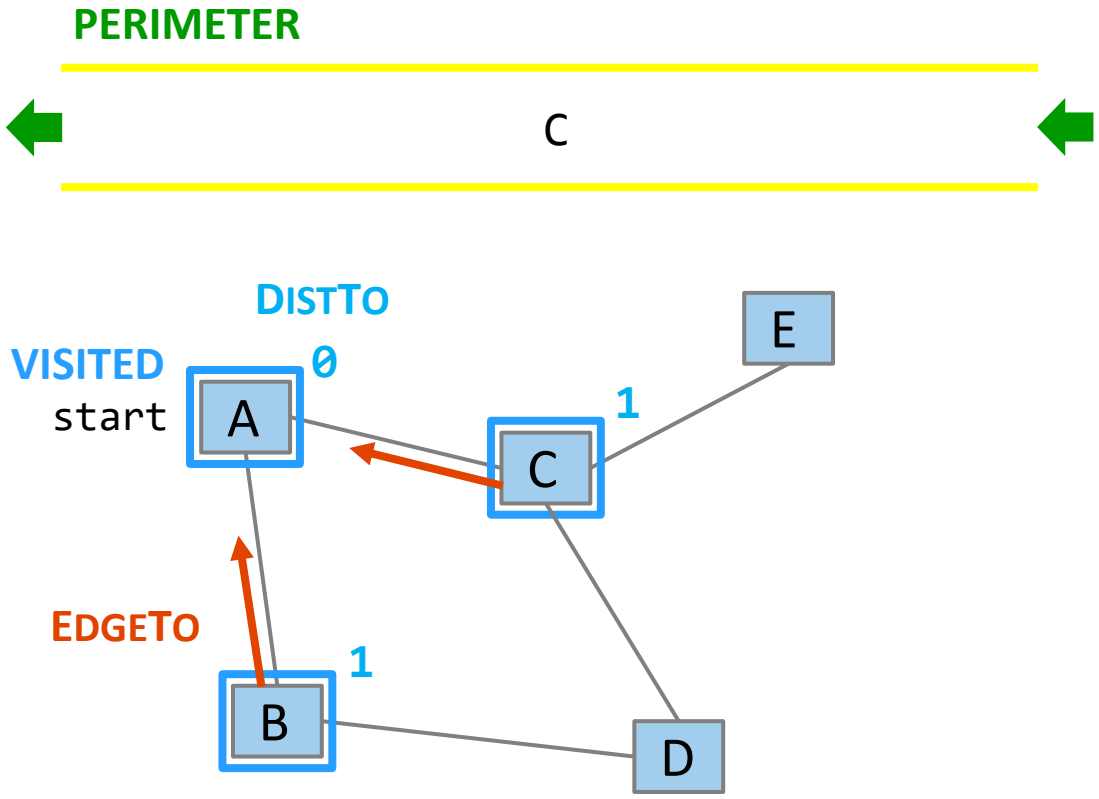
while (!perimeter.isEmpty()) {
  Vertex from = perimeter.remove();
  for (Edge edge : graph.edgesFrom(from)) {
    Vertex to = edge.to();
    if (!visited.contains(to)) {
      edgeTo.put(to, edge);
      distTo.put(to, distTo.get(from) + 1);
      perimeter.add(to);
      visited.add(to);
    }
  }
}
return edgeTo, distTo;
}
```


Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

BFS zur Suche des ungewichteten kürzesten Pfads

Beispiel



Die edgeTo-Map speichert Backpointer: Jeder Knoten merkt sich, welcher Knoten benutzt wurde, um ihn zu erreichen!

```
perimeter.add(start);
visited.add(start);
```

```
...
Map<Vertex, Edge> edgeTo = ...
Map<Vertex, Integer> distTo = ...

edgeTo.put(start, null);
distTo.put(start, 0);

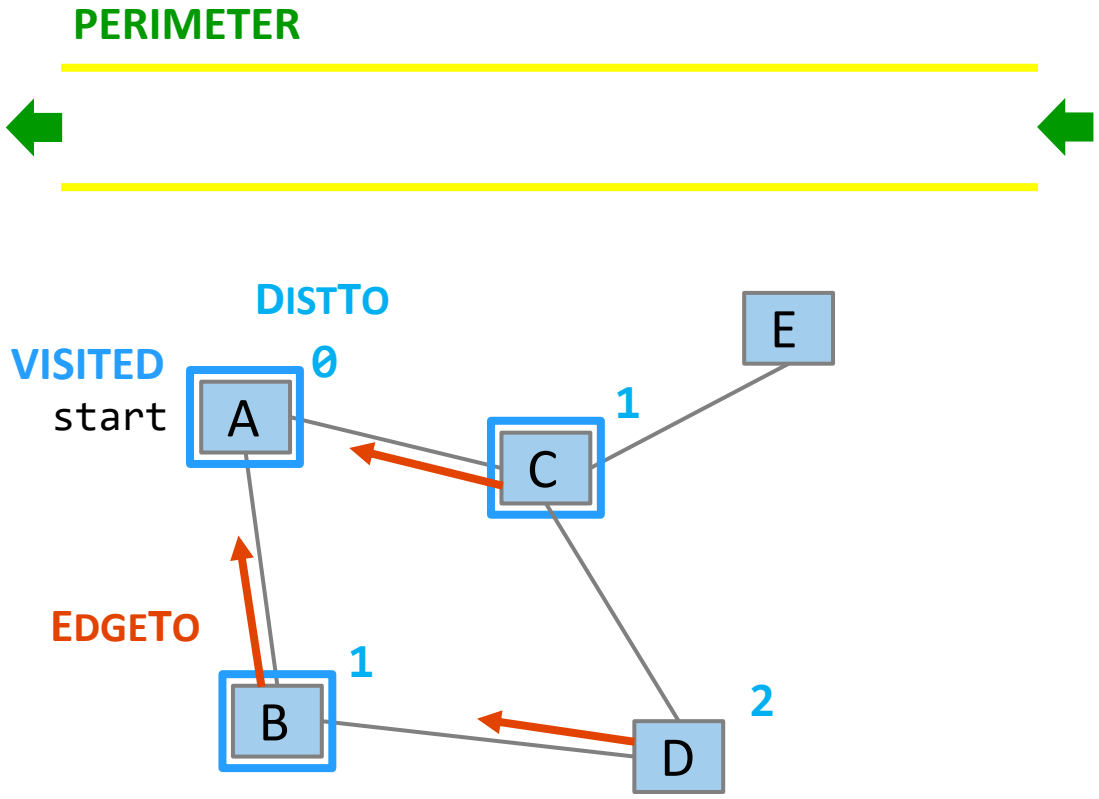
while (!perimeter.isEmpty()) {
  Vertex from = perimeter.remove();
  for (Edge edge : graph.edgesFrom(from)) {
    Vertex to = edge.to();
    if (!visited.contains(to)) {
      edgeTo.put(to, edge);
      distTo.put(to, distTo.get(from) + 1);
      perimeter.add(to);
      visited.add(to);
    }
  }
}
return edgeTo, distTo;
}
```

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

BFS zur Suche des ungewichteten kürzesten Pfads

Beispiel



Die edgeTo-Map speichert Backpointer: Jeder Knoten merkt sich, welcher Knoten benutzt wurde, um ihn zu erreichen!

```
perimeter.add(start);
visited.add(start);
```

```
...
Map<Vertex, Edge> edgeTo = ...
Map<Vertex, Integer> distTo = ...

edgeTo.put(start, null);
distTo.put(start, 0);

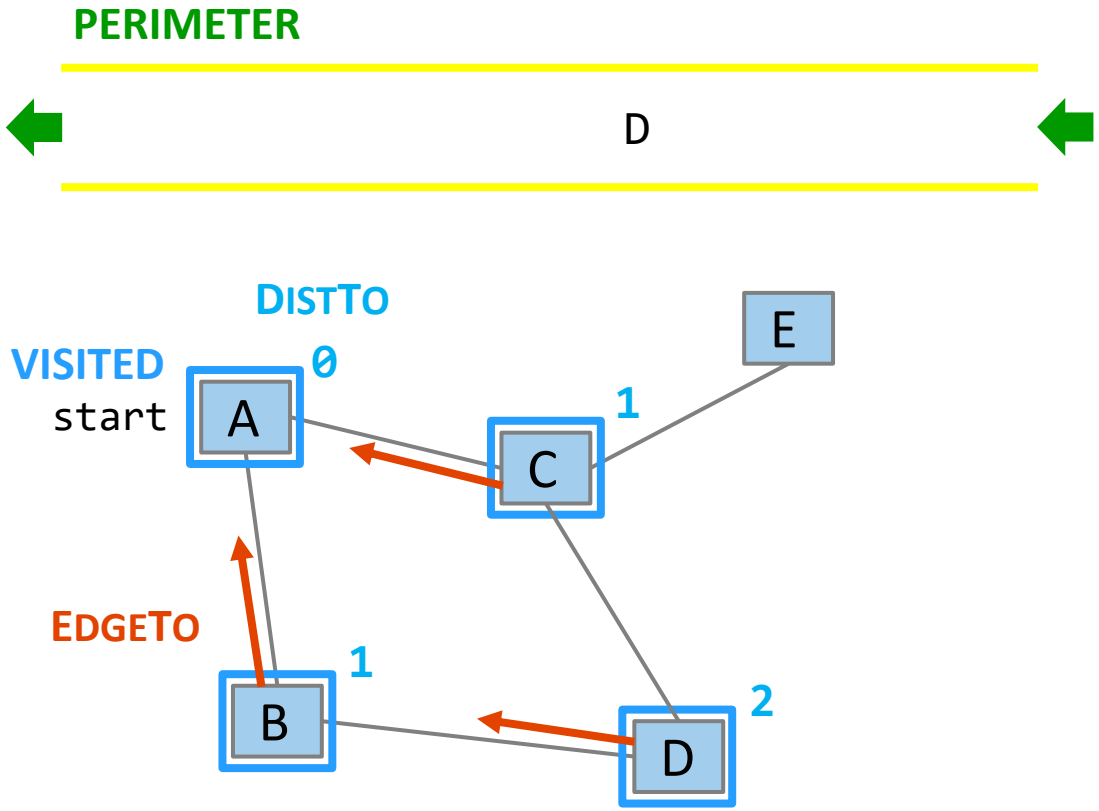
while (!perimeter.isEmpty()) {
  Vertex from = perimeter.remove();
  for (Edge edge : graph.edgesFrom(from)) {
    Vertex to = edge.to();
    if (!visited.contains(to)) {
      edgeTo.put(to, edge);
      distTo.put(to, distTo.get(from) + 1);
      perimeter.add(to);
      visited.add(to);
    }
  }
}
return edgeTo, distTo;
}
```


Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

BFS zur Suche des ungewichteten kürzesten Pfads

Beispiel



Die edgeTo-Map speichert Backpointer: Jeder Knoten merkt sich, welcher Knoten benutzt wurde, um ihn zu erreichen!

```
perimeter.add(start);
visited.add(start);
```

```
...
Map<Vertex, Edge> edgeTo = ...
Map<Vertex, Integer> distTo = ...

edgeTo.put(start, null);
distTo.put(start, 0);

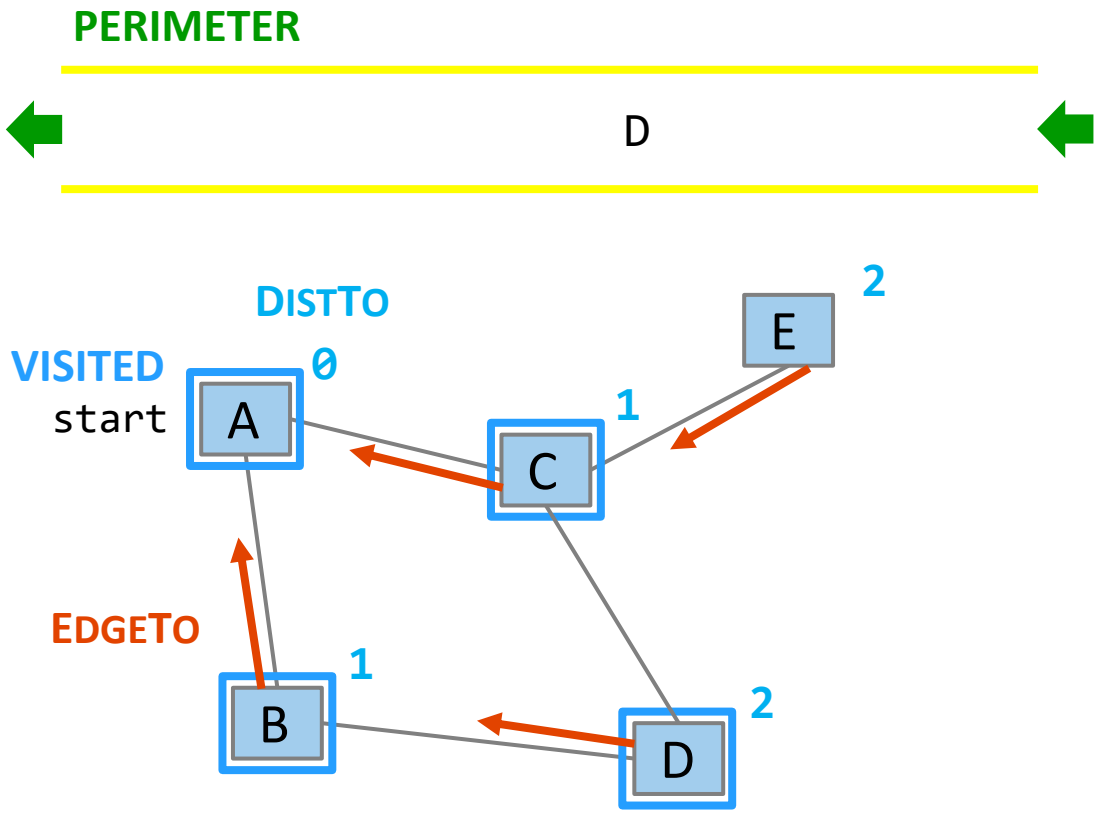
while (!perimeter.isEmpty()) {
  Vertex from = perimeter.remove();
  for (Edge edge : graph.edgesFrom(from)) {
    Vertex to = edge.to();
    if (!visited.contains(to)) {
      edgeTo.put(to, edge);
      distTo.put(to, distTo.get(from) + 1);
      perimeter.add(to);
      visited.add(to);
    }
  }
}
return edgeTo, distTo;
}
```


Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

BFS zur Suche des ungewichteten kürzesten Pfads

Beispiel



Die edgeTo-Map speichert Backpointer: Jeder Knoten merkt sich, welcher Knoten benutzt wurde, um ihn zu erreichen!

```
perimeter.add(start);
visited.add(start);
```

```
...
Map<Vertex, Edge> edgeTo = ...
Map<Vertex, Integer> distTo = ...

edgeTo.put(start, null);
distTo.put(start, 0);

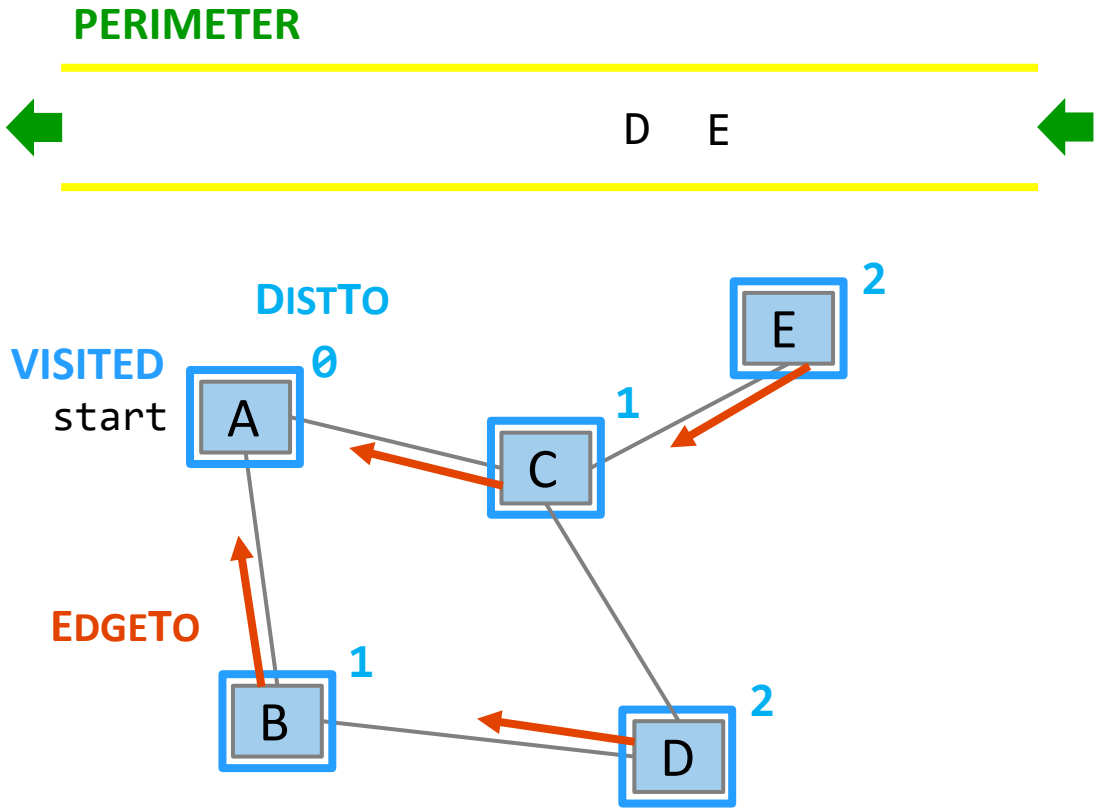
while (!perimeter.isEmpty()) {
  Vertex from = perimeter.remove();
  for (Edge edge : graph.edgesFrom(from)) {
    Vertex to = edge.to();
    if (!visited.contains(to)) {
      edgeTo.put(to, edge);
      distTo.put(to, distTo.get(from) + 1);
      perimeter.add(to);
      visited.add(to);
    }
  }
}
return edgeTo, distTo;
}
```

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

BFS zur Suche des ungewichteten kürzesten Pfads

Beispiel



Die edgeTo-Map speichert Backpointer: Jeder Knoten merkt sich, welcher Knoten benutzt wurde, um ihn zu erreichen!

```
perimeter.add(start);
visited.add(start);
```

```
...
Map<Vertex, Edge> edgeTo = ...
Map<Vertex, Integer> distTo = ...

edgeTo.put(start, null);
distTo.put(start, 0);

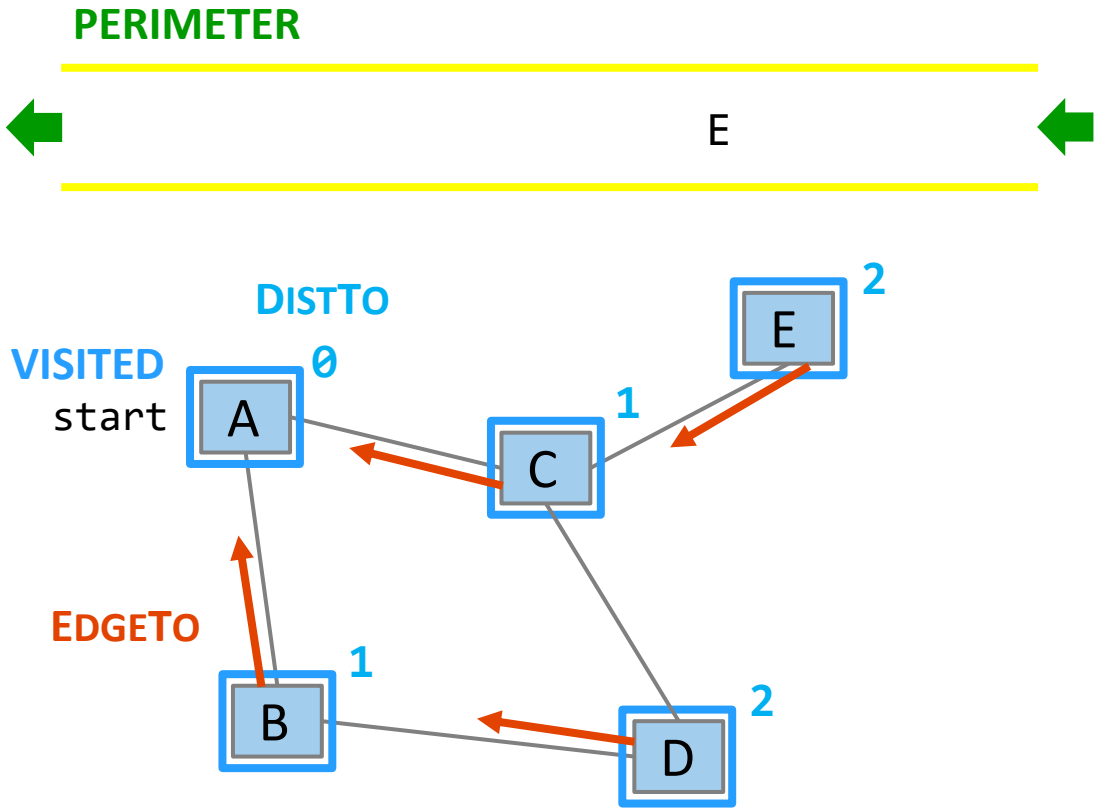
while (!perimeter.isEmpty()) {
  Vertex from = perimeter.remove();
  for (Edge edge : graph.edgesFrom(from)) {
    Vertex to = edge.to();
    if (!visited.contains(to)) {
      edgeTo.put(to, edge);
      distTo.put(to, distTo.get(from) + 1);
      perimeter.add(to);
      visited.add(to);
    }
  }
}
return edgeTo, distTo;
}
```

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

BFS zur Suche des ungewichteten kürzesten Pfads

Beispiel



Die edgeTo-Map speichert Backpointer: Jeder Knoten merkt sich, welcher Knoten benutzt wurde, um ihn zu erreichen!

```
perimeter.add(start);
visited.add(start);
```

```
...
Map<Vertex, Edge> edgeTo = ...
Map<Vertex, Integer> distTo = ...

edgeTo.put(start, null);
distTo.put(start, 0);

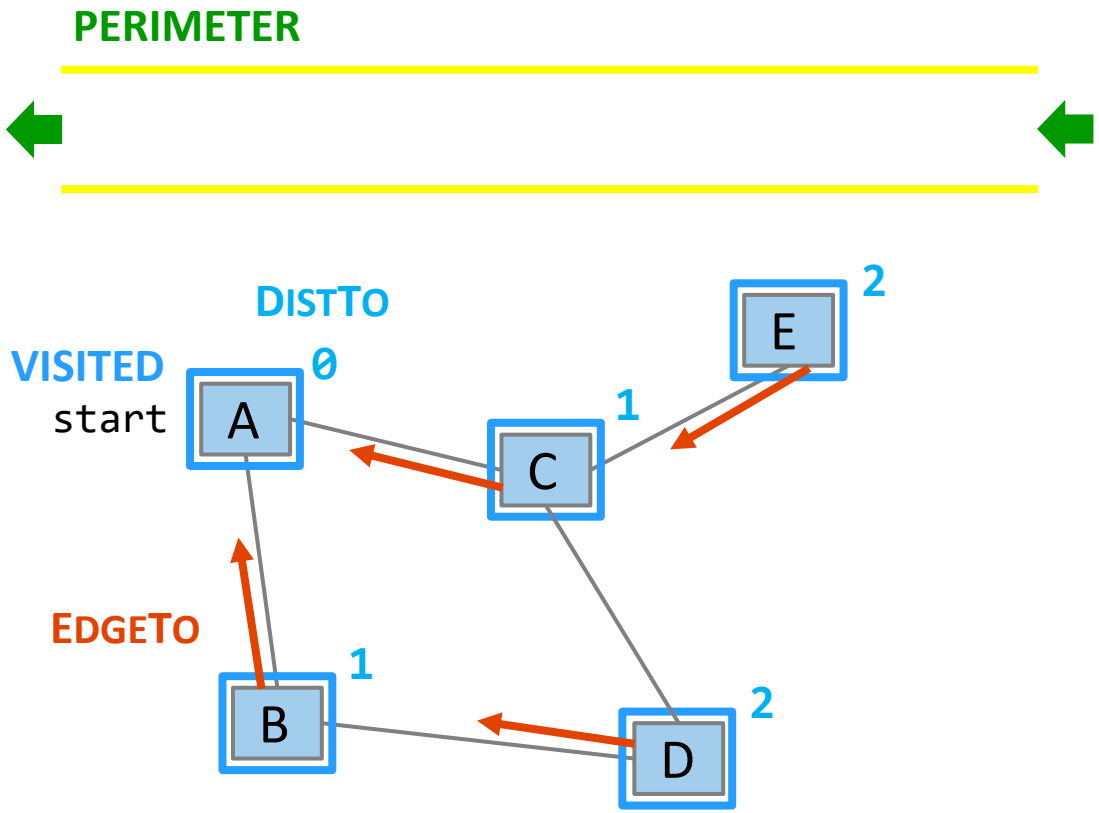
while (!perimeter.isEmpty()) {
  Vertex from = perimeter.remove();
  for (Edge edge : graph.edgesFrom(from)) {
    Vertex to = edge.to();
    if (!visited.contains(to)) {
      edgeTo.put(to, edge);
      distTo.put(to, distTo.get(from) + 1);
      perimeter.add(to);
      visited.add(to);
    }
  }
}
return edgeTo, distTo;
}
```


Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

BFS zur Suche des ungewichteten kürzesten Pfads

Beispiel



Die edgeTo-Map speichert Backpointer: Jeder Knoten merkt sich, welcher Knoten benutzt wurde, um ihn zu erreichen!

```
perimeter.add(start);
visited.add(start);
```

```
...
Map<Vertex, Edge> edgeTo = ...
Map<Vertex, Integer> distTo = ...

edgeTo.put(start, null);
distTo.put(start, 0);

while (!perimeter.isEmpty()) {
  Vertex from = perimeter.remove();
  for (Edge edge : graph.edgesFrom(from)) {
    Vertex to = edge.to();
    if (!visited.contains(to)) {
      edgeTo.put(to, edge);
      distTo.put(to, distTo.get(from) + 1);
      perimeter.add(to);
      visited.add(to);
    }
  }
}
return edgeTo, distTo;
}
```


Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

BFS zur Suche des ungewichteten kürzesten Pfads: Beispiel

Note: this code stores `visited`, `edgeTo`, and `distTo` as **external maps** (only drawn on graph for convenience). Another implementation option: store them as fields of the nodes themselves

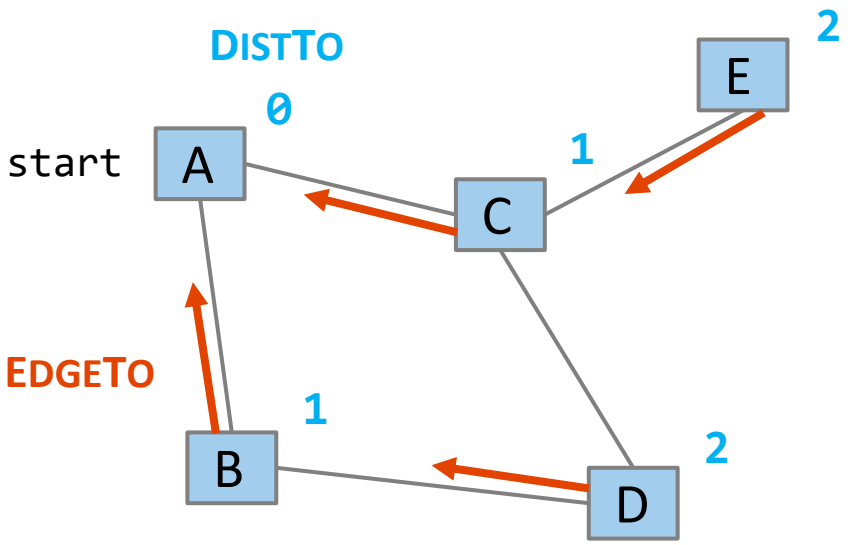
```
  ...
  Map<Vertex, Edge> edgeTo = ...
  Map<Vertex, Integer> distTo = ...

  edgeTo.put(start, null);
  distTo.put(start, 0);

  while (!perimeter.isEmpty()) {
    Vertex from = perimeter.remove();
    for (Edge edge : graph.edgesFrom(from)) {
      Vertex to = edge.to();
      if (!visited.contains(to)) {
        edgeTo.put(to, edge);
        distTo.put(to, distTo.get(from) + 1);
        perimeter.add(to);
        visited.add(to);
      }
    }
  }
  return edgeTo, distTo;
}
```

Kürzester Pfad: Baum kürzester Pfade

Baum kürzester Pfade:

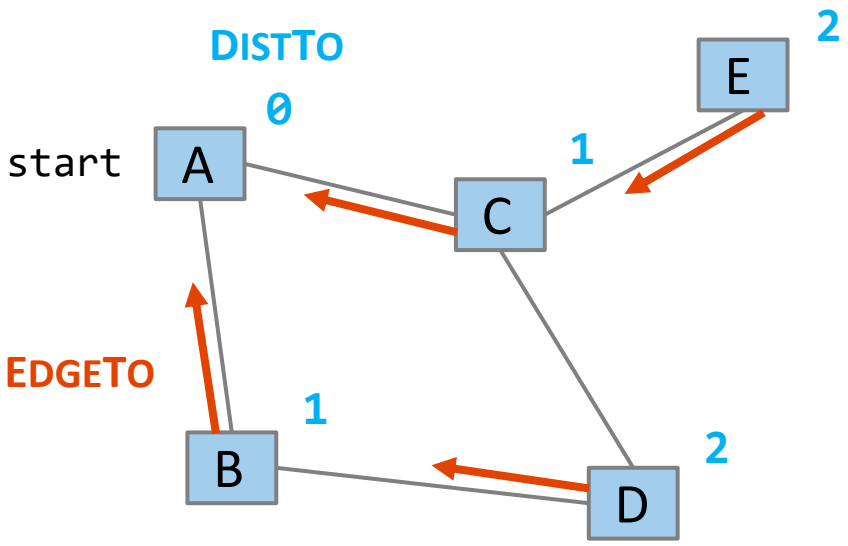


Diese Abwandlung des BFS erwähnte den Zielknoten überhaupt nicht!

Stattdessen wurde der kürzeste Pfad und die Entfernung vom Startknoten zu jedem anderen Knoten berechnet

Kürzester Pfad: Baum kürzester Pfade

Baum kürzester Pfade:



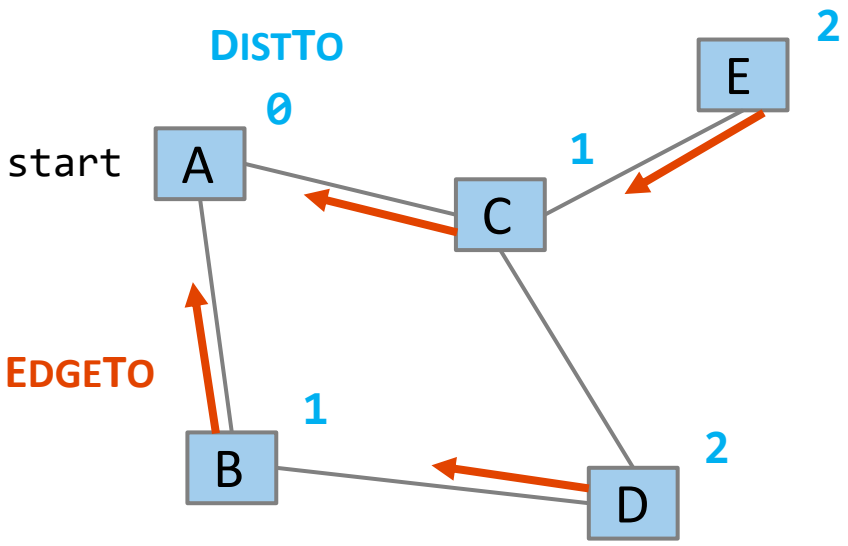
Diese Abwandlung des BFS erwähnte den Zielknoten überhaupt nicht!

Stattdessen wurde der kürzeste Pfad und die Entfernung vom Startknoten zu jedem anderen Knoten berechnet

- Dies nennt man den Baum kürzester Pfade
  - Ein allgemeines Konzept: In dieser Implementierung besteht er aus Entfernungen und Backpointern
-

Kürzester Pfad: Baum kürzester Pfade

Baum kürzester Pfade:



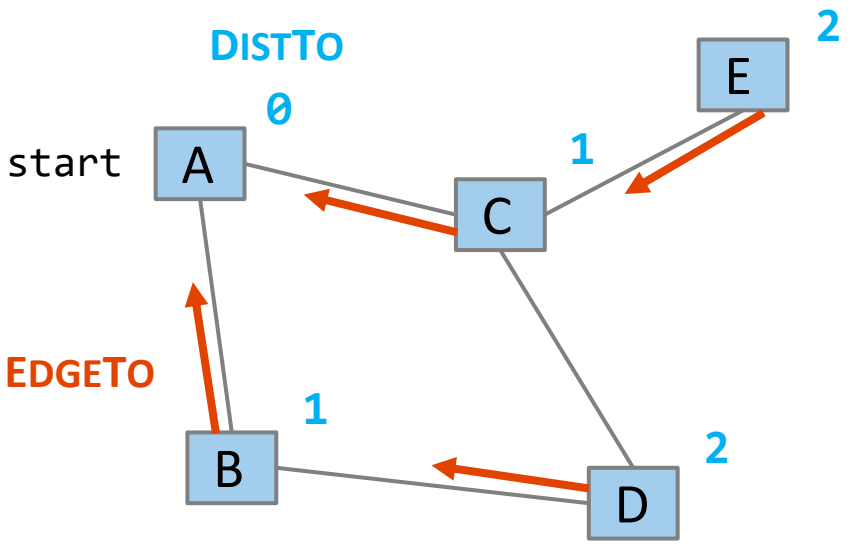
Diese Abwandlung des BFS erwähnte den Zielknoten überhaupt nicht!

Stattdessen wurde der kürzeste Pfad und die Entfernung vom Startknoten zu jedem anderen Knoten berechnet

- Dies nennt man den Baum kürzester Pfade
  - Ein allgemeines Konzept: In dieser Implementierung besteht er aus Entfernungen und Backpointern
- Warum wird das Baum genannt?

Kürzester Pfad: Baum kürzester Pfade

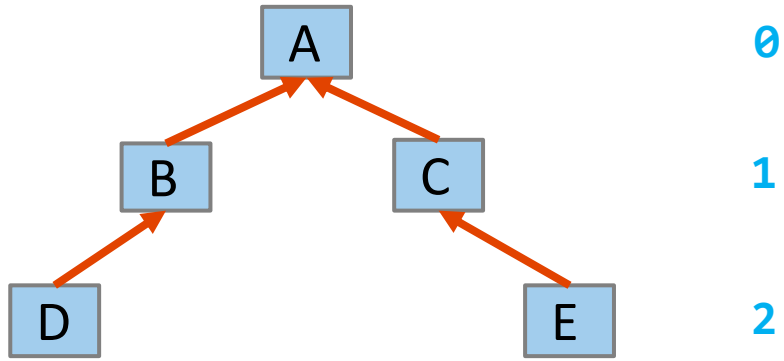
Baum kürzester Pfade:



Diese Abwandlung des BFS erwähnte den Zielknoten überhaupt nicht!

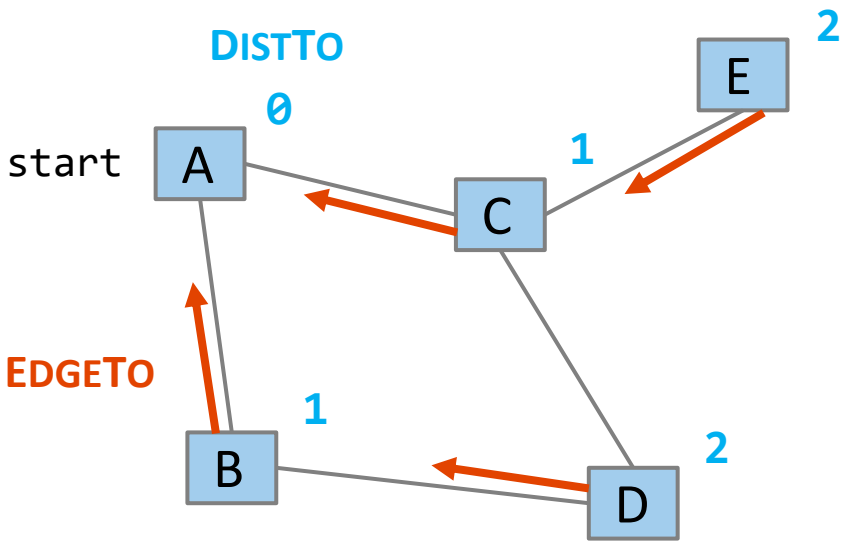
Stattdessen wurde der kürzeste Pfad und die Entfernung vom Startknoten zu jedem anderen Knoten berechnet

- Dies nennt man den Baum kürzester Pfade
  - Ein allgemeines Konzept: In dieser Implementierung besteht er aus Entfernungen und Backpointern
- Warum wird das Baum genannt?



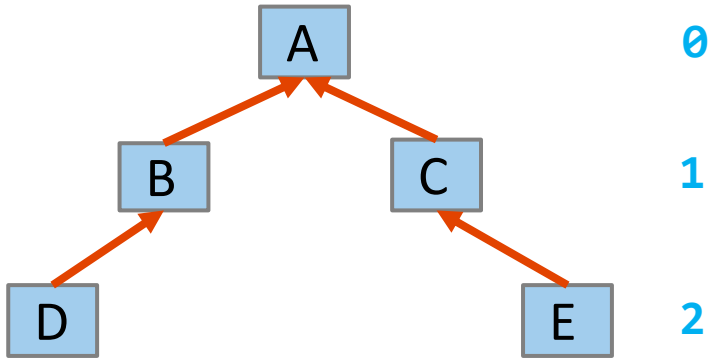
Kürzester Pfad: Baum kürzester Pfade

Baum kürzester Pfade:



Der Baum kürzester Pfade hat alle Antworten!

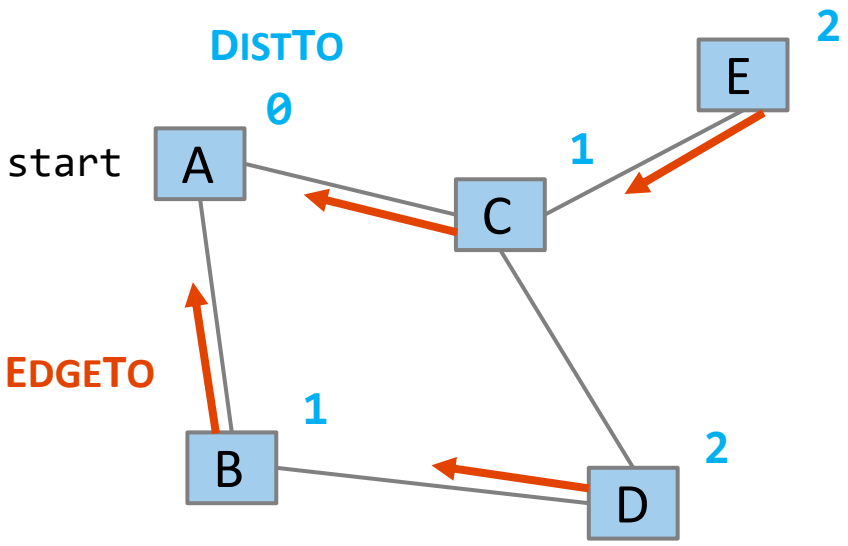
- Länge des kürzesten Pfades von A nach D?
  - Nachschlagen in der Tabelle distTo(D): 2
-
  -



Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

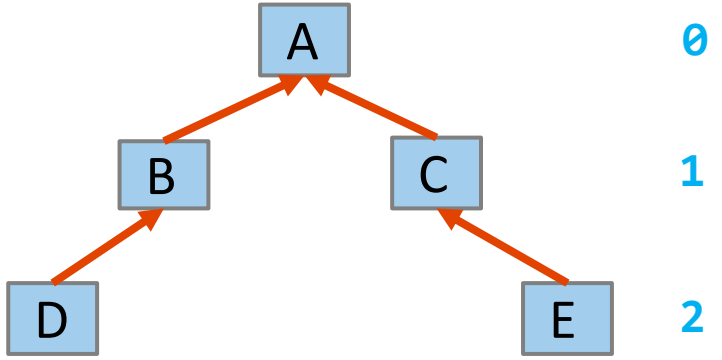
Kürzester Pfad: Baum kürzester Pfade

Baum kürzester Pfade:



Der Baum kürzester Pfade hat alle Antworten!

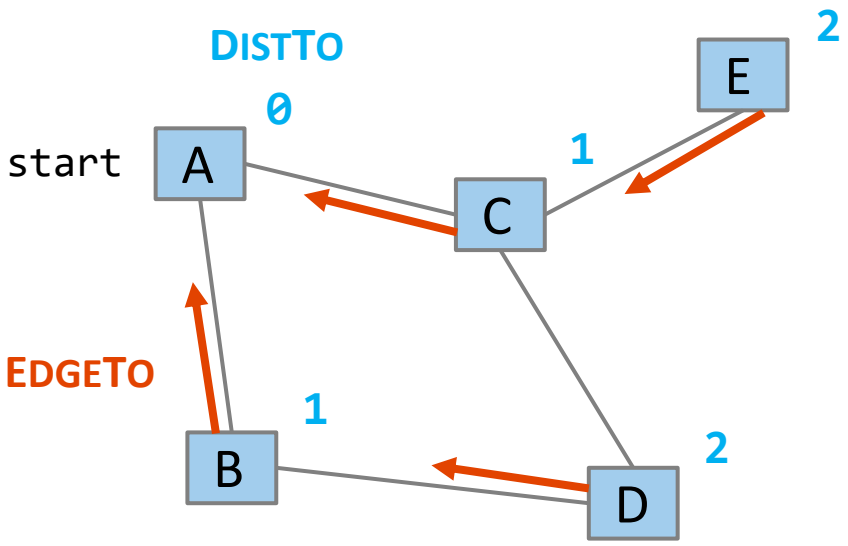
- Länge des kürzesten Pfades von A nach D?
  - Nachschlagen in der Tabelle distTo(D): 2
- Was ist der kürzeste Pfad von A nach D?
  - Von der edgeTo-Map aus rückwärts aufbauen: bei D beginnen, dem Backpointer zu B folgen, dem Backpointer zu A folgen - unser kürzester Pfad ist A → B → D



Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Kürzester Pfad: Baum kürzester Pfade

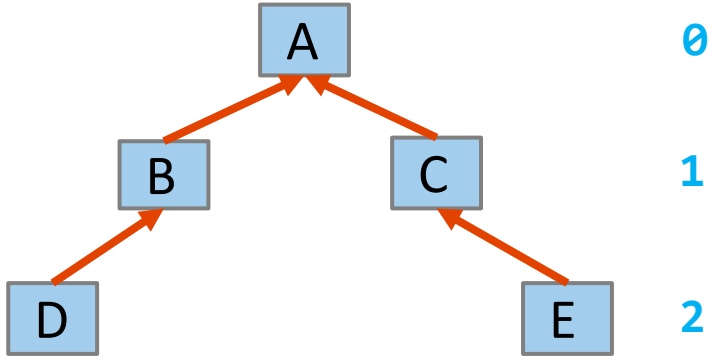
Baum kürzester Pfade:



Der Baum kürzester Pfade hat alle Antworten!

- Länge des kürzesten Pfades von A nach D?
  - Nachschlagen in der Tabelle distTo(D): 2
- Was ist der kürzeste Pfad von A nach D?
  - Von der edgeTo-Map aus rückwärts aufbauen: bei D beginnen, dem Backpointer zu B folgen, dem Backpointer zu A folgen - unser kürzester Pfad ist A → B → D

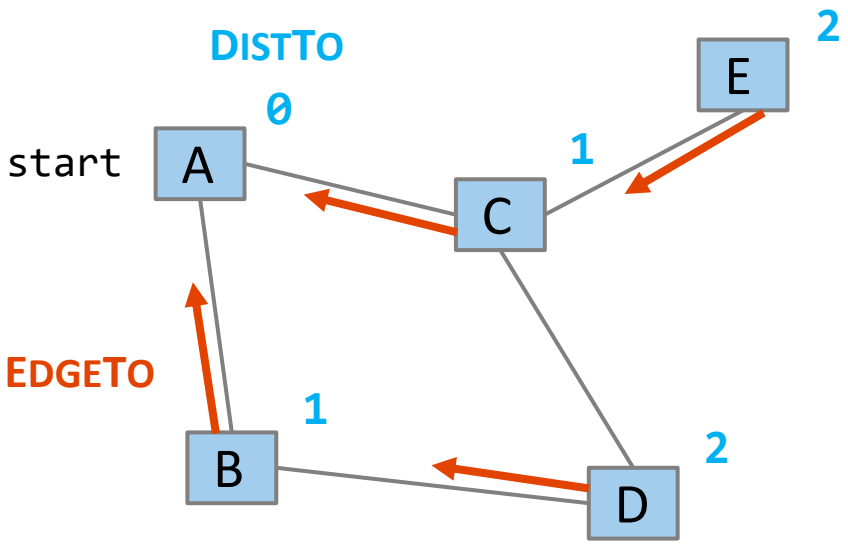
Wenn man sich nur für den kürzesten Pfad zu Knoten t interessiert, kann man manchmal früher abbrechen!



Kürzester Pfad: Baum kürzester Pfade

Übungsblatt 7: Aufgabe 4

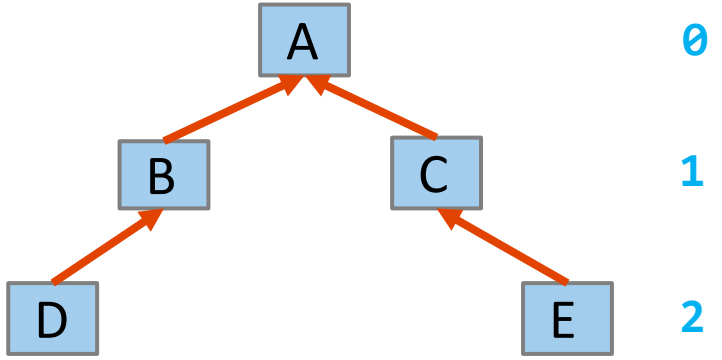
Baum kürzester Pfade:



Der Baum kürzester Pfade hat alle Antworten!

- Länge des kürzesten Pfades von A nach D?
  - Nachschlagen in der Tabelle distTo(D): 2
- Was ist der kürzeste Pfad von A nach D?
  - Von der edgeTo-Map aus rückwärts aufbauen: bei D beginnen, dem Backpointer zu B folgen, dem Backpointer zu A folgen - unser kürzester Pfad ist A → B → D

Wenn man sich nur für den kürzesten Pfad zu Knoten t interessiert, kann man manchmal früher abbrechen!



Zusammenfassung: Graphenprobleme

So wie alles Graphen sind, ist jedes Problem ein Graphenproblem

BFS und DFS sind sehr nützliche Werkzeuge, um diese zu lösen! Wir werden noch viele weitere sehen.

EINFACH



Erreichbarkeitsproblem

Gibt es bei einem Startknoten s und einem Zielknoten t einen Pfad zwischen s und t?

BFS oder DFS + prüfen ob wir t erreicht haben

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Zusammenfassung: Graphenprobleme

So wie alles Graphen sind, ist jedes Problem ein Graphenproblem

BFS und DFS sind sehr nützliche Werkzeuge, um diese zu lösen! Wir werden noch viele weitere sehen.



EINFACH

Erreichbarkeitsproblem

Gibt es bei einem Startknoten s und einem Zielknoten t einen Pfad zwischen s und t?

BFS oder DFS + prüfen ob wir t erreicht haben



MITTEL

Ungewichteter kürzester Pfad

Wie lang ist der kürzeste Pfad von einem Startknoten s zu einem Zielknoten t? Aus welchen Kanten setzt sich dieser Pfad zusammen?

BFS + den Baum der kürzesten Pfade nach und nach erstellen

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Zusammenfassung: Graphenprobleme

So wie alles Graphen sind, ist jedes Problem ein Graphenproblem

BFS und DFS sind sehr nützliche Werkzeuge, um diese zu lösen! Wir werden noch viele weitere sehen.



EINFACH

Erreichbarkeitsproblem

Gibt es bei einem Startknoten s und einem Zielknoten t einen Pfad zwischen s und t?

BFS oder DFS + prüfen ob wir t erreicht haben



MITTEL

Ungewichteter kürzester Pfad

Wie lang ist der kürzeste Pfad von einem Startknoten s zu einem Zielknoten t? Aus welchen Kanten setzt sich dieser Pfad zusammen?

BFS + den Baum der kürzesten Pfade nach und nach erstellen

Was ist mit dem kürzesten Pfad in einem gewichteten Graphen?

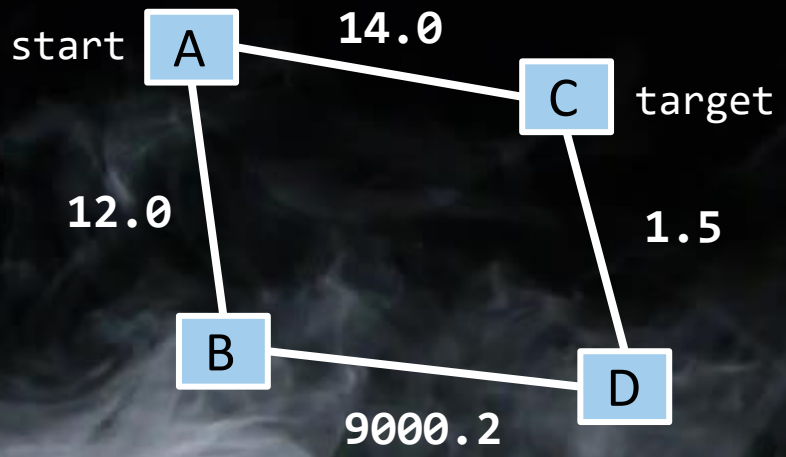
Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Gewichteter kürzester Pfad

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

SCHWIERIGER (IM MOMENT)

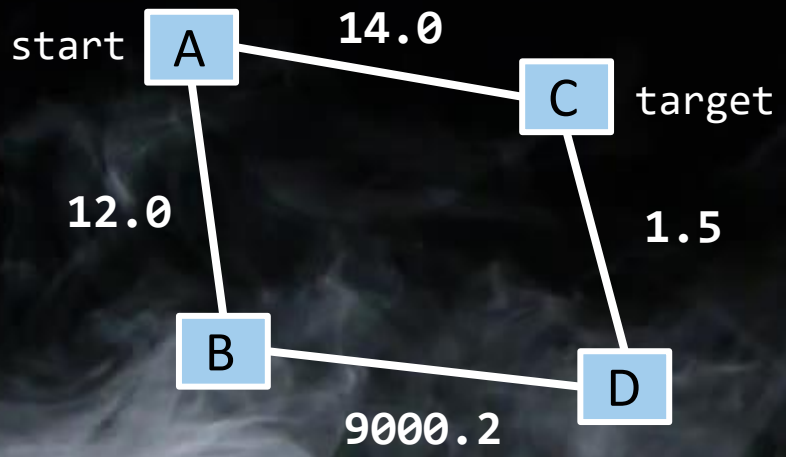


- Angenommen, wir wollen den kürzesten Pfad von A nach C finden, indem wir das Gewicht jeder Kante als „Entfernung“ verwenden.
- Wird uns BFS hier das richtige Ergebnis liefern?

Gewichteter kürzester Pfad

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

SCHWIERIGER (IM MOMENT)



- Angenommen, wir wollen den kürzesten Pfad von A nach C finden, indem wir das Gewicht jeder Kante als „Entfernung“ verwenden.
- Wird uns BFS hier das richtige Ergebnis liefern?

Anwendungen:

- Routenplanung
- Routing-Protokolle für Netzwerke
  - Open Shortest Path First (OSPF)

Dijkstras Algorithmus

Benannt nach seinem Erfinder, Edsger Dijkstra (1930-2002)

- Wahrlich einer der „Begründer“ der Computerwissenschaft
- 1972 Turing Award
- Dieser Algorithmus ist nur einer seiner vielen Beiträge!

Dijkstras Algorithmus

Benannt nach seinem Erfinder, Edsger Dijkstra (1930-2002)

- Wahrlich einer der „Begründer“ der Computerwissenschaft
- 1972 Turing Award
- Dieser Algorithmus ist nur einer seiner vielen Beiträge!

Die Idee:

- Erinnert an BFS, aber angepasst, um mit Gewichten umzugehen
- Beginnt mit den nächsten (kürzeste Entfernung) Nachbarn
-
-

Dijkstras Algorithmus

Benannt nach seinem Erfinder, Edsger Dijkstra (1930-2002)

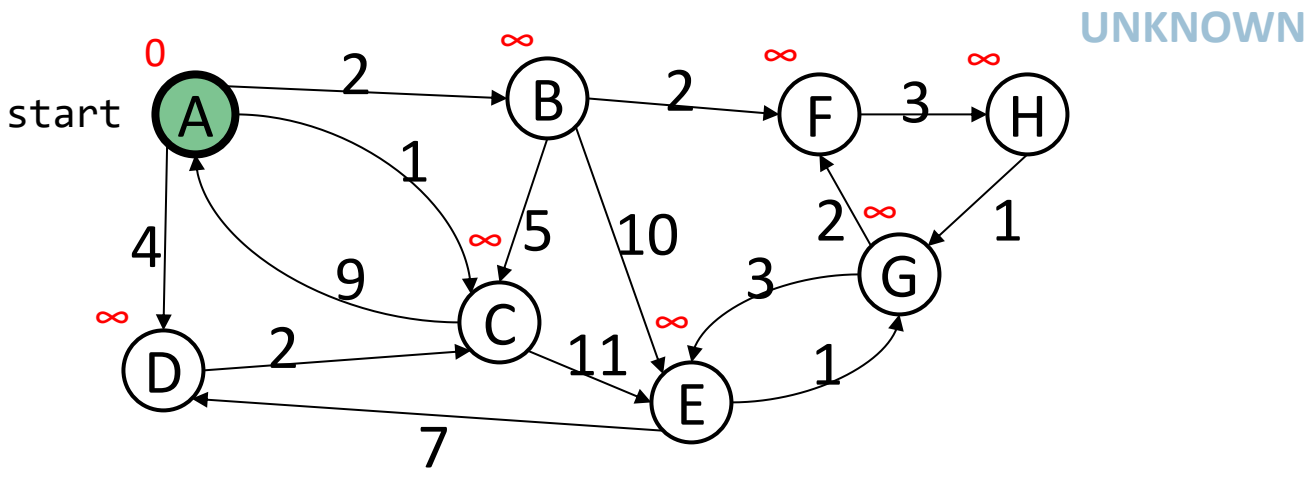
- Wahrlich einer der „Begründer“ der Computerwissenschaft
- 1972 Turing Award
- Dieser Algorithmus ist nur einer seiner vielen Beiträge!

Die Idee:

- Erinnert an BFS, aber angepasst, um mit Gewichten umzugehen
- Beginnt mit den nächsten (kürzeste Entfernung) Nachbarn
- Knoten, für die die kürzeste Entfernung berechnet wurde, werden in einer Menge gespeichert
- Knoten, die noch nicht in dieser Menge enthalten sind, haben eine „bestmöglich geschätzte Entfernung“.

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

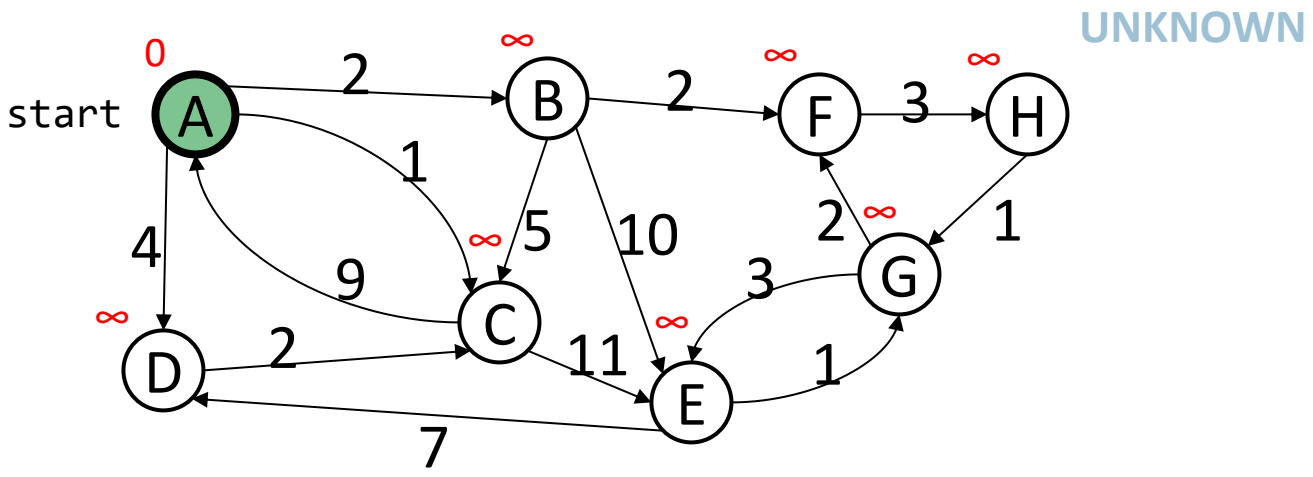
Dijkstras Algorithmus: Idee



Initialisierung:

- Startknoten hat Abstand 0; alle anderen Knoten haben Abstand ∞

Dijkstras Algorithmus: Idee



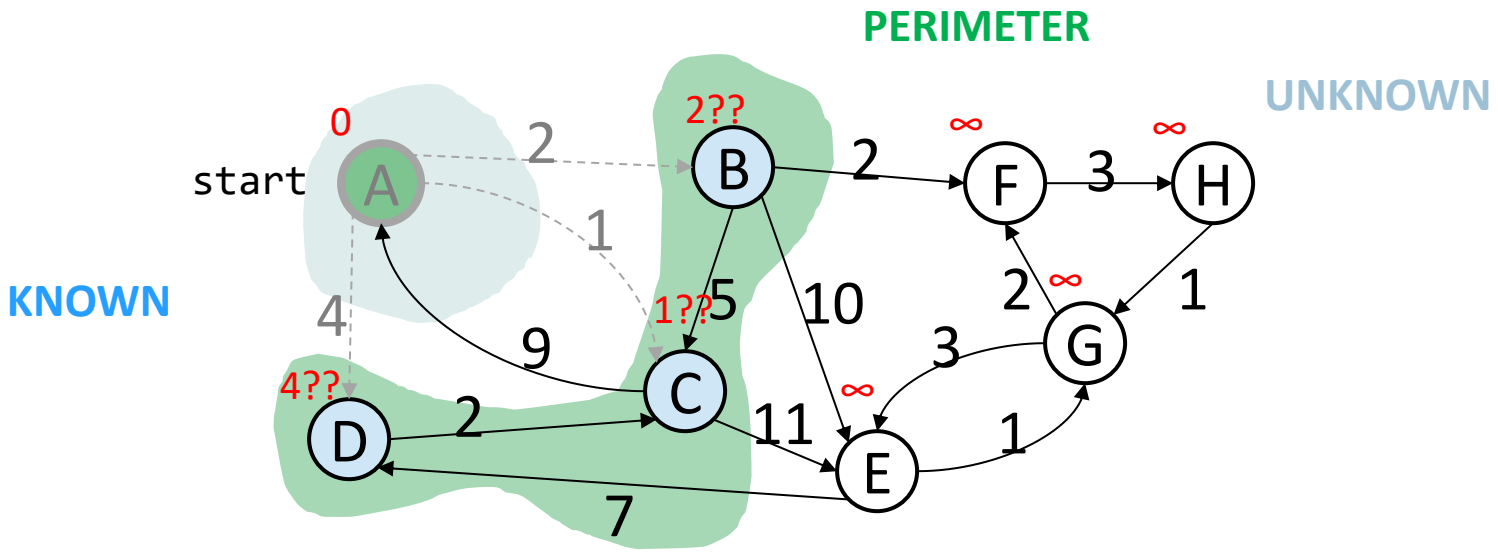
Initialisierung:

- Startknoten hat Abstand 0; alle anderen Knoten haben Abstand ∞

Bei jedem Iterationsschritt:

- Nächstgelegenen unbekannten Knoten u auswählen

Dijkstras Algorithmus: Idee



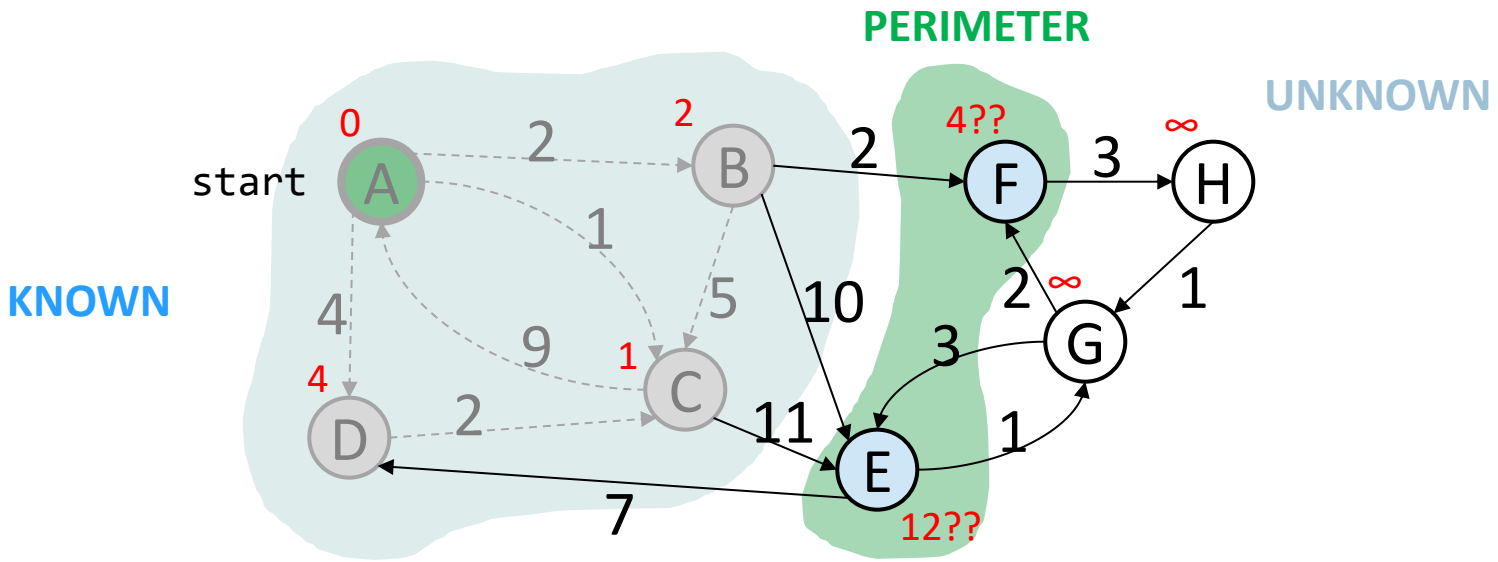
Initialisierung:

- Startknoten hat Abstand 0; alle anderen Knoten haben Abstand ∞

Bei jedem Iterationsschritt:

- Nächstgelegenen unbekannten Knoten u auswählen
- Hinzufügen zur „Wolke“ der bekannten Knoten
- Aktualisierung der „bisher besten“ Abstände für Knoten, die Kanten von Knoten u haben

Dijkstras Algorithmus: Idee



Initialisierung:

- Startknoten hat Abstand 0; alle anderen Knoten haben Abstand ∞

Bei jedem Iterationsschritt:

- Nächstgelegenen unbekannten Knoten u auswählen
- Hinzufügen zur „Wolke“ der bekannten Knoten
- Aktualisierung der „bisher besten“ Abstände für Knoten, die Kanten von Knoten u haben

Dijkstras Pseudocode (High-Level)

```
dijkstraShortestPath(G graph, V start)
```

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Syst
Fakultät für Informatik und Ingenieu

Dijkstras Pseudocode (High-Level)

Ähnlich wie bei „visited“ in BFS, sind „known“ Knoten, die final sind (wir kennen ihren kürzestes Pfad)

```
dijkstraShortestPath(G graph, V start)
  Set known; Map edgeTo, distTo;
```

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Syst
Fakultät für Informatik und Ingenieu

Dijkstras Pseudocode (High-Level)

Ähnlich wie bei „visited“ in BFS, sind „known“ Knoten, die final sind (wir kennen ihren kürzestes Pfad)

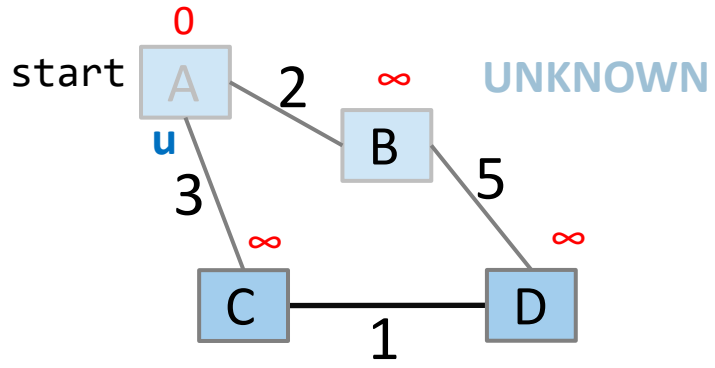
Dijkstra’s algorithm is all about updating “best-so-far” in distTo if we find shorter path! Init all paths to infinite.

```
dijkstraShortestPath(G graph, V start)
  Set known; Map edgeTo, distTo;

  initialize distTo with all nodes mapped to ∞, except start to 0
```


Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Pseudocode (High-Level)



Ähnlich wie bei „visited“ in BFS, sind „known“ Knoten, die final sind (wir kennen ihren kürzestes Pfad)

Dijkstra’s algorithm is all about updating “best-so-far” in distTo if we find shorter path! Init all paths to infinite.

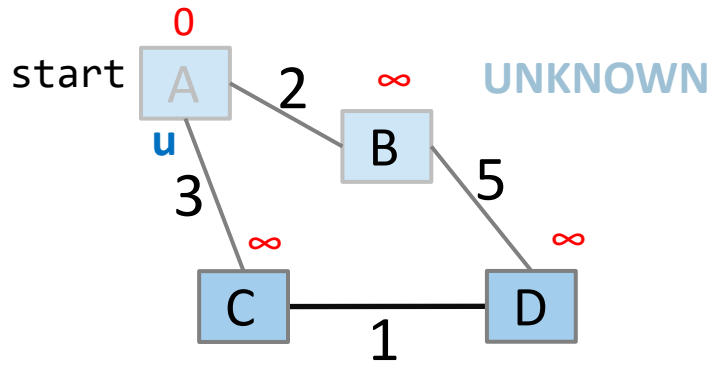
Reihenfolge ist wichtig: Besuche immer zuerst den nächsten Knoten!

```
dijkstraShortestPath(G graph, V start)
  Set known; Map edgeTo, distTo;
  initialize distTo with all nodes mapped to ∞, except start to 0

  while (there are unknown vertices):
    let u be the closest unknown vertex
    known.add(u);
```


Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Pseudocode (High-Level)



Ähnlich wie bei „visited“ in BFS, sind „known“ Knoten, die final sind (wir kennen ihren kürzestes Pfad)

Dijkstra’s algorithm is all about updating “best-so-far” in distTo if we find shorter path! Init all paths to infinite.

Reihenfolge ist wichtig: Besuche immer zuerst den nächsten Knoten!

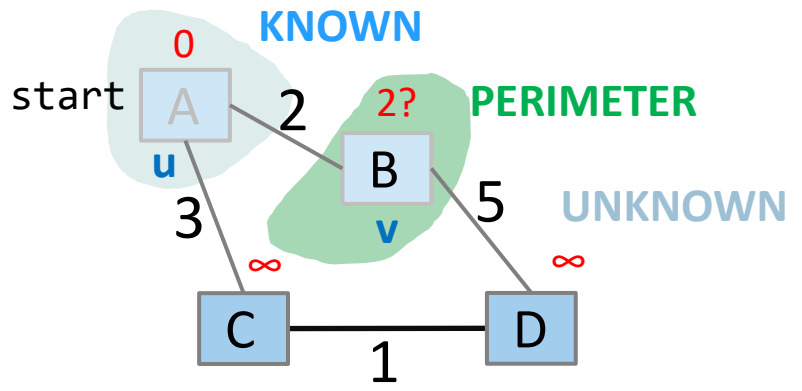
Betrachte alle Nachbarknoten von u: Wäre der Pfad dorthin über u kürzer als der, den Sie derzeit kennen?

```
dijkstraShortestPath(G graph, V start)
  Set known; Map edgeTo, distTo;
  initialize distTo with all nodes mapped to ∞, except start to 0

  while (there are unknown vertices):
    let u be the closest unknown vertex
    known.add(u);
   for each edge (u,v) to unknown v with weight w:
     oldDist = distTo.get(v)      // previous best path to v
     newDist = distTo.get(u) + w  // what if we went through u?
    if (newDist < oldDist):
      distTo.put(v, newDist)
      edgeTo.put(v, u)
```

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Pseudocode (High-Level)



Ähnlich wie bei „visited“ in BFS, sind „known“ Knoten, die final sind (wir kennen ihren kürzesten Pfad)

Dijkstra’s algorithm is all about updating “best-so-far” in distTo if we find shorter path! Init all paths to infinite.

Reihenfolge ist wichtig: Besuche immer zuerst den nächsten Knoten!

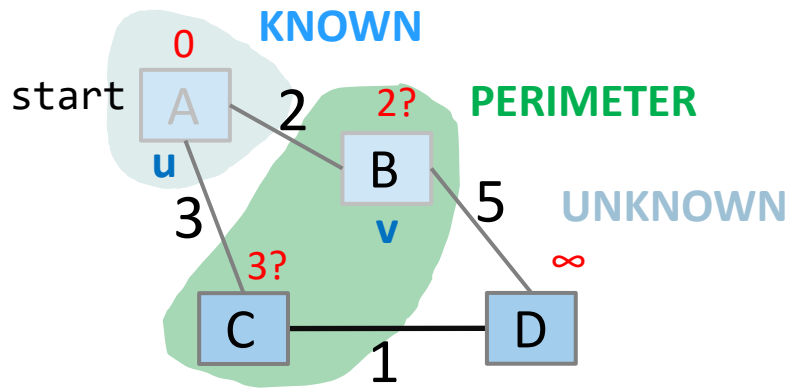
Betrachte alle Nachbarknoten von u: Wäre der Pfad dorthin über u kürzer als der, den Sie derzeit kennen?

```
dijkstraShortestPath(G graph, V start)
  Set known; Map edgeTo, distTo;
  initialize distTo with all nodes mapped to ∞, except start to 0

  while (there are unknown vertices):
    let u be the closest unknown vertex
    known.add(u);
   for each edge (u,v) to unknown v with weight w:
     oldDist = distTo.get(v)      // previous best path to v
     newDist = distTo.get(u) + w  // what if we went through u?
     if (newDist < oldDist):
       distTo.put(v, newDist)
       edgeTo.put(v, u)
```

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Pseudocode (High-Level)



Ähnlich wie bei „visited“ in BFS, sind „known“ Knoten, die final sind (wir kennen ihren kürzesten Pfad)

Dijkstra’s algorithm is all about updating “best-so-far” in distTo if we find shorter path! Init all paths to infinite.

Reihenfolge ist wichtig: Besuche immer zuerst den nächsten Knoten!

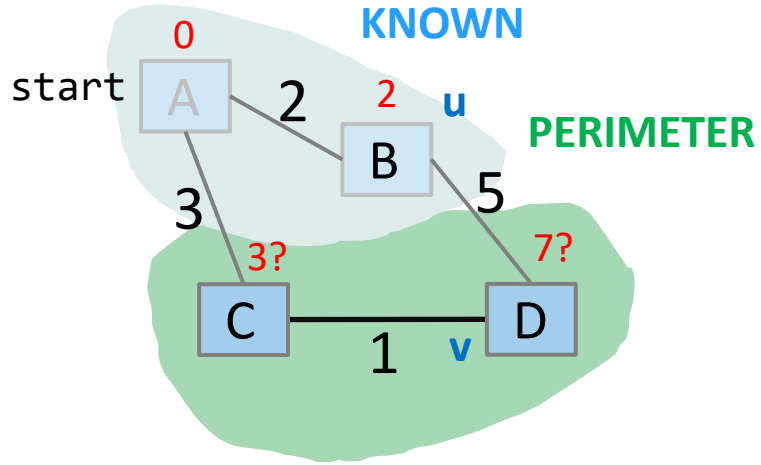
Betrachte alle Nachbarknoten von u: Wäre der Pfad dorthin über u kürzer als der, den Sie derzeit kennen?

```
dijkstraShortestPath(G graph, V start)
  Set known; Map edgeTo, distTo;
  initialize distTo with all nodes mapped to ∞, except start to 0

  while (there are unknown vertices):
    let u be the closest unknown vertex
    known.add(u);
   for each edge (u,v) to unknown v with weight w:
     oldDist = distTo.get(v)      // previous best path to v
     newDist = distTo.get(u) + w  // what if we went through u?
    if (newDist < oldDist):
      distTo.put(v, newDist)
      edgeTo.put(v, u)
```

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Pseudocode (High-Level)



Ähnlich wie bei „visited“ in BFS, sind „known“ Knoten, die final sind (wir kennen ihren kürzesten Pfad)

Dijkstra’s algorithm is all about updating “best-so-far” in distTo if we find shorter path! Init all paths to infinite.

Reihenfolge ist wichtig: Besuche immer zuerst den nächsten Knoten!

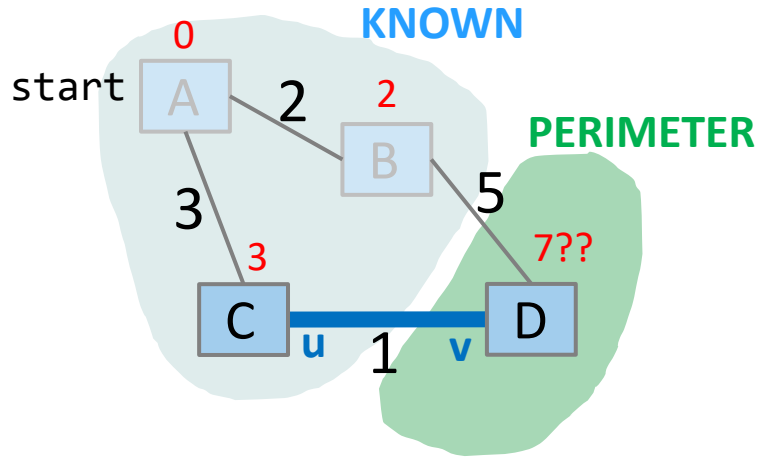
Betrachte alle Nachbarknoten von u: Wäre der Pfad dorthin über u kürzer als der, den Sie derzeit kennen?

```
dijkstraShortestPath(G graph, V start)
  Set known; Map edgeTo, distTo;
  initialize distTo with all nodes mapped to ∞, except start to 0

  while (there are unknown vertices):
    let u be the closest unknown vertex
    known.add(u);
   for each edge (u,v) to unknown v with weight w:
     oldDist = distTo.get(v)      // previous best path to v
     newDist = distTo.get(u) + w  // what if we went through u?
    if (newDist < oldDist):
      distTo.put(v, newDist)
      edgeTo.put(v, u)
```

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Pseudocode (High-Level)



Nehmen Sie an wir haben Knoten B besucht, distTo[D] = 7

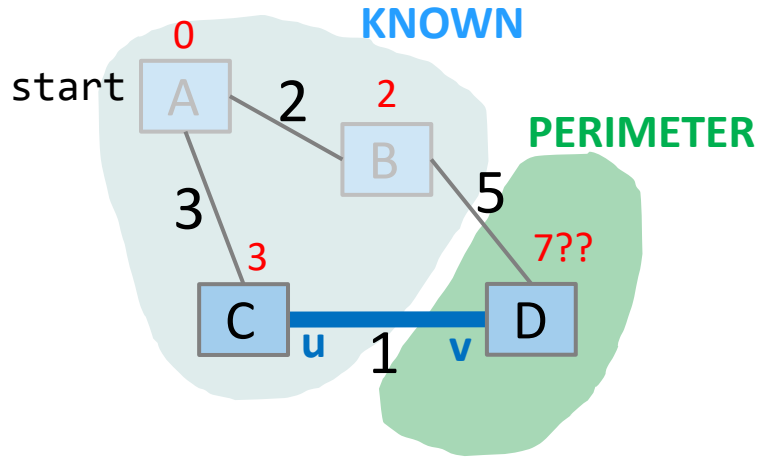
-
-
-

```
dijkstraShortestPath(G graph, V start)
  Set known; Map edgeTo, distTo;
  initialize distTo with all nodes mapped to ∞, except start to 0

  while (there are unknown vertices):
    let u be the closest unknown vertex
    known.add(u);
    for each edge (u,v) to unknown v with weight w:
      oldDist = distTo.get(v)      // previous best path to v
      newDist = distTo.get(u) + w  // what if we went through u?
      if (newDist < oldDist):
        distTo.put(v, newDist)
        edgeTo.put(v, u)
```

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Pseudocode (High-Level)



Nehmen Sie an wir haben Knoten B besucht, distTo[D] = 7

Betrachten Sie nun Kante (C, D):

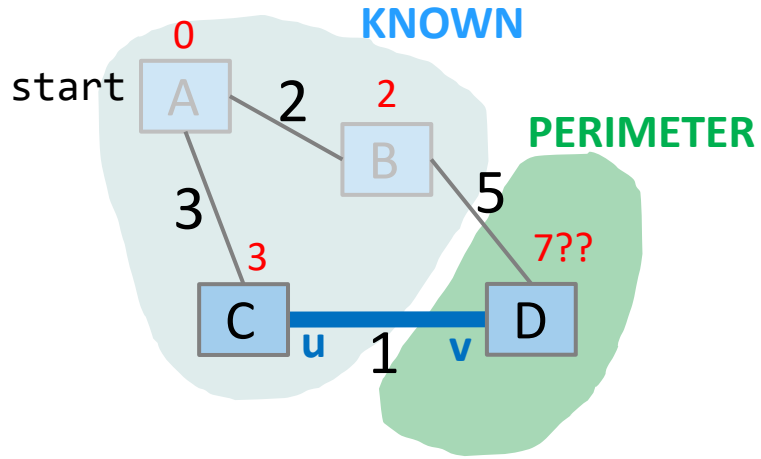
- oldDist = 7
- newDist = 3 + 1
-

```
dijkstraShortestPath(G graph, V start)
  Set known; Map edgeTo, distTo;
  initialize distTo with all nodes mapped to ∞, except start to 0

  while (there are unknown vertices):
    let u be the closest unknown vertex
    known.add(u);
   for each edge (u,v) to unknown v with weight w:
     oldDist = distTo.get(v)      // previous best path to v
     newDist = distTo.get(u) + w  // what if we went through u?
     if (newDist < oldDist):
       distTo.put(v, newDist)
       edgeTo.put(v, u)
```

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Pseudocode (High-Level)



Nehmen Sie an wir haben Knoten B besucht, distTo[D] = 7

Betrachten Sie nun Kante (C, D):

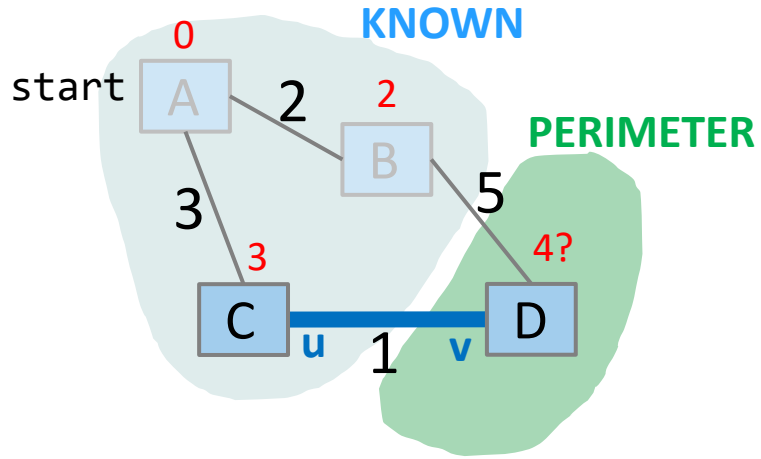
- oldDist = 7
- newDist = 3 + 1
- Das ist besser! Aktualisiere distTo[D], edgeTo[D]

```
dijkstraShortestPath(G graph, V start)
  Set known; Map edgeTo, distTo;
  initialize distTo with all nodes mapped to ∞, except start to 0

  while (there are unknown vertices):
    let u be the closest unknown vertex
    known.add(u);
   for each edge (u,v) to unknown v with weight w:
     oldDist = distTo.get(v)      // previous best path to v
     newDist = distTo.get(u) + w  // what if we went through u?
     if (newDist < oldDist):
       distTo.put(v, newDist)
       edgeTo.put(v, u)
```

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Pseudocode (High-Level)



Nehmen Sie an wir haben Knoten B besucht, distTo[D] = 7

Betrachten Sie nun Kante (C, D):

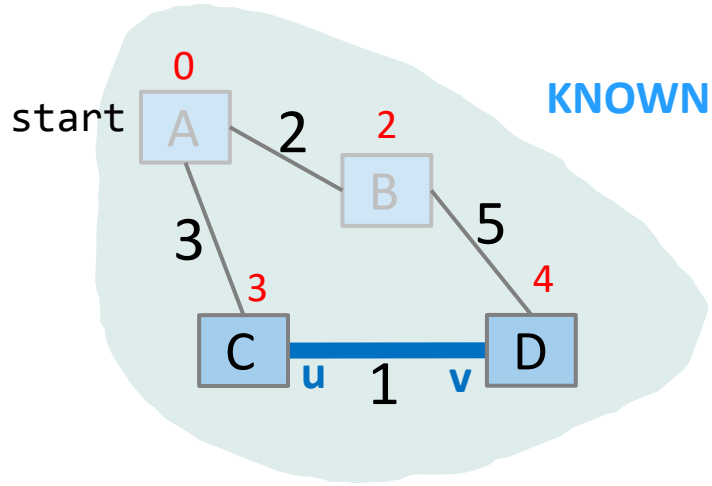
- oldDist = 7
- newDist = 3 + 1
- Das ist besser! Aktualisiere distTo[D], edgeTo[D]

```
dijkstraShortestPath(G graph, V start)
  Set known; Map edgeTo, distTo;
  initialize distTo with all nodes mapped to ∞, except start to 0

  while (there are unknown vertices):
    let u be the closest unknown vertex
    known.add(u);
   for each edge (u,v) to unknown v with weight w:
     oldDist = distTo.get(v)      // previous best path to v
     newDist = distTo.get(u) + w  // what if we went through u?
     if (newDist < oldDist):
       distTo.put(v, newDist)
       edgeTo.put(v, u)
```

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Pseudocode (High-Level)



Nehmen Sie an wir haben Knoten B besucht, distTo[D] = 7

Betrachten Sie nun Kante (C, D):

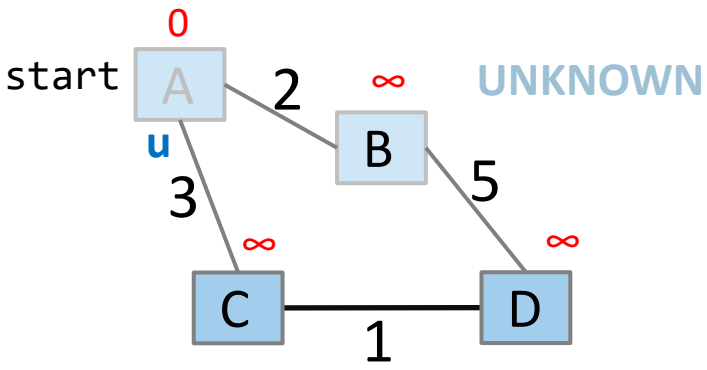
- oldDist = 7
- newDist = 3 + 1
- Das ist besser! Aktualisiere distTo[D], edgeTo[D]

```
dijkstraShortestPath(G graph, V start)
  Set known; Map edgeTo, distTo;
  initialize distTo with all nodes mapped to ∞, except start to 0

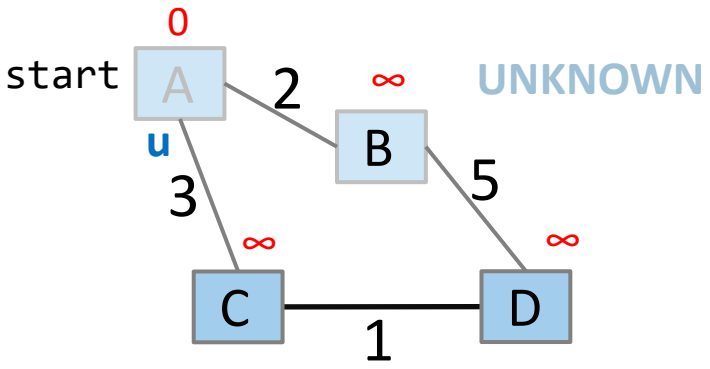
  while (there are unknown vertices):
    let u be the closest unknown vertex
    known.add(u);
   for each edge (u,v) to unknown v with weight w:
     oldDist = distTo.get(v)      // previous best path to v
     newDist = distTo.get(u) + w  // what if we went through u?
     if (newDist < oldDist):
       distTo.put(v, newDist)
       edgeTo.put(v, u)
```

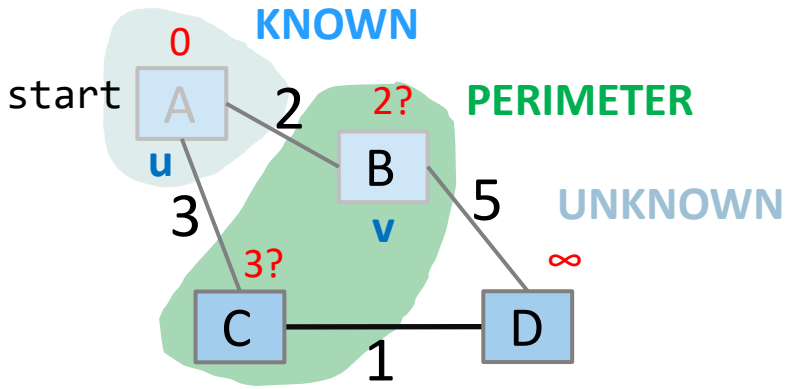
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Algorithmus in Bildern

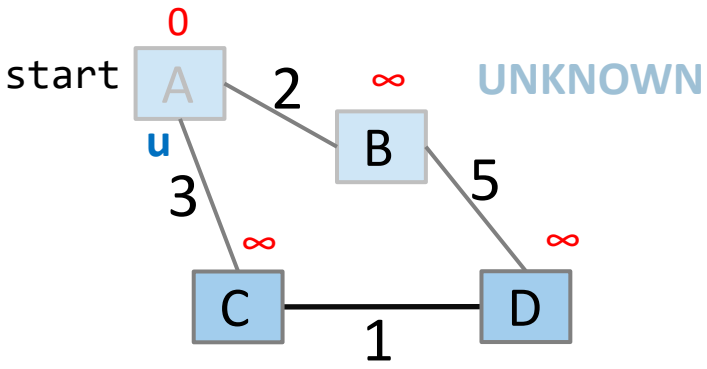


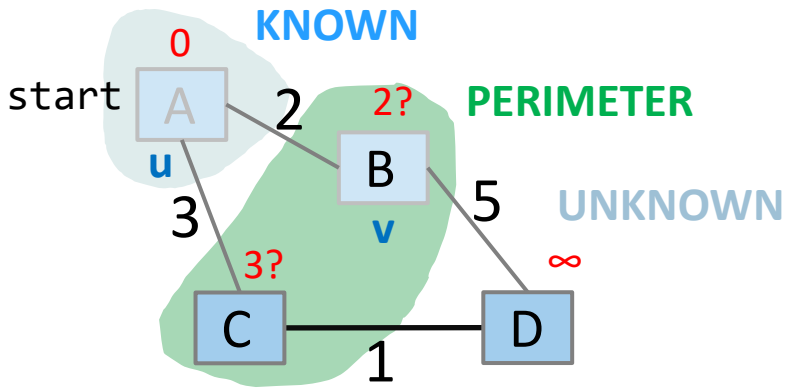
Dijkstras Algorithmus in Bildern

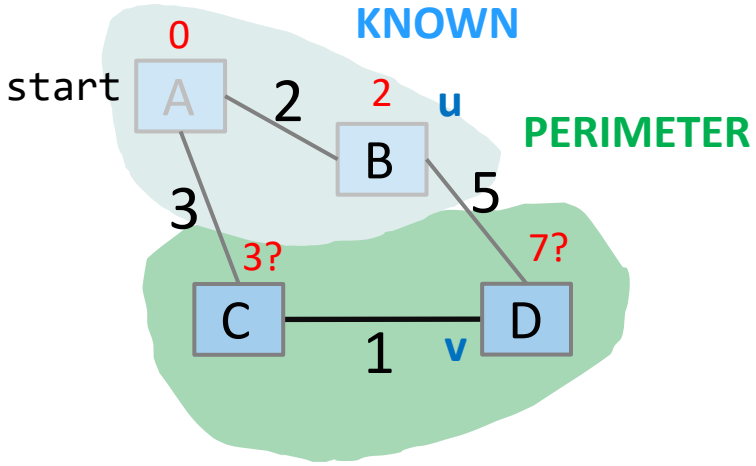




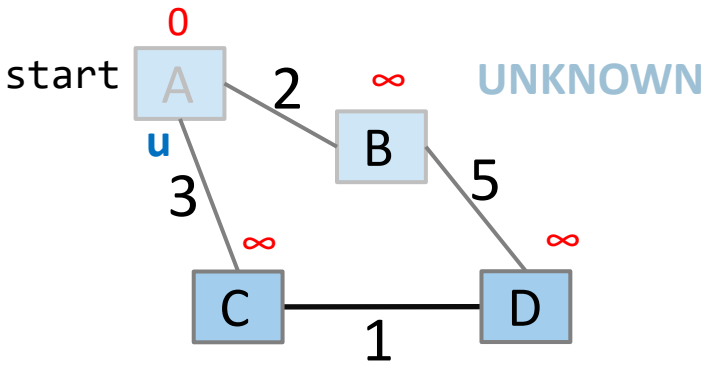
Dijkstras Algorithmus in Bildern

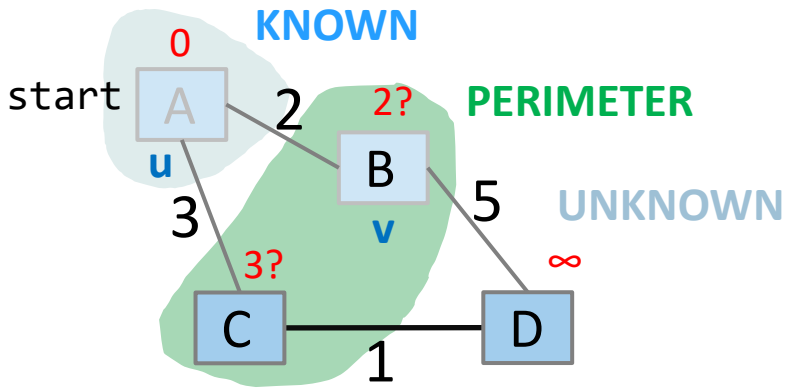


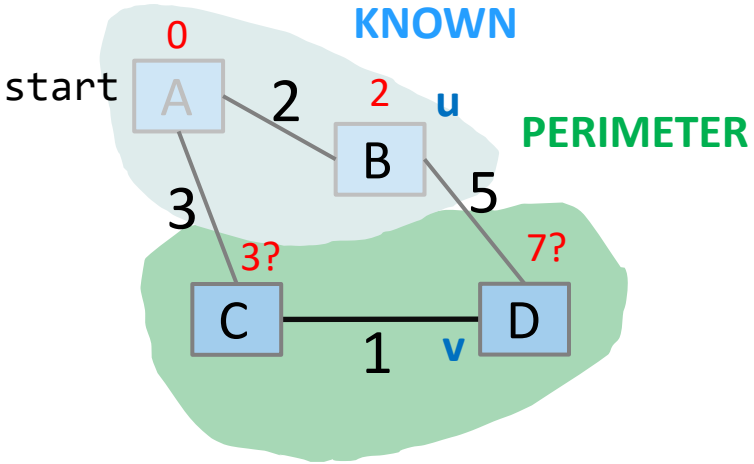


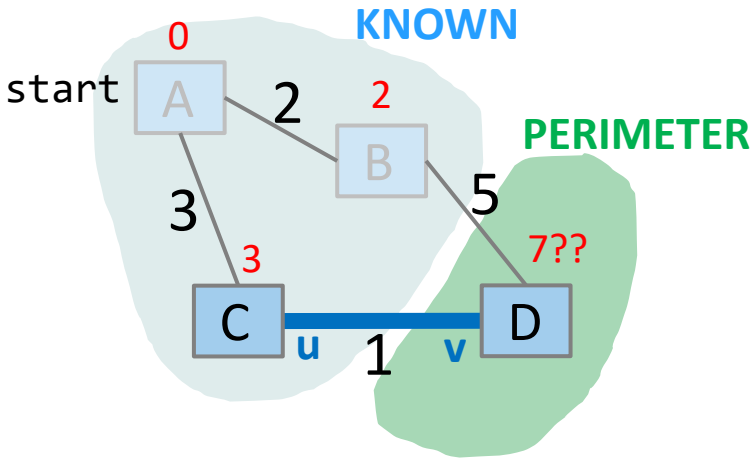


Dijkstras Algorithmus in Bildern

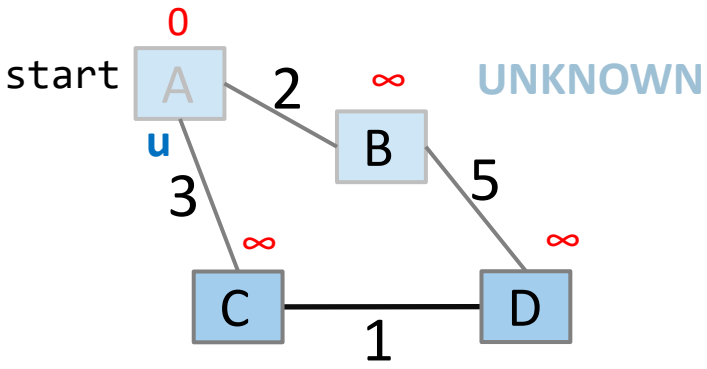


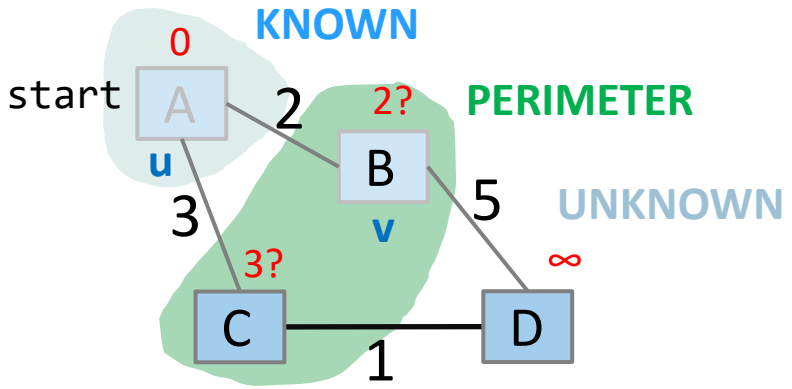


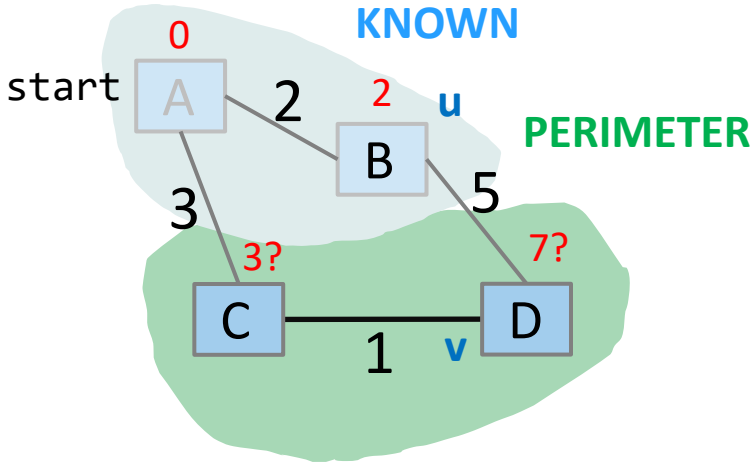


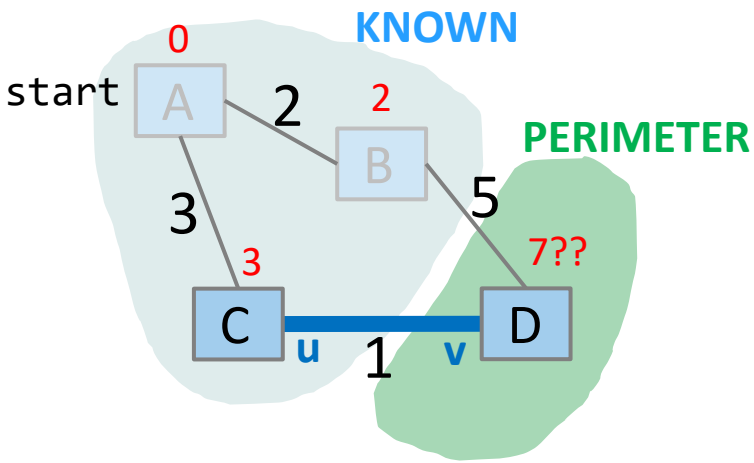


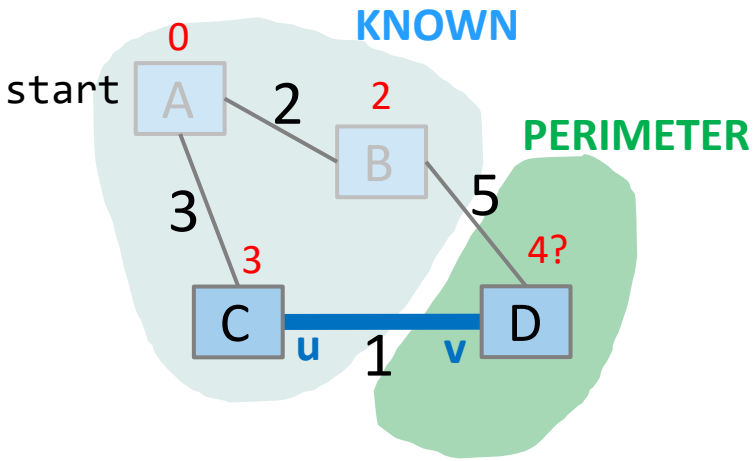
Dijkstras Algorithmus in Bildern



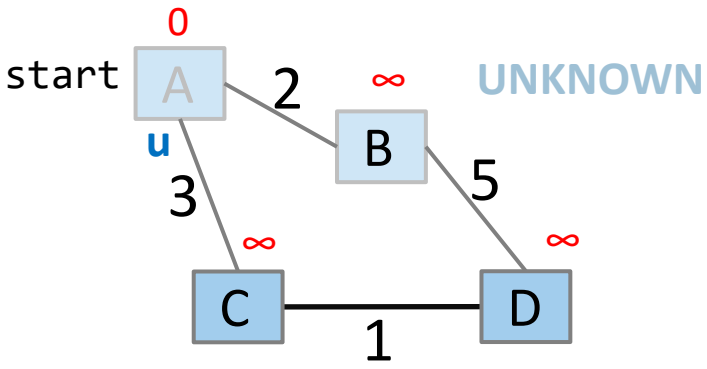


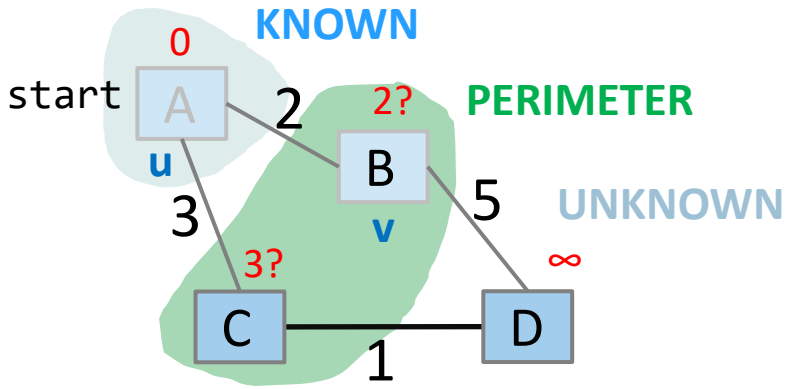


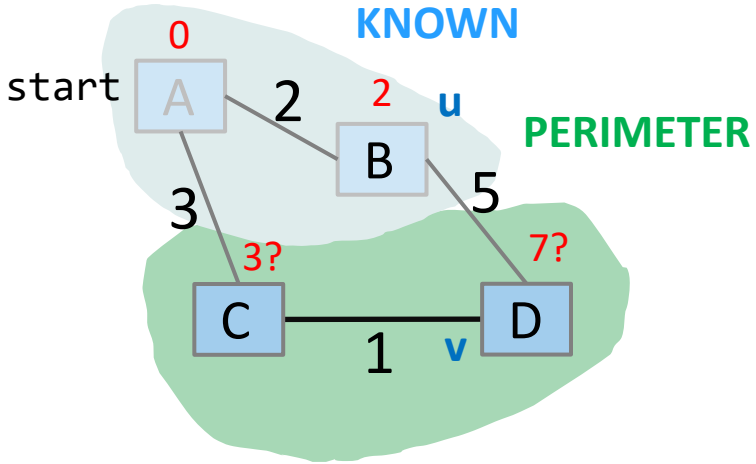


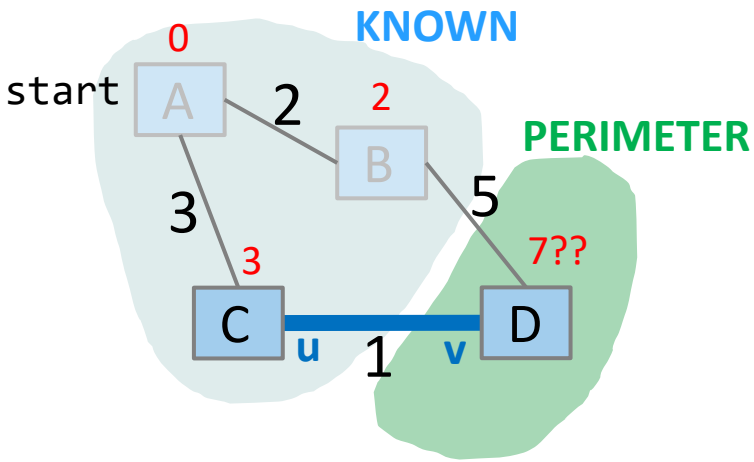


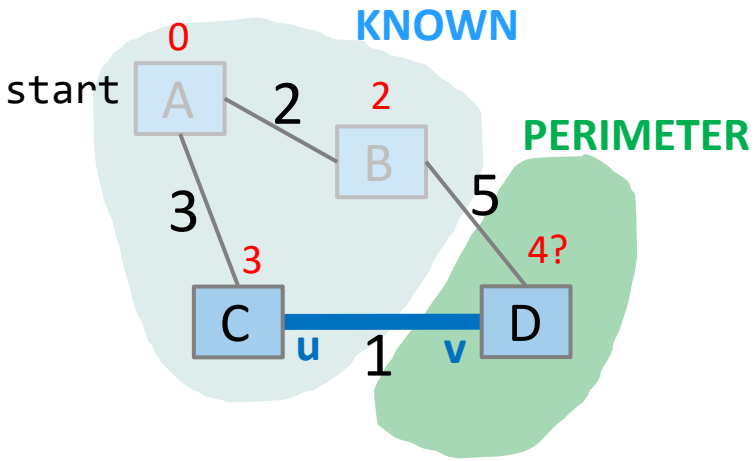
Dijkstras Algorithmus in Bildern

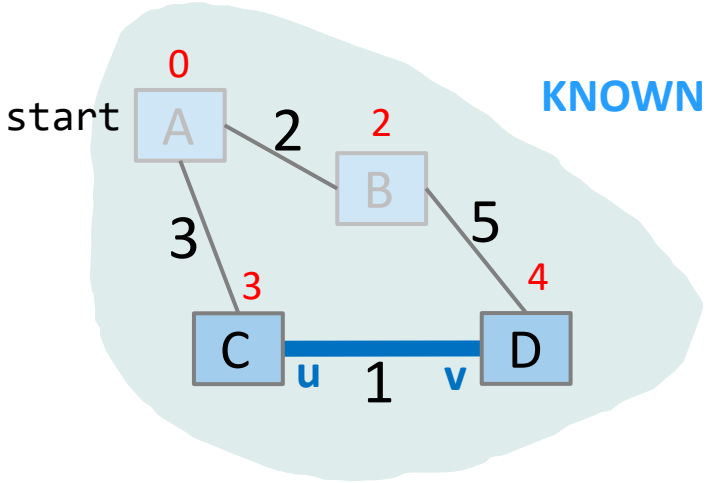












Dijkstras Algorithmus: Wichtigste Eigenschaften

Sobald ein Knoten markiert ist (d.h. in Menge „known“ ist), ist sein kürzester Pfad bekannt

- Rekonstruktion des Pfades durch Verfolgung von Back-Pointern (in der edgeTo-Map)

Dijkstras Algorithmus: Wichtigste Eigenschaften

Sobald ein Knoten markiert ist (d.h. in Menge „known“ ist), ist sein kürzester Pfad bekannt

- Rekonstruktion des Pfades durch Verfolgung von Back-Pointern (in der edgeTo-Map)

Solange ein Knoten nicht bekannt ist, kann ein anderer kürzerer Pfad gefunden werden.

- Wir nennen diese Aktualisierung „Relaxierung der Distanz", weil sie immer nur den aktuell besten Pfad verkürzt

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

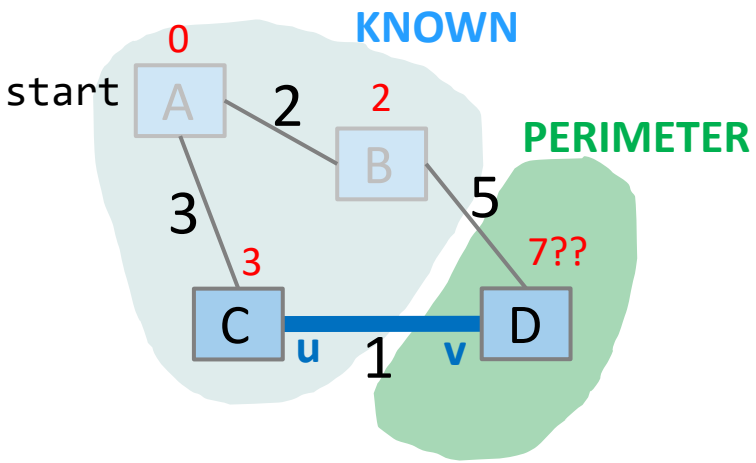
Dijkstras Algorithmus: Wichtigste Eigenschaften

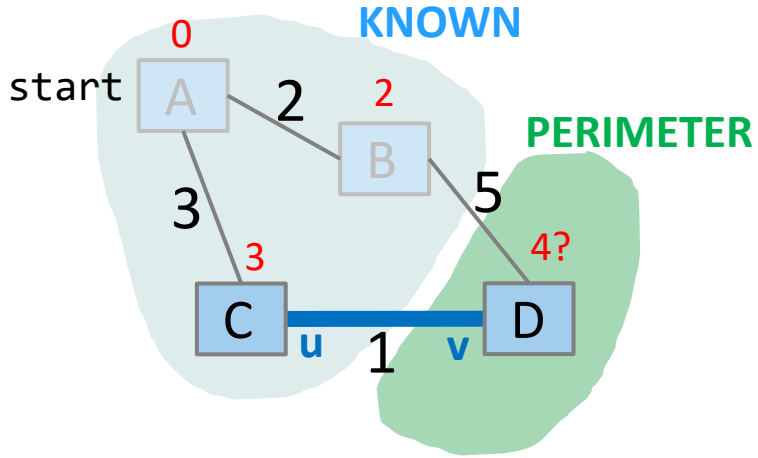
Sobald ein Knoten markiert ist (d.h. in Menge „known“ ist), ist sein kürzester Pfad bekannt

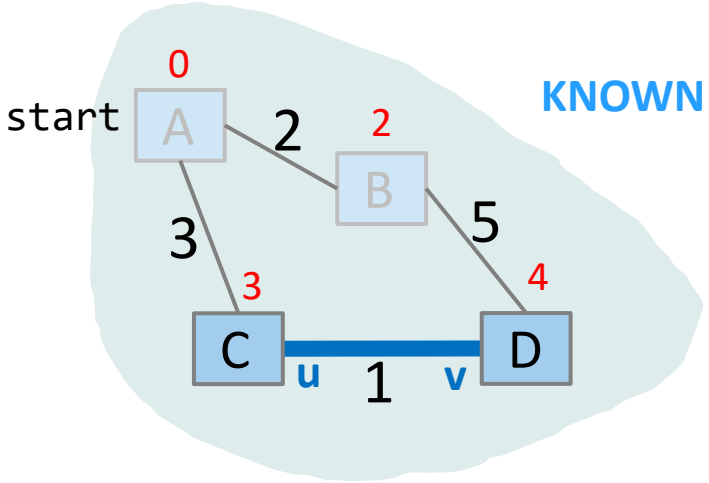
- Rekonstruktion des Pfades durch Verfolgung von Back-Pointern (in der edgeTo-Map)

Solange ein Knoten nicht bekannt ist, kann ein anderer kürzerer Pfad gefunden werden.

- Wir nennen diese Aktualisierung „Relaxierung der Distanz", weil sie immer nur den aktuell besten Pfad verkürzt







Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Algorithmus: Wichtigste Eigenschaften

Da wir zuerst die nächstgelegenen Knoten durchgehen, können wir mit Sicherheit sagen, dass kein kürzerer Pfad gefunden werden kann, sobald der endgültige kürzeste Pfad bekannt ist

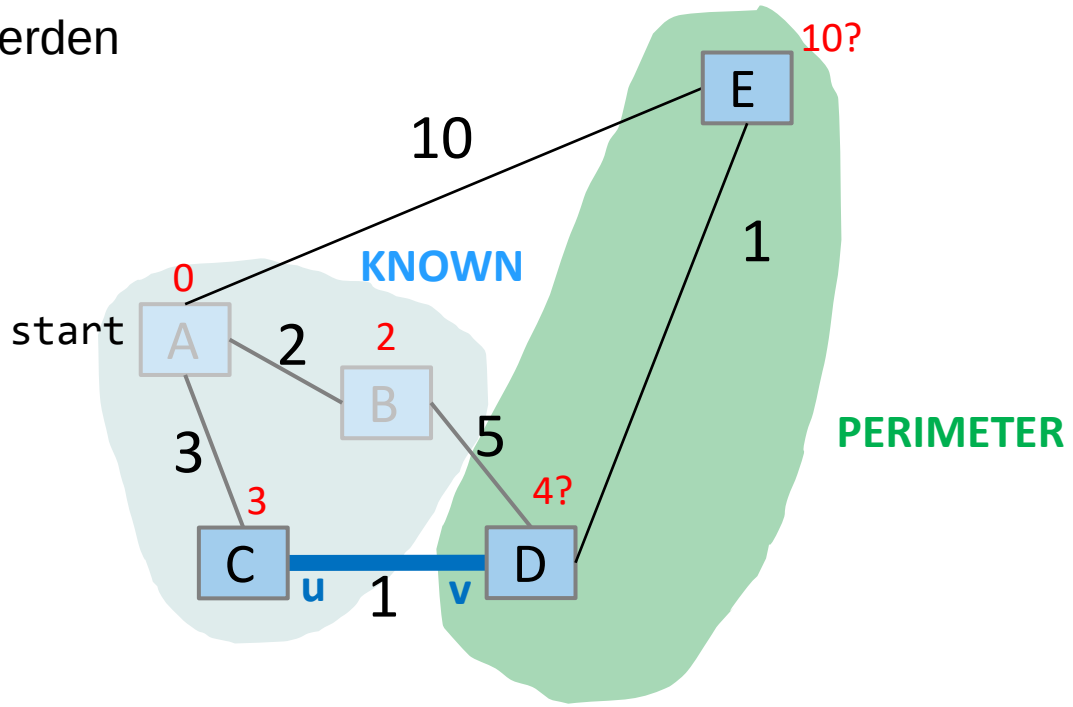
- Weil es nicht möglich ist, einen kürzeren Pfad zu finden, der einen weiter entfernten Knoten benutzt, den wir später betrachten werden

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Algorithmus: Wichtigste Eigenschaften

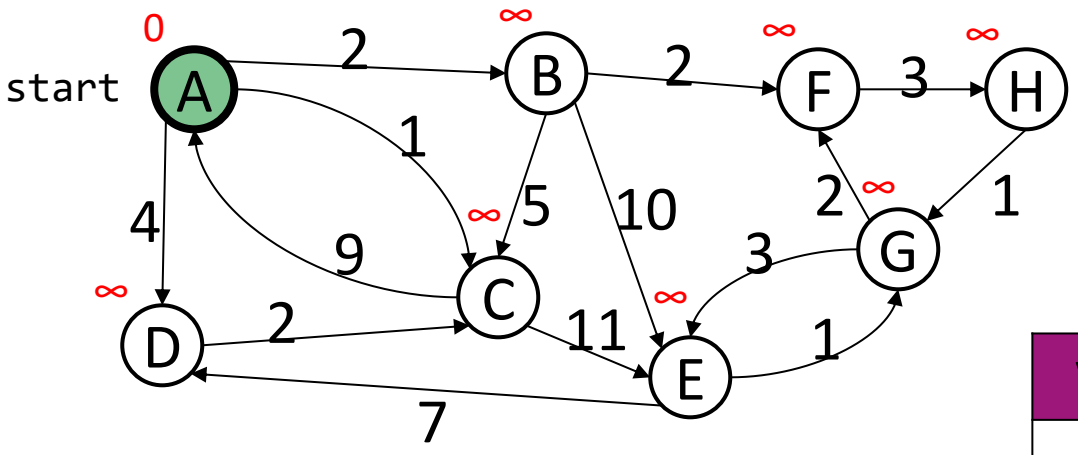
Da wir zuerst die nächstgelegenen Knoten durchgehen, können wir mit Sicherheit sagen, dass kein kürzerer Pfad gefunden werden kann, sobald der endgültige kürzeste Pfad bekannt ist

- Weil es nicht möglich ist, einen kürzeren Pfad zu finden, der einen weiter entfernten Knoten benutzt, den wir später betrachten werden



Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Algorithmus: Beispiel




```
while (there are unknown vertices):
  let u be the closest unknown vertex
  known.add(u);
  for each edge (u,v) to unknown v with w:
    if distTo.get(u) + w < distTo.get(v):
```

Vertex	Known?	distTo	edgeTo
A		0	
B		∞	
C		∞	
D		∞	
E		∞	
F		∞	
G		∞	
H		∞	

Order Added to Known Set:

Prof. Dr. Daniel Galda

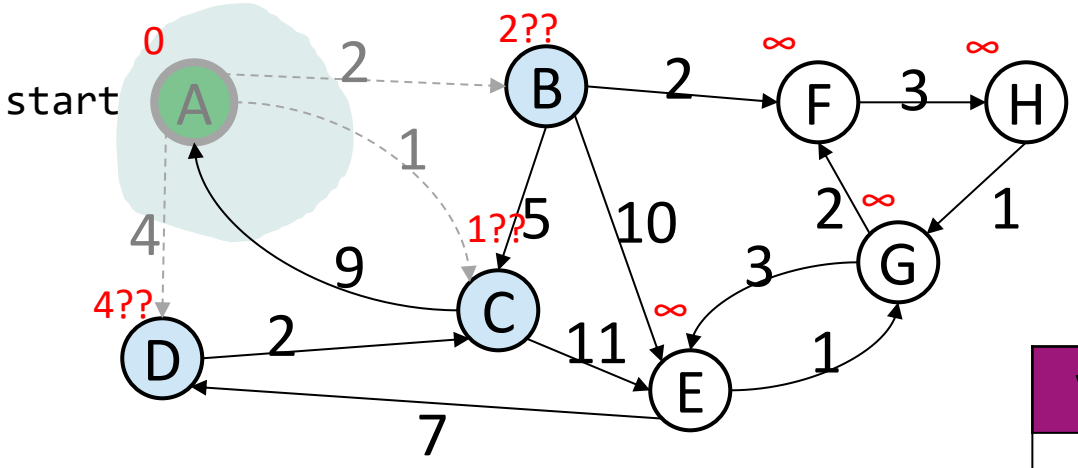
Professor für Cyber-Physische Systeme

Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Technology Arts Sciences TH Köln

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Algorithmus: Beispiel




```
while (there are unknown vertices):
  let u be the closest unknown vertex
  known.add(u);
  for each edge (u,v) to unknown v with w:
    if distTo.get(u) + w < distTo.get(v):
```

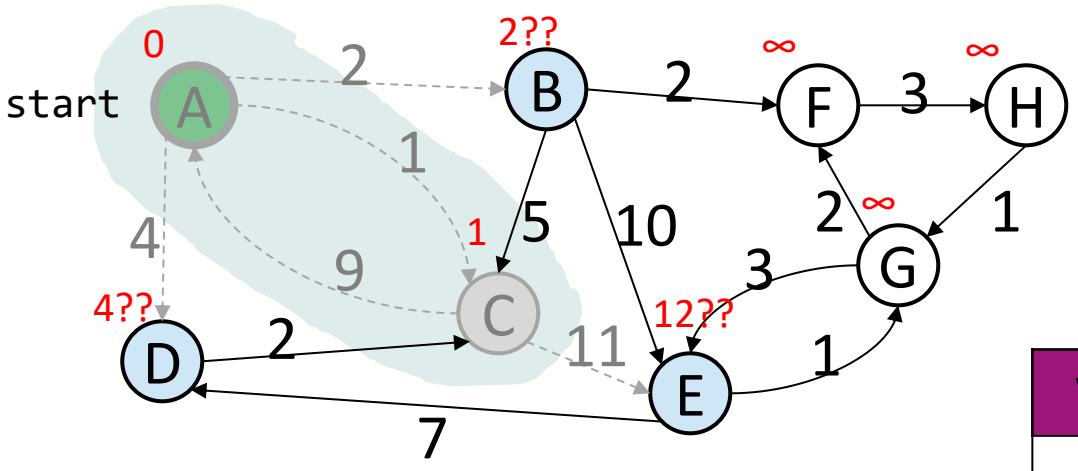
Vertex	Known?	distTo	edgeTo
A	Y	0	/
B		≤ 2	A
C		≤ 1	A
D		≤ 4	A
E		∞	
F		∞	
G		∞	
H		∞	

Order Added to Known Set:
A

Prof. Dr. Daniel Galda
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Technology Arts Sciences TH Köln

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Algorithmus: Beispiel



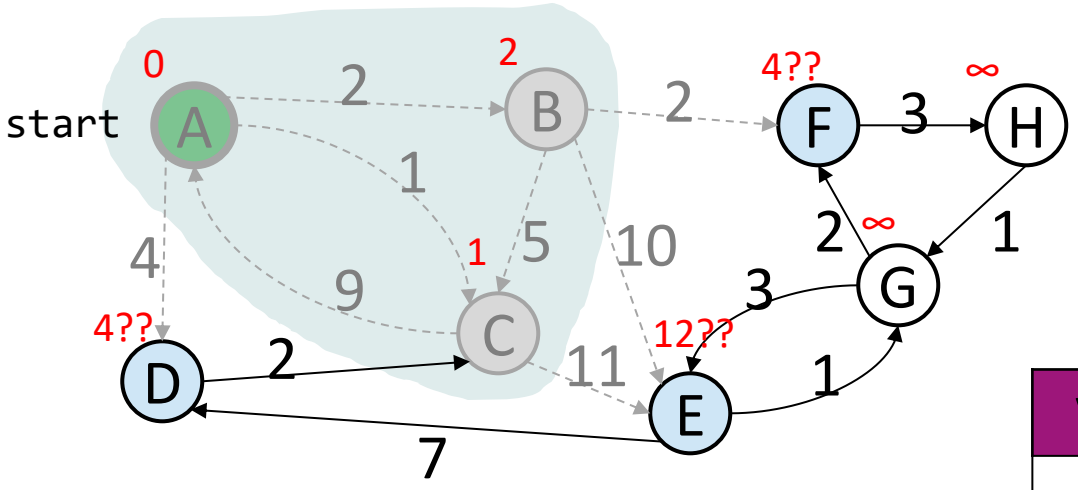

```
while (there are unknown vertices):
  let u be the closest unknown vertex
  known.add(u);
  for each edge (u,v) to unknown v with w:
    if distTo.get(u) + w < distTo.get(v):
```

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B		≤ 2	A
C	Y	1	A
D		≤ 4	A
E		**≤ 12**	**C**
F		∞	
G		∞	
H		∞	

Order Added to Known Set:
A, C

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Algorithmus: Beispiel




```
while (there are unknown vertices):
  let u be the closest unknown vertex
  known.add(u);
  for each edge (u,v) to unknown v with w:
    if distTo.get(u) + w < distTo.get(v):
```

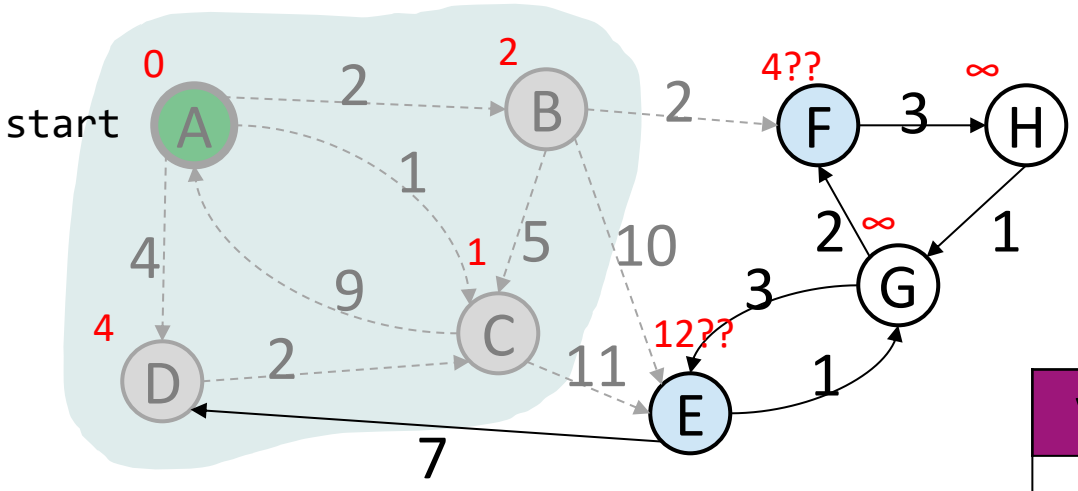
Vertex	Known?	distTo	edgeTo
A	Y	0	/
B	Y	2	A
C	Y	1	A
D		≤ 4	A
E		≤ 12	C
F		**≤ 4**	**B**
G		∞	
H		∞	

Order Added to Known Set:
A, C, B

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Algorithmus: Beispiel




```
while (there are unknown vertices):
  let u be the closest unknown vertex
  known.add(u);
  for each edge (u,v) to unknown v with w:
    if distTo.get(u) + w < distTo.get(v):
```

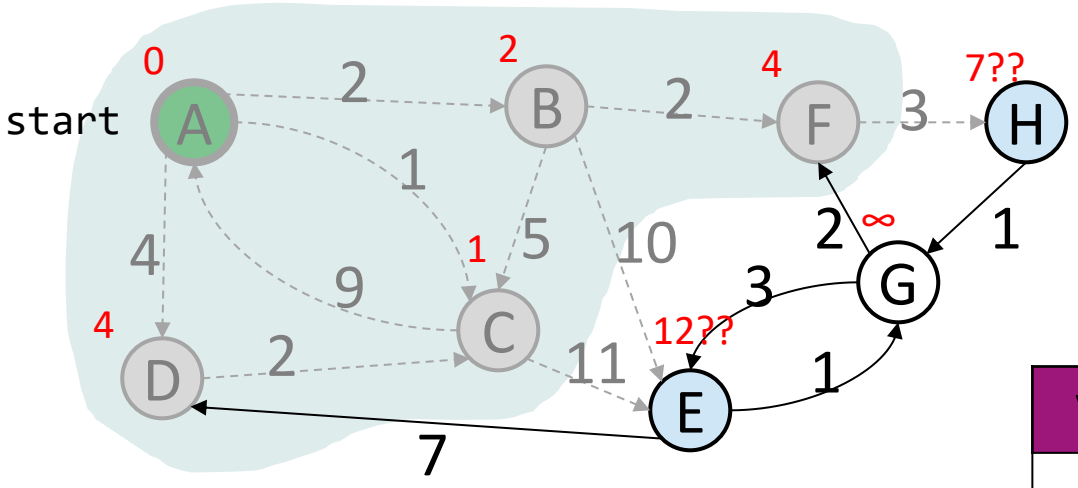
Vertex	Known?	distTo	edgeTo
A	Y	0	/
B	Y	2	A
C	Y	1	A
D	Y	4	A
E		≤ 12	C
F		≤ 4	B
G		∞	
H		∞	

Order Added to Known Set:
A, C, B, D

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Algorithmus: Beispiel




```
while (there are unknown vertices):
  let u be the closest unknown vertex
  known.add(u);
  for each edge (u,v) to unknown v with w:
    if distTo.get(u) + w < distTo.get(v):
```

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B	Y	2	A
C	Y	1	A
D	Y	4	A
E		≤ 12	C
F	Y	4	B
G		∞	
H		**≤ 7**	**F**

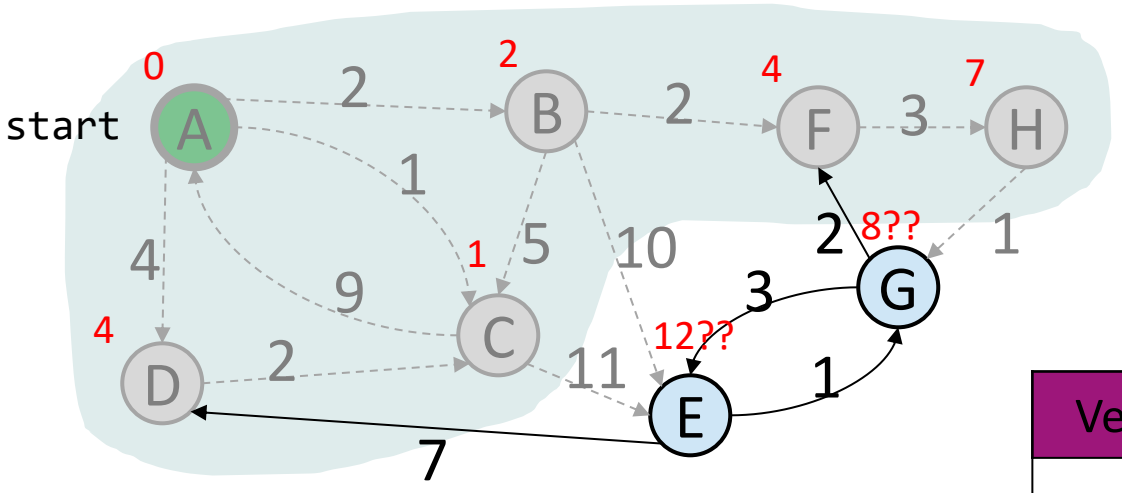
Order Added to Known Set:
A, C, B, D, F

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Technology Arts Sciences TH Köln

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Algorithmus: Beispiel




```
while (there are unknown vertices):
  let u be the closest unknown vertex
  known.add(u);
  for each edge (u,v) to unknown v with w:
    if distTo.get(u) + w < distTo.get(v):
```

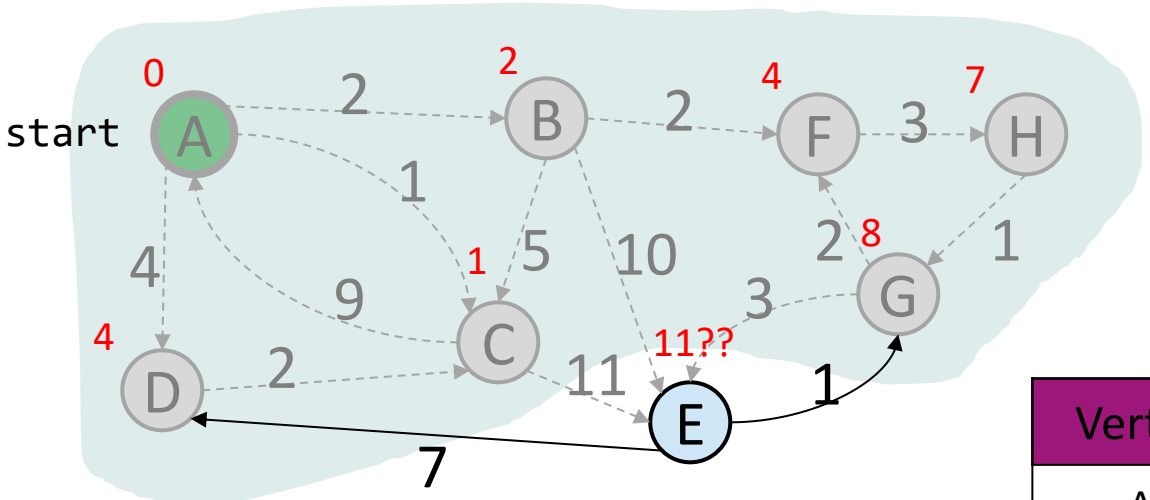
Vertex	Known?	distTo	edgeTo
A	Y	0	/
B	Y	2	A
C	Y	1	A
D	Y	4	A
E		≤ 12	C
F	Y	4	B
G		**≤ 8**	**H**
H	Y	7	F

Order Added to Known Set:
A, C, B, D, F, H

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Technology Arts Sciences TH Köln

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Algorithmus: Beispiel




```
while (there are unknown vertices):
  let u be the closest unknown vertex
  known.add(u);
  for each edge (u,v) to unknown v with w:
    if distTo.get(u) + w < distTo.get(v):
```

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B	Y	2	A
C	Y	1	A
D	Y	4	A
E		**≤ 11**	**G**
F	Y	4	B
G	Y	8	H
H	Y	7	F

Order Added to Known Set:
A, C, B, D, F, H, G

Prof. Dr. Daniel Gaida

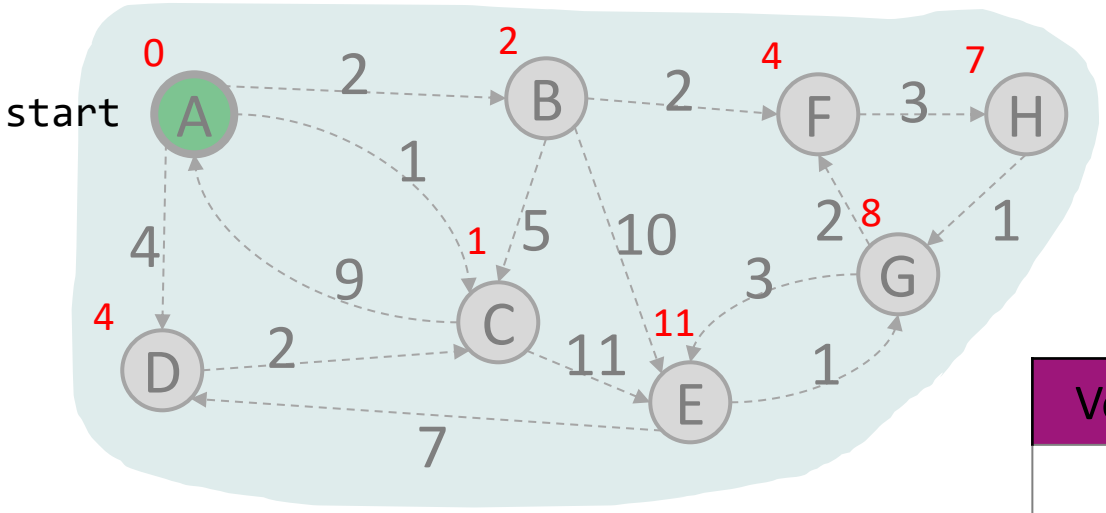
Professor für Cyber-Physische Systeme

Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Technology Arts Sciences TH Köln

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Algorithmus: Beispiel




```
while (there are unknown vertices):
  let u be the closest unknown vertex
  known.add(u);
  for each edge (u,v) to unknown v with w:
    if distTo.get(u) + w < distTo.get(v):
```

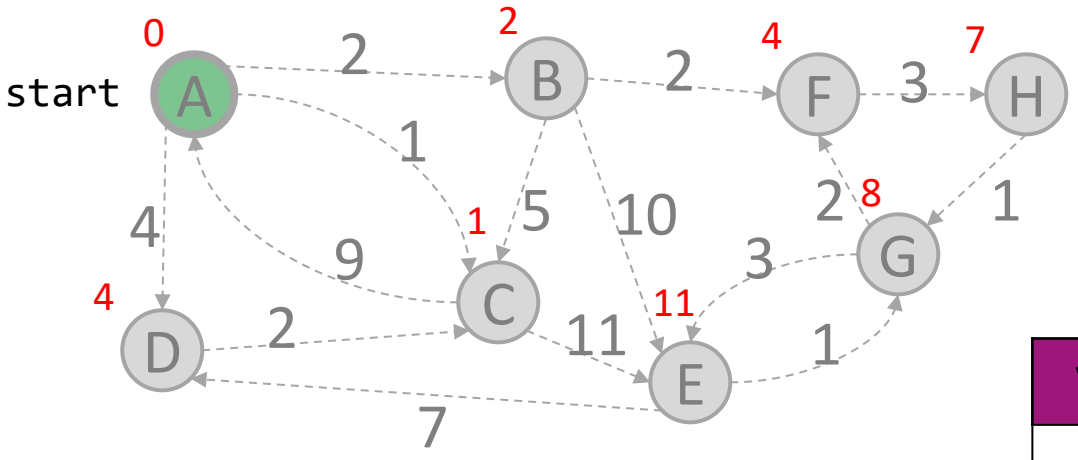
Vertex	Known?	distTo	edgeTo
A	Y	0	/
B	Y	2	A
C	Y	1	A
D	Y	4	A
E	Y	11	G
F	Y	4	B
G	Y	8	H
H	Y	7	F

Order Added to Known Set:
A, C, B, D, F, H, G, E

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Algorithmus: Interpretation der Ergebnisse



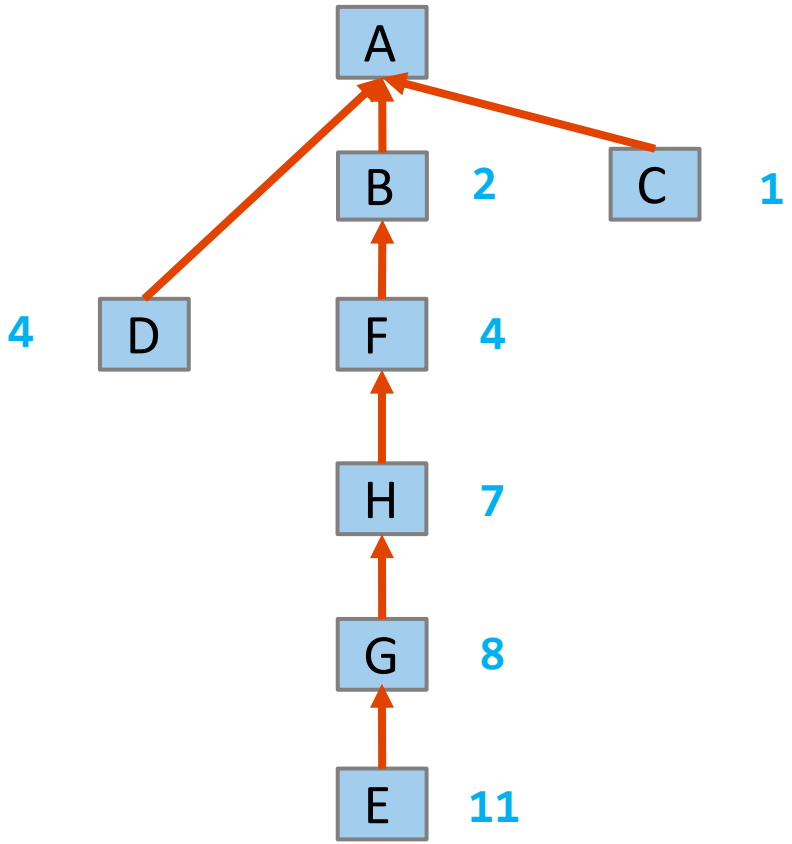
Wie erhalten wir den kürzesten Pfad von A nach E? → Folgen Sie edgeTo Backpointers!

distTo und edgeTo bilden den Baum kürzester Pfade

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B	Y	2	**A**
C	Y	1	A
D	Y	4	A
E	Y	11	**G**
F	Y	4	**B**
G	Y	8	**H**
H	Y	7	**F**

Pfad entlang edgeTo von E zu A:
E, G, H, F, B, A

Dijkstras Algorithmus: Baum kürzester Pfade



Vertex	Known?	distTo	edgeTo
A	Y	0	/
B	Y	2	**A**
C	Y	1	A
D	Y	4	A
E	Y	11	**G**
F	Y	4	**B**
G	Y	8	**H**
H	Y	7	**F**

Dijkstras Algorithmus: Wichtigste Eigenschaften (cont‘d)

Die Reihenfolge, wie zu der „known“ Menge bei Gleichheit der Distanz hinzugefügt wird, ist unwichtig

Dijkstras Algorithmus: Wichtigste Eigenschaften (cont‘d)

Die Reihenfolge, wie zu der „known“ Menge bei Gleichheit der Distanz hinzugefügt wird, ist unwichtig

Wenn wir nur den kürzesten Pfad zu einem bestimmten Knoten benötigen, können wir aufhören, sobald dieser Knoten bekannt ist

- Weil sich sein kürzester Pfad nicht ändern kann!
- Rückgabe eines Teilbaums kürzester Pfade

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Algorithmus: Implementierung

Wie implementieren wir `let u be the closest unknown vertex`?

Es wäre sicher praktisch, Knoten in einer Datenstruktur zu speichern, die ...

-
-
-

```
dijkstraShortestPath(G graph, V start)
  Set known; Map edgeTo, distTo;
  initialize distTo with all nodes mapped to ∞, except start to 0

  while (there are unknown vertices):
    let u be the closest unknown vertex
    known.add(u)
    for each edge (u,v) to unknown v with weight w:
      oldDist = distTo.get(v)      // previous best path to v
      newDist = distTo.get(u) + w  // what if we went through u?
      if (newDist < oldDist):
        distTo.put(v, newDist)
        edgeTo.put(v, u)
```

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Technology Arts Sciences TH Köln

Dijkstras Algorithmus: Implementierung

Wie implementieren wir `let u be the closest unknown vertex`?

Es wäre sicher praktisch, Knoten in einer Datenstruktur zu speichern, die ...

- jedem Knoten einen Abstandswert zuweist
- Den Knoten mit dem geringsten Abstand schnell auswählt
-

```
dijkstraShortestPath(G graph, V start)
  Set known; Map edgeTo, distTo;
  initialize distTo with all nodes mapped to ∞, except start to 0

  while (there are unknown vertices):
    let u be the closest unknown vertex
    known.add(u)
    for each edge (u,v) to unknown v with weight w:
      oldDist = distTo.get(v)      // previous best path to v
      newDist = distTo.get(u) + w  // what if we went through u?
      if (newDist < oldDist):
        distTo.put(v, newDist)
        edgeTo.put(v, u)
```

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Technology Arts Sciences TH Köln

Dijkstras Algorithmus: Implementierung

Wie implementieren wir `let u be the closest unknown vertex`?

Es wäre sicher praktisch, Knoten in einer Datenstruktur zu speichern, die ...

- jedem Knoten einen Abstandswert zuweist
- Den Knoten mit dem geringsten Abstand schnell auswählt
- diesen Abstand aktualisieren lässt, wenn wir neue, bessere Pfade entdecken

```
dijkstraShortestPath(G graph, V start)
  Set known; Map edgeTo, distTo;
  initialize distTo with all nodes mapped to ∞, except start to 0

  while (there are unknown vertices):
    let u be the closest unknown vertex
    known.add(u)
    for each edge (u,v) to unknown v with weight w:
      oldDist = distTo.get(v)      // previous best path to v
      newDist = distTo.get(u) + w  // what if we went through u?
      if (newDist < oldDist):
        distTo.put(v, newDist)
        edgeTo.put(v, u)
```

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Technology Arts Sciences TH Köln

Dijkstras Algorithmus: Implementierung

Wie implementieren wir `let u be the closest unknown vertex`?

Es wäre sicher praktisch, Knoten in einer Datenstruktur zu speichern, die ...

- jedem Knoten einen Abstandswert zuweist
- Den Knoten mit dem geringsten Abstand schnell auswählt
- diesen Abstand aktualisieren lässt, wenn wir neue, bessere Pfade entdecken

```
dijkstraShortestPath(G graph, V start)
  Set known; Map edgeTo, distTo;
  initialize distTo with all nodes mapped to ∞, except start to 0

  while (there are unknown vertices):
    let u be the closest unknown vertex
    known.add(u)
    for each edge (u,v) to unknown v with weight w:
      oldDist = distTo.get(v)      // previous best path to v
      newDist = distTo.get(u) + w  // what if we went through u?
      if (newDist < oldDist):
        distTo.put(v, newDist)
        edgeTo.put(v, u)
```

MIN PRIORITÄTSWARTESCHLANGE ADT

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Algorithmus: Implementierung

Verwendung einer Min-Prioritätswarteschlange zur Erfassung des Perimeters

- Es muss nicht der gesamte Graph erfasst werden
-

```
dijkstraShortestPath(G graph, V start)
  Map edgeTo, distTo;
  initialize distTo with all nodes mapped to ∞, except start to 0

  PriorityQueue<V> perimeter; perimeter.add(start);

  while (!perimeter.isEmpty()):
    u = perimeter.removeMin()

    for each edge (u,v) to v with weight w:
      oldDist = distTo.get(v)      // previous best path to v
      newDist = distTo.get(u) + w  // what if we went through u?
      if (newDist < oldDist):
        distTo.put(v, newDist)
        edgeTo.put(v, u)
        if (perimeter.contains(v)):
          perimeter.changePriority(v, newDist)
        else:
          perimeter.add(v, newDist)
```


Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Arts Sciences
TH Köln

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Algorithmus: Implementierung

Verwendung einer Min-Prioritätswarteschlange zur Erfassung des Perimeters

- Es muss nicht der gesamte Graph erfasst werden
- Keine separate „known" Menge erforderlich – newDist wird für einen „known“ Knoten nie kleiner sein als oldDist

```
dijkstraShortestPath(G graph, V start)
  Map edgeTo, distTo;
  initialize distTo with all nodes mapped to ∞, except start to 0

  PriorityQueue<V> perimeter; perimeter.add(start);

  while (!perimeter.isEmpty()):
    u = perimeter.removeMin()

    for each edge (u,v) to v with weight w:
      oldDist = distTo.get(v)      // previous best path to v
      newDist = distTo.get(u) + w  // what if we went through u?
      if (newDist < oldDist):
        distTo.put(v, newDist)
        edgeTo.put(v, u)
        if (perimeter.contains(v)):
          perimeter.changePriority(v, newDist)
        else:
          perimeter.add(v, newDist)
```


Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Arts Sciences
TH Köln

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Algorithmus: Implementierung für Praktikum (alt)

Verwendung einer Min-Prioritätswarteschlange zur Erfassung des Perimeters als Tabelle (enthält distTo)

```
dijkstraShortestPath(G graph, V start)
  Set known; Map edgeTo;

  PriorityQueue<V, dist> perimeter; perimeter.add(start, 0);

  while (!perimeter.isEmpty()):
    u, u_dist = perimeter.removeMin()
    known.add(u)

    for each edge (u,v) to unknown v with weight w:
      oldDist = perimeter.get_dist(v)      // previous best path to v
      newDist = u_dist + w  // what if we went through u?
      if (newDist < oldDist):
        edgeTo.put(v, u)
        if (perimeter.contains(v)):
          perimeter.changePriority(v, newDist)
        else:
          perimeter.add(v, newDist)
```


Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Arts Sciences
TH Köln

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Algorithmus: Laufzeit

$\Theta(|V|)$ →

```
dijkstraShortestPath(G graph, V start)
  Map edgeTo, distTo;
  initialize distTo with all nodes mapped to ∞, except start to 0

  PriorityQueue<V> perimeter; perimeter.add(start);

  while (!perimeter.isEmpty()):
    u = perimeter.removeMin()

    for each edge (u,v) to unknown v with weight w:
      oldDist = distTo.get(v)      // previous best path to v
      newDist = distTo.get(u) + w  // what if we went through u?
      if (newDist < oldDist):
        distTo.put(v, newDist)
        edgeTo.put(v, u)
        if (perimeter.contains(v)):
          perimeter.changePriority(v, newDist)
        else:
          perimeter.add(v, newDist)
```

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Algorithmus: Laufzeit

$\Theta(|V|)$ →

$\Theta(|V|)$ iterations →

```
dijkstraShortestPath(G graph, V start)
  Map edgeTo, distTo;
  initialize distTo with all nodes mapped to ∞, except start to 0

  PriorityQueue<V> perimeter; perimeter.add(start);

  while (!perimeter.isEmpty()):
    u = perimeter.removeMin()

    for each edge (u,v) to unknown v with weight w:
      oldDist = distTo.get(v)      // previous best path to v
      newDist = distTo.get(u) + w  // what if we went through u?
      if (newDist < oldDist):
        distTo.put(v, newDist)
        edgeTo.put(v, u)
        if (perimeter.contains(v)):
          perimeter.changePriority(v, newDist)
        else:
          perimeter.add(v, newDist)
```

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Algorithmus: Laufzeit

$\Theta(|V|)$ →
$\Theta(|V|)$ iterations →
$\Theta(\log |V|)$ →

```
dijkstraShortestPath(G graph, V start)
  Map edgeTo, distTo;
  initialize distTo with all nodes mapped to ∞, except start to 0

  PriorityQueue<V> perimeter; perimeter.add(start);

  while (!perimeter.isEmpty()):
    u = perimeter.removeMin()

    for each edge (u,v) to unknown v with weight w:
      oldDist = distTo.get(v)      // previous best path to v
      newDist = distTo.get(u) + w  // what if we went through u?
      if (newDist < oldDist):
        distTo.put(v, newDist)
        edgeTo.put(v, u)
        if (perimeter.contains(v)):
          perimeter.changePriority(v, newDist)
        else:
          perimeter.add(v, newDist)
```

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Algorithmus: Laufzeit

$\Theta(|V|)$ →
$\Theta(|V|)$ iterations →
$\Theta(\log |V|)$ →
<u>total</u> $\Theta(|E|)$ iterations →

```
dijkstraShortestPath(G graph, V start)
  Map edgeTo, distTo;
  initialize distTo with all nodes mapped to ∞, except start to 0

  PriorityQueue<V> perimeter; perimeter.add(start);

  while (!perimeter.isEmpty()):
    u = perimeter.removeMin()

    for each edge (u,v) to unknown v with weight w:
      oldDist = distTo.get(v)      // previous best path to v
      newDist = distTo.get(u) + w  // what if we went through u?
      if (newDist < oldDist):
        distTo.put(v, newDist)
        edgeTo.put(v, u)
        if (perimeter.contains(v)):
          perimeter.changePriority(v, newDist)
        else:
          perimeter.add(v, newDist)
```

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Algorithmus: Laufzeit

$\Theta(|V|)$ →
$\Theta(|V|)$ iterations →
$\Theta(\log |V|)$ →
total $\Theta(|E|)$ iterations →
$\Theta(1)$ →

```
dijkstraShortestPath(G graph, V start)
  Map edgeTo, distTo;
  initialize distTo with all nodes mapped to ∞, except start to 0

  PriorityQueue<V> perimeter; perimeter.add(start);

  while (!perimeter.isEmpty()):
    u = perimeter.removeMin()

    for each edge (u,v) to unknown v with weight w:
      oldDist = distTo.get(v)      // previous best path to v
      newDist = distTo.get(u) + w  // what if we went through u?
      if (newDist < oldDist):
        distTo.put(v, newDist)
        edgeTo.put(v, u)
        if (perimeter.contains(v)):
          perimeter.changePriority(v, newDist)
        else:
          perimeter.add(v, newDist)
```

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Algorithmus: Laufzeit

$\Theta(|V|)$ →
$\Theta(|V|)$ iterations →
$\Theta(\log |V|)$ →
total $\Theta(|E|)$ iterations →
$\Theta(1)$ →

```
dijkstraShortestPath(G graph, V start)
  Map edgeTo, distTo;
  initialize distTo with all nodes mapped to ∞, except start to 0

  PriorityQueue<V> perimeter; perimeter.add(start);

  while (!perimeter.isEmpty()):
    u = perimeter.removeMin()

    for each edge (u,v) to unknown v with weight w:
      oldDist = distTo.get(v)      // previous best path to v
      newDist = distTo.get(u) + w  // what if we went through u?
      if (newDist < oldDist):
        distTo.put(v, newDist)
        edgeTo.put(v, u)
        if (perimeter.contains(v)):
          perimeter.changePriority(v, newDist)
        else:
          perimeter.add(v, newDist)
```

if oldDist < ∞:

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Algorithmus: Laufzeit

```
dijkstraShortestPath(G graph, V start)
  Map edgeTo, distTo;
  initialize distTo with all nodes mapped to ∞, except start to 0

  PriorityQueue<V> perimeter; perimeter.add(start);

  while (!perimeter.isEmpty()):
    u = perimeter.removeMin()

    for each edge (u,v) to unknown v with weight w:
      oldDist = distTo.get(v)      // previous best path to v
      newDist = distTo.get(u) + w  // what if we went through u?
      if (newDist < oldDist):
        distTo.put(v, newDist)
        edgeTo.put(v, u)
        if (perimeter.contains(v)):
          perimeter.changePriority(v, newDist)
        else:
          perimeter.add(v, newDist)
```

$\Theta(|V|)$

$\Theta(|V|)$ iterations

$\Theta(\log |V|)$

total $\Theta(|E|)$ iterations

$\Theta(1)$

$\Theta(\log |V|)$

if oldDist < ∞:

log(n) wegen add()

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Algorithmus: Laufzeit

```
dijkstraShortestPath(G graph, V start)
  Map edgeTo, distTo;
  initialize distTo with all nodes mapped to ∞, except start to 0

  PriorityQueue<V> perimeter; perimeter.add(start);

  while (!perimeter.isEmpty()):
    u = perimeter.removeMin()

    for each edge (u,v) to unknown v with weight w:
      oldDist = distTo.get(v)      // previous best path to v
      newDist = distTo.get(u) + w  // what if we went through u?
      if (newDist < oldDist):
        distTo.put(v, newDist)
        edgeTo.put(v, u)
        if (perimeter.contains(v)):
          perimeter.changePriority(v, newDist)
        else:
          perimeter.add(v, newDist)
```

$\Theta(|V|)$

$\Theta(|V|)$ iterations

$\Theta(\log |V|)$

total $\Theta(|E|)$ iterations

$\Theta(1)$

$\Theta(\log |V|)$

$\Theta(\log |V|)$

if oldDist < ∞:

log(n) wegen add()

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Algorithmus: Laufzeit

$\Theta(|V|)$ →
$\Theta(|V|\log|V|)$ { $\Theta(|V|)$ iterations →
$\Theta(\log|V|)$ →
total $\Theta(|E|)$ iterations →
$\Theta(1)$ →
$\Theta(\log|V|)$ →
$\Theta(\log|V|)$ →

```
dijkstraShortestPath(G graph, V start)
  Map edgeTo, distTo;
  initialize distTo with all nodes mapped to ∞, except start to 0

  PriorityQueue<V> perimeter; perimeter.add(start);

  while (!perimeter.isEmpty()):
    u = perimeter.removeMin()

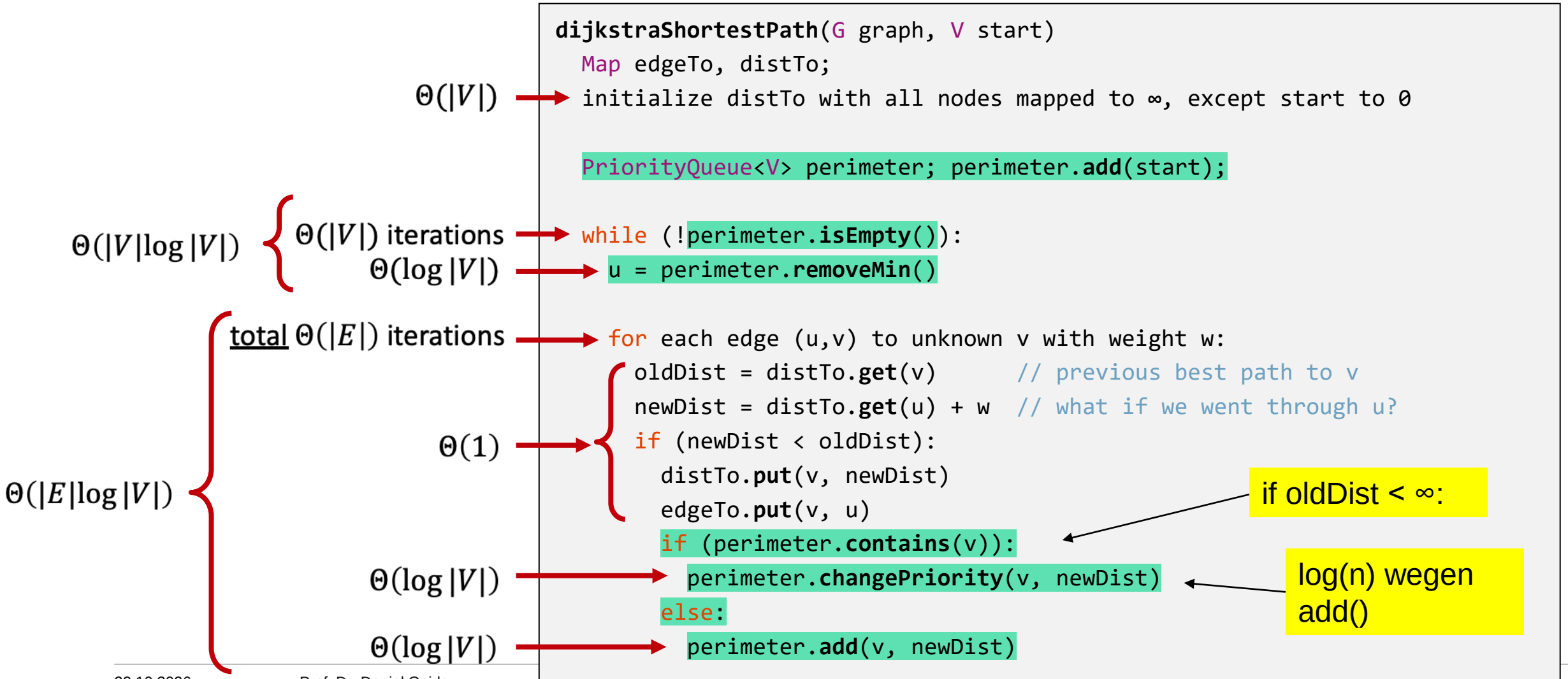
    for each edge (u,v) to unknown v with weight w:
      oldDist = distTo.get(v)      // previous best path to v
      newDist = distTo.get(u) + w  // what if we went through u?
      if (newDist < oldDist):
        distTo.put(v, newDist)
        edgeTo.put(v, u)
        if (perimeter.contains(v)):
          perimeter.changePriority(v, newDist)
        else:
          perimeter.add(v, newDist)
```

if oldDist < ∞:

log(n) wegen add()

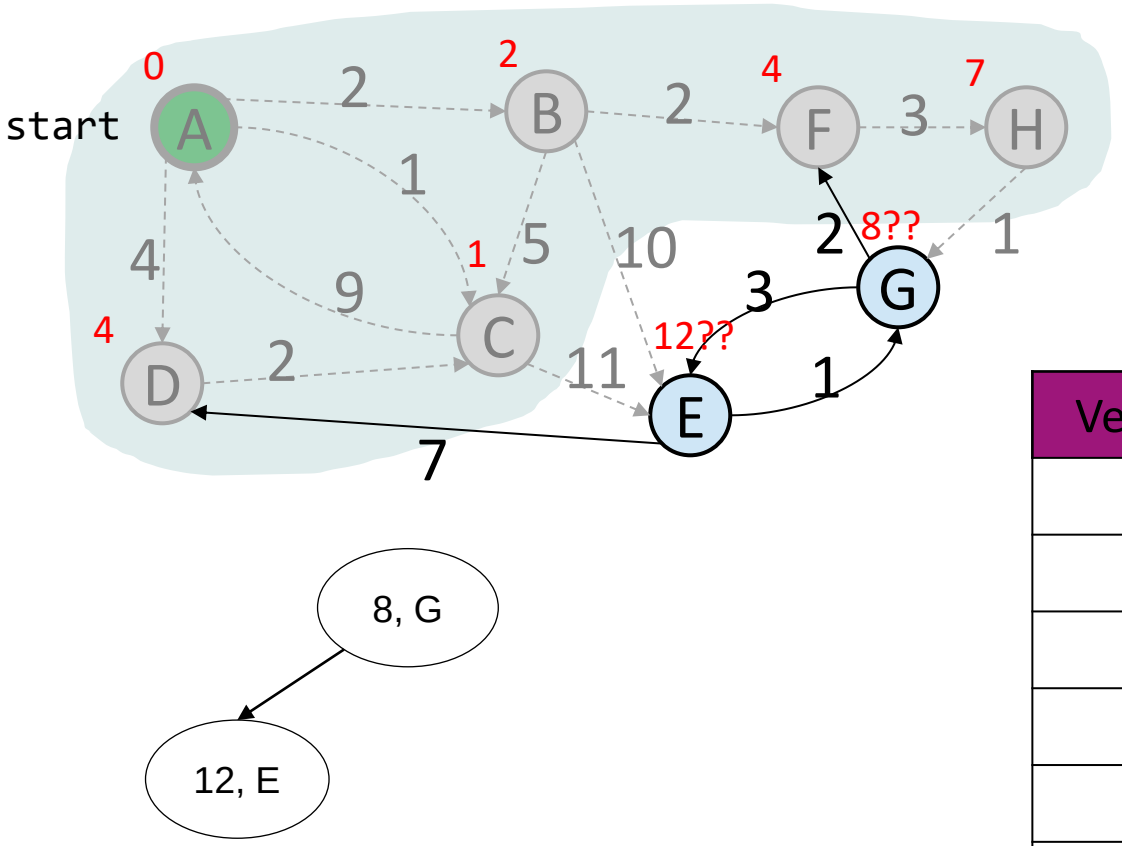
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Algorithmus: Laufzeit



Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Algorithmus: Beispiel

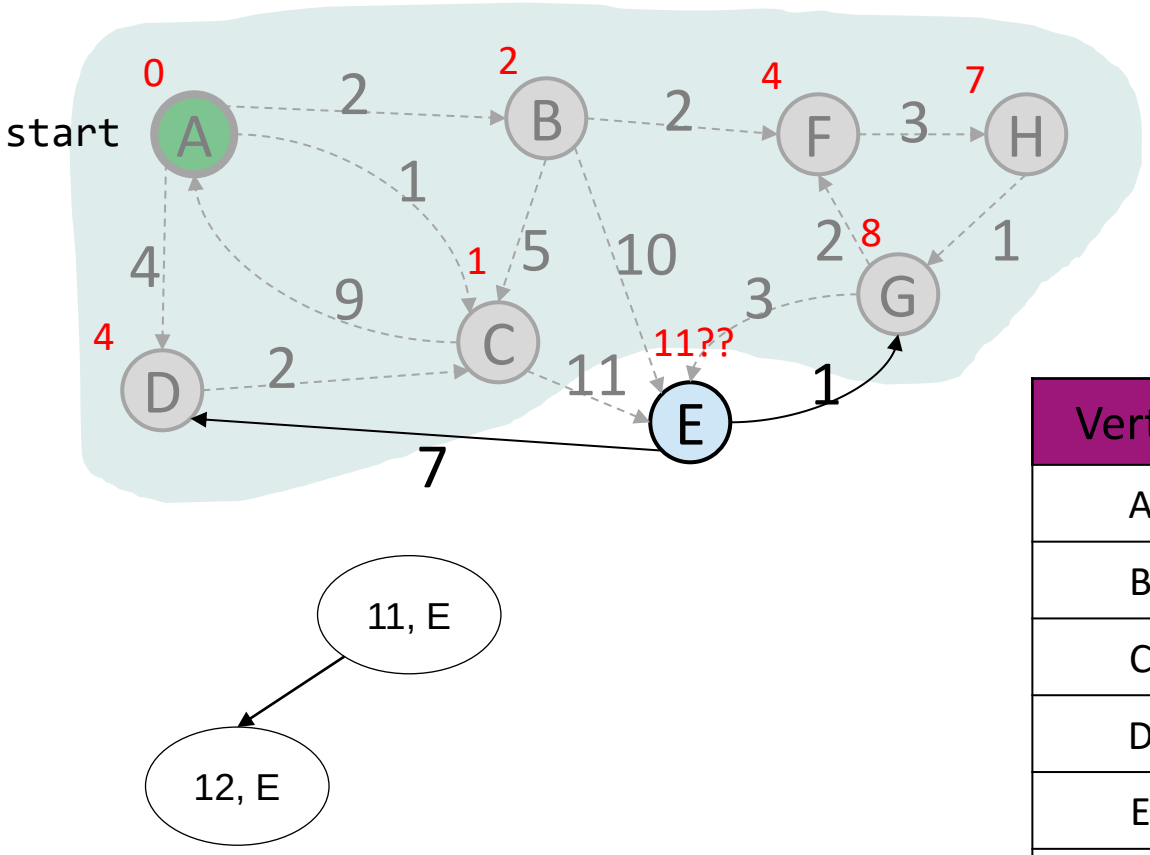



```
while (there are unknown vertices):
  let u be the closest unknown vertex
  known.add(u);
  for each edge (u,v) to unknown v with w:
    if distTo.get(u) + w < distTo.get(v):
```

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B	Y	2	A
C	Y	1	A
D	Y	4	A
E		≤ 12	C
F	Y	4	B
G		**≤ 8**	**H**
H	Y	7	F

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Algorithmus: Beispiel

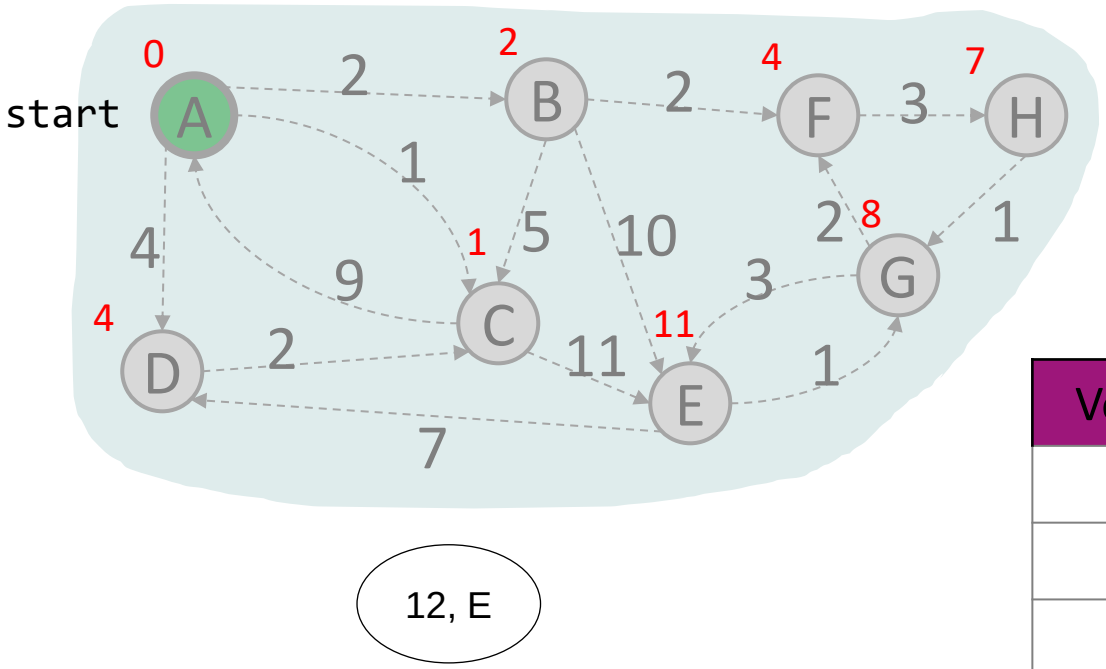



```
while (there are unknown vertices):
  let u be the closest unknown vertex
  known.add(u);
  for each edge (u,v) to unknown v with w:
    if distTo.get(u) + w < distTo.get(v):
```

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B	Y	2	A
C	Y	1	A
D	Y	4	A
E		**≤ 11**	**G**
F	Y	4	B
G	Y	8	H
H	Y	7	F

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Algorithmus: Beispiel



12, E

u: E, v: G → 12 + 1 < 8 Falsch

u: E, v: D → 12 + 7 < 4 Falsch

```
while (there are unknown vertices):
  let u be the closest unknown vertex
  known.add(u);
  for each edge (u,v) to unknown v with w:
    if distTo.get(u) + w < distTo.get(v):
```

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B	Y	2	A
C	Y	1	A
D	Y	4	A
E	Y	11	G
F	Y	4	B
G	Y	8	H
H	Y	7	F

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Dijkstras Algorithmus: Laufzeit

Finales Ergebnis:

$$\Theta(|V|\log|V| + |E|\log|V|)$$

Bei zusammenhängenden Graph:

- Annahme, dass |E| größer als |V| ist, also dominiert |E|log|V|.
- Aber das stimmt nicht immer!

$\Theta(|V|)$

$\Theta(|V|\log|V|)$ = $\Theta(|V|)$ iterations × $\Theta(\log|V|)$

$\Theta(|E|\log|V|)$ = total $\Theta(|E|)$ iterations × ($\Theta(1)$ + $\Theta(\log|V|)$ + $\Theta(\log|V|)$)

```
dijkstraShortestPath(G graph, V start)
  Map edgeTo, distTo;
  initialize distTo with all nodes mappe

  PriorityQueue<V> perimeter; perimeter.

  while (!perimeter.isEmpty()):
    u = perimeter.removeMin()

    for each edge (u,v) to unknown v wit
      oldDist = distTo.get(v)       // pr
      newDist = distTo.get(u) + w   // wh
      if (newDist < oldDist):
        distTo.put(v, newDist)
        edgeTo.put(v, u)
        if (perimeter.contains(v)):
          perimeter.changePriority(v, ne
        else:
          perimeter.add(v, newDist)
```


Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
TH Köln

Fragen?

Übungsblatt 7: Aufgabe (5), 6, (9), 14

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Technology Arts Sciences TH Köln

Greedy Algorithmus

Tun in jeder Iteration das, was ihnen in dieser Iteration am besten erscheint.

- „Sofortige Befriedigung“ - gierig
- „in jeder Iteration wird die lokal optimale Wahl getroffen“

Greedy Algorithmus

Tun in jeder Iteration das, was ihnen in dieser Iteration am besten erscheint.

- „Sofortige Befriedigung“ - gierig
- „in jeder Iteration wird die lokal optimale Wahl getroffen“

Dijkstra ist „gierig“ (greedy), denn sobald ein Knoten als „known“ markiert ist, wird er nie wieder besucht.

Aus diesem Grund funktioniert Dijkstra nicht mit negativen Kantengewichten.

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2023

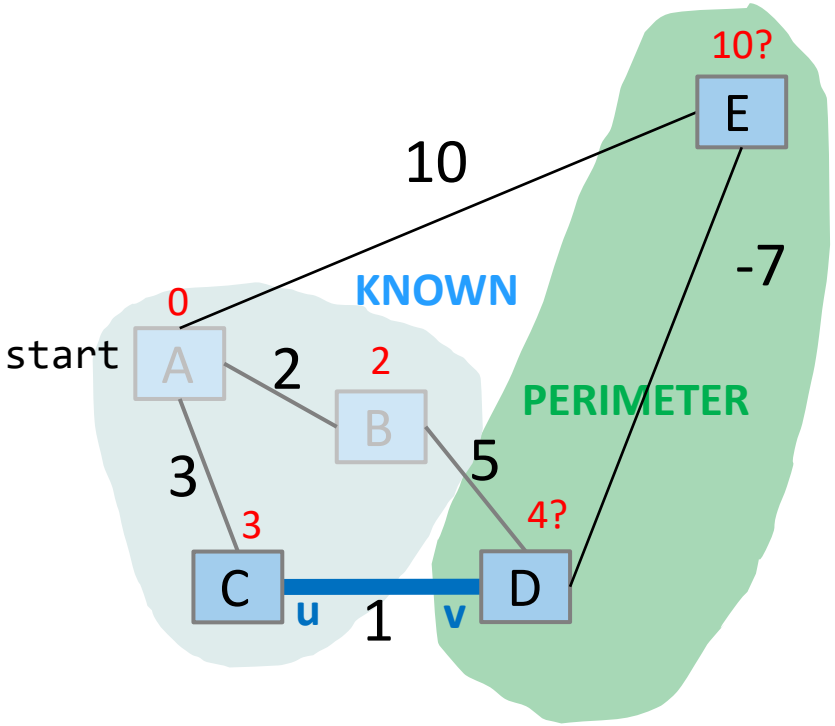
Greedy Algorithmus

Tun in jeder Iteration das, was ihnen in dieser Iteration am besten erscheint.

- „Sofortige Befriedigung“ - gierig
- „in jeder Iteration wird die lokal optimale Wahl getroffen“

Dijkstra ist „gierig“ (greedy), denn sobald ein Knoten als „known“ markiert ist, wird er nie wieder besucht.

Aus diesem Grund funktioniert Dijkstra nicht mit negativen Kantengewichten.



Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2023

Greedy Algorithmus

Andere Beispiele für Greedy Algorithmen sind:

- Algorithmus von Kruskal und Algorithmus von Prim für minimale Spannbäume (nächste Woche)
- Gradientenabstieg (Mathematik oder KI)

Bellman-Ford-Algorithmus zur Berechnung des kürzesten Pfads

- Ein Algorithmus für den kürzesten Pfad, der mit negativen Kantengewichten funktioniert
- Funktioniert nicht, wenn ein negativer Zyklus existiert (ein Zyklus dessen Summe der Kantengewichte negativ ist) - in diesem Fall gibt es keinen kürzesten Pfad
-
-

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2023

Bellman-Ford-Algorithmus zur Berechnung des kürzesten Pfads

- Ein Algorithmus für den kürzesten Pfad, der mit negativen Kantengewichten funktioniert
- Funktioniert nicht, wenn ein negativer Zyklus existiert (ein Zyklus dessen Summe der Kantengewichte negativ ist) - in diesem Fall gibt es keinen kürzesten Pfad

- Kein Greedy Algorithmus

- Ursprünglich vorgeschlagen von Alfonso Shimbel, dann veröffentlicht von Edward F. Moore, dann wieder veröffentlicht von Lester Ford Jr. und schließlich benannt nach Richard Bellman (Erfinder der dynamischen Programmierung), dessen Veröffentlichung auf der von Ford aufbaute

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2023

Bellman-Ford-Algorithmus: Pseudocode

```
für jedes v aus V
    Distanz(v) := unendlich, Vorgänger(v) := keiner
Distanz(s) := 0            // Distanz von Startknoten auf 0 setzen

wiederhole n − 1 mal
    für jedes (u, v) aus E
        wenn (Distanz(u) + Gewicht(u, v)) < Distanz(v) dann
            Distanz(v) := Distanz(u) + Gewicht(u, v)              // distTo
            Vorgänger(v) := u          // edgeTo

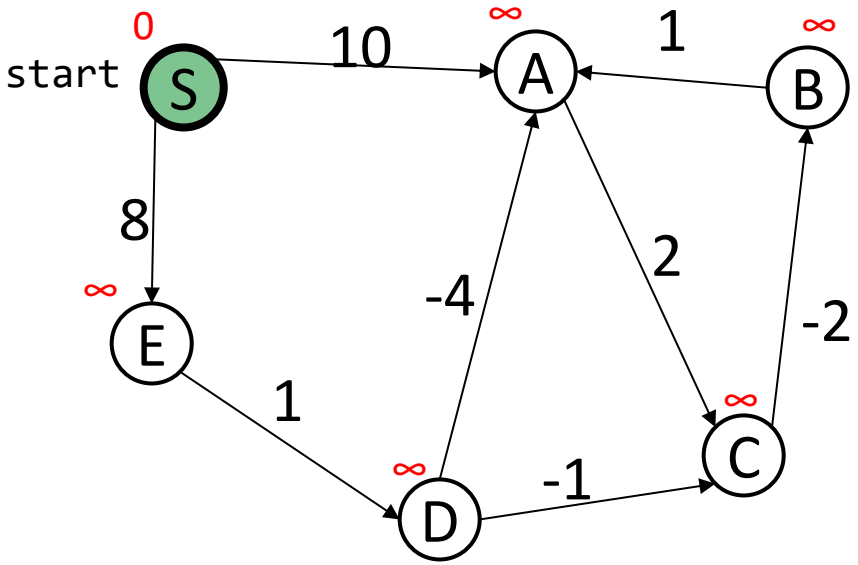
Ausgabe Distanz, Vorgänger
```

Bellman-Ford-Algorithmus Basics

- There can be at most |V| - 1 edges in our shortest path
  - If there are |V| or more edges in a path that means there's a cycle/repeated Vertex
- Run |V| - 1 iterations of shortest path analysis through the graph
  - This means we will repeatedly revisit the "distance from" selected per vertex
- Look at each vertex's outgoing edges in each iteration
- It is slower than Dijkstra's for the same problem because it will revisit previously assessed vertices

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2023

Bellman-Ford-Algorithmus: Beispiel (1 von 5)

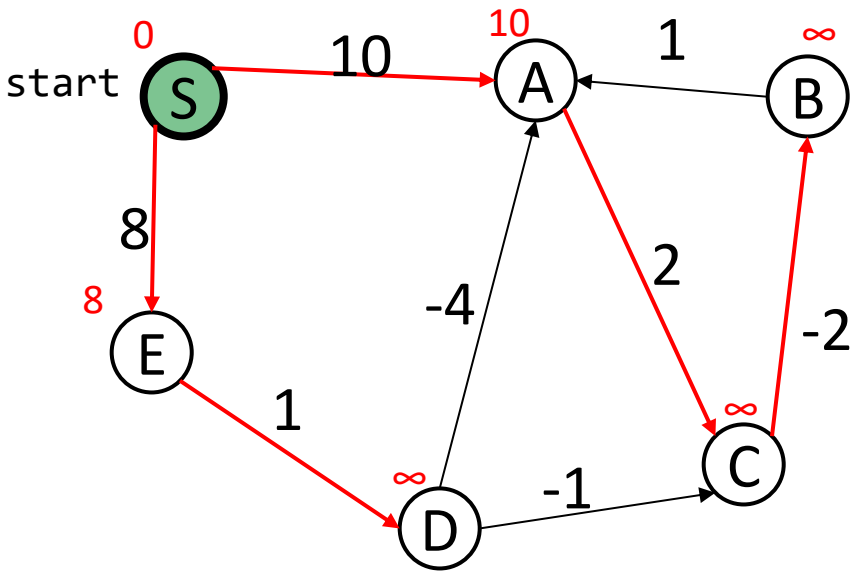


Vertex	distTo	edgeTo
S	0	
A	∞	
B	∞	
C	∞	
D	∞	
E	∞	

Example Walk Through Video (5min)

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2023

Bellman-Ford-Algorithmus: Beispiel (2 von 5)



Iteration 1 - für jede ausgehende Kante eines jeden Knotens: Erhalten wir einen kürzeren Pfad zu einem unbekannten Knoten?

Vertex	distTo	edgeTo
S	0	-
A	10	S
B	∞	
C	∞	
D	∞	
E	8	S

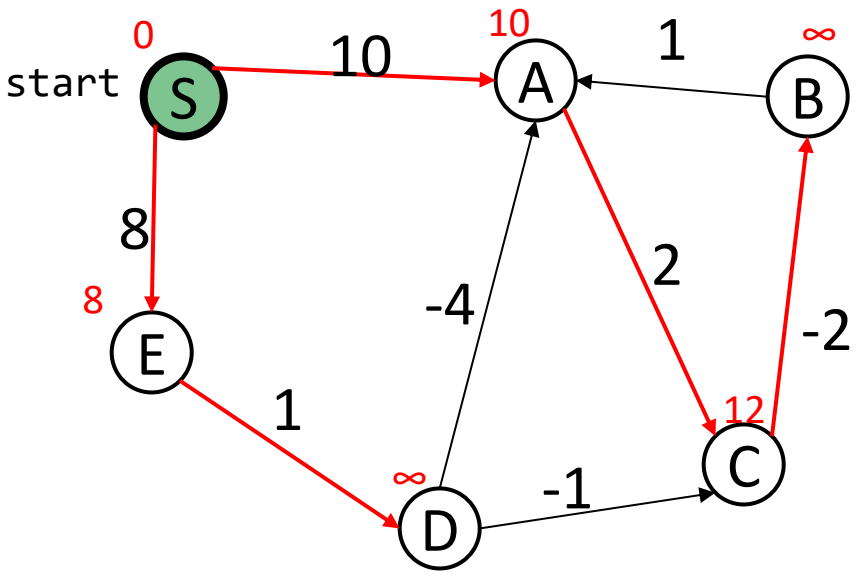
für jedes (u, v) aus E

wenn (Distanz(u) + Gewicht(u, v)) < Distanz(v) dann

Distanz(v) := Distanz(u) + Gewicht(u, v) // distTo

Vorgänger(v) := u // edgeTo

Bellman-Ford-Algorithmus: Beispiel (2 von 5)



Iteration 1 - für jede ausgehende Kante eines jeden Knotens: Erhalten wir einen kürzeren Pfad zu einem unbekannten Knoten?

Vertex	distTo	edgeTo
S	0	-
A	10	S
B	∞	
C	12	A
D	∞	
E	8	S

für jedes (u, v) aus E

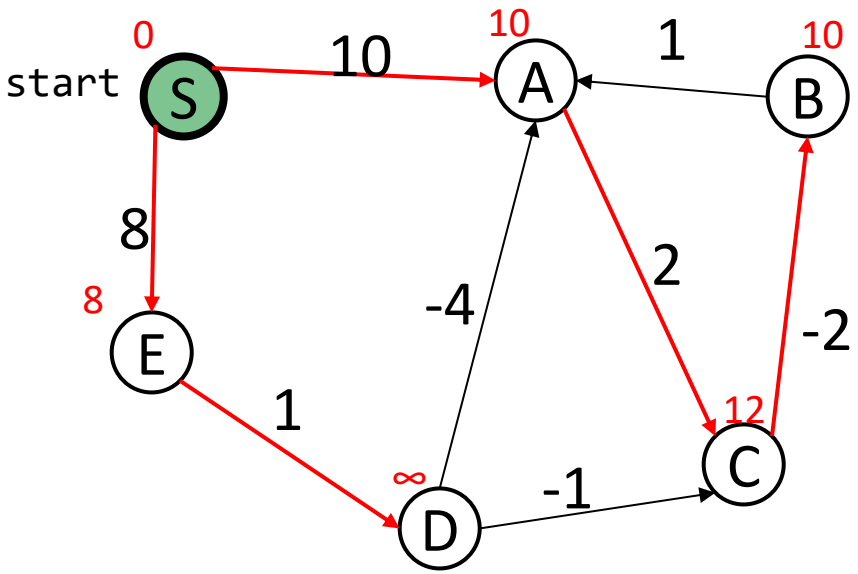
wenn (Distanz(u) + Gewicht(u, v)) < Distanz(v) dann

Distanz(v) := Distanz(u) + Gewicht(u, v) // distTo

Vorgänger(v) := u // edgeTo

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2023

Bellman-Ford-Algorithmus: Beispiel (2 von 5)



Iteration 1 - für jede ausgehende Kante eines jeden Knotens: Erhalten wir einen kürzeren Pfad zu einem unbekannten Knoten?

Vertex	distTo	edgeTo
S	0	-
A	10	S
B	10	C
C	12	A
D	∞	
E	8	S

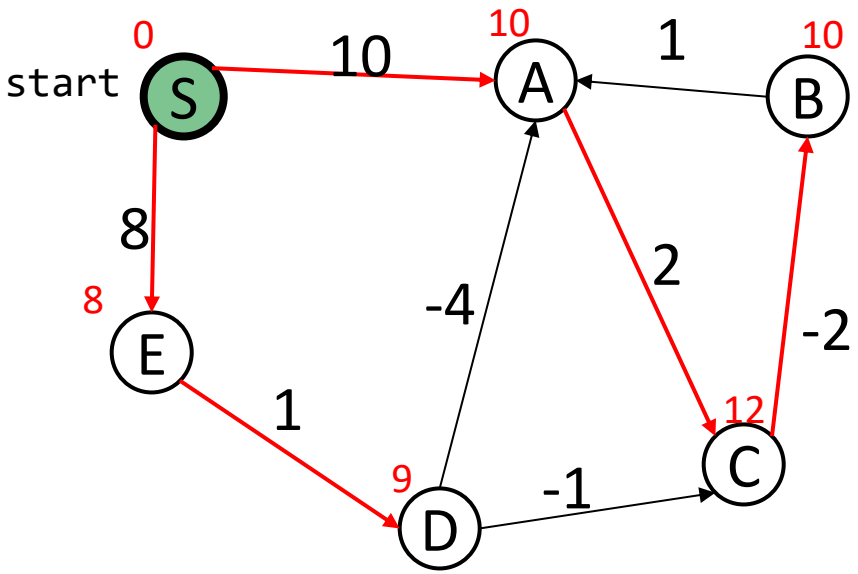
für jedes (u, v) aus E

wenn (Distanz(u) + Gewicht(u, v)) < Distanz(v) dann

Distanz(v) := Distanz(u) + Gewicht(u, v) // distTo

Vorgänger(v) := u // edgeTo

Bellman-Ford-Algorithmus: Beispiel (2 von 5)



Iteration 1 - für jede ausgehende Kante eines jeden Knotens: Erhalten wir einen kürzeren Pfad zu einem unbekannten Knoten?

Vertex	distTo	edgeTo
S	0	-
A	10	S
B	10	C
C	12	A
D	9	E
E	8	S

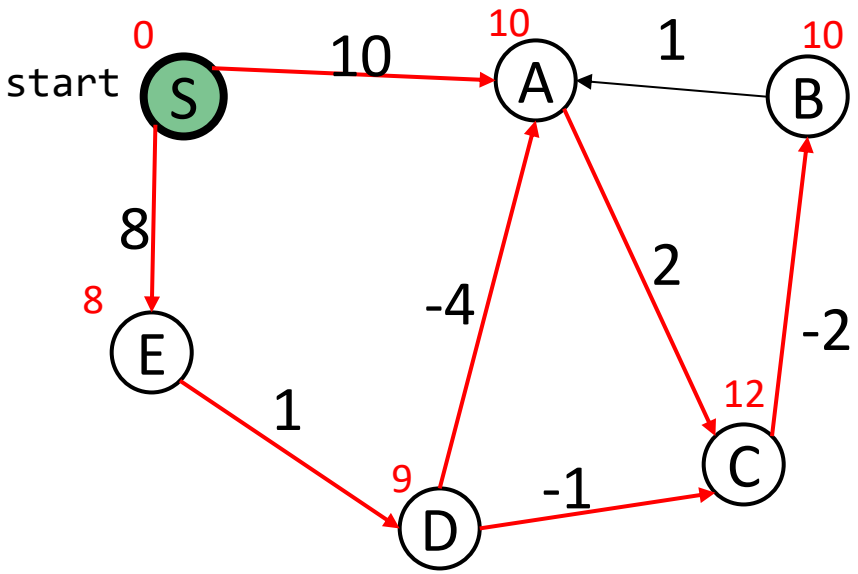
für jedes (u, v) aus E

wenn (Distanz(u) + Gewicht(u, v)) < Distanz(v) dann

Distanz(v) := Distanz(u) + Gewicht(u, v) // distTo

Vorgänger(v) := u // edgeTo

Bellman-Ford-Algorithmus: Beispiel (3 von 5)



Iteration 2 - erneute Überprüfung der ausgehenden Kanten: Können wir den Abstand zu einem gegebenen Knoten verbessern?

Vertex	distTo	edgeTo
S	0	-
A	~~10~~ **5**	~~S~~ **D**
B	10	C
C	~~12~~ **8**	~~A~~ **D**
D	9	E
E	8	S

Da die Entfernung zu D zum Zeitpunkt der Verarbeitung von D bekannt ist, können wir die ausgehenden Kanten von D in die Betrachtung einbeziehen

für jedes (u, v) aus E

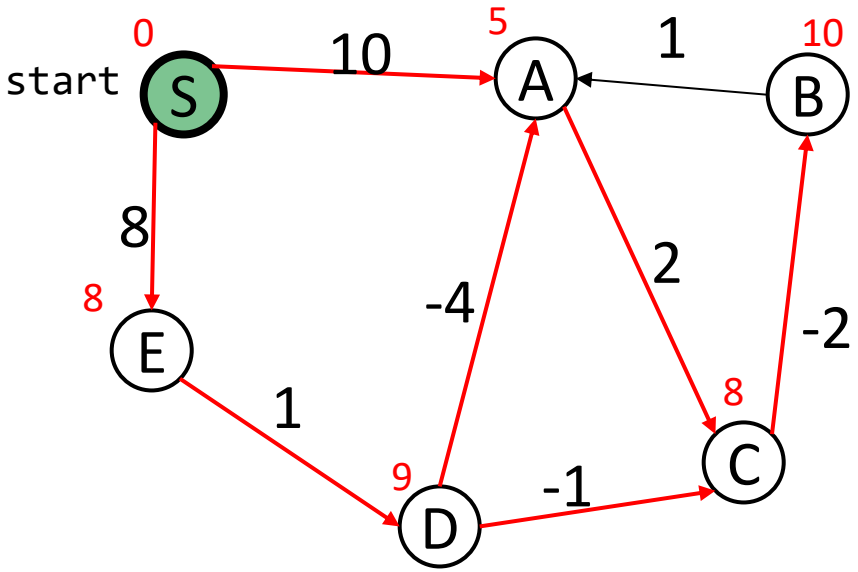
    wenn (Distanz(u) + Gewicht(u, v)) < Distanz(v) dann

        Distanz(v) := Distanz(u) + Gewicht(u, v) // distTo

        Vorgänger(v) := u // edgeTo

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2023

Bellman-Ford-Algorithmus: Beispiel (4 von 5)



Iteration 3 – Wiederhole!

Vertex	distTo	edgeTo
S	0	-
A	5	D
B	10	C
C	~~8~~ **7**	A
D	9	E
E	8	S

für jedes (u, v) aus E

  wenn (Distanz(u) + Gewicht(u, v)) < Distanz(v) dann

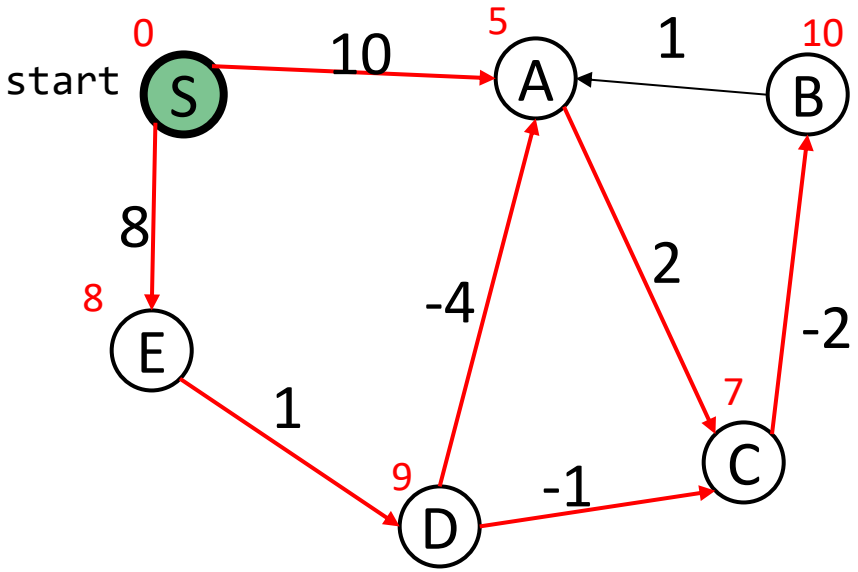
    Distanz(v) := Distanz(u) + Gewicht(u, v) // distTo

    Vorgänger(v) := u // edgeTo

Mit einem verkürzten Abstand zu A aus Iteration 2 können wir den Abstand zu C verbessern

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2023

Bellman-Ford-Algorithmus: Beispiel (4 von 5)



Iteration 3 – Wiederhole!

Vertex	distTo	edgeTo
S	0	-
A	5	D
B	~~10~~ **5**	C
C	~~8~~ **7**	A
D	9	E
E	8	S

Mit einem verkürzten Abstand zu C aus dieser Iteration können wir den Abstand zu B verbessern

Mit einem verkürzten Abstand zu A aus Iteration 2 können wir den Abstand zu C verbessern

für jedes (u, v) aus E

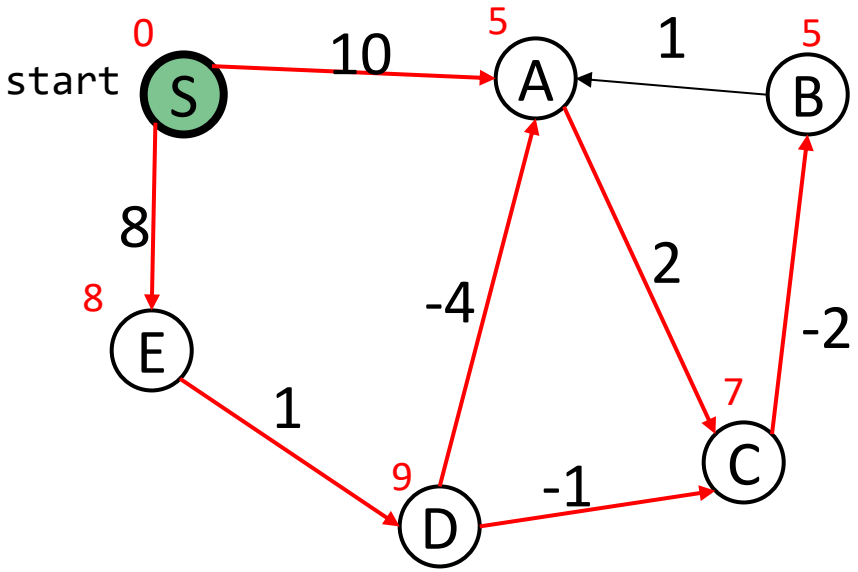
  wenn (Distanz(u) + Gewicht(u, v)) < Distanz(v) dann

    Distanz(v) := Distanz(u) + Gewicht(u, v) // distTo

    Vorgänger(v) := u // edgeTo

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2023

Bellman-Ford-Algorithmus: Beispiel (5 von 5)



Iteration 4 – Wiederhole!

Vertex	distTo	edgeTo
S	0	-
A	5	D
B	5	C
C	7	A
D	9	E
E	8	S

Keine Änderungen!
Das bedeutet, dass wir die Iteration früher abbrechen können

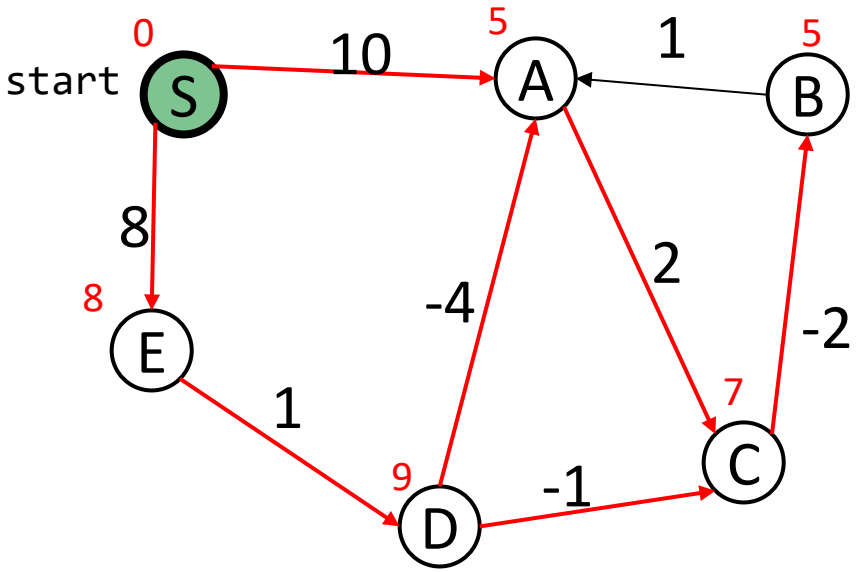
für jedes (u, v) aus E

    wenn (Distanz(u) + Gewicht(u, v)) < Distanz(v) dann

        Distanz(v) := Distanz(u) + Gewicht(u, v) // distTo

        Vorgänger(v) := u // edgeTo

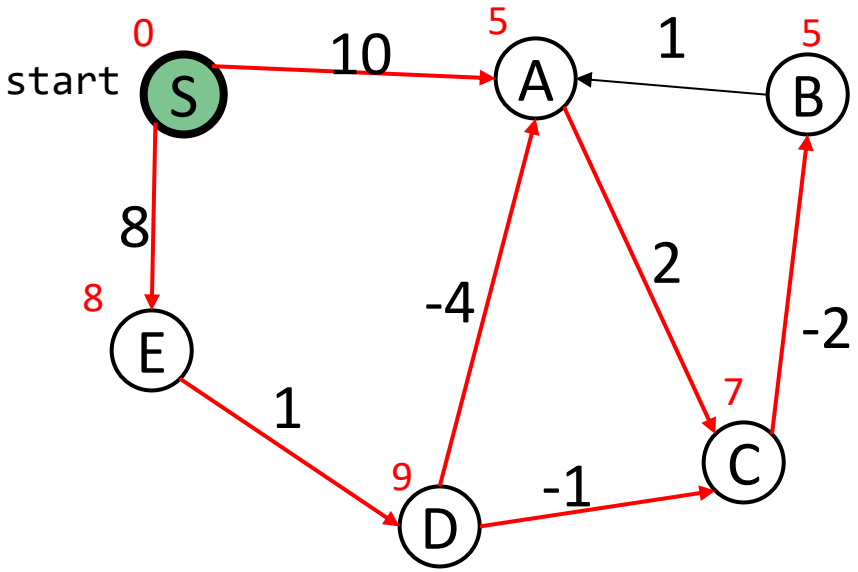
Bellman-Ford-Algorithmus: Beispiel (5 von 5)



Kürzester Pfad von S zu C ist:
C -> A -> D -> E -> S

Vertex	distTo	edgeTo
S	0	-
A	5	D
B	5	C
C	7	A
D	9	E
E	8	S

Bellman-Ford-Algorithmus: Beispiel (5 von 5)



Kürzester Pfad von S zu C ist:
C -> A -> D -> E -> S
Distanz berechnet sich über:
2 – 4 + 1 + 8 = 7

Vertex	distTo	edgeTo
S	0	-
A	5	D
B	5	C
C	7	A
D	9	E
E	8	S

Graphenprobleme

Es gibt viele interessante Fragen, die man einem Graphen stellen kann:

- Was ist der kürzeste Pfad von S nach T?
- Was ist der längste Pfad ohne Zyklen?
- Gibt es Zyklen?
- Gibt es eine Tour (einen Zyklus), bei der man jeden Knoten (Station) genau einmal besucht?
- Gibt es eine Tour (einen Zyklus), die jede Kante genau einmal benutzt?

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022
CSE 373 20 WI – HANNAH TANG

Graphenprobleme

Einige bekannte Graphenprobleme und ihre gebräuchlichen Namen:

- Erreichbarkeitsproblem: Gibt es einen Pfad zwischen den Knoten s und t?
- Konnektivität: Ist der Graph verbunden?
- Bikonnektivität: Gibt es einen Knoten, dessen Beseitigung die Konnektivität des Graphen aufhebt?
- Kürzester Pfad s-t: Welches ist der kürzeste Pfad zwischen den Knoten s und t?
- Zyklus-Erkennung: Enthält der Graph irgendwelche Zyklen?
- Eulertour: Gibt es einen Zyklus, der jede Kante genau einmal benutzt?
- Hamiltonkreis: Gibt es einen Zyklus, bei dem jeder Knoten genau einmal verwendet wird?
- Planarer Graph: Kann man den Graphen auf Papier zeichnen, ohne dass sich die Kanten kreuzen?
- Isomorphie: Haben zwei Graphen dieselbe Struktur?

Graphenprobleme gehören zu den mathematisch interessantesten Gebieten der Computerwissenschaft!

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022
CSE 373 20 WI – HANNAH TANG

Graphenprobleme - Forschung

Es gibt auch noch Forschung in diesem Bereich:

- Ran Duan, Jiayi Mao, Xiao Mao, Xinkai Shu, Longhui Yin (2025): „Breaking the Sorting Barrier for Directed Single-Source Shortest Paths“. 57th Annual ACM Symposium on Theory of Computing (STOC).
  „Our new algorithm recursively shrinks the size of the frontier being considered by mixing Dijkstra's and another textbook algorithm, called Bellman-Ford, along with a cleverly designed data structure that allows insertion and extraction in groups“

- „We give a deterministic $O(m \log^{2/3} n)$-time algorithm for single-source shortest paths (SSSP) on directed graphs with real non-negative edge weights in the comparison-addition model. This is the first result to break the O(m + n logn) time bound of Dijkstra’s algorithm on sparse graphs, showing that Dijkstra’s algorithm is not optimal for SSSP.“, https://dl.acm.org/doi/10.1145/3717823.3718179

Quelle: https://www.mpi-inf.mpg.de/news/detail/stoc-best-paper-award-how-to-find-the-shortest-path-faster

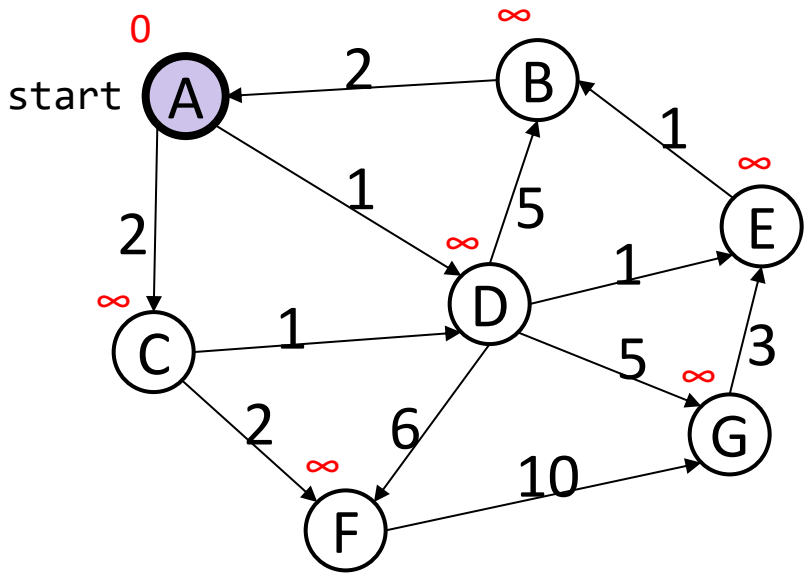
Lernziele von heute und Fragen zur Überprüfung der Lernziele

Die Studierenden können Problemstellungen als Graphen formulieren und einige Fragestellungen zu den Graphen beantworten, wie bspw. dem kürzesten (gewichteten) Pfad, Erreichbarkeit und Topologische Sortierung, indem sie Algorithmen wie Tiefen-, Breitensuche und Dijkstras Algorithmus anwenden, um später komplexe Problemstellungen mit Hilfe von Graphen lösen zu können.

s. Übungsblatt 7 und Praktikum 3

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Technology Arts Sciences TH Köln

Dijkstras Algorithmus: Übung




```
while (there are unknown vertices):
  let u be the closest unknown vertex
  known.add(u);
  for each edge (u,v) to unknown v with w:
    if distTo.get(u) + w < distTo.get(v):
```

Vertex	Known?	distTo	edgeTo
A		0	
B		∞	
C		∞	
D		∞	
E		∞	
F		∞	
G		∞	

Order Added to Known Set:

Prof. Dr. Daniel Gaida

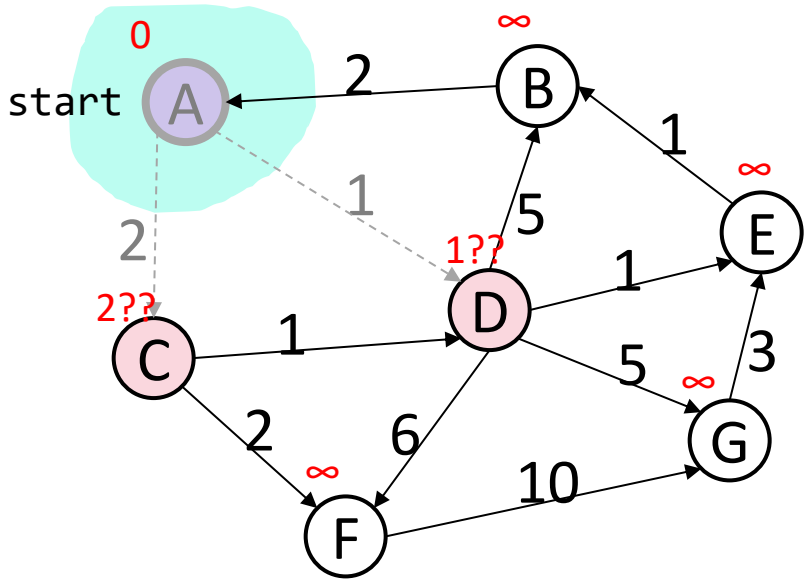
Professor für Cyber-Physische Systeme

Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

CSE 373 20 SU – Aaron Johnston

Dijkstras Algorithmus: Übung




```
while (there are unknown vertices):
  let u be the closest unknown vertex
  known.add(u);
  for each edge (u,v) to unknown v with w:
    if distTo.get(u) + w < distTo.get(v):
```

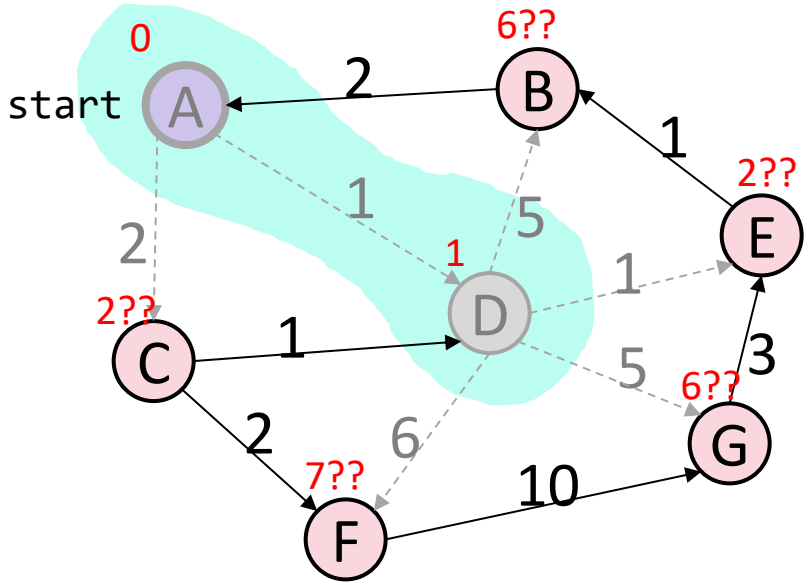
Vertex	Known?	distTo	edgeTo
A	Y	0	/
B		∞	
C		**≤ 2**	**A**
D		**≤ 1**	**A**
E		∞	
F		∞	
G		∞	

Order Added to Known Set:
A

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022
CSE 373 20 SU – Aaron Johnston

Dijkstras Algorithmus: Übung




```
while (there are unknown vertices):
  let u be the closest unknown vertex
  known.add(u);
  for each edge (u,v) to unknown v with w:
    if distTo.get(u) + w < distTo.get(v):
```

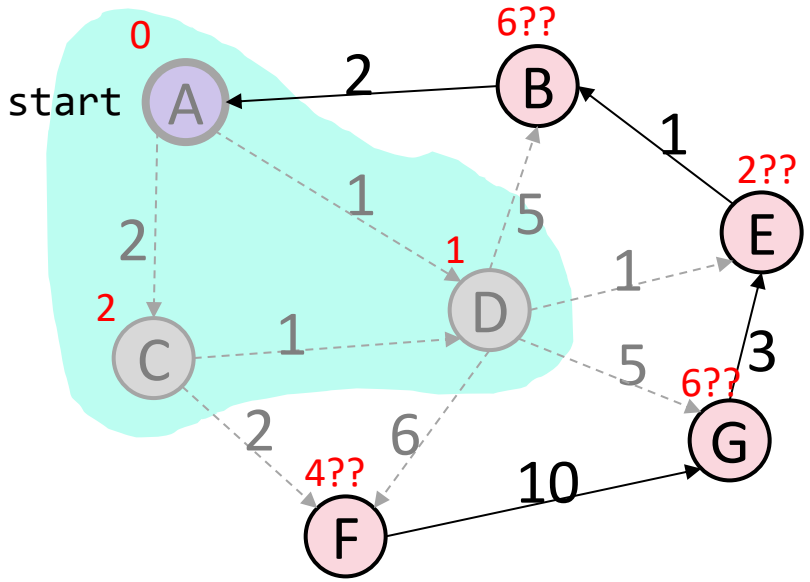
Vertex	Known?	distTo	edgeTo
A	Y	0	/
B		**≤ 6**	**D**
C		≤ 2	A
D	Y	1	A
E		**≤ 2**	**D**
F		**≤ 7**	**D**
G		**≤ 6**	**D**

Order Added to Known Set:
A, D

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

CSE 373 20 SU – Aaron Johnston

Dijkstras Algorithmus: Übung




```
while (there are unknown vertices):
  let u be the closest unknown vertex
  known.add(u);
  for each edge (u,v) to unknown v with w:
    if distTo.get(u) + w < distTo.get(v):
```

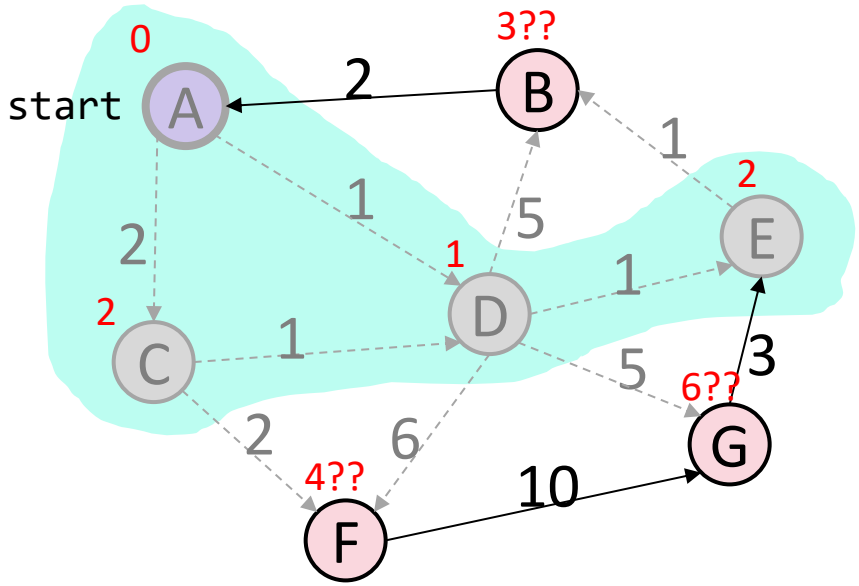
Vertex	Known?	distTo	edgeTo
A	Y	0	/
B		≤ 6	D
C	Y	2	A
D	Y	1	A
E		≤ 2	D
F		**≤ 4**	**C**
G		≤ 6	D

Order Added to Known Set:
A, D, C

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

CSE 373 20 SU – Aaron Johnston

Dijkstras Algorithmus: Übung




```
while (there are unknown vertices):
  let u be the closest unknown vertex
  known.add(u);
  for each edge (u,v) to unknown v with w:
    if distTo.get(u) + w < distTo.get(v):
```

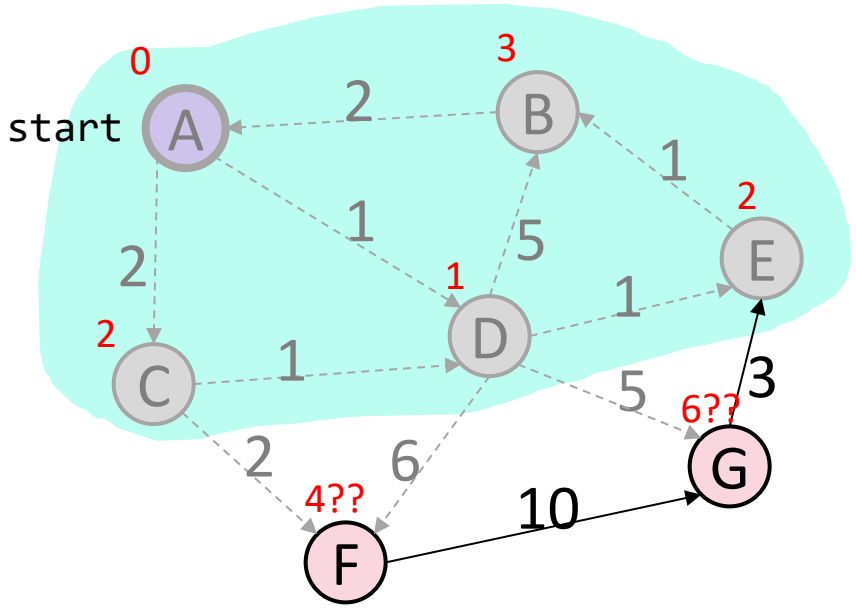
Vertex	Known?	distTo	edgeTo
A	Y	0	/
B		**≤ 3**	**E**
C	Y	2	A
D	Y	1	A
E	Y	2	D
F		≤ 4	C
G		≤ 6	D

Order Added to Known Set:
A, D, C, E

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022
CSE 373 20 SU – Aaron Johnston

Dijkstras Algorithmus: Übung




```
while (there are unknown vertices):
  let u be the closest unknown vertex
  known.add(u);
  for each edge (u,v) to unknown v with w:
    if distTo.get(u) + w < distTo.get(v):
```

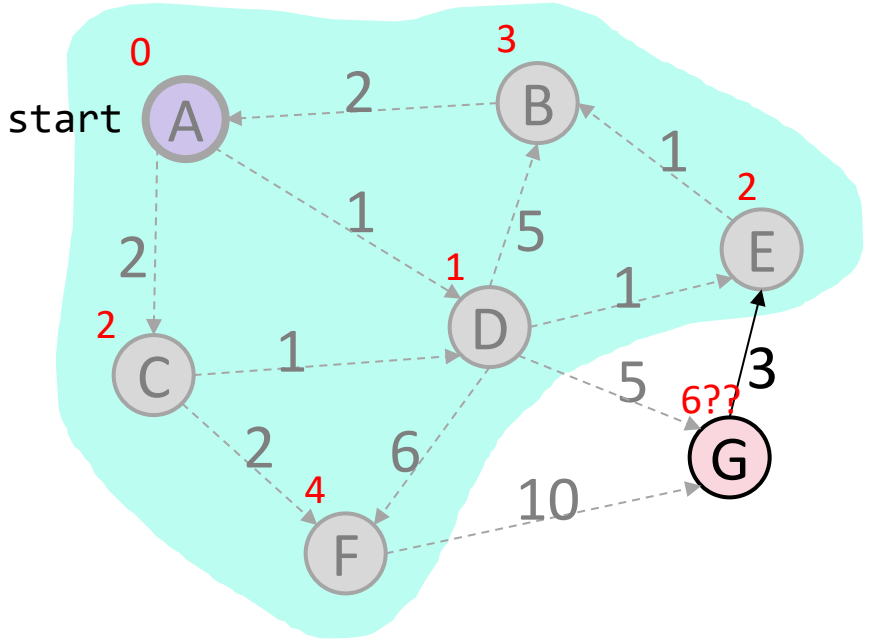
Vertex	Known?	distTo	edgeTo
A	Y	0	/
B	Y	3	E
C	Y	2	A
D	Y	1	A
E	Y	2	D
F		≤ 4	C
G		≤ 6	D

Order Added to Known Set:
A, D, C, E, B

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

CSE 373 20 SU – Aaron Johnston

Dijkstras Algorithmus: Übung




```
while (there are unknown vertices):
  let u be the closest unknown vertex
  known.add(u);
  for each edge (u,v) to unknown v with w:
    if distTo.get(u) + w < distTo.get(v):
```

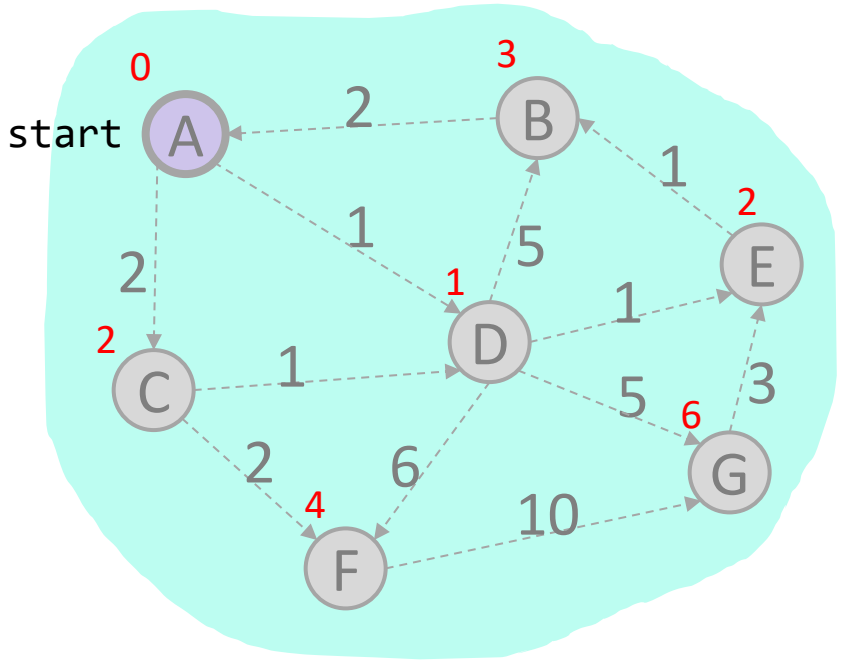
Vertex	Known?	distTo	edgeTo
A	Y	0	/
B	Y	3	E
C	Y	2	A
D	Y	1	A
E	Y	2	D
F	Y	4	C
G		≤ 6	D

Order Added to Known Set:
A, D, C, E, B, F

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

CSE 373 20 SU – Aaron Johnston

Dijkstras Algorithmus: Übung




```
while (there are unknown vertices):
  let u be the closest unknown vertex
  known.add(u);
  for each edge (u,v) to unknown v with w:
    if distTo.get(u) + w < distTo.get(v):
```

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B	Y	3	E
C	Y	2	A
D	Y	1	A
E	Y	2	D
F	Y	4	C
G	Y	6	D

Order Added to Known Set:
A, D, C, E, B, F, G

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

CSE 373 20 SU – Aaron Johnston

Nächste Vorlesung und Fragen

- Minimale Spannbäume
- Fragen?